OBSIDIA / X108
CORPUS MAÎTRE INTÉGRAL - V3

Architecture, boucles fermées, gouvernance, Brody, preuves, domaines, conformité, trajectoire et industrialisation.

Document interne de référence - 3 août 2026

Statut : recherche appliquée / POC local avancé / pré-industrialisation.

Cette V3 conserve la matière des supports sources. Elle ne doit pas être utilisée comme déclaration de conformité, certification ou preuve de production.

Pourquoi cette V3 existe

Les versions précédentes ont cherché à rendre Obsidia immédiatement lisible. Elles ont réussi la compression visuelle, mais elles ont supprimé une partie essentielle de la construction : les raisonnements, les boucles conceptuelles, le passage à l'implémentation, la trajectoire du fondateur, les dossiers métier et les limites détaillées.

La V3 sépare donc deux fonctions : une synthèse narrative pour comprendre rapidement, puis un corpus complet pour vérifier, approfondir et dériver de futurs supports.

Règle de lecture

Un élément présent dans ce corpus peut être un concept, une spécification, un artefact, un test, un branchement ou une preuve. Son inclusion ne change pas son statut réel. Les calibrations les plus récentes priment sur les formulations historiques.

La colonne vertébrale

• Réalité → perception → information située → mémoire qualifiée.

• Cognition → intention → décision → action → preuve → conséquence.

• DomainState → Agents → Aggregate → Meta → GuardX108 → CanonicalDecisionEnvelope.

• Brody parle, recherche et éduque. Sigma observe et alerte. X108 décide.

• KX108_ONLY ; aucun ACT implicite ; BLOCK > HOLD > ALLOW.

Carte du corpus

Bloc	Contenu	Usage
A	Support narratif V2	Lecture générale et cadrage honnête
B	Discussion maîtresse intégrale	Origine des boucles et fermeture conceptuelle
C	Boucles fermées et passage à l'implémentation	Transformation de la vision en ordre de branchement
D	Positionnement, vente et industrialisation	Marché, offres, publics, WorkCenter, roadmap
E	Trajectoire Étienne / Brody V3	Récit public, rôle fondateur, éducation et interface cognitive
F	Pitch investisseur	Scripts, objections, preuves et structure de rendez-vous
G	FRIA scoring crédit	Droits fondamentaux, risques, garde-fous et obligations déployeur
H	État IA Act	Alignement actuel et travaux réglementaires
I	Addendum conversation	Gouvernance personnelle, agents, NUD, GPS et hypothèse Niel

Addendum - Autorité, rôle et prérogatives

Étienne Aubin occupe la fonction de source de trajectoire : il définit l'intention stratégique, les frontières éthiques, les priorités et les conditions de Freeze. Cette position ne doit pas être confondue avec une autorité technique cachée dans chaque exécution. Dans un déploiement, les rôles humains doivent être contractualisés ; X108 conserve l'autorité canonique du système et la validation humaine critique reste définie par le domaine.

Séparation des pouvoirs

Acteur	Fonction	Frontière
Étienne / gouvernance	Vision, intention, arbitrage stratégique, Freeze	Ne remplace pas les contrats runtime
Agents	Exploration, proposition, analyse spécialisée	Aucun ACT
Brody	Dialogue, traduction, recherche, éducation	Non souverain
Sigma	Stabilité, incohérence, alerte, rétention	N'autorise jamais ACT
X108	Qualification canonique ALLOW/HOLD/BLOCK	KX108_ONLY
Opérateur autorisé	Validation critique selon domaine	Identité, compétence et responsabilité tracées

Règle souveraine

L'humain reste l'origine de l'intention et l'autorité finale sur sa trajectoire. Les agents proposent ; Sigma observe ; X108 qualifie structurellement le passage à l'action ; les validations humaines exigées restent explicites et tracées.

Addendum - Cockpit personnel et agents sentinelles

Le Personal Operations Center est l'origine opérationnelle d'Obsidia, pas l'angle produit de la V0.1 entreprise. Il observe la friction humaine - intention, valeur produite, temps consommé, fatigue décisionnelle - afin de protéger la trajectoire du décideur.

Agent	Fonction	État à conserver
#50 ANTI_DISPERSION	Filtrer le bruit et maintenir les priorités	Opérationnel selon le registre fourni
#51 ENERGIE_PRIORITE	Évaluer l'effet des sollicitations sur l'énergie disponible	Non branché / à formaliser
#52 EXTRACTEUR_MATIERE_BRUTE	Transformer les flux techniques en matière décisionnelle	Non branché / connecteurs à valider

Mémoire silencieuse

La couche Interaction Pattern distingue strictement Observé, Interprété et Confirmé. Elle peut mémoriser des habitudes de format ou des contraintes explicites, mais elle ne doit jamais halluciner une personnalité, une intention cachée ou une volonté attribuée à Étienne.

Dette décisionnelle

Lorsqu'un arbitrage humain manque sur un point de rupture logiciel, éthique ou économique, la trajectoire reste en HOLD. Cette stagnation volontaire est une propriété de sûreté, pas une panne.

Addendum - Décisions en attente et frontières

Identifiant	Enjeu	Risque si non gouverné
NUD-N01 / Gencoin	Autoriser seulement une exploration readonly/shadow après preuve	Transformer un concept de valeur cognitive en token ou promotion non autorisée
NUD-N02 / Watchtower	Définir supervision et intégration OS3	Créer une surveillance sans frontière d’autorité claire
NUD-N03 / Cadastre	Définir le cadre opérationnel	Activer une capacité sans mandat explicite
D23 / Timeverse	Valider la logique temporelle et son raccord kernel	Instabilité ou confusion entre projection et autorité
D29 / NPL Action	Autoriser explicitement toute action domaine	Laisser une couche linguistique agir en Banque/Trading
NSC-01	Confirmer l’emplacement de generated_code_authorization_gate	Question documentaire et technique, distincte d’un arbitrage stratégique

Calibration Gencoin

Gencoin reste un mécanisme shadow/readonly de mesure de valeur cognitive. Aucun token, mint, actif financier ou mécanisme de promotion ne doit être présenté comme acquis ou autorisé.

Addendum - Ce que le démonstrateur GPS doit prouver

Le hackathon GPS ne doit pas seulement montrer un fichier hostile qui traverse une API. Il doit démontrer une chaîne causale reproductible allant du signal RF réel à une décision X108, avec comparaison nominale, rupture située et absence de fuite de labels.

Étape	Démonstration attendue	Preuve
1. Corpus	FGI UT_DFMC officiel, licence et hash	Manifest et SHA-256
2. Nominal	NAV/PVT stable avec le même récepteur et la même configuration	Logs et métriques de stabilité
3. Hostile	Divergence après l’onset documenté vers 135 s	Observables alignés temporellement
4. Traduction	Observables → DomainState sans connaissance du label	Config et traces de transformation
5. Autorité	HOLD ou BLOCK produit par X108 selon invariants	CanonicalDecisionEnvelope / receipt
6. Replay	Même entrée, même verdict et mêmes frontières	Rapport reproductible

État honnête au 3 août 2026

Le corpus UTD_L1_E1.dat de 9,82 GiB est présent et une chaîne réelle partielle atteint X108 live. Le niveau RECORDED_RF_ATTACK n’est pas acquis. La perte de lock vers 14 s ne prouve pas l’attaque, car elle précède l’onset hostile attendu.

Addendum - IA Act et FRIA : ce qui reste à fermer

Obsidia apporte des mécanismes utiles à la conformité - séparation d’autorité, gestion de l’incertitude, supervision, logs et replay - mais la conformité dépend d’un produit, d’une finalité, d’un acteur et d’un déploiement. Le dossier doit donc parler d’alignement architectural et de préparation, jamais de conformité acquise.

Priorité	Livrable restant
P0	Registre des produits, finalités prévues et qualification article 6 / Annexes
P0	Dossier technique Annexe IV et evidence registry claim → version → test → rapport
P0	QMS, responsabilités, gestion des risques et critères d’acceptation
P1	Gouvernance des données, conservation, suppression et cybersécurité
P1	Supervision humaine effective, compétences, indépendance et temps disponible
P1	Procédure incidents graves, post-market, recours et coopération autorités
P1	FRIA/DPIA par cas de déploiement réel
P2	Évaluation de conformité, déclaration UE, marquage CE et base UE si applicables

Addendum - Hypothèse Étienne Aubin × Xavier Niel

Cette partie décrit une cible stratégique. Aucun accord, intégration Free, collaboration Scaleway/Mistral/Kyutai ou accès réseau n’existe à ce stade.

La thèse de rencontre

L’écosystème européen peut réunir infrastructure de calcul, modèles et réseau. Obsidia propose une quatrième couche : l’autorité vérifiable entre intelligence et action. L’approche doit être “sniper” : un problème, une rupture, une démonstration bornée, une demande claire.

Message possible

Vous avez l’infrastructure et les modèles. J’ai construit la couche qui empêche un modèle de devenir souverain lorsqu’il commence à agir.

Physical Signal World Model

L’idée consiste à transformer des observables RF agrégés en information physique située, sans identifier les personnes ni cartographier la vie privée. Les réseaux pourraient devenir des capteurs de perturbations du monde physique ; X108 gouvernerait l’usage de cette information. Cette capacité reste une hypothèse de recherche et doit être présentée comme telle.

Couche	Apport possible
Infrastructure européenne	Calcul, hébergement, distribution
Modèles	Exploration, génération, compréhension
Télécom / signal terrain	Observables physiques autorisés et agrégés
Obsidia X108	Autorité, HOLD/BLOCK, preuve et replay
Opérateur humain	Finalité, validation critique et responsabilité

Addendum - Doctrine publique et trajectoire

Étienne doit être présenté comme architecte IA indépendant et entrepreneur, pas comme développeur principal omniscient. Son avantage est la liaison des couches : humain, métier, décision, mémoire, preuve et action réelle.

Le parcours public

• Terrain et ambulance : confrontation à l’irréversibilité et aux conséquences.

• Entrepreneuriat et e-commerce : usage, test, packaging et compréhension utilisateur.

• IA depuis fin 2024 : accélération de la structuration et apparition de la question d’autorité.

• Février 2025 : origine du Personal Operations Center puis mutation vers Obsidia.

• 2026 : consolidation Kernel, domaines, Brody, mémoire, preuves et pré-industrialisation.

Doctrine

• Séparer : le modèle propose, l’autorité décide.

• Prouver : chaque claim est borné et rattaché à une preuve.

• Traduire : Brody et Reverse rendent la structure intelligible sans la modifier.

• Oublier activement : conserver la capacité de reconstruire sans activer tout le passé.

• Fermer : passer de freestyle à structuration, test et Freeze.

Formule finale

Le parcours explique pourquoi cette architecture existe. Les preuves expliquent pourquoi elle est crédible. La roadmap explique comment elle peut devenir industrielle.

OBSIDIA / X108
SUPPORT MAÎTRE INTÉGRAL

Vision, architecture, boucles fermées, preuves, domaines, conformité et industrialisation

Version V2 narrative — 3 août 2026

Statut : recherche appliquée / POC local avancé / pré-industrialisation

Gouverner le droit d’agir des intelligences artificielles.

MODE D’EMPLOI

Un document canonique, plusieurs lectures

Comprenable par un enfant, crédible pour un investisseur, inspectable par un développeur senior.

Trois niveaux de lecture

- Lecture simple : problème, solution, exemples.
- Lecture architecture : couches, rôles, frontières, flux.
- Lecture preuve : tests, formel, métriques, statuts et limites.

Publics

- Étienne Aubin et équipe interne.
- Développeurs senior, CTO, auditeurs.
- Incubateurs, investisseurs, banques, partenaires GPS/défense.

Ce support ne remplace ni les spécifications, ni les rapports de tests, ni les dossiers réglementaires détaillés.

FIL DE LECTURE	POINT DE CONTRÔLE
Le support part du problème réel - une IA qui commence à agir - puis descend vers l’architecture, les preuves et les domaines. Il remonte ensuite vers le produit, la conformité et la sortie industrielle. Chaque claim doit rester lisible à trois niveaux : intuition, mécanisme et preuve.	Ce support ne remplace ni les spécifications, ni les rapports de tests, ni les dossiers réglementaires détaillés.

DISCIPLINE DE PREUVE

La légende qui protège tout le support

Un concept documenté n’est pas automatiquement un composant branché ; un test ne vaut pas autorisation.

Statut	Signification
CONCEPT	Idée, doctrine ou capacité décrite.
PRÉSENT	Artefact retrouvé dans les sources.
TESTÉ	Comportement exécuté dans une campagne bornée.
BRANCHÉ	Composant relié au runtime concerné.
PROUVÉ	Claim exact soutenu par une preuve reproductible.
OUVERT	Donnée, procédure ou intégration encore manquante.

POURQUOI C’EST CENTRAL	POINT DE CONTRÔLE
Obsidia contient beaucoup de concepts et d’artefacts. La crédibilité vient donc moins du volume que de la capacité à dire exactement ce qui existe, ce qui tourne, ce qui a été testé et ce qui reste ouvert. Cette légende empêche un glissement entre vision, implémentation et preuve.	Le claim public doit rester rattaché à un périmètre, une version, une exécution reproductible et une limite connue.

POURQUOI MAINTENANT

Les IA passent de la réponse à l’action

Une sortie probabiliste devient dangereuse lorsqu’elle déclenche une conséquence irréversible.

- Les agents utilisent des outils, modifient des fichiers, lancent des transactions et pilotent des workflows.
- La puissance des modèles augmente plus vite que la clarté de leur autorité.
- Les politiques ajoutées après coup peuvent être contournées ou mal branchées.
- La réponse n’est que le début de la chaîne de conséquences.

Le problème n’est plus seulement : “Est-ce que la réponse est correcte ?” Il devient : “Qui possède le droit d’agir ?”

LE BASCULEMENT	POINT DE CONTRÔLE
Tant qu’un modèle ne faisait que répondre, une erreur restait souvent informationnelle. Lorsqu’un agent utilise un outil, une mauvaise sortie peut devenir transaction, ordre, modification ou trajectoire. Obsidia intervient précisément au moment où le langage devient conséquence.	Le claim public doit rester rattaché à un périmètre, une version, une exécution reproductible et une limite connue.

IMAGE SIMPLE

Le feu rouge, l’aiguilleur et la boîte noire

Obsidia se place entre la recommandation et l’action.

Image	Fonction Obsidia
Feu rouge	Autoriser, suspendre ou bloquer avant l’action.
Aiguilleur	Diriger localement, escalader ou demander une validation.
Boîte noire	Tracer, prouver et rejouer la décision.

Obsidia n’est pas le véhicule et n’est pas le conducteur. C’est l’infrastructure qui gouverne le passage.

SCÈNE CONCRÈTE	POINT DE CONTRÔLE
Un agent recommande une action. Avant que l’outil ne l’exécute, X108 vérifie les autorisations, les contradictions, les inconnues et les preuves disponibles. Si le dossier est incomplet, le feu reste orange : HOLD. Si une règle est violée, il passe au rouge : BLOCK. Si tout est qualifié, l’action encadrée peut poursuivre.	Le claim public doit rester rattaché à un périmètre, une version, une exécution reproductible et une limite connue.

PROMESSE

Ce qu’Obsidia est

Une infrastructure ex ante de gouvernance décisionnelle pour agents IA et workflows critiques.

- Couche placée avant l’action réelle.
- Séparation stricte entre cognition, intention, décision, action et preuve.
- Autorité canonique KX108_ONLY.
- HOLD en cas d’incertitude conséquente.
- BLOCK sur structure interdite.
- Receipts, replay et audit.

Obsidia gouverne le droit d’agir, pas seulement la qualité d’une réponse.

LA RUPTURE	POINT DE CONTRÔLE
Obsidia ne cherche pas à rendre le modèle infaillible. Il part du principe qu’un modèle restera faillible et lui retire donc la souveraineté. L’innovation est une frontière d’autorité indépendante de l’intelligence qui propose.	Le claim public doit rester rattaché à un périmètre, une version, une exécution reproductible et une limite connue.

NON-OBJECTIFS

Ce qu’Obsidia n’est pas

- Pas un assistant généraliste.
- Pas un nouveau LLM plus gros.
- Pas une simple surcouche de prompts.
- Pas un orchestrateur autonome sans autorité claire.
- Pas un framework de sécurité générique.
- Pas une promesse de sécurité absolue.
- Pas un produit déclaré production-ready.

Statut honnête : recherche appliquée / POC local avancé / pré-industrialisation.

POURQUOI DIRE NON	POINT DE CONTRÔLE
Ce cadrage protège le produit contre la dispersion. Obsidia n’a pas besoin de remplacer les modèles, les outils métier ou les opérateurs. Il doit devenir la couche commune qui gouverne leur passage à l’action et rend ce passage auditable.	Statut honnête : recherche appliquée / POC local avancé / pré-industrialisation.

THÈSE

Les équations doctrinales

Formule	Conséquence
Signal ≠ vérité	Une donnée reste candidate tant qu’elle n’est pas qualifiée.
Contenu ≠ autorité	Un document consulté ne peut pas gouverner l’action.
Confiance ≠ permission	Un score élevé ne crée pas un droit d’exécution.

Formule	Conséquence
Mémoire ≠ vérité canonique	La mémoire informe ; elle ne décide pas.
Test PASS ≠ AUTHORIZED	Une réussite technique n’est pas une autorisation souveraine.
LLM ≠ décideur	Le modèle propose, traduit ou explique.

L’attaque peut entrer comme donnée. Elle ne doit pas pouvoir gouverner l’action.

LOGIQUE PROFONDE	POINT DE CONTRÔLE
Chaque équation casse un raccourci dangereux : une donnée n’est pas une vérité, une confiance n’est pas une permission et un test réussi n’est pas une autorisation. En les séparant, Obsidia empêche qu’une qualité probabiliste se transforme automatiquement en pouvoir opérationnel.	Le claim public doit rester rattaché à un périmètre, une version, une exécution reproductible et une limite connue.

BOUCLE MAÎTRESSE

De la réalité à l’apprentissage

Obsidia ferme la chaîne connaissance → action → conséquence.

- Réalité → perception → information située.
- Monde structuré → recherche → comparaison.
- Contradictions → vérification → intention.
- Décision → action → preuve.
- Conséquence → correction → apprentissage.

Une civilisation ne se définit pas seulement par ce qu’elle sait, mais par ce qu’elle retient des conséquences de ce qu’elle fait.

NARRATION DE LA BOUCLE	POINT DE CONTRÔLE
Le système ne s’arrête pas au verdict. Il observe ce que l’action a produit, rattache la conséquence à la décision, puis nourrit la correction et l’apprentissage. La preuve n’est donc pas une archive morte : elle devient la mémoire contrôlée de la trajectoire.	Le claim public doit rester rattaché à un périmètre, une version, une exécution reproductible et une limite connue.

MÉTHODE DE R&D

Freestyle → structuration → Freeze

Étape	Fonction
1. Friction	Une anomalie, une intuition ou une contradiction apparaît.
2. Freestyle	Les domaines sont reliés librement.
3. Motif	Une structure commune se répète.
4. Formalisation	Le motif devient invariant, protocole ou loi candidate.

Étape	Fonction
5. Test	La proposition rencontre le code, les scénarios et les preuves.
6. Freeze	Ce qui est validé devient référence ; le reste reste candidat.

Fermer une boucle ne signifie pas tout implémenter. Cela signifie connaître sa place, son rôle et sa condition de preuve.

COMMENT LE CHAOS DEVIENT SYSTÈME	POINT DE CONTRÔLE
La phase freestyle autorise les connexions inattendues. La structuration extrait les motifs reproductibles. Le Freeze interdit ensuite de réécrire silencieusement ce qui a été validé. Cette tension permet de créer librement sans confondre intuition et canon.	Le claim public doit rester rattaché à un périmètre, une version, une exécution reproductible et une limite connue.

MACHINERIE

Le flux complet d’Obsidia

- Entrée → OS de traduction → Intent.
- Plan actif → graphe de capacités → résolution corpus/outils.
- Domaines → périphérie → agents.
- Aggregate → Meta → GuardX108.
- CanonicalDecisionEnvelope → HOLD / BLOCK / ACT encadré.
- Receipt → replay → audit.

La réflexion peut se produire dans les transformations explicites entre couches, pas uniquement dans l’inférence interne d’un modèle.

CE QUI CIRCULE	POINT DE CONTRÔLE
Une demande humaine devient une intention structurée. Les domaines traduisent le contexte réel. Les agents proposent des signaux, l’agrégation les consolide, puis GuardX108 tranche sur la structure d’autorité. L’action et son reçu restent liés jusqu’au replay.	Le claim public doit rester rattaché à un périmètre, une version, une exécution reproductible et une limite connue.

PIPELINE CANONIQUE

Le chemin décisionnel minimal

Couche	Produit	Ne peut pas
DomainState	Réalité métier traduite	Décider seul
Agents	Signaux spécialisés	Émettre ACT
Aggregate	Consolidation	Contourner les frontières
Meta	Qualification structurelle	Devenir souverain
GuardX108	Verdict d’autorité	Déléguer implicitement

Couche	Produit	Ne peut pas
Envelope	Décision canonique et preuves	Masquer son origine

EXEMPLE DE PASSAGE	POINT DE CONTRÔLE
Dans un dossier bancaire, le score ne va jamais directement vers le refus ou l’acceptation. Il devient une entrée de DomainState, rejoint les signaux de provenance et de risque, puis traverse Aggregate, Meta et GuardX108. La sortie finale n’est plus “le modèle pense oui”, mais une décision qualifiée et traçable.	Le claim public doit rester rattaché à un périmètre, une version, une exécution reproductible et une limite connue.

SÉPARATION DES POUVOIRS

Qui fait quoi

Composant	Rôle	Autorité
Brody	Dialogue, recherche, éducation, contextualisation	Non souverain
Obsidure	Proposition de mutations de code	Non souverain
Lean / TLA+	Vérification de propriétés	Ne décide pas
Graphiti / mémoire	Contexte et relations	Readonly dans le live gelé
Domaines	Traduction métier	emits_act=false
Sigma	Stabilité, incohérence, alerte	N’autorise jamais ACT
X108	Décision canonique	KX108_ONLY
Humain autorisé	Intention et validation critique	Selon contrat domaine

POURQUOI PLUSIEURS RÔLES	POINT DE CONTRÔLE
Brody peut être brillant sans disposer du droit d’agir. Sigma peut détecter une dérive sans devenir juge. Lean peut prouver un invariant sans autoriser une transaction. Cette spécialisation empêche qu’une seule brique cumule connaissance, interprétation et souveraineté.	Le claim public doit rester rattaché à un périmètre, une version, une exécution reproductible et une limite connue.

AUTORITÉ HUMAINE

L’humain comme origine de l’intention

Étienne Aubin est l’architecte fondateur, pas une autorité technique cachée dans chaque décision.

Niveau	Autorité
Vision et trajectoire	Fondateur / organisation souveraine
Intention opérationnelle	Opérateur humain autorisé
Décision canonique système	X108
Action réelle critique	Validation humaine prévue par le domaine

L’humain fixe l’intention et les limites. X108 qualifie structurellement la décision.

LA PLACE D’ÉTIENNE	POINT DE CONTRÔLE
Étienne définit la trajectoire, les frontières et les intentions stratégiques. Dans un déploiement réel, l’autorité opérationnelle doit toutefois être contractualisée par rôle, domaine et niveau de risque. X108 ne remplace pas cette responsabilité : il l’exécute sous une forme vérifiable.	Le claim public doit rester rattaché à un périmètre, une version, une exécution reproductible et une limite connue.

DÉCISIONS

HOLD, BLOCK et ACT

Sortie	Condition	Effet
HOLD	Preuve, fraîcheur, confiance ou validation insuffisante	Suspendre et demander ce qui manque
BLOCK	Invariant violé, structure interdite ou risque non admissible	Interdire le passage
ACT encadré	Conditions satisfaites et autorisations présentes	Poursuivre selon le contrat

Aucun ACT implicite.

UNE TROISIÈME VOIE	POINT DE CONTRÔLE
Les systèmes classiques opposent souvent exécuter ou refuser. HOLD ajoute une suspension structurée : le système sait qu’il ne sait pas assez. Il précise la pièce manquante, attend une donnée ou une validation, puis reprend sans masquer l’incertitude initiale.	Le claim public doit rester rattaché à un périmètre, une version, une exécution reproductible et une limite connue.

INFORMATION

CONTENT_IS_NOT_AUTHORITY

- Tout contenu consulté reste une donnée non souveraine.
- La provenance, l’auteur, la version et la fraîcheur doivent être visibles.
- Une réponse de modèle ne remplace pas sa source.
- La promotion canonique exige une autorité identifiée.
- Conserver tout ne signifie pas activer tout.

Une information peut être vraie et néanmoins ne pas posséder le droit de déclencher une action.

SCÉNARIO D’ATTAQUE	POINT DE CONTRÔLE

Un document hostile peut entrer dans le corpus et même contenir une instruction convaincante. Parce qu'il reste marqué comme contenu non souverain, il ne peut pas modifier seul la politique ni déclencher ACT. L'attaque est observable, mais son pouvoir reste nul.	Le claim public doit rester rattaché à un périmètre, une version, une exécution reproductible et une limite connue.

MÉMOIRE

Une mémoire structurée, non souveraine

- BRUT → candidat → provenance → validation → canonique → projection.
- memory_write=false dans la configuration live gelée.
- Graphiti informe Brody et les domaines ; Graphiti ne décide pas.
- Fraîcheur, obsolescence, contradiction et rejet doivent être explicites.
- L'oubli actif trie, condense et permet la reconstruction.

La mémoire n'est pas la vérité. Elle est une carte de ce qui a été observé, qualifié et retenu.

CE QUE LA MÉMOIRE APPORTE	POINT DE CONTRÔLE
La mémoire conserve les relations, les sources et les décisions passées sans devenir arbitre. Une information nouvelle entre comme matière brute, reçoit une provenance et peut être rejetée, corrigée ou promue. Cette chaîne rend possible l'apprentissage sans corruption silencieuse du canon.	Le claim public doit rester rattaché à un périmètre, une version, une exécution reproductible et une limite connue.

TEMPS

Passé → Présent → Futur → Sigma → X108

- Le passé fournit traces et conséquences.
- Le présent compile l'état actif.
- Le futur projette les trajectoires candidates.
- Sigma qualifie stabilité et dérive.
- X108 garde l'autorité finale.

Sigma ne doit jamais être représenté après X108 comme une seconde autorité de validation.

TRAJECTOIRE PLUTÔT QU'INSTANT	POINT DE CONTRÔLE
Une décision n'est pas seulement jugée sur une photographie présente. Elle est comparée au passé, aux conséquences connues et aux futurs plausibles. Sigma observe la stabilité de cette trajectoire ; X108 conserve seul la décision d'autorité.	Sigma ne doit jamais être représenté après X108 comme une seconde autorité de validation.

BRODY

Parler, comprendre, apprendre — sans gouverner

- Brody est un LLM propriétaire et une interface cognitive.
- Il relie la mémoire ; il n’est pas la mémoire.
- Il peut être non appelé lorsque la route est déjà connue.
- Il traduit et explique sans devenir l’autorité.
- Sa valeur cible est l’éducation traçable, pas seulement la génération.

Brody parle. X108 décide.

LE VISAGE SANS LE POUVOIR	POINT DE CONTRÔLE
Brody donne une voix humaine à la machinerie : il explique, enseigne, recherche et reconstruit le contexte. Mais sa formulation ne modifie jamais le verdict. Même très convaincante, sa réponse reste une projection intelligible d’éléments déjà qualifiés.	Le claim public doit rester rattaché à un périmètre, une version, une exécution reproductible et une limite connue.

ÉDUCATION

Élever Brody plutôt que remplir un corpus

- Recherche → leçon → exercice → tentative.
- Correction → compréhension → validation.
- Expérience → mémoire → capacité → réflexe.
- Anomalie, ambiguïté ou fort enjeu → sortie du réflexe → règle explicite ou HOLD.

La compréhension doit finir par devenir capacité, puis réflexe.

DE LA RÉPONSE AU RÉFLEXE	POINT DE CONTRÔLE
Le but n’est pas seulement d’ingérer des documents. Brody doit parcourir des leçons, produire des tentatives, recevoir des corrections et démontrer une capacité. Une compétence stable peut ensuite devenir un réflexe ; toute anomalie force le retour vers une règle explicite.	Le claim public doit rester rattaché à un périmètre, une version, une exécution reproductible et une limite connue.

OBSIDURE ET PREUVES

Proposer n’est pas décider

Acteur	Opération	Frontière
Obsidure	Génère une proposition dans une sandbox	Aucune souveraineté
Lean	Compile et vérifie un énoncé	lean_decides=false
TLA+/TLC	Explore états et transitions	Ne prouve pas tout le runtime
Git / CI	Trace l’intégration	Commit seulement après critères

Acteur	Opération	Frontière
X108	Qualifie la décision	KX108_ONLY

Workflow validé : sandbox → vérification → forbidden tokens → diff → statut → intégration conditionnelle.

MUTER SANS SE DONNER RAISON	POINT DE CONTRÔLE
Obsidure peut proposer du code, mais la proposition reste isolée. Lean ou les tests vérifient un périmètre défini. Le diff montre ce qui change. Le commit n’arrive qu’après les critères. Ainsi, la capacité à créer ne devient jamais le droit de s’auto-valider.	Workflow validé : sandbox → vérification → forbidden tokens → diff → statut → intégration conditionnelle.

SIGMA

La stabilité n’est pas l’autorité

Sigma peut	Sigma ne peut pas
Évaluer entropie, dissipation, instabilité	Autoriser ACT
Produire un rapport de cohérence	Remplacer GuardX108
Alerter et demander un gel	Réécrire la décision canonique
Qualifier une trajectoire	Devenir un second kernel

LE RÔLE DE LA SENTINELLE	POINT DE CONTRÔLE
Sigma regarde les tendances qu’une décision instantanée ne voit pas : instabilité, dissipation, incohérence ou dérive. Il peut demander un gel et attirer l’attention humaine. Il ne peut ni inverser X108 ni créer une autorisation.	Le claim public doit rester rattaché à un périmètre, une version, une exécution reproductible et une limite connue.

COCKPIT

Rendre la machinerie visible

- Le cockpit n’est pas un chatbot.
- Il n’est pas un second kernel.
- Il n’est pas un tableau décoratif.
- Il ne raccourcit jamais la chaîne d’autorité.
- Il expose statuts, preuves, limites, freezes, mémoires, branches, agents, routes, logs et validations.

Le cockpit compresse la présentation ; il ne compresse pas la preuve.

UNE INTERFACE DE SOUVERAINETÉ	POINT DE CONTRÔLE

Le cockpit doit permettre de voir pourquoi une action est retenue, quelle preuve manque, quel composant est gelé et qui doit intervenir. Il réduit la charge cognitive de l’opérateur sans cacher la chaîne causale derrière un simple voyant vert.	Le claim public doit rester rattaché à un périmètre, une version, une exécution reproductible et une limite connue.

PRODUIT ENTREPRISE

V0.1 : se brancher sur toute la stack

- Systèmes, documents, processus, données et outils.
- Agents, validations, frontières et autorisations.
- Immersion dans l’état client.
- Cartographie des flux et frictions.
- Propositions d’amélioration.
- Décisions gouvernées et conséquences traçables.

La V0.1 n’est pas le cockpit personnel d’Étienne : c’est une infrastructure branchable à l’entreprise.

L’ENTRÉE CLIENT	POINT DE CONTRÔLE
La V0.1 commence en lecture contrôlée : elle cartographie la stack, les processus, les outils et les frontières humaines. Elle révèle où une recommandation devient action et où la preuve disparaît. Les premières gates sont ensuite posées sur les passages les plus critiques.	Le claim public doit rester rattaché à un périmètre, une version, une exécution reproductible et une limite connue.

UNIVERSALITÉ

Une machinerie commune, des domaines traducteurs

Commun à tous les secteurs	Spécifique au domaine
Intent, gates, autorité, receipts	Variables et vocabulaire métier
HOLD/BLOCK/ACT	Invariants sectoriels
Provenance et temporalité	Sources de données
Replay et audit	Conséquences et recours

Le kernel n’a pas besoin de comprendre directement le monde : les domaines traduisent la réalité.

POURQUOI LE MÊME KERNEL PEUT SERVIR PLUSIEURS DOMAINES	POINT DE CONTRÔLE

La banque, le trading et le GPS n’emploient ni les mêmes données ni les mêmes conséquences. Ils partagent toutefois une question : le dossier présenté possède-t-il la structure nécessaire pour agir ? Le domaine traduit ; X108 applique la grammaire commune de l’autorité.	Le claim public doit rester rattaché à un périmètre, une version, une exécution reproductible et une limite connue.

DOMAINES DE PREUVE

Banque, Trading, GPS

Domaine	Question gouvernée	Risque principal
Banque	Peut-on poursuivre une décision affectant une personne ?	Biais, données, recours
Trading	Peut-on émettre ou poursuivre une action de marché ?	Volatilité, limites, kill switch
GPS / GNSS	Peut-on faire confiance à une trajectoire pour agir ?	Spoofing, brouillage, ambiguïté

TROIS ÉPREUVES COMPLÉMENTAIRES	POINT DE CONTRÔLE
La banque teste les droits des personnes, le trading teste la vitesse et l’irréversibilité, le GPS teste la fidélité au monde physique sous signal hostile. Ensemble, ces pilotes démontrent que la séparation d’autorité n’est pas limitée à un seul métier.	Le claim public doit rester rattaché à un périmètre, une version, une exécution reproductible et une limite connue.

BANQUE

Scoring crédit : gouverner avant la décision

- Le moteur de scoring produit une estimation candidate.
- Obsidia vérifie provenance, fraîcheur, inconnues, contradictions et autorisations.
- Une décision défavorable exige une supervision humaine effective.
- Les personnes doivent disposer d’une explication et d’un recours.
- Le pilote reste hypothétique : aucune donnée bancaire réelle.

Obsidia ne remplace pas la banque ; il structure la preuve et les conditions de décision.

DÉMONSTRATION ATTENDUE	POINT DE CONTRÔLE
Le pilote doit montrer qu’un score élevé ne suffit jamais. Variables interdites, données périmées, absence d’explication ou supervision insuffisante doivent provoquer HOLD ou BLOCK. Chaque décision doit pouvoir être expliquée, contestée et rejouée.	Le claim public doit rester rattaché à un périmètre, une version, une exécution reproductible et une limite connue.

FRIA

Du document déclaratif au dispositif vérifiable

Risque	Garde-fou Obsidia	Reste déployeur
Discrimination	HOLD/BLOCK, invariants candidats	Tests de biais et seuils
Données périmées	Provenance, fraîcheur, replay	Correction et information
Opacité	Receipts et diagnostic	Explication intelligible
Automation bias	Validation humaine	Formation et audit
Absence de recours	Traces	Canal et SLA
Défaillance systémique	Sandbox, tests, freeze	Incident et continuité

La FRIA réelle appartient au déployeur concerné ; Obsidia fournit un kit et des preuves.

CE QU'OBSIDIA CHANGE	POINT DE CONTRÔLE
Une FRIA classique décrit les risques et les mesures. Obsidia peut relier chaque mesure à une gate, un test, un reçu et une version. Le déployeur garde la responsabilité juridique, mais il dispose d'un dossier dont une partie des affirmations devient vérifiable techniquement.	La FRIA réelle appartient au déployeur concerné ; Obsidia fournit un kit et des preuves.

TRADING

Contrôler l'irréversible

- Les signaux de marché restent des verdicts candidats.
- Les limites de risque, positions, volatilité et kill switch sont des entrées de domaine.
- HOLD protège lorsque le régime est ambigu.
- BLOCK interdit les structures explicitement prohibées.
- Toute activation réelle reste hors du runtime actuel.

L'exactitude prédictive ne suffit pas : l'autorité doit rester distincte du signal.

DÉMONSTRATION ATTENDUE	POINT DE CONTRÔLE
Une stratégie peut rester performante tout en devenant inadmissible lorsque la volatilité explose ou qu'un kill switch est actif. Le pilote doit montrer que X108 bloque ou suspend l'action même si le moteur prédictif continue de recommander la transaction.	Le claim public doit rester rattaché à un périmètre, une version, une exécution reproductible et une limite connue.

GPS / DÉFENSE

Le signal hostile ne doit pas devenir souverain

- Signal GNSS → récepteur → observables → DomainState.

- Contrôles de cohérence physique et temporelle.
- Spoofing brutal : BLOCK attendu.
- Dérive progressive ou ambiguïté : HOLD attendu.
- Receipt et replay de la décision.

L’attaquant peut falsifier le signal. Il ne doit pas pouvoir falsifier l’autorité qui décide quoi en faire.

DÉMONSTRATION ATTENDUE	POINT DE CONTRÔLE
La chaîne doit comparer un nominal stable et un signal hostile avec le même récepteur et la même configuration. Après l’onset documenté, la divergence doit apparaître dans les observables, être traduite en DomainState puis provoquer le verdict X108 attendu.	Le claim public doit rester rattaché à un périmètre, une version, une exécution reproductible et une limite connue.

GPS — ÉTAT RÉEL

FGI UT_DFMC : preuve partielle jusqu’à X108 live

Élément	État
Corpus officiel FGI-SpoofRepo UT_DFMC	Présent, licence CC-BY-4.0, hashé
UTD_L1_E1.dat	9,82 GiB
Chaîne réelle partielle	Atteint X108 live
RECORDED_RF_ATTACK	Non acquis
Nominal NAV/PVT stable	À obtenir avec même récepteur/config
Rupture hostile après onset 135 s	À démontrer
Perte de lock vers 14 s	Ne constitue pas une preuve d’attaque

Claim final seulement si nominal + hostile décodés, rupture post-onset, X108, reproductibilité et absence de fuite de labels.

CE QUI EST DÉJÀ DÉMONTRÉ	POINT DE CONTRÔLE
Le corpus officiel est présent, hashé et une chaîne réelle partielle atteint X108 live. Ce résultat prouve l’intégration d’une matière RF réelle, pas encore la détection complète d’une attaque enregistrée. La différence est essentielle pour un jury technique.	Claim final seulement si nominal + hostile décodés, rupture post-onset, X108, reproductibilité et absence de fuite de labels.

HACKATHON AMD

Éviter l’inférence lorsque la route est connue

Mesure	Campagne bornée
Tokens	6 939 → 1 740, environ 75 % évités
Appels distants évités	15 / 18

Mesure	Campagne bornée
Level 0	61 %
Exactitude	100 % sur la campagne
Invariants	180 / 180
Violations	2 / 8 → 0 / 8
Tests	369 PASS
Latence locale moyenne	0,211 ms sur 18 tâches

Quand la route est connue, prédire devient plus lent que vérifier.

HISTOIRE DE LA DÉMO	POINT DE CONTRÔLE
Sur les tâches où la route était déjà connue, le routeur local a évité des appels distants et réduit les tokens sans dégrader l'exactitude de la campagne. Le message n'est pas "plus jamais de LLM", mais "n'inférons pas lorsque la structure suffit".	Le claim public doit rester rattaché à un périmètre, une version, une exécution reproductible et une limite connue.

PREUVE

La stack qui transforme une promesse en claim

- Tests unitaires et d'intégration.
- Campagnes adversariales.
- Lean pour les propriétés formalisées.
- TLA+/TLC pour les transitions.
- Manifests et hashes.
- Receipts et replay.
- Freezes et CI.
- Rapports rattachés à une version.

Une preuve sur un modèle ne prouve pas automatiquement le runtime complet.

CHAÎNE DE CUSTODY	POINT DE CONTRÔLE
Une affirmation solide doit relier l'intention testée, le scénario, les données, la version du code, le résultat et les limites. Les différentes familles de preuve se complètent : le test observe une exécution, Lean prouve un énoncé et le receipt rattache la décision au runtime.	Le claim public doit rester rattaché à un périmètre, une version, une exécution reproductible et une limite connue.

MÉTRIQUES

Discipline des chiffres

À faire	À interdire
Rattacher chaque nombre à une campagne	Généraliser une campagne à tout Obsidia
Séparer latence et throughput	Convertir une mesure sans base
Nommer le commit et le périmètre	Employer des pourcentages de scénario comme preuve
Afficher les limites	Confondre test PASS et autorisation

LECTURE OPÉRATIONNELLE	POINT DE CONTRÔLE
Cette section situe discipline des chiffres dans la chaîne complète : intention, traduction, décision, action, preuve et conséquence.	Le claim public doit rester rattaché à un périmètre, une version, une exécution reproductible et une limite connue.

IA ACT

Alignement architectural, pas conformité déclarée

- Forces : séparation intelligence/autorité, gestion de l’incertitude, refus, supervision, logs et replay.
- L’architecture facilite la gestion des risques et la documentation.
- La qualification haut risque dépend de la finalité précise et du produit.
- Obsidia n’est pas encore conforme juridiquement comme système mis sur le marché.

Obsidia est conçu pour rendre la conformité vérifiable, pas pour la déclarer automatiquement.

POSITION HONNÊTE	POINT DE CONTRÔLE
L’architecture couvre déjà plusieurs capacités utiles au règlement, mais la conformité dépend d’un produit, d’une finalité, d’acteurs et de procédures réelles. Le support peut donc parler d’alignement by design et de préparation, jamais de conformité acquise.	Le claim public doit rester rattaché à un périmètre, une version, une exécution reproductible et une limite connue.

IA ACT — RESTE À FAIRE

Fermer la couche réglementaire et organisationnelle

Priorité	Livrable
P0	Registre des produits et finalités prévues
P0	Qualification article 6 / Annexes
P0	Dossier Annexe IV et evidence registry
P0	QMS et plan de gestion des risques
P1	Données, cybersécurité, supervision humaine

Priorité	Livrable
P1	Conservation, incidents, post-market
P1	Instructions déployeur et transparence
P1	FRIA/DPIA par déploiement
P2	Évaluation, déclaration UE, CE et base UE si applicables

PASSAGE DU CODE À L'ORGANISATION	POINT DE CONTRÔLE
La couche technique ne suffit pas. Il faut nommer les responsabilités, conserver les preuves, qualifier les incidents, surveiller le système après déploiement et produire la documentation de chaque configuration. C'est ce chantier qui transforme une architecture prometteuse en produit industrialisable.	Le claim public doit rester rattaché à un périmètre, une version, une exécution reproductible et une limite connue.

ÉTIENNE AUBIN

Architecte fondateur et source de trajectoire

- Vision macro et conception des séparations d'autorité.
- Origine de l'intention stratégique.
- Manager/architecte, pas développeur principal.
- Garant des frontières, du Freeze et de l'alignement long terme.
- Besoin désormais de développeurs senior, auditeurs et spécialistes métier pour corriger, stabiliser et industrialiser.

Obsidia n'automatise pas la volonté humaine. Il protège le passage entre intention, décision et conséquence.

TRAJECTOIRE DU FONDATEUR	POINT DE CONTRÔLE
Le projet est né d'une capacité à observer les frictions, relier des domaines et construire une architecture globale. La prochaine étape change la fonction du fondateur : moins porter seul chaque correction, davantage préserver la vision, recruter les validateurs et diriger la convergence.	Le claim public doit rester rattaché à un périmètre, une version, une exécution reproductible et une limite connue.

ORIGINE

Du Personal Operations Center à X108

Phase	Transformation
Cockpit personnel	Organiser APIs, mails, agents et travail réel
Observation des frictions	Mesurer intention, valeur, temps et fatigue
Agents spécialisés	Répondre aux capacités manquantes
Séparation des rôles	Empêcher l'assistance de devenir souveraine
Kernel X108	Formaliser l'autorité décisionnelle universelle

LE MOUVEMENT FONDATEUR	POINT DE CONTRÔLE

Le cockpit personnel a d’abord servi à organiser le travail réel. L’observation des frictions a fait émerger des agents spécialisés. Leur multiplication a créé un nouveau problème : qui possède l’autorité ? X108 est la réponse structurelle à cette montée en puissance.	Le claim public doit rester rattaché à un périmètre, une version, une exécution reproductible et une limite connue.

INTERACTION

Observé ≠ Interprété ≠ Confirmé

Niveau	Contenu	Autorité
Observé	Habitudes, formats, corrections	Signal
Interprété	Vocabulaire et motifs candidats	Non canonique
Confirmé	Contraintes et décisions explicites	Promouvable selon règles

Interdiction d’inventer une personnalité ou des intentions cachées.

LECTURE OPÉRATIONNELLE	POINT DE CONTRÔLE
Interdiction d’inventer une personnalité ou des intentions cachées.	Le claim public doit rester rattaché à un périmètre, une version, une exécution reproductible et une limite connue.

PARTENARIAT STRATÉGIQUE

Hypothèse Étienne Aubin × Xavier Niel

Aucun accord n’existe : il s’agit d’une cible stratégique et d’un scénario de collaboration.

- Infrastructure européenne + modèles + réseau constituent un terrain d’industrialisation.
- Obsidia apporte une quatrième couche : l’autorité vérifiable entre intelligence et action.
- Le premier contact doit montrer une rupture démontrable, pas raconter tout le projet.

Vous avez l’infrastructure et les modèles. J’ai construit la couche qui empêche un modèle de devenir souverain lorsqu’il commence à agir.

POURQUOI NIEL	POINT DE CONTRÔLE
L’hypothèse n’est pas une demande de financement abstraite. Elle consiste à présenter une pièce manquante entre infrastructure souveraine et modèles puissants : une couche d’autorité avant action. Le contact doit commencer par une démonstration bornée, reproductible et impossible à confondre avec un simple wrapper.	Le claim public doit rester rattaché à un périmètre, une version, une exécution reproductible et une limite connue.

STACK SOUVERAINE

Muscles, cerveaux, autorité, opérateur

Couche	Exemple	Fonction
Infrastructure	Cloud européen / télécom	Calcul, hébergement, réseau
Modèles	LLM et modèles spécialisés	Analyse, génération, exploration
Gouvernance	Obsidia / X108	Autoriser, suspendre, bloquer, prouver
Opérateur	Organisation ou humain autorisé	Finalité et validation critique

Les liens précis entre Xavier Niel, Scaleway, Mistral ou Kyutai doivent être vérifiés avant publication externe.

LECTURE OPÉRATIONNELLE	POINT DE CONTRÔLE
Les liens précis entre Xavier Niel, Scaleway, Mistral ou Kyutai doivent être vérifiés avant publication externe.	Le claim public doit rester rattaché à un périmètre, une version, une exécution reproductible et une limite connue.

PHYSICAL SIGNAL WORLD MODEL

Recherche télécom et information située

- Transformer des observables RF agrégés en information physique située.
- Apprendre des perturbations du monde, pas des identités humaines.
- Périmètre : infrastructures autorisées, routes, forêts, lacs, espaces non privés.
- Privacy by design, agrégation, finalité bornée.
- Aucune intégration Free actuelle ; aucune détection réseau démontrée.

Le réseau pourrait devenir un capteur physique gouverné — hypothèse de recherche, pas capacité acquise.

VISION DE RECHERCHE	POINT DE CONTRÔLE
Les réseaux télécom observent déjà des perturbations du monde physique. L'enjeu serait d'en extraire des motifs agrégés et non identifiants, puis de les situer dans le temps et l'espace. X108 gouvernerait alors l'usage de cette information plutôt que de laisser le capteur devenir une autorité.	Le claim public doit rester rattaché à un périmètre, une version, une exécution reproductible et une limite connue.

POSITIONNEMENT

La différence de marché

Approche typique	Obsidia
Modèle au centre	Autorité séparée du modèle
Conformité après coup	Gouvernance pensée en amont
Un score de confiance	Contradictions, unknowns, conséquences et preuves

Approche typique	Obsidia
Toujours inférer	Éviter l’inférence quand structurellement inutile
Logs après action	Gate et receipt avant/après action

CATÉGORIE DE MARCHÉ	POINT DE CONTRÔLE
Obsidia se place entre l’IA, les outils et le réel. Sa valeur n’est pas une nouvelle prédiction, mais la maîtrise du passage de la prédiction à la conséquence. Cette catégorie devient plus importante à mesure que les entreprises donnent des permissions opérationnelles aux agents.	Le claim public doit rester rattaché à un périmètre, une version, une exécution reproductible et une limite connue.

VENTE

Ce que l’on vend réellement

- Une couche de gouvernance branchable.
- Un diagnostic de stack et de flux.
- Des domaines traducteurs.
- Des gates d’autorité.
- Un cockpit de preuve et de validation.
- Un kit de conformité et de déploiement.
- Une trajectoire d’industrialisation.

Le produit n’est pas “une IA qui sait tout”. C’est une infrastructure qui sait quand une action peut être permise.

OFFRE CONCRÈTE	POINT DE CONTRÔLE
Le premier engagement peut prendre la forme d’un diagnostic, d’une cartographie d’autorité et d’un pilote readonly. L’entreprise voit où ses agents peuvent agir, quelles preuves manquent et quelles gates poser. L’industrialisation vient ensuite, sur un périmètre choisi.	Le claim public doit rester rattaché à un périmètre, une version, une exécution reproductible et une limite connue.

DÉCLINAISONS

Un support maître, plusieurs dérivés

Public	Support dérivé
Investisseur	Problème, rupture, preuve, marché, équipe
Incubateur	Trajectoire, état, besoins, jalons
Développeur senior	Architecture, contrats, backlog, tests
Banque	Scoring, FRIA, supervision, données
GPS / défense	RF hostile, trajectoire, preuves physiques
CTO / conformité	Intégration, QMS, Annexe IV, sécurité

LECTURE OPÉRATIONNELLE	POINT DE CONTRÔLE

Cette section situe un support maître, plusieurs dérivés dans la chaîne complète : intention, traduction, décision, action, preuve et conséquence.	Le claim public doit rester rattaché à un périmètre, une version, une exécution reproductible et une limite connue.

ROADMAP PRODUIT

De la V0.1 à la V1 structurelle

Phase	Objectif
V0.1	Brancher la stack entreprise en lecture contrôlée
Assemblage	Relier machinerie, agents, domaines, mémoire et routes
Éducation	Corpus minimal et progression Brody
Pilotes	Banque, Trading, GPS bornés
V1 structurelle	Architecture connectée, testée, freeze et auditable
Industrialisation	Sécurité, QMS, équipe, conformité, clients

LOGIQUE DE PROGRESSION	POINT DE CONTRÔLE
Chaque phase ferme une incertitude différente : visibilité de la stack, branchement des composants, capacité cognitive, démonstration métier, puis qualité industrielle. Sauter directement au client critique créerait une dette de preuve et de responsabilité.	Le claim public doit rester rattaché à un périmètre, une version, une exécution reproductible et une limite connue.

ORDRE MAÎTRE

Séquence de branchement à préserver

- AGENTS → MODULES → CORTEX → LOIS.
- PÉPITES → PROTOCOLES → BALANCES → CANON.
- DOMAINES → ROUTES_API → BOUNDARIES.
- MÉMOIRE_GRAPHITI → PREUVES.

État consolidé : audit de 4 353 items clos ; reprise par lots ; Terminal encore incomplet.

LECTURE OPÉRATIONNELLE	POINT DE CONTRÔLE
État consolidé : audit de 4 353 items clos ; reprise par lots ; Terminal encore incomplet.	Le claim public doit rester rattaché à un périmètre, une version, une exécution reproductible et une limite connue.

ÉQUIPE

Le passage du solo à l’organisation

Rôle	Mission
Étienne Aubin	Vision, architecture, direction, arbitrages
Lead backend/kernel	Runtime, contrats, intégration
Formal methods	Lean, TLA+, correspondance runtime
Sécurité	Threat model, supply chain, pentest
Domain experts	Banque, Trading, GNSS
Produit/UX	Cockpit et supervision humaine
Conformité/qualité	QMS, IA Act, RGPD, incidents

La R&D conceptuelle existe ; le besoin actuel est correction, fusion, stabilisation, contrôle et validation senior.

POURQUOI MAINTENANT	POINT DE CONTRÔLE
La vision et une partie importante de la R&D existent déjà. Le goulot n’est plus l’imagination, mais la validation indépendante, la fusion des briques et la sécurité du runtime. Une petite équipe senior et transversale peut convertir ce capital documentaire en système contrôlé.	Le claim public doit rester rattaché à un périmètre, une version, une exécution reproductible et une limite connue.

ÉTAT RÉEL

Fermé, partiel, ouvert

Bloc	Statut	Lecture
Doctrine d’autorité	FERMÉ	KX108_ONLY, no bypass, no ACT implicite
HOLD/BLOCK	FERMÉ	Contrats et tests présents
AMD router	PROUVÉ BORNÉ	Campagne publique mesurée
Lean/Obsidure	PARTIEL	Capacités actives, couverture encore large à fermer
Mémoire/Terminal	PARTIEL	Readonly, branchements incomplets
GPS hostile	PARTIEL	Corpus réel jusqu’à X108, claim RF complet ouvert
IA Act/FRIA	PARTIEL	Alignement et pilote, conformité ouverte
Production client	OUVERT	Aucun déploiement critique réel

LECTURE DU TABLEAU	POINT DE CONTRÔLE
Les blocs fermés définissent le socle sur lequel construire. Les blocs partiels possèdent des artefacts mais exigent encore une preuve de branchement ou de couverture. Les blocs ouverts sont des chantiers assumés, pas des capacités à présenter comme acquises.	Le claim public doit rester rattaché à un périmètre, une version, une exécution reproductible et une limite connue.

RISQUES

Ce qui peut faire échouer Obsidia

- Confondre profondeur documentaire et fonctionnement réel.
- Multiplier les concepts avant de fermer les branchements.
- Employer des claims non rattachés à une preuve.
- Transformer Brody ou Sigma en souverain par glissement.
- Présenter les domaines comme produits finis.
- Sous-estimer sécurité, QMS et responsabilité.
- Porter seul trop longtemps la validation technique.

Le prochain avantage compétitif ne vient plus d’une nouvelle idée : il vient de la fermeture vérifiable de ce qui existe.

RISQUE MAÎTRE	POINT DE CONTRÔLE
La plus grande menace est le décalage entre l’immensité de la vision et le nombre de boucles réellement fermées. La réponse n’est pas de réduire Obsidia, mais de choisir des claims bornés, les prouver complètement et construire la crédibilité de proche en proche.	Le claim public doit rester rattaché à un périmètre, une version, une exécution reproductible et une limite connue.

90 JOURS

Priorités de sortie

Lot	Résultat
1. GPS	Nominal NAV/PVT stable, hostile post-onset, rapport reproductible
2. Assemblage	Fermer les lots AGENTS → PREUVES
3. Terminal	Interface unique, état réel, no auto-exec
4. Evidence registry	Claims reliés aux commits, tests et rapports
5. Équipe	Identifier dev senior, sécurité et domaine
6. Pilote	Choisir une configuration readonly bornée
7. Sortie	Pitch, incubateurs, Niel, partenaires terrain

OBJECTIF DE LA SÉQUENCE	POINT DE CONTRÔLE
Les sept lots convergent vers une même sortie : un démonstrateur incontestable, un runtime lisible, une evidence registry et une équipe capable de soutenir la revue. Cette base permet ensuite le pitch, le pilote entreprise et les discussions stratégiques.	Le claim public doit rester rattaché à un périmètre, une version, une exécution reproductible et une limite connue.

PHRASES CANONIQUES

Le langage qui ne doit plus bouger

- Brody parle. X108 décide.
- Le domaine parle métier. X108 parle autorité.

- L’attaque peut entrer. Elle ne doit pas gouverner.
- Quand la route est connue, prédire devient plus lent que vérifier.
- L’humain reste l’origine de l’intention et l’autorité finale sur sa trajectoire.
- Conserver tout ne signifie pas activer tout.
- La réponse est le début des conséquences.
- Liberté cognitive, souveraineté décisionnelle, responsabilité traçable.

Obsidia gouverne le passage entre intelligence et action.

LECTURE OPÉRATIONNELLE	POINT DE CONTRÔLE
Obsidia gouverne le passage entre intelligence et action.	Le claim public doit rester rattaché à un périmètre, une version, une exécution reproductible et une limite connue.

SOURCES

Documents consolidés dans cette V2

Famille	Sources
Boucles	Discussion maîtresse intégrale ; support intégral ; passage à l’implémentation
Stratégie personnelle	Support personnel V3, avec V2 comme historique
Vente	Support positionnement/vente/industrialisation ; pitch investisseur
Conformité	État IA Act corrigé ; tableau article par article
FRIA	FRIA scoring crédit 16 pages et mapping
Conversation	Rôle d’Étienne, hypothèse Niel, GPS/FGI, calibrations de claims
Projet	Statuts runtime, Lean/Obsidure, Terminal, AMD, mémoire et domaines

Les anciens supports deviennent des sources ; ce document devient la colonne vertébrale.

CONCLUSION

La vision est fermée ; la machinerie doit se prouver

Obsidia possède une architecture cohérente, des briques exécutables, des preuves bornées et une trajectoire industrielle identifiable.

- La priorité n’est plus d’ajouter des couches.
- La priorité est de brancher, tester, mesurer, faire auditer et déployer un pilote borné.
- La puissance d’Obsidia vient de la structure : intelligence libre, autorité séparée, conséquence traçable.

Une IA peut explorer largement. Elle ne doit jamais s’attribuer seule le droit d’agir.

Fin du support maître intégral V2 — document de pré-industrialisation, non production-ready.

OBSIDIA

La discussion qui referme l'architecture

Version intégrale, expliquée simplement et orientée implémentation

On n'a pas ajouté une couche de plus. On a enfin relié les couches existantes dans une circulation complète.

Ce document reprend toute la discussion, les anciennes boucles retrouvées, les fonctionnalités qui les matérialisent, les éléments encore ouverts et la décision de phase : terminer l'état actuel avant d'élargir le corpus.

But Comprendre simplement ce qui a été refermé et pourquoi cela change la phase du projet.	**Contenu** Philosophie, architecture, fonctionnalités, rôles, exemples, statuts et ordre d'implémentation.
Point de départ Plusieurs semaines de brainstorming, de retours sur les anciennes discussions et de consolidation documentaire.	**Décision finale** Finir, brancher, tester et geler. Les documents sources attendront après la V1 structurelle.

Version 2.0 - 29 juillet 2026

SOMMAIRE GENERAL

1. Ce que la discussion a changé
2. La méthode réelle de découverte : du brainstorming au Freeze
3. La boucle maîtresse expliquée simplement
4. Les grandes boucles conceptuelles fermées
5. Les fonctionnalités qui rendent ces boucles utilisables
6. Les rôles : qui fait quoi et qui ne décide pas
7. Un scénario complet de bout en bout
8. Ce qui est vraiment fermé, ce qui reste partiel et ce qui reste ouvert
9. L'ordre d'implémentation retenu
10. La politique des documents sources
11. Le critère réaliste de V1
12. Les phrases canoniques à conserver
13. Conclusion : la vision est fermée, la machination doit se prouver

> **Comment lire ce support**
>
> Les trois premiers chapitres donnent la compréhension générale. Le chapitre 4 explique chaque boucle avec des exemples. Le chapitre 5 traduit ces idées en fonctionnalités. Les derniers chapitres fixent le plan concret pour terminer la V1 sans repartir dans tous les sens.

1. CE QUE LA DISCUSSION A CHANGE

Au départ, le projet contenait déjà énormément de choses : le Kernel X-108, Brody, Obsidur, Sigma, la mémoire, les domaines, Reverse, Lean, les preuves, les chemins rapides, la périphérie scientifique, l'éducation, les images, la causalité et plusieurs branches futures. Le problème n'était plus l'absence d'idées. Le problème était que ces idées étaient dispersées dans des conversations, des documents et des couches différentes.

La discussion a servi à reprendre ces éléments un par un et à vérifier s'ils formaient seulement de belles intuitions ou s'ils fermaient réellement une boucle. Une boucle est considérée comme fermée lorsque l'on sait d'où elle part, ce qu'elle transforme, qui intervient, où se trouve l'autorité, comment elle se termine et dans quelles conditions elle doit se rouvrir.

Avant Beaucoup de pépites justes, mais encore perçues comme des sujets séparés : vérité, image, physique, mémoire, apprentissage, preuve, agents, code.	**Après** Une seule architecture où toutes ces fonctions se répondent dans un cycle complet.
Ancien risque Continuer à inventer des couches, ouvrir des sources ou lancer l'éducation avant de savoir ce qui est réellement branché.	**Nouvelle discipline** Capturer les nouvelles idées sans les brancher, terminer le Terminal et l'audit, puis implémenter la carte stabilisée.

La vision est fermée. La machination doit maintenant prouver qu'elle correspond à cette vision.

Ce que signifie réellement « fermer l'architecture »

Cela ne signifie pas que tout le code existe déjà ni que toutes les capacités sont actives. Cela signifie que les grandes questions de structure ont maintenant une réponse cohérente : comment le système reçoit le monde, comment il distingue le connu de l'inconnu, comment il apprend, comment il utilise un modèle, comment il prouve, comment il agit et comment il se corrige.

> **Attention**
>
> Architecture conceptuellement fermée ne veut pas dire produit terminé. Il faut encore distinguer : concept défini, artefact présent, composant branché, effet causal démontré, testé, prouvé, Freeze et production.

2. DU BRAINSTORMING AU FREEZE : LA METHODE REELLE DE R&D

Tu as raison : nous n'étions pas toujours en « recherche » au sens classique. Nous étions souvent dans un brainstorming libre. Une vidéo, une analogie, un problème de parking, une discussion sur le blanchiment, une image, AlphaProof ou un détail de code faisait apparaître une intuition. Ensuite, en la confrontant au reste du projet, une loi transversale finissait par émerger.

La méthode qui s'est répétée pendant plusieurs semaines

1	**Une friction, une intuition ou une contradiction apparaît.**
2	Le brainstorming relie librement plusieurs domaines.
3	Un motif commun commence à se répéter.
4	Le motif est reformulé comme invariant ou loi candidate.
5	On cherche où cette loi se place dans l'architecture existante.
6	On sépare ce qui relève du concept, de l'hypothèse, de la preuve et de l'implémentation.
7	On attribue les rôles et les droits : qui observe, qui propose, qui prouve, qui décide.
8	On définit les tests, les preuves ou les limites nécessaires.
9	**La loi est gelée conceptuellement ou renvoyée en branche future.**

Pourquoi cette méthode est importante

Elle explique pourquoi certaines discussions semblaient partir dans plusieurs directions tout en produisant finalement une architecture très cohérente. Le brainstorming n'était pas le résultat final. Il était la zone d'exploration. La fermeture arrivait lorsque les contradictions trouvaient toutes une place dans la même circulation.

> **Fonctionnalité à construire**
>
> Brody devra pouvoir détecter qu'une loi transversale vient d'apparaître dans un flux chaotique, extraire sa formulation, retrouver ses liens avec les couches existantes et proposer un statut : pépite, hypothèse, loi candidate, à prouver, à intégrer ou à Freeze.

Freestyle -> détection d'invariant -> qualification -> formalisation -> preuve -> Freeze.

3. LA BOUCLE MAITRESSE EXPLIQUEE SIMPLEMENT

La grande fermeture de cette discussion est la suivante : Obsidia n'est plus seulement une collection de modules. Il devient un système capable de recevoir une partie du monde, de la comprendre progressivement, de préparer une action, de gouverner cette action, puis d'apprendre des conséquences.

La circulation générale d'Obsidia

1	**Réalité : un état existe dans le monde, dans une entreprise, dans un logiciel ou dans un domaine.**
2	Perception : des données, images, signaux, documents ou événements rendent cet état observable.
3	Ancrage : le système vérifie la provenance, le temps, le contexte et la relation avec le réel.
4	Education : les observations sont reliées à des concepts, des distinctions, des erreurs et des corrections.
5	Structure scientifique : mathématiques, géométrie, logique, physique et causalité donnent des limites et des relations.
6	Mémoire : les éléments qualifiés assurent une continuité entre les sessions et les organes.
7	Compilation du contexte : chaque organe reçoit seulement la partie du monde utile à sa mission.
8	Routage : le système détermine quelle capacité appeler et si une inférence est réellement nécessaire.
9	Génération ciblée : Brody, Obsidur, un modèle visuel ou un solveur produisent une proposition bornée.
10	Simulation et confrontation : la proposition est testée dans un sandbox logique, physique, visuel, métier ou logiciel.
11	Preuve : les invariants, tests, traces et preuves formelles qualifient ce qui a été produit.
12	X-108 : l'autorité décide ACT, HOLD ou BLOCK selon les risques, inconnues, contradictions et preuves.
13	Action : l'effet autorisé est exécuté dans le monde réel ou dans le système.
14	Observation du résultat : les conséquences et écarts sont capturés.
15	**Correction et nouvelle éducation : l'expérience qualifiée enrichit ou corrige la compréhension.**

Le monde éduque Obsidia. Obsidia agit sous gouvernance. Les conséquences rééduquent Obsidia.

Ce que cette boucle change

Dans une architecture centrée uniquement sur un grand modèle, une grande partie du monde doit être reconstruite dans les poids ou dans une fenêtre de contexte. Ici, le monde, la mémoire, les règles, les preuves et les droits existent hors du modèle. Le modèle devient un organe remplaçable qui reçoit une projection préparée.

> **En très simple**
>
> Le modèle n'a pas besoin de porter tout Obsidia dans sa tête. Obsidia prépare le terrain, lui montre la bonne portion du problème, vérifie sa proposition et garde le droit final d'agir.

4. LES GRANDES BOUCLES CONCEPTUELLES FERMEES

Les chapitres suivants reprennent les grandes boucles retrouvées dans les anciennes conversations et dans la consolidation du 29 juillet. Chaque boucle est expliquée avec son problème initial, sa logique complète, un exemple et son statut réel.

> **Règle de lecture**
>
> Une boucle peut être fermée philosophiquement tout en restant partiellement implémentée. Le document ne confond jamais une bonne architecture avec un runtime déjà terminé.

4.1. TEMPS, HOLD ET IRRÉVERSIBILITÉ

Le problème que cette boucle résout

Une action peut être techniquement possible tout en étant prématurée. Une information peut sembler correcte mais être encore trop fraîche, instable ou contradictoire. Sans gestion du temps, le système risque de transformer une incertitude temporaire en conséquence irréversible.

Ce que le brainstorming a permis de comprendre

La décision ne se réduit pas à agir ou ne pas agir. Attendre consciemment est lui-même un acte de stabilisation. HOLD devient une décision active, traçable et rejouable. Le système compare le coût d'agir, le coût de ne pas agir et le coût d'attendre.

La boucle complete

1	**Etat ou information reçue**
2	Evaluation de la fraîcheur et de la stabilité
3	Mesure du coût d'action
4	Mesure du coût d'inaction
5	Mesure du coût de HOLD
6	Analyse de la réversibilité
7	X-108 décide ACT, HOLD ou BLOCK
8	**Le passage du temps et les nouvelles preuves peuvent rouvrir la décision**

HOLD n'est pas une absence d'action. HOLD est une action de stabilisation temporelle.

Exemple simple

Une banque détecte une transaction inhabituelle. La transaction pourrait être légitime, mais la provenance d'une information manque et l'action est difficilement réversible. Obsidia ne valide pas par défaut et ne bloque pas définitivement : il place la décision en HOLD, demande la preuve manquante et conserve une trace du raisonnement.

Ce que cela change dans Obsidia

- **Capacité et autorité sont séparées :** pouvoir exécuter ne signifie pas avoir le droit d'exécuter.
- **Le temps devient une donnée du raisonnement :** une vérité périssable ne doit pas être utilisée comme une vérité stable.

- **HOLD devient productif :** il crée une fenêtre pour obtenir de nouvelles preuves au lieu de forcer une réponse.

FERMEE PHILOSOPHIQUEMENT	La doctrine est stable. La temporalité signée et les coûts causaux doivent encore être uniformisés dans toutes les couches.

Ce qu'il reste à implémenter ou vérifier

- **Formaliser partout les fenêtres de fraîcheur, les TTL et les preuves de temps.**
- **Tester que HOLD reste actif, traçable, replayable et non contournable.**
- **Relier le coût action / inaction / attente aux domaines critiques.**

4.2. PENSER LIBREMENT, AGIR SOUS PREUVE

Le problème que cette boucle résout

Une intelligence utile doit pouvoir imaginer, proposer, créer des ponts et explorer. Mais si chaque proposition peut devenir directement une vérité, une mémoire ou une action, la créativité devient dangereuse.

Ce que le brainstorming a permis de comprendre

La solution n'est pas de supprimer l'exploration probabiliste. Elle consiste à séparer clairement les statuts. Brody ou Obsidur peuvent produire une hypothèse, une formulation ou un patch, mais cette sortie reste non souveraine tant qu'elle n'a pas traversé les vérifications nécessaires.

La boucle complete

1	**Exploration libre ou génération**
2	Sortie marquée : validé, candidate, hypothèse, à tester
3	Confrontation aux sources, règles et invariants
4	Tests, preuves ou demande humaine
5	X-108 gouverne l'admission et l'action
6	**La proposition est intégrée, rejetée ou renvoyée en exploration**

Penser librement. Agir sous preuve.

Exemple simple

Brody invente un nouveau nom pour une couche ou propose une nouvelle relation entre deux concepts. Cette invention peut être très utile. Mais elle sort avec le statut HYPOTHESE ou CANDIDATE. Elle ne modifie ni la mémoire canonique ni le Kernel avant validation.

Ce que cela change dans Obsidia

- **La créativité reste possible :** le système ne devient pas une machine rigide incapable d'inventer.
- **Les statuts deviennent visibles :** une hypothèse ne se déguise plus en fait.
- **La sortie et l'action sont séparées :** produire une idée ne donne aucun droit d'exécution.

FERMEE CONCEPTUELLEMENT	La doctrine est stable. Les contrôles de statut doivent être appliqués uniformément à Brody, Obsidur, la mémoire et les automatisations.

Ce qu'il reste à implémenter ou vérifier

- **Créer des statuts de sortie communs à tous les organes.**
- **Tester qu'aucune hypothèse ne peut être promue silencieusement.**
- **Brancher le boundary check du langage et de la mémoire.**

4.3. LE KERNEL NE DOIT PAS APPRENDRE LE MONDE

Le problème que cette boucle résout

Un Kernel qui devrait comprendre chaque métier, chaque image, chaque loi physique et chaque situation deviendrait gigantesque, mouvant et difficile à prouver. Son autorité serait mélangée à une intelligence générale impossible à stabiliser.

Ce que le brainstorming a permis de comprendre

Les domaines traduisent le monde. Le Kernel ne juge pas une transaction bancaire comme un banquier ni une route comme un GPS. Il reçoit une forme commune : proposition, inconnues, contradictions, risques, preuves, irréversibilité et autorité demandée.

La boucle complete

1	**Réalité complexe**
2	Domaine spécialisé ou adaptateur
3	Traduction dans un alphabet décisionnel commun
4	Etat borné et traçable
5	Guard / Kernel X-108
6	**ACT, HOLD ou BLOCK**

On ne forme pas le Kernel. On informe le Kernel.

Exemple simple

Le domaine GPS détecte une incohérence entre le GNSS, l'inertiel et une carte. Il ne demande pas au Kernel de comprendre toute la navigation. Il lui transmet : contradiction critique, confiance faible, action irréversible possible, preuves disponibles. Le Kernel peut alors placer la trajectoire en HOLD ou la bloquer.

Ce que cela change dans Obsidia

- **Le Kernel peut rester stable :** les domaines évoluent sans déplacer l'autorité.
- **La preuve devient localisable :** on sait si l'erreur vient de la perception, de la traduction ou de la décision.
- **La V0.1 peut se brancher à plusieurs secteurs :** chaque domaine traduit son réel dans la même structure de gouvernance.

FERMEE ARCHITECTURALEMENT	La doctrine est nette. Le branchement réel et les adaptateurs de tous les domaines doivent encore être audités.

Ce qu'il reste à implémenter ou vérifier

- **Finir les contrats de traduction des domaines.**
- **Vérifier que les domaines ne peuvent pas émettre ACT eux-mêmes.**
- **Tester la priorité BLOCK > HOLD > ACT sur des cas multi-sources.**

4.4. STRUCTURE AVANT INFÉRENCE

Le problème que cette boucle résout

Les architectures centrées sur le modèle ont tendance à envoyer presque toutes les demandes vers une inférence coûteuse, même lorsqu'une route, une règle ou une preuve locale suffit. Le modèle reconstruit alors des éléments déjà connus.

Ce que le brainstorming a permis de comprendre

Obsidia commence par comprendre la forme de la demande : intention, domaine, risque, besoins, inconnues et route possible. Il n'appelle un LLM que lorsque le manque exige réellement une exploration probabiliste.

La boucle complete

1	**Demande utilisateur**
2	Reverse et normalisation
3	Intentions, contexte, domaine et risque
4	Recherche d'une route locale connue
5	Règle, solveur, mémoire ou chemin rapide si suffisant

6	LLM ou organe spécialisé seulement si nécessaire
7	**Validation, preuve et receipt**

Quand la route est connue, prédire devient plus lent que vérifier.

Exemple simple

Un utilisateur demande l'état du Terminal. Si les endpoints health et status existent et que la route est définie, il n'est pas nécessaire qu'un LLM invente une réponse. Le système interroge directement les états, vérifie les invariants et produit un compte rendu.

Ce que cela change dans Obsidia

- **L'inférence devient locale :** elle explore le manque au lieu de reconstruire le monde entier.
- **Les coûts deviennent mesurables :** tokens, appels distants et latence peuvent être évités.
- **Le contexte est préparé avant l'appel :** le modèle reçoit seulement ce dont il a besoin.

FERMEE ET PARTIELLEMENT DEMONTREE	La doctrine est stable et le routeur public en démontre une tranche bornée. La généralisation à toute la stack reste à prouver.

Ce qu'il reste à implémenter ou vérifier

- **Brancher OIE à toutes les routes importantes.**
- **Mesurer appel évité, qualité restante, latence et coût.**
- **Vérifier qu'une route rapide ne contourne jamais les gates.**

4.5. L'INCONNU, L'ABSENCE ET LE REFUS COMME SIGNAUX

Le problème que cette boucle résout

Un système génératif classique tend à remplir les vides par une réponse plausible. Pourtant, une donnée manquante peut être précisément l'information la plus importante : elle indique que le système ne dispose pas du droit ou du contexte nécessaire pour agir.

Ce que le brainstorming a permis de comprendre

L'inconnu devient un objet gouverné. Le système décrit exactement ce qui manque, pourquoi cela compte, quelles conséquences sont possibles et quelle route peut résoudre le manque. Le refus devient une boussole plutôt qu'un échec.

La boucle complete

1	**Manque, contradiction ou ambiguïté détectée**

2	Création d'un objet de manque avec contexte et conséquences
3	Routage vers KNOWN, CONFLICT, RESEARCH, EXPERIMENT ou HUMAN
4	HOLD ou refus si le manque empêche une décision sûre
5	Nouvelle source, expérience ou correction
6	**Réévaluation et promotion éventuelle**

Le vide n'est pas un espace à remplir par génération. C'est un signal de gouvernance.

Exemple simple

Obsidur reçoit une demande de modification mais les tests du module n'existent pas. Au lieu de produire un patch et de prétendre qu'il est sûr, le système marque le manque, place l'intégration en HOLD et demande soit la création des tests, soit une validation humaine.

Ce que cela change dans Obsidia

- **Le système sait dire « je ne sais pas » :** sans transformer l'incertitude en faiblesse.
- **Le manque guide la prochaine action :** recherche, expérience, clarification ou abstention.
- **Les unknowns deviennent auditables :** on peut vérifier pourquoi une décision n'a pas été prise.

FERMEE CONCEPTUELLEMENT	Les objets de manque et leurs routes doivent encore être uniformisés dans le runtime.

Ce qu'il reste à implémenter ou vérifier

- **Créer un Knowledge Gap Envelope canonique.**
- **Tester que les unknowns critiques empêchent ACT.**
- **Relier les refus aux prochaines étapes utiles sans les transformer en recherche automatique illimitée.**

4.6. RECHERCHE DE VÉRITÉ ET RÉEL ADMISSIBLE

Le problème que cette boucle résout

Le monde extérieur ne fournit pas directement une vérité propre. Une page, un document, un capteur ou un message peut être faux, périmé, manipulé ou sorti de son contexte. Si une entrée externe devient immédiatement opérable, un attaquant peut essayer de fabriquer une fausse réalité cohérente.

Ce que le brainstorming a permis de comprendre

Obsidia distingue observation brute, information qualifiée et vérité opérable. Une affirmation passe par la provenance, la fraîcheur, les sources, les contradictions, les limites, les tests et la canonisation avant de pouvoir être utilisée comme base d'une action.

La boucle complete

1	**Observation ou affirmation brute**
2	Source, auteur, date, provenance et contexte
3	Confrontation à plusieurs sources ou attestations
4	Contradictions, limites et durée de validité
5	Zone latente : brute, sourcée, provisoire, testée ou rejetée
6	Expérience, test ou preuve
7	Canonisation bornée
8	**Utilisation comme réel admissible**

Obsidia ne possède pas la vérité. Il gouverne la manière dont une affirmation devient admissible.

Exemple simple

Un document affirme qu'une configuration est la version active du système. Obsidia ne le croit pas simplement parce que le texte est bien rédigé. Il compare le document aux fichiers, au build, aux tests, aux signatures et au runtime. Si les éléments divergent, l'entrée reste provisoire ou passe en HOLD.

Ce que cela change dans Obsidia

- **La sortie d'un LLM n'est jamais une source :** elle doit pointer vers des preuves indépendantes.
- **Le réel opérable est canonisé :** ce qui n'est pas qualifié ne peut pas gouverner une action.
- **La sécurité remonte avant le Kernel :** il faut protéger la fabrication de l'entrée présentée comme réelle.

FERMEE EPISTEMIQUEMENT	La boucle est définie. La multi-attestation, l'anti-replay et le journal de canonisation restent à renforcer.

Ce qu'il reste à implémenter ou vérifier

- **Distinguer BRUT, QUALIFIE, CANONISE et QUARANTAINE.**
- **Ajouter preuve de fraîcheur, nonce, provenance signée et anti-replay.**
- **Empêcher une trace exploratoire de devenir silencieusement décisionnaire.**

4.7. L'INFORMATION SITUÉE EST UNE CAUSE

Le problème que cette boucle résout

Une donnée n'a pas seulement un contenu. Lorsqu'elle arrive à une personne, un agent ou un marché, elle peut modifier une croyance, déclencher une action, provoquer une inaction ou rendre une prédiction auto-réalisatrice. Une même information n'a donc pas le même effet partout et à tout moment.

Ce que le brainstorming a permis de comprendre

L'information devient causale lorsqu'elle est située dans un lieu, un temps, un récepteur, un état du système et une possibilité d'action. Il faut modéliser ce qu'elle peut faire au réel, pas seulement ce qu'elle dit.

La boucle complete

1	**Information reçue**
2	Situation spatiale, temporelle, relationnelle et décisionnelle
3	Perception ou croyance modifiée
4	Décision possible
5	Action, inaction ou HOLD
6	Trace produite dans le monde
7	**Feedback et nouvelle information**

La data ne décrit pas seulement le monde. Elle participe à produire le monde suivant.

Exemple simple

Une application indique qu'une place de parking est libre. Plusieurs conducteurs reçoivent cette information et se dirigent vers la place. La diffusion de l'information modifie immédiatement la réalité qu'elle décrivait et peut la rendre fausse. La donnée doit donc inclure sa fraîcheur, sa propagation et son risque de réflexivité.

Ce que cela change dans Obsidia

- **Le world model devient causal :** il représente les effets possibles de l'information sur le monde.
- **La mémoire fraîche devient utile :** elle suit les tensions et bifurcations récentes.
- **Le coût d'action et d'inaction devient central :** chaque option transforme un futur différent.

FERMEE CONCEPTUELLEMENT	La couche est cadrée mais ses objets et métriques doivent encore être construits.

Ce qu'il reste à implémenter ou vérifier

- **Implémenter SituatedInformationPacket.**
- **Implémenter CausalCostMap et ReverseCausalChain.**
- **Ajouter freshness_score, reflexivity_risk, causal_stability_score et proof_readiness.**

4.8. COMPRÉHENSION ANCRÉE PAR LE RÉEL, L'IMAGE ET LA PHYSIQUE

Le problème que cette boucle résout

Une structure peut être cohérente sans correspondre correctement au monde. Un système peut connaître des mots, des relations et des équations sans savoir à quoi ils se rattachent réellement. L'image seule ne suffit pas non plus : elle peut montrer une apparence sans expliquer les causes ni les limites.

Ce que le brainstorming a permis de comprendre

Les mathématiques et la géométrie donnent la forme. L'image donne une représentation observée. La physique apporte les contraintes du réel. Le contexte précise la situation. La causalité explique les transformations. La compréhension devient la capacité à circuler entre ces niveaux et à corriger les déformations.

La boucle complete

1	**Symbole ou mot**
2	Structure relationnelle
3	Image ou représentation
4	Contexte actif
5	Contraintes physiques et causales
6	Conséquence observée
7	Comparaison avec l'intention ou le réel
8	**Correction et compréhension opérationnelle**

Les mathématiques donnent la forme du savoir. L'image, le réel, le contexte et le langage donnent ce à quoi cette forme correspond.

Exemple simple

Le système reçoit une image d'un objet, la décompose, construit une représentation interne, puis génère une nouvelle vue de cet objet. Il observe ensuite sa propre reconstruction et la compare à l'intention initiale. Ce qui a été perdu ou déformé devient un signal d'apprentissage.

Ce que cela change dans Obsidia

- **L'image devient un exercice :** pas seulement un contenu à générer.
- **La physique fournit une résistance :** une représentation doit respecter des contraintes extérieures au langage.
- **Le contexte devient un état structuré hors du LLM :** il peut survivre aux sessions et être transmis à plusieurs organes.

FERMEE CONCEPTUELLEMENT	Le principe est stable. L'éducation visuelle et le moteur physique complet ne sont pas encore démontrés.

Ce qu'il reste à implémenter ou vérifier

- **Construire les paquets de contexte spécialisés pour les moteurs visuels et physiques.**
- **Définir les tests de reconstruction, d'invariance et de causalité.**
- **Conserver le statut SYNTHETIQUE pour les exercices générés.**

4.9. DE LA CONNAISSANCE AU RÉFLEXE CORRIGIBLE

Le problème que cette boucle résout

Un système qui doit relire toutes ses règles à chaque décision reste lent et dépendant d'une recherche permanente. À l'inverse, un réflexe opaque et impossible à expliquer peut devenir dangereux lorsqu'il rencontre un cas nouveau.

Ce que le brainstorming a permis de comprendre

Une connaissance peut être progressivement compilée en capacité rapide après compréhension, exercices, erreurs, corrections, variations et transfert. Mais le réflexe doit rester décompilable : une anomalie, une contradiction ou une forte conséquence doit forcer le retour à la règle explicite.

La boucle complete

1	**Définition ou règle**
2	Compréhension
3	Exercices
4	Erreurs et corrections
5	Variations de contexte

6	Transfert à des cas nouveaux
7	Automatisation
8	Réflexe
9	Anomalie ou ambiguïté
10	**Retour à la règle, recherche et correction**

La connaissance devient réflexe sans perdre la possibilité d'être expliquée, retracée et corrigée.

Exemple simple

Brody apprend une distinction grammaticale. Au début, il consulte la règle et réalise des exercices. Après suffisamment de cas validés, la distinction devient rapide. Mais si une phrase ambiguë ou contradictoire apparaît, il sort du mode réflexe, réexamine la règle et peut demander une correction.

Ce que cela change dans Obsidia

- **L'éducation ne se réduit pas à ingérer des documents :** elle exige exercices, correction et transfert.
- **Le système distingue plusieurs régimes :** inné, acquis, expérimental et réflexe.
- **Le réflexe reste gouverné :** il ne peut pas contourner HOLD ou les preuves.

FERMEE EDUCATIVEMENT	La boucle est bien définie. Son acquisition réelle par Brody doit encore être démontrée.

Ce qu'il reste à implémenter ou vérifier

- **Créer des progressions éducatives et des paquets d'exercices.**
- **Tracer le passage règle -> capacité -> réflexe.**
- **Tester la décompilation du réflexe lors des anomalies.**

4.10. CROISSANCE MATHÉMATIQUE CUMULATIVE ET ALPHAPROOF

Le problème que cette boucle résout

Une preuve isolée peut résoudre un problème sans enrichir durablement l'architecture. À l'inverse, un corpus de lois non exploré reste statique. La question était de relier l'exploration mathématique, la formalisation, la preuve et l'admission dans le système.

Ce que le brainstorming a permis de comprendre

La comparaison avec AlphaProof a permis de répartir clairement les rôles. Obsidia construit et contraint le territoire. Un moteur explore plusieurs chemins. Obsidur formalise et construit les connecteurs. Lean vérifie. KX108 gouverne l'admission. Le corpus absorbe le résultat et ouvre de nouvelles questions.

La boucle complete

1	**Corpus validé**
2	Manque, contradiction ou zone ouverte
3	Conjecture ou obligation de preuve
4	Exploration de plusieurs chemins
5	Obsidur formalise
6	Lean vérifie ou rejette
7	KX108 gouverne l'admission
8	Le corpus absorbe le résultat
9	**De nouvelles capacités et questions apparaissent**

Obsidia cadre. Le moteur mathématique explore. Obsidur propose. Lean valide. KX108 gouverne. Le corpus absorbe.

Exemple simple

Une propriété candidate est découverte dans une structure. Obsidur la transforme en énoncé Lean. Lean prouve ou rejette la proposition. Même si elle est prouvée, elle ne devient pas automatiquement active dans le runtime : KX108 doit encore gouverner son admission et son périmètre.

Ce que cela change dans Obsidia

- **La preuve devient une infrastructure :** elle agrandit progressivement le territoire sûr.
- **Lean reste non souverain :** prouver n'est pas décider.
- **L'exploration peut rester probabiliste :** la validation et l'admission restent déterministes.

FERMEE CONCEPTUELLEMENT	La boucle des rôles est fermée. Le moteur autonome complet de conjecture et de réintégration reste à construire.

Ce qu'il reste à implémenter ou vérifier

- **Relier les obligations de preuve aux gaps et contradictions.**

- **Automatiser la génération de propositions sans automatiser leur admission.**
- **Conserver une trace complète des preuves, rejets et corrections.**

4.11. CARTE DU SYSTÈME ET RÉALITÉ DU RUNTIME

Le problème que cette boucle résout

Une architecture peut être très cohérente sur le papier tout en décrivant mal le système réellement actif. Un README récent peut parler de modules non branchés. Un composant peut être appelé sans influencer le résultat. Une preuve peut exister sans être reliée au runtime.

Ce que le brainstorming a permis de comprendre

La carte doit devenir une projection vérifiable du système réel : fichiers, fonctions, routes, tests, mesures, versions, dépendances et effets. Cette boucle n'est pas encore fermée ; elle constitue précisément l'objet de l'audit de branchement en cours.

La boucle complete

1	**Carte ou documentation**
2	Inventaire des artefacts réels
3	Vérification des appels et dépendances
4	Test de l'effet causal de chaque couche
5	Comparaison carte / runtime
6	Correction de la carte ou du branchement
7	**Freeze d'un état réellement démontré**

Un fichier présent ne signifie pas qu'un composant est branché. Un composant appelé ne signifie pas qu'il produit l'effet annoncé.

Exemple simple

La documentation affirme que Brody utilise une source mémoire. L'audit vérifie non seulement que le fichier existe et que l'appel apparaît dans le code, mais également que retirer cette source modifie réellement la réponse de la manière attendue.

Ce que cela change dans Obsidia

- **Le projet évite de se raconter une fausse fermeture :** chaque capacité doit être reliée à une preuve d'effet.
- **L'audit devient causal :** il ne vérifie pas seulement la présence des fichiers.

- **Le Terminal peut refléter un état réel :** et non une carte décorative ou obsolète.

IDENTIFIEE MAIS PAS ENCORE FERMEE	C'est l'un des deux grands verrous actuels avec l'attribution causale des capacités.

Ce qu'il reste à implémenter ou vérifier

- **Terminer l'audit de branchement et classer chaque élément.**
- **Tester par ablation quelle couche produit quelle capacité.**
- **Synchroniser automatiquement les receipts, métriques et états avec la carte.**

4.12. TRACES SILENCIEUSES ET MÉMOIRE NON SOUVERAINE

Le problème que cette boucle résout

Une trace peut rester dans la mémoire, un ancien contexte, un replay ou un réflexe. Le danger n'est pas son existence. Le danger apparaît lorsqu'elle influence une décision sans origine, sans statut et sans que le système sache qu'elle agit.

Ce que le brainstorming a permis de comprendre

La doctrine devient : aucune trace silencieuse ne doit devenir décisionnaire. Chaque trace doit avoir une origine, une version, un niveau de confiance, une fraîcheur, un périmètre et un statut souverain ou non souverain.

La boucle complete

1	**Trace produite**
2	Identification de l'origine
3	Statut et niveau de confiance
4	Version, fraîcheur et périmètre
5	Autorisation d'usage ou quarantaine
6	Projection readonly vers un organe
7	**Décision toujours gouvernée par X108**

La mémoire conserve. Elle ne gouverne jamais.

Exemple simple

Une ancienne hypothèse réapparaît dans une recherche sémantique. Brody peut l'utiliser pour rappeler qu'une piste a existé, mais il doit la marquer comme PROVISIONAL ou DEPRECATED. Elle ne peut pas devenir un fait actuel ni déclencher une action.

Ce que cela change dans Obsidia

- **La mémoire devient explicable :** on sait pourquoi un élément a été remonté.
- **Le replay reste utile sans devenir une vérité :** une ancienne situation ne vaut pas automatiquement pour le présent.
- **Les organes reçoivent des projections readonly :** la mémoire ne prend jamais l'autorité.

FERMEE COMME DOCTRINE	La typologie et les gardes de traces doivent encore être généralisées.

Ce qu'il reste à implémenter ou vérifier

- **Ajouter TTL, origine, version, confiance et statut à chaque trace.**
- **Interdire la promotion automatique d'une trace exploratoire.**
- **Tester les fuites entre mémoire, réflexe, replay et décision.**

5. LES FONCTIONNALITES QUI MATERIALISENT LES BOUCLES

Les boucles précédentes donnent le sens. Les fonctionnalités suivantes permettent à cette philosophie de devenir un système concret. Elles ne sont pas toutes terminées, mais leur rôle est désormais clair.

5.1 Capture des pépites

But. Faire de tes discussions une source de R&D structurée sans confondre intuition et canon.

> **Fonctionnement simple**
>
> Flux libre -> détection d'une formulation forte -> rapprochement avec les lois existantes -> statut -> obligation de preuve -> Freeze.

Brody ne résume plus seulement. Il signale qu'une loi vient peut-être d'apparaître, montre les conversations liées et sépare pépite, hypothèse, preuve et implémentation.

STATUT	A spécifier après le Terminal de base.

5.2 Terminal unifié

But. Donner une seule entrée lisible à toute la machination.

> **Fonctionnement simple**
>
> Demande -> plan actif -> Capability Graph -> Corpus Resolver -> organes utiles -> gates -> résultat -> receipt.

Le Terminal ne doit pas appeler tout le monde. Il choisit la route, explique ce qui est actif, montre ce qui manque et fournit soit un résultat, soit une commande, soit une décision HOLD/BLOCK.

STATUT	P0 - finalisation prioritaire.

5.3 Audit de branchement

But. Faire correspondre la carte du projet au système vivant.

> **Fonctionnement simple**
>
> Inventaire -> ACTIVE / READY_TO_CONNECT / DORMANT / BROKEN / DEPRECATED -> preuve de l'effet -> action.

L'audit doit vérifier les mémoires, domaines, routes, endpoints, tests, automatisations, droits et modules dormants.

STATUT	P0 - en cours.

5.4 Sigma introspection

But. Permettre au système d'observer ses propres incohérences sans prendre le pouvoir.

> **Fonctionnement simple**
>
> Etat ou résultat -> contradictions -> localisation probable -> guidance -> alerte ou gel -> retour vers la bonne couche.

Sigma peut dire où regarder et pourquoi une divergence est suspecte. Il ne décide pas et ne répare pas silencieusement.

STATUT	P1 - consolider le guidage.

5. LES FONCTIONNALITES - SUITE

5.5 OIE économie d'inférence

But. Mesurer quand la structure a réellement évité une inférence.

> **Fonctionnement simple**
>
> Route -> appel effectué ou évité -> tokens, latence, coût et qualité -> amélioration du prochain routage.

OIE montre la valeur réelle de Structure Before Inference. Il ne décide pas si une route est vraie ; il mesure son efficacité.

STATUT	P1 - partiellement démontré.

5.6 Memory Readiness

But. Transformer les documents et traces en mémoire utilisable.

> **Fonctionnement simple**
>
> Source -> provenance -> statut -> branchement ou quarantaine -> projection readonly -> réévaluation.

Les statuts CANON, PROVISIONAL, NO_TRAIN, READONLY, DEPRECATED, NEEDS_AUDIT et READY_TO_CONNECT empêchent de mélanger présence et fiabilité.

STATUT	P0/P1 - audit prioritaire.

5.7 Education Bus

But. Eduquer progressivement sans donner l'autorité aux sources éducatives.

> **Fonctionnement simple**
>
> Source -> filtre NO_TRAIN -> paquet -> exercice -> réponse -> correction -> transfert -> validation -> réflexe.

Les sources peuvent être toi, des documents, des tests, un LLM externe, Lean ou le repo. Aucune ne décide seule de ce qui devient canon.

STATUT	Après le Freeze structurel.

5.8 Automation Governance

But. Rendre les processus répétables sans automatiser l'autorisation.

> **Fonctionnement simple**
>
> Evénement -> ticket -> contexte -> proposition -> tests -> verdict -> trace -> Freeze éventuel.

La couche peut router, déclencher, documenter et préparer. Elle ne peut pas transformer automatiquement une proposition en action autorisée.

STATUT	P1/P2.

5. LES FONCTIONNALITES - SUITE

5.9 Obsidure Code Mutation Gate

But. Gouverner les modifications de code produites par les IA.

> **Fonctionnement simple**
>
> Demande dev -> patch IA -> diff -> Reverse de l'intention -> impact -> invariants -> tests -> ALLOW / HOLD / BLOCK.

Claude Code, Codex ou Cursor génèrent. Obsidure juge le droit d'intégration. Chaque session devient aussi un futur cas d'apprentissage.

STATUT	P1.

5.10 Agent Rights Orchestration

But. Faire croître les agents sans créer des super-agents opaques.

> **Fonctionnement simple**
>
> Mission -> droits -> sortie typée -> vote -> agrégation -> contradictions -> Guard -> décision.

Chaque agent reste simple. La complexité se trouve dans les droits, les flux, les votes, les agrégations et les preuves.

STATUT	P1.

5.11 Brody Native Language

But. Permettre un langage vivant sans transformer l'invention en vérité.

> **Fonctionnement simple**
>
> Contexte -> mode de réponse -> composition -> marquage des affirmations -> boundary check -> sortie.

Brody distingue restitution, structuration, projection contrôlée, invention bornée et refus structurel.

STATUT	P1/P2.

5.12 Physical Signal World Model

But. Ajouter plus tard une perception physique complémentaire.

> **Fonctionnement simple**
>
> Wi-Fi, RF, GNSS, radar ou UWB -> features agrégées -> cohérence ou contradiction -> preuve candidate.

La couche observe les phénomènes physiques, pas les individus. Elle reste EVIDENCE_ONLY, NO_ACT et KX108_ONLY.

STATUT	Branche future, hors chemin critique.

6. QUI FAIT QUOI : CARTE SIMPLE DES ROLES ET DES AUTORITES

Une grande partie de la fermeture vient de la séparation des pouvoirs. Plusieurs organes peuvent être très capables sans posséder le droit final d'agir. Cette carte évite les confusions les plus fréquentes.

Organe	Rôle	Ce qu'il fait	Ce qu'il ne fait pas
Terminal	Point d'entrée et cockpit	Comprend la demande, sélectionne les routes, montre les états et rassemble les sorties.	Ne devient pas une autorité cachée.
Brody	Langage, compréhension et contextualisation	Formule, explique, relie, structure et propose.	N'est ni la mémoire, ni le Kernel, ni une autorité d'action.
Obsidur	Agent de code et formalisation	Propose des patches, formalise, construit et prépare des mutations.	Ne décide pas seul que son code peut être intégré.
Domaines	Traducteurs du réel spécialisé	Comprennent banque, trading, GPS ou autres terrains et produisent un état borné.	Ne possèdent pas le droit final d'agir.
Mémoire	Continuité et contexte readonly	Conserve les traces qualifiées et fournit des projections adaptées.	Ne canonise ni ne décide seule.
Sigma	Introspection et contradiction	Détecte les divergences, alerte, guide et peut demander un gel.	Ne tranche pas ACT / HOLD / BLOCK.
OIE	Mesure de l'inférence	Mesure les appels évités, les coûts, la latence et la qualité.	Ne détermine pas la vérité d'une route.
Lean	Vérification formelle	Prouve ou rejette des énoncés formalisés.	Prouver ne signifie pas autoriser l'activation.
Automation Layer	Circulation et répétabilité	Déclenche, route, crée des tickets et prépare des rapports.	Automatiser n'est pas autoriser.
X-108 / Guard	Autorité décisionnelle	Evalue risques, inconnues, contradictions, preuves et irréversibilité.	Seul le cadre souverain rend ACT, HOLD ou BLOCK.

La capacité de proposer est distribuée. Le droit d'agir reste isolé.

7. UN SCENARIO COMPLET DE BOUT EN BOUT

Pour rendre l'architecture plus simple à comprendre, voici un exemple où une entreprise demande à une IA de modifier un composant logiciel important. Cet exemple montre comment presque toutes les boucles se rejoignent.

1. La demande arrive

Le Terminal reçoit : « ajoute cette fonctionnalité au système de paiement ». La phrase n'est pas encore une autorisation.

2. Reverse extrait l'intention

Le système identifie l'objectif, les fichiers probables, le domaine, le risque financier et les inconnues.

3. Le contexte est compilé

Obsidur reçoit l'architecture active, les dépendances, les tests, les invariants, les anciens incidents et les fichiers protégés.

4. L'agent produit une proposition

Claude Code ou Obsidur génère un patch. Le patch reste CANDIDATE et NON SOUVERAIN.

5. Le diff est lu comme une mutation

Code Mutation Gate compare l'intention déclarée à l'intention réelle du diff, repère les routes, droits et couches touchées.

6. Les inconnues sont explicitées

Si un test manque ou si une dépendance est ambiguë, le système crée un gap au lieu d'inventer une assurance.

7. Tests et preuves

Les tests, les invariants et éventuellement Lean vérifient ce qui peut l'être. Sigma recherche les contradictions.

8. Le coût temporel est évalué

Si l'intégration est irréversible ou si une preuve est encore fraîche, HOLD peut être préférable à ACT.

9. X-108 gouverne

Le Kernel reçoit proposition, risques, inconnues, contradictions, preuves et autorité demandée. Il rend ACT, HOLD ou BLOCK.

10. L'intégration laisse une preuve

Si ACT est autorisé, le patch est intégré avec receipt, diff, tests, version et trace.

11. Le résultat rééduque le système

Les conséquences observées deviennent une expérience qualifiée. Elles peuvent améliorer les règles, les tests ou les futurs réflexes d'Obsidur.

Générer n'est pas intégrer. Modifier n'est pas muter sans preuve.

8. CE QUI EST FERME, PARTIEL OU ENCORE OUVERT

La discussion a aussi permis de ne pas confondre trois types d'état. Cette distinction protège le projet contre les annonces trop rapides et aide à définir le vrai chemin critique.

Fermé conceptuellement La loi, les rôles, les limites et la boucle sont suffisamment clairs pour ne plus devoir être réinventés.	**Partiellement démontré** Une tranche fonctionne ou a été testée, mais la généralisation à toute la stack reste à prouver.
Identifié mais ouvert Le problème et la direction sont connus, mais la boucle n'est pas encore stabilisée ou implémentée.	**Branche future** Le concept est cadré et conservé, mais il ne doit pas entrer dans le chemin critique actuel.

Fermé comme doctrine

- Séparation pensée / preuve / action.
- Kernel informé par les domaines, non entraîné au monde.
- Structure avant inférence.
- Inconnu et refus comme signaux.
- Vérité qualifiée et réel admissible.
- Information située comme cause.
- Compréhension ancrée par image, contexte et physique.
- Connaissance vers réflexe corrigible.
- Rôles Obsidur / Lean / KX108 dans la croissance formelle.
- Mémoire et automatisation non souveraines.

Partiellement démontré

- Router et économie d'inférence sur une surface bornée.
- Terminal V0 en lecture et guidance.
- Lean / Obsidure sur plusieurs théorèmes et lots.
- Stack locale, domaines et mémoires partiellement reconnectés.

Encore ouvert ou à prouver

- Synchronisation automatique carte / runtime.
- Attribution causale précise : quelle couche produit quelle capacité.
- Education visuelle et acquisition réelle de réflexes.
- Moteur physique complet.
- Exploration mathématique autonome et réintégration cumulative.
- Tous les organes branchés et régressés comme un ensemble.

Branches futures à ne pas ouvrir maintenant

- Physical Signal World Model.
- Scraping causal large.

- Corpus éducatif massif.
- Nouvelles extensions cosmologiques ou multidimensionnelles non formalisées.
- Automatisation forte avant stabilisation des droits et des flux.

9. L'ORDRE D'IMPLEMENTATION RETENU

La grande valeur pratique de cette discussion est d'avoir fixé un ordre. Tu n'as plus besoin de choisir chaque jour entre des dizaines de couches. Le chemin critique est maintenant clair.

Le chemin critique

1	**Terminer le Terminal comme entrée canonique et observable.**
2	Finir l'audit de branchement : mémoire, domaines, automatisations, droits, routes, preuves et modules dormants.
3	Classer chaque élément : ACTIVE, READY_TO_CONNECT, DORMANT, BROKEN, DEPRECATED ou FUTURE.
4	Synchroniser la carte du projet avec le runtime réel.
5	Brancher ce qui est déjà prêt sans réinventer de nouvelles couches.
6	Intégrer les fermetures définies dans cette discussion : statuts, gaps, traces, droits, Code Gate, introspection et mesure.
7	Tester les flux complets et l'attribution causale des capacités.
8	Produire des receipts, métriques et freezes successifs.
9	Déclarer la V1 structurelle lorsque tout est relié, statué, traçable et prêt pour une éducation contrôlée.
10	**Seulement ensuite ouvrir les documents sources, l'Education Bus et le corpus large.**

> **Règle de phase**
>
> Une nouvelle idée peut être capturée et classée, mais elle ne doit pas interrompre le Terminal, l'audit ou les branchements déjà décidés sauf si elle révèle une contradiction critique.

Ne plus agrandir la matière tant que la machination existante n'est pas entièrement reliée, vérifiée et figée.

10. POLITIQUE DES DOCUMENTS SOURCES

Tu as réuni beaucoup de documents sources importants. Ils auront de la valeur pour l'éducation, la vérification, la recherche ciblée et la reconstruction de certaines idées. Mais ils ne sont pas nécessaires dans le chemin critique immédiat.

Les ouvrir maintenant pourrait mélanger des documents anciens, provisoires, redondants ou non qualifiés avec la reconstruction en cours. Cela risquerait également de rouvrir des milliers de pistes avant d'avoir fini les branchements déjà préparés.

> **Statut recommandé**
>
> SOURCE_ARCHIVE + READONLY + NOT_CONNECTED + NO_TRAIN_FOR_NOW + REVIEW_AFTER_V1_FREEZE

Ce que cela veut dire

- **Ils ne sont pas abandonnés :** ils sont préservés pour la phase suivante.
- **Ils ne sont pas encore éducatifs :** aucun document ne doit entrer dans un training packet sans audit.
- **Ils ne sont pas canon automatiquement :** leur présence ne leur donne aucun statut de vérité.
- **Ils ne doivent pas distraire le chantier actuel :** le Terminal et l'audit restent prioritaires.

On capture les nouvelles matières. On ne les branche pas tant que la structure actuelle n'est pas gelée.

11. LE CRITERE REALISTE DE V1

La V1 ne signifie pas que tout est intelligent, autonome ou capable de comprendre le monde entier. Elle signifie que l'architecture essentielle fonctionne comme un ensemble cohérent et qu'elle est prête à recevoir une éducation contrôlée.

- **Brody contextualise :** il peut comprendre, structurer et expliquer sans prendre l'autorité.
- **Obsidur propose et formalise :** ses mutations passent par le Code Gate.
- **Les domaines sont branchés :** ils traduisent le réel sans émettre ACT eux-mêmes.
- **La mémoire est statutée :** les sources sont qualifiées, readonly et non souveraines.
- **Les automatisations sont gouvernées :** elles circulent sans autoriser.
- **Les flux sont auditables :** on peut rejouer une demande, une proposition, un verdict et une action.
- **Les métriques sont vérifiables :** latence, appels, coûts, qualité, contradictions et preuves.
- **La carte correspond au runtime :** les composants annoncés ont un effet causal démontré.
- **Les learning packets sont propres :** le système est prêt pour l'éducation sans ingérer un corpus sauvage.

> **Définition courte**
>
> La V1 est atteinte lorsque tout est relié, statué, traçable et prêt pour une éducation contrôlée, sans confusion de pouvoir.

12. PHRASES CANONIQUES A CONSERVER

1	**Penser librement. Agir sous preuve.**
2	**On ne forme pas le Kernel. On informe le Kernel.**
3	**Quand la route est connue, prédire devient plus lent que vérifier.**
4	**Le vide n'est pas un espace à remplir. C'est un signal de gouvernance.**
5	**Obsidia ne possède pas la vérité. Il gouverne la manière dont une affirmation devient admissible.**
6	**Ce qui n'est pas canonisé n'existe pas pour Obsidia comme vérité opérable.**
7	**La data ne décrit pas seulement le monde. Elle participe à produire le monde suivant.**
8	**HOLD n'est pas une absence d'action. HOLD est une action de stabilisation temporelle.**

12. PHRASES CANONIQUES - SUITE

9	**Les mathématiques donnent la forme du savoir. L'image, le réel, le contexte et le langage donnent ce à quoi cette forme correspond.**
10	**La connaissance devient réflexe sans perdre la possibilité d'être expliquée, retracée et corrigée.**
11	**Obsidia cadre. Le moteur explore. Obsidur propose. Lean valide. KX108 gouverne. Le corpus absorbe.**
12	**L'agent est simple. L'orchestration est souveraine.**
13	**La capacité de proposer est séparée du droit d'agir.**
14	**Les IA génèrent. Obsidure autorise.**
15	**Générer n'est pas intégrer.**
16	**Modifier n'est pas muter sans preuve.**

12. PHRASES CANONIQUES - SUITE

17	**Apprendre n'est pas autoriser.**
18	**Expliquer n'est pas valider.**

19	**Contextualiser n'est pas décider.**
20	**Automatiser n'est pas autoriser.**
21	**La mémoire conserve. Elle ne gouverne jamais.**
22	**On n'automatise pas le chaos. On automatise une structure déjà stabilisée.**
23	**Un fichier présent ne signifie pas qu'un composant est branché.**
24	**La vision est fermée. La machination doit maintenant prouver qu'elle correspond à cette vision.**

13. CONCLUSION : LA DISCUSSION MAITRESSE

Cette discussion est structurante parce qu'elle ne s'est pas contentée de produire une nouvelle idée brillante. Elle a repris les pépites des dernières semaines, retrouvé les boucles plus anciennes, distingué les doctrines des fonctionnalités et fixé une frontière nette entre ce qui est fermé, ce qui est partiellement prouvé et ce qui doit attendre.

Le résultat est une architecture dans laquelle le monde, l'éducation, le contexte, les modèles, la mémoire, les preuves et l'action ne sont plus des blocs indépendants. Ils forment une circulation cohérente. Le modèle peut comprendre et proposer sans posséder l'autorité. La mémoire peut conserver sans décider. Lean peut prouver sans activer. L'automatisation peut circuler sans autoriser. X-108 garde le droit de passage.

On n'a pas ajouté une nouvelle couche à Obsidia. On a refermé l'architecture entière autour de ce qui existait déjà.

La prochaine étape n'est donc pas une nouvelle phase de brainstorming massif. Elle est plus simple à énoncer et plus exigeante à exécuter : terminer le Terminal, finir l'audit de branchement, connecter les éléments prêts, prouver leurs effets, geler un état structurel cohérent, puis seulement ouvrir l'éducation et les documents sources.

> **La phrase de phase**
>
> Finir l'état actuel. Brancher la carte stabilisée. Tester les boucles complètes. Prouver les effets. Geler la V1 structurelle. Puis éduquer.

Sources internes utilisées

Discussion de consolidation du 29 juillet 2026, anciennes conversations Obsidia rappelées dans cette discussion, document consolidé « bsidia suite 29.07 », et première version du support « Boucles fermées et passage à l'implémentation ».

OBSIDIA

Boucles fermées et passage à l'implémentation

Support de référence issu de la discussion du 29 juillet 2026

On n'a pas ajouté une nouvelle couche à Obsidia. On a refermé l'architecture entière autour de ce qui existait déjà.

Objet du document. Rassembler les pépites des dernières semaines, distinguer les boucles réellement fermées des pistes encore ouvertes, traduire ces fermetures en fonctionnalités et fixer l'ordre d'implémentation sans rouvrir la dispersion.

Nature Support interne de synthèse, d'architecture et de pilotage.	**Périmètre** Philosophie, boucles cognitives, fonctions, autorités, statuts et feuille de route.
Source principale Discussion en cours et document consolidé « bsidia suite 29.07 ».	**Doctrine de phase** Finir, brancher, tester et geler avant d'ouvrir le corpus source et l'éducation.

Version 1.0 | 29 juillet 2026

SOMMAIRE

1. Ce que cette discussion a réellement fermé
2. La méthode de R&D : du freestyle au Freeze
3. La boucle maîtresse d'Obsidia
4. Les grandes boucles conceptuelles fermées
5. Les fermetures fonctionnelles
6. Carte des rôles et des autorités
7. Ce qui est fermé, partiel ou encore ouvert
8. Passage à l'implémentation
9. Politique des documents sources
10. Critère réaliste de V1 structurelle
11. Formules canoniques à conserver
12. Conclusion de phase

> **Lecture recommandée**
>
> Les sections 1 à 4 posent le sens. Les sections 5 à 8 traduisent ce sens en modules et en ordre de travail. Les sections 9 à 12 fixent la discipline de phase.

1. CE QUE CETTE DISCUSSION A REELLEMENT FERME

Cette discussion marque une transition de phase. Le projet ne se trouve plus principalement dans l'invention de grandes couches. Les principales lois, séparations de pouvoir, boucles cognitives et familles fonctionnelles ont désormais une place cohérente dans un système global.

> **Résultat central**
>
> Obsidia dispose maintenant d'un schéma fonctionnel complet : perception, monde structuré, éducation, contexte, capacités, génération, simulation, preuve, autorité et apprentissage.

Avant	Après
Accumulation de pépites, intuitions, modules, documents, sous-couches et branches conceptuelles.	Carte consolidée des boucles, des droits, des fonctions, des statuts et des dépendances.
Risque ancien	**Règle nouvelle**
Réinventer, rouvrir des chantiers et brancher trop tôt des sources ou des idées non qualifiées.	Capturer les idées, mais terminer l'état actuel avant toute nouvelle expansion.

La vision est fermée. La machination doit maintenant prouver qu'elle correspond à cette vision.

- **Ce qui est acquis :** les grandes boucles conceptuelles, la séparation proposition / décision / action, la place du Kernel, la vérité qualifiée, l'éducation, le contexte, la preuve et le retour d'expérience.
- **Ce qui devient prioritaire :** Terminal, audit de branchement, synchronisation carte-runtime, attribution causale, tests transversaux et freezes.
- **Ce qui doit attendre :** corpus source massif, scraping, nouvelles branches, entraînement large et fonctionnalités futures non nécessaires au chemin critique.

2. LA METHODE DE R&D : DU FREESTYLE AU FREEZE

Le mot « recherche » décrit mal le point de départ. La plupart des découvertes sont nées d'un brainstorming libre, parfois chaotique, où une friction ou une intuition faisait apparaître progressivement une structure stable.

Boucle naturelle de découverte

1	**Friction, intuition, analogie ou contradiction**
2	Brainstorming libre entre plusieurs domaines
3	Détection d'un invariant transversal
4	Formulation d'une loi candidate
5	Rattachement aux couches existantes
6	Séparation concept / hypothèse / preuve / implémentation
7	Répartition des rôles et des autorités
8	Tests, obligations de preuve ou statut d'attente
9	**Freeze conceptuel ou technique**

Fonctionnalité à institutionnaliser

Brody doit pouvoir détecter qu'une loi vient d'apparaître, extraire la formulation brute, la rattacher aux couches existantes et proposer un statut : pépite, hypothèse, loi candidate, à prouver, à intégrer ou à Freeze.

Pépites détectées Formulations ou relations nouvelles apparues dans un flux de discussion.	**Lois candidates** Pépites qui expliquent plusieurs cas et ferment une contradiction ancienne.
A prouver Concepts suffisamment précis pour produire des tests ou une formalisation.	**A Freeze** Blocs cohérents, bornés, reliés et assez stables pour devenir canoniques.

Une boucle est fermée lorsque chaque contradiction, chaque inconnu, chaque acteur et chaque conséquence trouvent une place cohérente dans une circulation complète.

3. LA BOUCLE MAITRESSE D'OBSIDIA

Toutes les boucles retrouvées convergent vers une circulation générale. Cette boucle relie réception du monde, construction de la compréhension, production de possibilités, gouvernance de l'action et réintégration des conséquences.

Circulation générale

1	**Réalité et état du monde**
2	Perception, signaux, images et observations
3	Ancrage, provenance, fraîcheur et contexte
4	Education, primitives, langage, maths et physique
5	Mémoire qualifiée et continuité
6	Compilation du contexte avant inférence
7	Routage vers les capacités nécessaires
8	Génération, exploration et simulation bornées
9	Preuves, contradictions, coûts et risques
10	X-108 : HOLD / BLOCK / ACT
11	Action contrôlée, receipt et replay
12	Observation des conséquences
13	**Correction, mise à jour et nouvelle éducation**

Le monde éduque Obsidia. Obsidia agit sous gouvernance. Les conséquences rééduquent Obsidia.

> **Ce que cette fermeture change**
>
> Obsidia n'est plus une collection de briques. Il devient une architecture circulaire où chaque sortie revient comme une trace qualifiable, sans que la mémoire, le modèle, l'automatisation ou la preuve ne puissent prendre seuls le pouvoir.

4. LES GRANDES BOUCLES CONCEPTUELLES FERMEES

Les pages suivantes présentent les boucles considérées comme fermées au niveau philosophique et architectural. Leur implémentation peut rester partielle ; leur logique, leur frontière et leur répartition des rôles ne sont plus à réinventer.

4.1 Temps, friction, HOLD et irréversibilité

1	**Etat incomplet ou instable**
2	Mesure de fraîcheur, contradictions et coût
3	Comparaison agir / ne pas agir / attendre
4	HOLD, BLOCK ou ACT
5	Trace temporelle et conséquences
6	**Réévaluation lorsque l'état change**

HOLD n'est pas une absence d'action. HOLD est une action de stabilisation temporelle et causale.

Ce que la boucle ferme

- Une capacité technique ne donne pas automatiquement le droit d'agir.
- Une bonne action exécutée trop tôt peut devenir une mauvaise action.
- L'inaction et l'attente ont elles aussi un coût qui doit être représenté.

FERMEE CONCEPTUELLEMENT	Temporalité signée, anti-replay et propagation complète restent à consolider dans toutes les couches.

4.2 Liberté de pensée, sortie gouvernée

1	**Exploration libre**
2	Hypothèses, formulations et pistes
3	Marquage des statuts
4	Confrontation aux sources, règles et preuves
5	Proposition non souveraine
6	**Autorisation séparée par X-108**

Penser librement. Agir sous preuve.

Ce que la boucle ferme

- Brody, Obsidur ou un modèle peuvent inventer et proposer.
- Une proposition ne devient jamais silencieusement vérité, mémoire canonique ou droit d'agir.
- Le probabiliste peut explorer ; il ne devient pas souverain.

FERMEE CONCEPTUELLEMENT	Le contrôle des statuts doit être appliqué uniformément aux sorties et aux mémoires.

4.3 Le Kernel ne doit pas apprendre le monde

1	**Réalité complexe**
2	Domaine spécialisé
3	Traduction dans un alphabet commun
4	Etat décisionnel borné
5	Guard / Kernel X-108
6	**HOLD, BLOCK ou ACT**

On ne forme pas le Kernel. On informe le Kernel.

Ce que la boucle ferme

- Les domaines comprennent et traduisent leurs terrains.
- Le Kernel évalue contradictions, inconnues, risques, preuves et irréversibilité.
- L'autorité peut rester stable pendant que les périphéries progressent.

FERMEE ARCHITECTURALEMENT	Le branchement de tous les domaines et adaptateurs reste un chantier technique.

4.4 Structure avant inférence

1	**Demande utilisateur**
2	Normalisation et langage interne
3	Domaine, risque et besoins
4	Route locale connue ou inconnue
5	Solveur / règle / mémoire si suffisant
6	LLM uniquement si nécessaire
7	**Validation et trace**

Quand la route est connue, prédire devient plus lent que vérifier.

Ce que la boucle ferme

- Le premier choix n'est pas quel modèle appeler, mais si un modèle est nécessaire.
- L'inférence devient un organe ciblé, pas le cerveau souverain.
- La qualité inclut coût évité, appels évités, vitesse et respect des invariants.

FERMEE ET PARTIELLEMENT DEMONTREE	Le router public démontre une tranche bornée ; la généralisation à toute la stack reste à prouver.

4.5 L'inconnu, l'absence et le refus deviennent des signaux

1	**Donnée manquante ou contradiction**
2	Qualification du manque
3	Route KNOWN / CONFLICT / RESEARCH / EXPERIMENT / HUMAN
4	HOLD ou refus si insuffisant
5	Recherche ou expérience ciblée
6	**Réouverture après nouvelle preuve**

Le vide n'est pas un espace à remplir par génération. C'est un signal de gouvernance.

Ce que la boucle ferme

- Un refus indique souvent une frontière ou une dépendance manquante.
- Les unknowns empêchent une fausse assurance.
- L'abstention devient une sortie légitime, traçable et utile.

FERMEE CONCEPTUELLEMENT	Les schémas de Knowledge Gap et les politiques de promotion doivent être branchés au runtime.

4.6 Recherche de vérité et admissibilité du réel

1	**Signal ou affirmation brute**
2	Provenance, temps et contexte
3	Sources et attestations
4	Contradictions et limites
5	Zone latente / statut provisoire
6	Tests et confrontation au réel
7	Canonisation bornée
8	**Usage opérable**

Obsidia ne possède pas la vérité. Il gouverne la manière dont une affirmation devient admissible.

Ce que la boucle ferme

- Une sortie LLM n'est jamais une source.
- Ce qui n'est pas canonisé n'existe pas comme vérité opérable.
- La sécurité protège aussi la fabrication de la réalité présentée au système.

FERMEE EPISTEMIQUEMENT	La multi-attestation, la quarantine et la canonisation automatisée restent à formaliser et tester.

4.7 L'information située est une cause

1	**Information reçue**
2	Lieu, temps, récepteur et état
3	Croyance ou perception modifiée
4	Décision
5	Action / inaction / HOLD
6	Trace et feedback
7	**Nouvelle information**

La donnée ne décrit pas seulement le monde. Elle participe à produire le monde suivant.

Ce que la boucle ferme

- Une même information n'a pas le même effet selon son contexte.
- Une prédiction peut modifier la réalité qu'elle décrit.
- Le world model doit représenter l'effet causal de ses propres informations.

FERMEE CONCEPTUELLEMENT	SituatedInformationPacket, CausalCostMap et ReverseCausalChain restent à implémenter.

4.8 Compréhension ancrée par le réel, l'image et la physique

1	**Symbole**
2	Structure
3	Représentation
4	Contexte
5	Réel et limites physiques
6	Conséquence observée
7	Correction
8	**Compréhension opérationnelle**

Les maths donnent la forme du savoir. L'image, le réel, le contexte et le langage donnent ce à quoi cette forme correspond.

Ce que la boucle ferme

- L'image devient leçon, reconstruction, test et preuve visuelle.
- La physique fournit une résistance extérieure aux représentations.
- Le contexte doit être compilé avant l'inférence, pas reconstruit après.

FERMEE CONCEPTUELLEMENT	L'éducation visuelle et le World Model physique complet ne sont pas encore démontrés.

4.9 De la connaissance au réflexe corrigible

1	**Définition explicite**
2	Compréhension
3	Exercices
4	Erreurs
5	Corrections et variations
6	Transfert
7	Automatisation
8	Réflexe
9	Anomalie
10	**Décompilation et correction**

Une connaissance comprise, répétée et validée finit par être compilée en réflexe sans perdre la possibilité d'être expliquée.

Ce que la boucle ferme

- La règle reste traçable même lorsque son usage devient rapide.
- Une ambiguïté ou une contradiction doit faire sortir du mode réflexe.
- L'éducation relie primitives, expérience, mémoire et contrôle.

FERMEE EDUCATIVEMENT	L'acquisition réelle et le transfert de réflexes par Brody doivent être démontrés.

4.10 Croissance mathématique cumulative

1	**Corpus validé**
2	Manque, contradiction ou conjecture
3	Exploration de plusieurs chemins
4	Obsidur formalise
5	Lean vérifie
6	KX-108 gouverne l'admission
7	Le corpus absorbe
8	**Nouvelles capacités et questions**

Obsidia cadre. Le moteur explore. Obsidur formalise. Lean valide. KX-108 gouverne. Le corpus absorbe.

Ce que la boucle ferme

- La preuve devient une infrastructure cumulative, pas seulement un résultat.
- Lean prouve mais ne décide pas.
- L'admission d'un théorème ou d'une capacité reste gouvernée.

FERMEE CONCEPTUELLEMENT	L'exploration autonome, la production de conjectures et la réintégration complète restent à construire.

5. LES FERMETURES FONCTIONNELLES

Certaines pépites ne forment pas une nouvelle philosophie autonome. Elles ferment le passage entre les doctrines et un système utilisable, observable et améliorable.

5.1 Capture des pépites

> **Boucle**
>
> Flux de discussion -> détection de loi -> statut -> rattachement -> obligation de preuve -> Freeze.

Fonction : Transforme le brainstorming naturel en pipeline de R&D reproductible.

Statut : P0 - spécification à formaliser

5.2 Terminal unifié

> **Boucle**
>
> IN -> plan actif -> Capability Graph -> Corpus Resolver -> organes -> sortie gouvernée.

Fonction : Transforme toute la machination en une capacité utilisable depuis une seule entrée.

Statut : P0 - finalisation en cours

5.3 Audit de branchement

> **Boucle**
>
> Inventaire -> ACTIVE / READY_TO_CONNECT / DORMANT / BROKEN / DEPRECATED -> preuve de causalité.

Fonction : Fait correspondre la carte du projet avec le runtime réel.

Statut : P0 - en cours

5.4 Sigma introspection

> **Boucle**
>
> Etat ou résultat -> contradictions -> guidance -> alerte / gel -> retour vers la bonne couche.

Fonction : Ferme la boucle d'observation du système par lui-même sans donner de pouvoir décisionnel à Sigma.

Statut : P1

5.5 OIE économie d'inférence

> **Boucle**
>
> Route choisie -> appel évité ou effectué -> coût, latence et qualité -> amélioration du routage.

Fonction : Mesure la valeur de Structure Before Inference.

Statut : P1 - partiellement mesuré

5.6 Memory Readiness

> **Boucle**
>
> Trace -> provenance -> statut -> branchement ou quarantine -> usage borné.

Fonction : Transforme une masse documentaire en mémoire gouvernée.

Statut : P0/P1

5.7 Education Bus

> **Boucle**
>
> Source -> filtre NO_TRAIN -> paquet -> exercice -> correction -> validation -> réflexe.

Fonction : Relie plusieurs sources éducatives sans leur donner d'autorité.

Statut : Après Freeze structurel

5.8 Automation Governance

> **Boucle**
>
> Evénement -> ticket -> contexte -> proposition -> tests -> verdict -> trace.

Fonction : Automatise les flux stabilisés sans automatiser l'autorisation.

Statut : P1/P2

5.9 Code Mutation Gate

> **Boucle**
>
> Prompt dev -> diff IA -> Reverse -> impact -> invariants -> tests -> ALLOW / HOLD / BLOCK.

Fonction : Gouverne les mutations produites par Claude Code, Codex ou Cursor.

Statut : P1

5.10 Agent Rights

> **Boucle**
>
> Mission -> droits -> sortie typée -> vote -> agrégation -> Guard -> décision.

Fonction : Permet la croissance multi-agents tout en gardant chaque nœud simple.

Statut : P1

5.11 Brody Native Language

> **Boucle**
>
> Contexte hydraté -> mode de réponse -> composition -> statuts -> boundary check -> sortie.

Fonction : Autorise un langage vivant et une invention bornée sans confusion entre nouveauté et vérité.

Statut : P1/P2

5.12 Physical Signal World Model

> **Boucle**
>
> Signaux ambiants -> features physiques agrégées -> contradiction -> preuve candidate.

Fonction : Ajoute une perception physique future sans cartographier les individus.

Statut : Branche future - hors chemin critique

6. CARTE DES ROLES ET DES AUTORITES

La cohérence globale dépend moins de la puissance de chaque organe que de la clarté des droits. Cette séparation doit rester visible dans le code, les schémas, les tests, le Terminal et les communications publiques.

Composant	Fonction	Frontière d'autorité
Utilisateur / professeur	Donne objectifs, corrections, arbitrages et enseignement.	Autorité humaine finale selon le périmètre.
Reverse / langage Uni	Traduit intention, contexte, relations, risques et besoins.	Ne décide pas.
Brody	Contextualise, explique, relie et formule ; peut proposer sous statut.	Non souverain ; pas de memory_write ni de kernel_action par défaut.
Agents de domaine	Observent, mesurent, votent et signalent dans un périmètre borné.	Ne peuvent pas autoriser seuls.
Sigma	Détecte contradictions, dérives et incohérences ; peut alerter ou geler.	Ne décide pas.
OIE	Mesure appels, coût, latence, tokens et routes alternatives.	Ne décide pas.
Obsidur / Obsidure	Propose du code, reverse les mutations et juge leur droit d'intégration.	Non souverain pour l'action réelle ; peut produire un verdict de gate selon cadre.
Lean / ProofKit	Vérifie les propriétés formelles et obligations de preuve.	Prouve mais ne décide pas.
Mémoire	Fournit continuité, traces et contexte qualifié.	Readonly et non souveraine.
Automatisation	Route, déclenche, documente et prépare les flux.	Automatiser n'est pas autoriser.
X-108 / Guard	Evalue contradictions, inconnues, risques, preuves et irréversibilité.	Seul détenteur du droit d'admission / HOLD / BLOCK / ACT dans son périmètre.

La capacité de proposer doit rester séparée du droit d'agir.

7. CE QUI EST FERME, PARTIEL OU ENCORE OUVERT

Le document doit empêcher une confusion récurrente : une architecture conceptuelle fermée ne prouve pas automatiquement que tous ses éléments sont actifs dans le runtime.

Echelle de maturité obligatoire

1	**Concept défini**
2	Artefact présent
3	Composant branché
4	Effet causal démontré
5	Testé
6	Prouvé
7	Freeze
8	**Production**

Règle de vérité technique

Un fichier présent ne signifie pas qu'un composant est branché. Un composant appelé ne signifie pas qu'il influence réellement le résultat. Une preuve isolée ne signifie pas que le runtime l'applique.

FERMEES	Boucles conceptuelles majeures
FERMEE	Séparation proposition / preuve / autorité
FERMEE	Place du Kernel et des domaines
PARTIELLEMENT DEMONTREE	Structure Before Inference
EN FINALISATION	Terminal unifié
EN COURS	Audit de branchement
OUVERTE	Synchronisation carte / runtime
OUVERTE	Attribution causale des compétences

APRES FREEZE	Education contrôlée complète
BRANCHE FUTURE	Physical Signal World Model
SPECULATIF	3-6-9 / multivers

8. PASSAGE A L'IMPLEMENTATION

La bonne stratégie est maintenant restrictive. Il faut terminer l'état actuel, intégrer les fermetures définies ici, mesurer ce qui agit réellement, puis geler la V1 structurelle avant d'ouvrir l'éducation et le corpus source.

P0 - Fermer le socle vivant

- Terminer le Terminal et l'entrée unique obsidia.
- Finir l'audit de branchement réel.
- Classer ACTIVE / READY_TO_CONNECT / DORMANT / BROKEN / DEPRECATED.
- Reconnecter les organes déjà prêts.
- Synchroniser la carte du projet avec les fichiers, fonctions, routes, tests et métriques.

> **Verrou**
>
> Ne rien ouvrir de massif avant que le Terminal et l'audit donnent une image fiable de la machination réelle.

P1 - Gouverner les flux et les mutations

- Memory Readiness et statuts CANON / PROVISIONAL / NO_TRAIN / READONLY.
- Matrice des autorités et des automatisations.
- Obsidure Code Gate et observation Claude Code / Codex.
- Sigma Guidance, OIE et Agent Rights.
- Tests d'attribution causale par retrait ou neutralisation de couches.

P2 - Prouver et geler

- Tests transversaux de bout en bout.
- Receipts, replay, anti-replay et provenance.
- Formalisation Lean des frontières prioritaires.
- Freezes par famille puis Freeze structurel V1.
- Démonstrations reproductibles et documentation d'audit.

P3 - Ouvrir l'éducation

- Education Bus multi-source contraint.
- Paquets éducatifs, exercices, corrections et transferts.
- Ingestion progressive des documents sources qualifiés.
- Mesure de l'acquisition, de la déformation et des réflexes.

P4 - Branches futures

- Physical Signal World Model.
- World Model causal-situé étendu.
- Gencoin et économie cognitive.
- Robotique mutualisée et autres expansions.

Auditer -> brancher -> statuer -> gouverner -> mesurer -> tester ->

Freeze.

9. POLITIQUE DES DOCUMENTS SOURCES

Les documents sources importants ont de la valeur, mais ils ne sont pas sur le chemin critique immédiat. Les brancher maintenant risquerait de réintroduire du bruit, de rouvrir des sujets déjà fermés et de mélanger le canon avec des matériaux non qualifiés.

> **Décision de phase**
>
> Réunir, indexer et protéger les sources sans les injecter dans la machination avant le Freeze structurel.

SOURCE_ARCHIVE	Document conservé comme matière potentielle, non actif.
READONLY	Aucune écriture ou transformation automatique dans le canon.
NOT_CONNECTED	Pas de branchement au Terminal, à Brody ou au runtime.
NO_TRAIN_FOR_NOW	Interdit à l'éducation tant que le corpus n'est pas audité.
REVIEW_AFTER_V1_FREEZE	Réouverture planifiée après stabilisation de la machination.

- **A faire maintenant :** inventaire minimal, dédoublonnage, provenance, date, thème et emplacement.
- **A ne pas faire maintenant :** ingestion massive, scraping, classification automatique non vérifiée, promotion canonique ou création de nouvelles couches à partir de chaque document.
- **Réouverture future :** éducation ciblée, vérification, recherche de contradiction ou résolution d'un manque démontré par l'audit.

On capture les nouvelles idées. On ne les branche pas immédiatement.

10. CRITERE REALISTE DE V1 STRUCTURELLE

La V1 ne signifie pas que tout est intelligent ou que toutes les ambitions sont déjà actives. Elle signifie que le système est relié, statué, traçable et prêt pour une éducation contrôlée.

- Brody contextualise et respecte ses frontières.
- Obsidure statue sur les mutations de code dans son périmètre.
- Les domaines actifs sont branchés et bridge-only lorsque requis.
- La mémoire est classée et gouvernée.
- Les automatisations sont enregistrées et non souveraines.
- Les flux sont auditables et replayables.
- Les métriques principales peuvent être reproduites.
- Claude Code / Codex peuvent être observés comme sources de mutations.
- Les learning packets sont propres et filtrés.
- Les sources éducatives restent non souveraines.
- La carte du système correspond au runtime réel.
- Les capacités principales ont une attribution causale suffisamment démontrée.

> **Ce que la V1 ne doit pas prétendre**
>
> Absence absolue d'hallucination, compréhension totale du monde, tous les organes branchés, moteur physique complet, zéro inférence pour tous les cas ou impossibilité absolue de prise de contrôle.

La V1 n'est pas finie quand Brody sait répondre. Elle est finie quand mémoire, automatisation, domaines et autorité sont reliés sans confusion de pouvoir.

11. FORMULES CANONIQUES A CONSERVER

1	**La vision est fermée. La machination doit maintenant prouver qu'elle correspond à cette vision.**
2	**Penser librement. Agir sous preuve.**
3	**On ne forme pas le Kernel. On informe le Kernel.**
4	**Quand la route est connue, prédire devient plus lent que vérifier.**
5	**Obsidia ne possède pas la vérité. Il gouverne la manière dont une affirmation devient admissible.**
6	**La donnée ne décrit pas seulement le monde. Elle participe à produire le monde suivant.**
7	**HOLD est une action de stabilisation temporelle et causale.**
8	**La stabilité se pose avant. La compréhension se construit par confrontation.**
9	**Une connaissance devient réflexe sans perdre la possibilité d'être expliquée, retracée et corrigée.**
10	**Obsidia cadre. Le moteur explore. Obsidur formalise. Lean valide. KX-108 gouverne. Le corpus absorbe.**
11	**L'agent est simple. L'orchestration est souveraine.**
12	**La capacité de proposer reste séparée du droit d'agir.**
13	**Les IA génèrent. Obsidure autorise.**
14	**Générer n'est pas intégrer. Modifier n'est pas muter sans preuve.**
15	**Apprendre n'est pas autoriser. Expliquer n'est pas valider. Contextualiser n'est pas décider.**
16	**On n'automatise pas le chaos. On automatise une structure déjà stabilisée.**

17	**Présent dans le repo ne signifie pas branché. Branché ne signifie pas causalement utile.**
18	**On ne cartographie pas les gens. On cartographie les contraintes physiques du monde.**
19	**On capture les nouvelles idées. On ne les branche pas immédiatement.**
20	**Auditer -> brancher -> statuer -> gouverner -> mesurer -> tester -> Freeze.**

12. CONCLUSION DE PHASE

Cette discussion ne crée pas une nouvelle promesse. Elle rassemble les promesses déjà dispersées, ferme leur circulation, distingue ce qui est canonique de ce qui reste futur et fixe une méthode de réalisation qui protège le projet contre sa propre richesse.

1	**Terminer le Terminal**
2	Finir l'audit de branchement
3	Reconnecter et statuer l'état actuel
4	Intégrer les boucles et fonctionnalités définies ici
5	Tester les flux et l'attribution causale
6	Geler la V1 structurelle
7	**Ouvrir ensuite les sources et l'éducation contrôlée**

Obsidia n'a plus besoin d'être réinventé. Il doit être récupéré dans ses propres recherches, unifié, branché, formalisé, testé, puis prouvé comme un ensemble.

SUPPORT POSITIONNEMENT, VENTE ET INDUSTRIALISATION - OBSIDIA / X108 - V1 COMPLETE

Sous-titre : De la gouvernance deterministe des agents IA au WorkCenter industriel.

Type de support : comprehension, vente, credibilite, pre-industrialisation, education, preparation interlocuteurs.

Public : investisseur, incubateur, banque, conformite, CTO, dev senior, Big Tech, partenaire terrain, auditeur, equipe interne.

Statut : support strategique. Ce document ne build rien, ne modifie aucun repo, ne lance aucun serveur et ne declare aucune production readiness.

Rappel de prudence : Obsidia / X108 est actuellement positionne comme recherche appliquee, POC local avance et pre-industrialisation. Aucun usage critique ne doit reposer dessus sans audit independant, durcissement, certification et validation domaine.

Sources internes utilisees

Source	Role dans ce support
README_OBSIDIA_X108_GITHUB_V2(2).md	Synthese projet, statut, architecture, runtime, domaines, preuves
README_REALITY_RECHECK_REPORT(2).md	Recheck realite 2026-06-17, runtime API, Lean, Brody, Graphiti, domaines
OBSIDIA_X108_GLOSSAIRE_INTEGRAL_AUDITE(2).md	Glossaire integral audite, statuts, definitions, autorite, invariants
CAHIER_DES_CHARGES_DEV_COCKPIT_OBSIDIA_X108_COMPLET_V1.md	Cahier des charges complet du Cockpit / WorkCenter
CHECKLIST_IMPLEMENTATION_COCKPIT_OBSIDIA_X108_V1_CORRIGEE.md	Checklist implementation V0 a V5
MATRICE_IMPLEMENTATION_COCKPIT_OBSIDIA_X108_V1_COMPLETE.md	Matrice tickets, pages, versions, routes, tests
CONTRAT_API_WORKCENTER_COCKPIT_OBSIDIA_X108_V1_COMPLETE.md	Contrat API WorkCenter
OBJECT_MODEL_DATA_REGISTRY_COCKPIT_OBSIDIA_X108_V1_COMPLETE.md	Object Model / Data Model Registry
SECURITY_AUTHORITY_BOUNDARY_MATRIX_COCKPIT_OBSIDIA_X108_V1_COMPLETE.md	Security & Authority Boundary Matrix
AUDIT_backend_minimal(1).txt	Audit backend minimal
AUDIT_connectors_minimal(1).txt	Audit connecteurs
AUDIT_sigma_files(1).txt	Audit Sigma
AUDIT_proofs_files(1).txt	Audit preuves
AUDIT_runtime_ports(1).txt	Audit ports runtime

Regle de lecture

Ce support fusionne deux approches : la presentation marche / vente et la presentation architecture / securite / preuves. Le but n est pas de noyer l interlocuteur dans la technique. Le but est de lui montrer assez d architecture pour susciter l interet, puis de garder la profondeur technique disponible si des questions arrivent.

La technique pure doit rester en deuxieme niveau. Le premier niveau doit faire comprendre le pourquoi, le probleme, la solution, le coeur Kernel, la securite, les terrains stables, les preuves, les limites et la trajectoire industrielle.

Principe narratif :

1. Montrer pourquoi le probleme existe maintenant.
2. Montrer pourquoi les agents IA ont besoin d une autorite.

3. Montrer que le Kernel X108 est le coeur de cette autorite.
4. Montrer que la securite vient de la separation des roles.
5. Montrer que Bank / Trading / GPS ne sont pas de simples idees.
6. Montrer que Brody, Graphiti, Sigma, Proof et WorkCenter forment un ecosysteme coherent.
7. Montrer que l avantage vient de la structure, pas de la masse brute.
8. Montrer les limites honnetes avant industrialisation.
9. Montrer le chemin vers un Workbench niveau Big Tech.

Carte rapide des preuves a pouvoir citer

Bloc	Statut a presenter	Phrase sobre
Kernel X108	CONFIRMED_BY_FREEZE, bridge runtime cable	Le Kernel est le verrou de decision ALLOW / HOLD / BLOCK.
API FastAPI 8000	CONFIRMED_BY_RUNTIME, 164 routes	L API est une surface bridge readonly, pas un decideur de secours.
Bank	Bridge runtime + decisions freeze	5 decisions ALLOW S1 capturees en freeze.
Trading	Bridge runtime + decisions freeze	5 decisions BLOCK S4 capturees en freeze.
GPS / Aviation	Bridge runtime + decisions freeze	5 decisions ALLOW S0 capturees en freeze.
Brody	124/124 tests PASS, readonly education pack	Brody assiste, explique, eduque, mais ne decide pas.
Graphiti	FROZEN_READONLY, 167 nodes, 477 rels, 20 episodes	La memoire aide le contexte mais ne gouverne pas.
Lean	57 theorems / lemmas, lake build PASS	Les invariants selectionnes sont formalises et verifies.
TLA+	Present, TLC non relance dans ce recheck	La spec existe mais ne doit pas etre sur-declaree.
RFC3161	Anchor present, openssl verify non execute dans ce recheck	L ancrage existe mais la verification externe reste a relancer.

Resume executif

Objectif de cette partie : Donner en quelques pages la these du projet, le statut reel, les actifs stabilises et la trajectoire.

Obsidia / X108 part d une question simple : quand une IA ou un agent peut proposer une action, qui controle le droit d agir ? La reponse n est pas un modele plus gros, ni un chatbot plus bavard. La reponse proposee est une architecture de gouvernance decisionnelle qui se place entre la recommandation et l action.

Le coeur du systeme est le Kernel X108. Il transforme une action candidate en decision gouvernee : ALLOW, HOLD ou BLOCK. Le principe de resolution est volontairement securitaire : BLOCK > HOLD > ALLOW. Cette priorite donne une logique fail-closed lisible pour des domaines critiques.

Autour du Kernel, Obsidia organise des couches : domaines terrain, agents, Sigma, API bridge, Brody, Graphiti, proof stack, audit, freeze et WorkCenter. Chaque couche a une fonction, mais l autorite reste separee. Ce qui explique ne decide pas. Ce qui memorise ne decide pas. Ce qui affiche ne decide pas.

Le projet n est pas vendu comme production-ready. Il est presente comme un POC local avance et une recherche appliquee en pre-industrialisation, avec des preuves techniques deja stabilisees et des limites clairement identifiees.

Message principal a faire passer

Obsidia doit etre raconte comme une infrastructure de gouvernance du passage a l action.

Version courte utilisable en rendez-vous

Phrase courte : Obsidia doit etre raconte comme une infrastructure de gouvernance du passage a l action.

Version orale : Obsidia met une couche d autorite entre ce qu une IA propose et ce qu un systeme a le droit de faire. Le projet ne cherche pas a parler plus fort qu un modele, il cherche a rendre l action controlable, tracable et verifiable.

Ce qu il faut montrer sans noyer dans la technique

- le role de la couche dans l architecture globale
- la valeur concrete pour l interlocuteur
- le statut reel : runtime, test, freeze, code, fichier, prior audit ou documented-only
- la limite honnete a ne pas masquer
- la question que cette partie doit susciter chez l interlocuteur

Preuve ou element interne mobilisable

Element	Statut	Utilisation
API 8000	CONFIRMED_BY_RUNTIME	surface bridge actuelle
Kernel 3001	CONFIRMED_BY_FREEZE	coeur de decision
WorkCenter docs	pre-build	base de construction industrielle

Limite a dire honnetement

Ne pas transformer un statut partiel en preuve totale. Ne pas dire production-ready. Ne pas dire certifie. Ne pas dire que les couches peripheriques decident. Ne pas dire que la presence d un fichier equivaut a un runtime actif. La credibilite d Obsidia vient aussi de cette discipline de statut.

Question a susciter

Question cible : si vos agents ou vos workflows pouvaient agir dans le reel, quelle couche separe aujourd hui la recommandation, l autorisation, la preuve et le replay ?

Notes de presentation

- garder les details code pour les questions techniques
- montrer le schema d architecture avant les fichiers
- faire ressortir le Kernel et la securite dans chaque explication
- lier chaque prouesse a une limite honnete
- conclure chaque partie par la valeur metier

Pourquoi maintenant : les agents IA passent a l action

Objectif de cette partie : Faire comprendre que le risque se deplace de la reponse vers l action.

Le moment est important parce que les IA ne restent plus seulement dans une zone de texte. Elles peuvent appeler des outils, declencher des workflows, modifier des fichiers, envoyer des requetes, interroger des bases, piloter des agents et preparer des actions externes.

Tant que l IA ne faisait que repondre, le risque principal etait une mauvaise reponse. Quand elle peut agir, le risque devient une mauvaise transformation du reel. Ce changement impose une couche d autorite distincte de la couche de raisonnement.

Le marche va donc se deplacer : il ne suffira plus de demander quel modele est le plus puissant. Il faudra demander quelle architecture controle le passage entre signal, intention, decision et action.

Message principal a faire passer

Obsidia doit etre raconte comme une infrastructure de gouvernance du passage a l action.

Version courte utilisable en rendez-vous

Phrase courte : Obsidia doit etre raconte comme une infrastructure de gouvernance du passage a l action.

Version orale : Obsidia met une couche d autorite entre ce qu une IA propose et ce qu un systeme a le droit de faire. Le projet ne cherche pas a parler plus fort qu un modele, il cherche a rendre l action controlable, tracable et verifiable.

Ce qu il faut montrer sans noyer dans la technique

- le role de la couche dans l architecture globale
- la valeur concrete pour l interlocuteur
- le statut reel : runtime, test, freeze, code, fichier, prior audit ou documented-only
- la limite honnete a ne pas masquer
- la question que cette partie doit susciter chez l interlocuteur

Preuve ou element interne mobilisable

Element	Statut	Utilisation
API 8000	CONFIRMED_BY_RUNTIME	surface bridge actuelle
Kernel 3001	CONFIRMED_BY_FREEZE	coeur de decision
WorkCenter docs	pre-build	base de construction industrielle

Limite a dire honnetement

Ne pas transformer un statut partiel en preuve totale. Ne pas dire production-ready. Ne pas dire certifie. Ne pas dire que les couches peripheriques decident. Ne pas dire que la presence d un fichier equivaut a un runtime actif. La credibilite d Obsidia vient aussi de cette discipline de statut.

Question a susciter

Question cible : si vos agents ou vos workflows pouvaient agir dans le reel, quelle couche separe aujourd hui la recommandation, l autorisation, la preuve et le replay ?

Notes de presentation

- garder les details code pour les questions techniques
- montrer le schema d architecture avant les fichiers
- faire ressortir le Kernel et la securite dans chaque explication
- lier chaque prouesse a une limite honnete
- conclure chaque partie par la valeur metier

Le probleme marche

Objectif de cette partie : Mettre en mots le manque d autorite claire entre recommandation et action.

Les organisations ajoutent des agents, de l automatisation, des dashboards et des assistants, mais la chaine d autorite reste souvent floue. Une recommandation devient parfois une execution sans verrou lisible. Une memoire peut influencer une action sans statut clair. Un agent peut appeler un outil sans preuve suffisante.

Le probleme n est pas seulement technique. Il est operationnel, juridique, humain, financier et organisationnel. Qui est responsable si un agent declenche une action ? Qui peut bloquer ? Qui peut suspendre ? Qui peut prouver pourquoi une action a ete autorisee ?

Obsidia adresse ce vide : il ne cherche pas seulement a generer une meilleure sortie, mais a gouverner le droit de transformer une situation.

Message principal a faire passer

Obsidia doit etre raconte comme une infrastructure de gouvernance du passage a l action.

Version courte utilisable en rendez-vous

Phrase courte : Obsidia doit etre raconte comme une infrastructure de gouvernance du passage a l action.

Version orale : Obsidia met une couche d autorite entre ce qu une IA propose et ce qu un systeme a le droit de faire. Le projet ne cherche pas a parler plus fort qu un modele, il cherche a rendre l action controlable, tracable et verifiable.

Ce qu il faut montrer sans noyer dans la technique

- le role de la couche dans l architecture globale
- la valeur concrete pour l interlocuteur
- le statut reel : runtime, test, freeze, code, fichier, prior audit ou documented-only
- la limite honnete a ne pas masquer
- la question que cette partie doit susciter chez l interlocuteur

Preuve ou element interne mobilisable

Element	Statut	Utilisation
API 8000	CONFIRMED_BY_RUNTIME	surface bridge actuelle
Kernel 3001	CONFIRMED_BY_FREEZE	coeur de decision
WorkCenter docs	pre-build	base de construction industrielle

Limite a dire honnetement

Ne pas transformer un statut partiel en preuve totale. Ne pas dire production-ready. Ne pas dire certifie. Ne pas dire que les couches peripheriques decident. Ne pas dire que la presence d un fichier equivaut a un runtime actif. La credibilite d Obsidia vient aussi de cette discipline de statut.

Question a susciter

Question cible : si vos agents ou vos workflows pouvaient agir dans le reel, quelle couche separe aujourd hui la recommandation, l autorisation, la preuve et le replay ?

Notes de presentation

- garder les details code pour les questions techniques
- montrer le schema d architecture avant les fichiers
- faire ressortir le Kernel et la securite dans chaque explication
- lier chaque prouesse a une limite honnete
- conclure chaque partie par la valeur metier

Le changement de paradigme IA

Objectif de cette partie : Passer du modele generatif a l infrastructure de gouvernance.

Le paradigme dominant augmente les capacites : plus de parametres, plus de donnees, plus de contexte, plus d outils. Obsidia suit une autre pente : plus de structure, plus d invariants, plus de traces, plus de verification, plus de separation d autorite.

Ce n est pas une opposition aux LLM. Les LLM restent utiles pour expliquer, assister, explorer et generer. Mais une decision critique ne doit pas dependre uniquement d une sortie probabiliste.

Le changement de paradigme est donc le passage du modele generatif au systeme gouverne. Dans un monde d agents capables, l avantage ne vient plus seulement de savoir produire. Il vient aussi de savoir refuser, suspendre, prouver et rejouer.

Message principal a faire passer

Obsidia doit etre raconte comme une infrastructure de gouvernance du passage a l action.

Version courte utilisable en rendez-vous

Phrase courte : Obsidia doit etre raconte comme une infrastructure de gouvernance du passage a l action.

Version orale : Obsidia met une couche d autorite entre ce qu une IA propose et ce qu un systeme a le droit de faire. Le projet ne cherche pas a parler plus fort qu un modele, il cherche a rendre l action controlable, tracable et verifiable.

Ce qu il faut montrer sans noyer dans la technique

- le role de la couche dans l architecture globale
- la valeur concrete pour l interlocuteur
- le statut reel : runtime, test, freeze, code, fichier, prior audit ou documented-only
- la limite honnete a ne pas masquer
- la question que cette partie doit susciter chez l interlocuteur

Preuve ou element interne mobilisable

Element	Statut	Utilisation
API 8000	CONFIRMED_BY_RUNTIME	surface bridge actuelle
Kernel 3001	CONFIRMED_BY_FREEZE	coeur de decision
WorkCenter docs	pre-build	base de construction industrielle

Limite a dire honnetement

Ne pas transformer un statut partiel en preuve totale. Ne pas dire production-ready. Ne pas dire certifie. Ne pas dire que les couches peripheriques decident. Ne pas dire que la presence d un fichier equivaut a un runtime actif. La credibilite d Obsidia vient aussi de cette discipline de statut.

Question a susciter

Question cible : si vos agents ou vos workflows pouvaient agir dans le reel, quelle couche separe aujourd hui la recommandation, l autorisation, la preuve et le replay ?

Notes de presentation

- garder les details code pour les questions techniques
- montrer le schema d architecture avant les fichiers
- faire ressortir le Kernel et la securite dans chaque explication
- lier chaque prouesse a une limite honnete
- conclure chaque partie par la valeur metier

Pourquoi les agents ont besoin d une autorite

Objectif de cette partie : Montrer que tool-use, memoire, workflows et automatisation exigent une autorite separee.

Un agent peut etre tres utile et rester dangereux si son autorite est mal definie. Il peut accumuler des signaux, produire des interpretations et proposer des actions, mais il ne doit pas necessairement etre celui qui autorise.

La couche d autorite sert a separer la competence de la permission. Un agent peut savoir quoi faire sans avoir le droit de le faire. Obsidia formalise cette separation.

Cette separation est centrale pour les domaines a impact : banque, trading, GPS, aviation, infrastructure, e-commerce, conformite, defense, recherche, workflows critiques.

Message principal a faire passer

La securite vient de la separation des roles, des boundaries explicites et de la non-substitution du Kernel par les couches peripheriques.

Version courte utilisable en rendez-vous

Phrase courte : La securite vient de la separation des roles, des boundaries explicites et de la non-substitution du Kernel par les couches peripheriques.

Version orale : Obsidia met une couche d autorite entre ce qu une IA propose et ce qu un systeme a le droit de faire. Le projet ne cherche pas a parler plus fort qu un modele, il cherche a rendre l action controlable, tracable et verifiable.

Ce qu il faut montrer sans noyer dans la technique

- le role de la couche dans l architecture globale
- la valeur concrete pour l interlocuteur
- le statut reel : runtime, test, freeze, code, fichier, prior audit ou documented-only
- la limite honnete a ne pas masquer
- la question que cette partie doit susciter chez l interlocuteur

Preuve ou element interne mobilisable

Element	Statut	Utilisation
API 8000	CONFIRMED_BY_RUNTIME	surface bridge actuelle
Kernel 3001	CONFIRMED_BY_FREEZE	coeur de decision
WorkCenter docs	pre-build	base de construction industrielle

Limite a dire honnetement

Ne pas transformer un statut partiel en preuve totale. Ne pas dire production-ready. Ne pas dire certifie. Ne pas dire que les couches peripheriques decident. Ne pas dire que la presence d un fichier equivaut a un runtime actif. La credibilite d Obsidia vient aussi de cette discipline de statut.

Question a susciter

Question cible : si vos agents ou vos workflows pouvaient agir dans le reel, quelle couche separe aujourd hui la recommandation, l autorisation, la preuve et le replay ?

Notes de presentation

- garder les details code pour les questions techniques
- montrer le schema d architecture avant les fichiers
- faire ressortir le Kernel et la securite dans chaque explication
- lier chaque prouesse a une limite honnete
- conclure chaque partie par la valeur metier

Ce qu est Obsidia / X108

Objectif de cette partie : Definir Obsidia comme architecture de gouvernance decisionnelle.

Obsidia / X108 est une architecture de gouvernance decisionnelle pour systemes IA, agents autonomes et workflows critiques. Elle organise le passage entre signal, intention, decision, action candidate, preuve et replay.

Le Kernel X108 est le coeur de la decision. Les couches peripheriques preparent le contexte, agregent les signaux, exposent les interfaces, expliquent le systeme ou memorisent les connaissances, mais ne remplacent pas le Kernel.

Obsidia est donc une infrastructure de controle. Son role n est pas de produire plus de texte, mais de rendre gouvernable le passage a l action.

Message principal a faire passer

Obsidia doit etre raconte comme une infrastructure de gouvernance du passage a l action.

Version courte utilisable en rendez-vous

Phrase courte : Obsidia doit etre raconte comme une infrastructure de gouvernance du passage a l action.

Version orale : Obsidia met une couche d autorite entre ce qu une IA propose et ce qu un systeme a le droit de faire. Le projet ne cherche pas a parler plus fort qu un modele, il cherche a rendre l action controlable, tracable et verifiable.

Ce qu il faut montrer sans noyer dans la technique

- le role de la couche dans l architecture globale
- la valeur concrete pour l interlocuteur

- le statut reel : runtime, test, freeze, code, fichier, prior audit ou documented-only
- la limite honnete a ne pas masquer
- la question que cette partie doit susciter chez l interlocuteur

Preuve ou element interne mobilisable

Element	Statut	Utilisation
API 8000	CONFIRMED_BY_RUNTIME	surface bridge actuelle
Kernel 3001	CONFIRMED_BY_FREEZE	coeur de decision
WorkCenter docs	pre-build	base de construction industrielle

Limite a dire honnetement

Ne pas transformer un statut partiel en preuve totale. Ne pas dire production-ready. Ne pas dire certifie. Ne pas dire que les couches peripheriques decident. Ne pas dire que la presence d un fichier equivaut a un runtime actif. La credibilite d Obsidia vient aussi de cette discipline de statut.

Question a susciter

Question cible : si vos agents ou vos workflows pouvaient agir dans le reel, quelle couche separe aujourd hui la recommandation, l autorisation, la preuve et le replay ?

Notes de presentation

- garder les details code pour les questions techniques
- montrer le schema d architecture avant les fichiers
- faire ressortir le Kernel et la securite dans chaque explication
- lier chaque prouesse a une limite honnete
- conclure chaque partie par la valeur metier

Ce qu Obsidia n est pas

Objectif de cette partie : Eviter les confusions : chatbot, wrapper, simple UI, modele plus gros, outil de monitoring seul.

Obsidia n est pas un chatbot. Un chatbot parle. Obsidia gouverne une chaine de decision.

Obsidia n est pas un wrapper autour d un LLM. Un wrapper transmet des prompts et des reponses. Obsidia separe les roles, les statuts, les preuves et les autorites.

Obsidia n est pas un simple tableau de bord. Un dashboard affiche. Le WorkCenter doit afficher, auditer, relier, valider, preparer et proteger, sans devenir souverain.

Obsidia n est pas une promesse de production critique immediate. Le support doit rester honnete : POC local avance, recherche appliquee, pre-industrialisation.

Message principal a faire passer

Obsidia doit etre raconte comme une infrastructure de gouvernance du passage a l action.

Version courte utilisable en rendez-vous

Phrase courte : Obsidia doit etre raconte comme une infrastructure de gouvernance du passage a l action.

Version orale : Obsidia met une couche d autorite entre ce qu une IA propose et ce qu un systeme a le droit de faire. Le projet ne cherche pas a parler plus fort qu un modele, il cherche a rendre l action controlable, tracable et verifiable.

Ce qu il faut montrer sans noyer dans la technique

- le role de la couche dans l architecture globale
- la valeur concrete pour l interlocuteur
- le statut reel : runtime, test, freeze, code, fichier, prior audit ou documented-only
- la limite honnete a ne pas masquer

- la question que cette partie doit susciter chez l interlocuteur

Preuve ou element interne mobilisable

Element	Statut	Utilisation
API 8000	CONFIRMED_BY_RUNTIME	surface bridge actuelle
Kernel 3001	CONFIRMED_BY_FREEZE	coeur de decision
WorkCenter docs	pre-build	base de construction industrielle

Limite a dire honnetement

Ne pas transformer un statut partiel en preuve totale. Ne pas dire production-ready. Ne pas dire certifie. Ne pas dire que les couches peripheriques decident. Ne pas dire que la presence d un fichier equivaut a un runtime actif. La credibilite d Obsidia vient aussi de cette discipline de statut.

Question a susciter

Question cible : si vos agents ou vos workflows pouvaient agir dans le reel, quelle couche separe aujourd hui la recommandation, l autorisation, la preuve et le replay ?

Notes de presentation

- garder les details code pour les questions techniques
- montrer le schema d architecture avant les fichiers
- faire ressortir le Kernel et la securite dans chaque explication
- lier chaque prouesse a une limite honnete
- conclure chaque partie par la valeur metier

Architecture en une page

Objectif de cette partie : Presenter la chaine Signal -> Domaines -> Sigma -> Kernel -> Proof -> Replay -> WorkCenter.

L architecture doit etre lue comme une chaine de gouvernance. Le signal arrive depuis un domaine. Les agents produisent des votes ou observations. Sigma structure les signaux. Le Kernel tranche. L API expose sans remplacer. Les preuves, freezes et replays rendent la decision auditables.

Le WorkCenter vient au-dessus pour rendre le systeme comprehensible et pilotable. Il ne remplace pas le Kernel. Il aide l humain, les devs et les auditeurs a voir l etat reel du projet.

Message principal a faire passer

Obsidia doit etre raconte comme une infrastructure de gouvernance du passage a l action.

Version courte utilisable en rendez-vous

Phrase courte : Obsidia doit etre raconte comme une infrastructure de gouvernance du passage a l action.

Version orale : Obsidia met une couche d autorite entre ce qu une IA propose et ce qu un systeme a le droit de faire. Le projet ne cherche pas a parler plus fort qu un modele, il cherche a rendre l action controlable, tracable et verifiable.

Ce qu il faut montrer sans noyer dans la technique

- le role de la couche dans l architecture globale
- la valeur concrete pour l interlocuteur
- le statut reel : runtime, test, freeze, code, fichier, prior audit ou documented-only
- la limite honnete a ne pas masquer
- la question que cette partie doit susciter chez l interlocuteur

Preuve ou element interne mobilisable

Element	Statut	Utilisation
API 8000	CONFIRMED_BY_RUNTIME	surface bridge actuelle

Kernel 3001	CONFIRMED_BY_FREEZE	coeur de decision
WorkCenter docs	pre-build	base de construction industrielle

Limite a dire honnetement

Ne pas transformer un statut partiel en preuve totale. Ne pas dire production-ready. Ne pas dire certifie. Ne pas dire que les couches peripheriques decident. Ne pas dire que la presence d un fichier equivaut a un runtime actif. La credibilite d Obsidia vient aussi de cette discipline de statut.

Question a susciter

Question cible : si vos agents ou vos workflows pouvaient agir dans le reel, quelle couche separe aujourd hui la recommandation, l autorisation, la preuve et le replay ?

Notes de presentation

- garder les details code pour les questions techniques
- montrer le schema d architecture avant les fichiers
- faire ressortir le Kernel et la securite dans chaque explication
- lier chaque prouesse a une limite honnete
- conclure chaque partie par la valeur metier

Le Kernel X108

Objectif de cette partie : Montrer le coeur de securite et d autorite.

Le Kernel est le coeur du support. S il n est pas visible, le projet semble devenir un ensemble de dashboards et d assistants. Il faut au contraire le montrer comme verrou souverain.

Le Kernel recoit une action candidate et retourne ALLOW, HOLD ou BLOCK. Il n est pas un simple module de scoring. Il porte l autorite finale dans l architecture X108.

Le statut a presenter est precis : Kernel 3001 confirme par freeze, endpoint POST /kernel/ragnarok, decisions capturees dans des freezes, bridge API cable. Au moment du recheck, le port etait ferme, donc il ne faut pas pretendre runtime live permanent.

Message principal a faire passer

Le Kernel X108 est le verrou. Il doit etre visible dans le support comme autorite finale et non comme composant secondaire.

Version courte utilisable en rendez-vous

Phrase courte : Le Kernel X108 est le verrou. Il doit etre visible dans le support comme autorite finale et non comme composant secondaire.

Version orale : Obsidia met une couche d autorite entre ce qu une IA propose et ce qu un systeme a le droit de faire. Le projet ne cherche pas a parler plus fort qu un modele, il cherche a rendre l action controlable, tracable et verifiable.

Ce qu il faut montrer sans noyer dans la technique

- le role de la couche dans l architecture globale
- la valeur concrete pour l interlocuteur
- le statut reel : runtime, test, freeze, code, fichier, prior audit ou documented-only
- la limite honnete a ne pas masquer
- la question que cette partie doit susciter chez l interlocuteur

Preuve ou element interne mobilisable

Element	Statut	Utilisation dans le discours
server.kernel.sealed.cjs	CONFIRMED_BY_FILE / CODE selon source	montrer qu il existe un coeur separe

POST /kernel/ragnarok	CONFIRMED_BY_FREEZE	montrer l endpoint decisionnel
15 decisions freeze	CONFIRMED_BY_FREEZE	montrer que des decisions ont ete capturees

Limite a dire honnetement

Ne pas transformer un statut partiel en preuve totale. Ne pas dire production-ready. Ne pas dire certifie. Ne pas dire que les couches peripheriques decident. Ne pas dire que la presence d un fichier equivaut a un runtime actif. La credibilite d Obsidia vient aussi de cette discipline de statut.

Question a susciter

Question cible : si vos agents ou vos workflows pouvaient agir dans le reel, quelle couche separe aujourd hui la recommandation, l autorisation, la preuve et le replay ?

Notes de presentation

- garder les details code pour les questions techniques
- montrer le schema d architecture avant les fichiers
- faire ressortir le Kernel et la securite dans chaque explication
- lier chaque prouesse a une limite honnete
- conclure chaque partie par la valeur metier

La regle BLOCK > HOLD > ALLOW

Objectif de cette partie : Expliquer la priorite fail-closed sans surcharge technique.

La regle explique la philosophie de securite. BLOCK l emporte sur HOLD, HOLD l emporte sur ALLOW. Cette priorite empeche une autorisation facile quand des signaux critiques existent.

Pour un public non-tech, il faut dire : en cas de doute critique, Obsidia ne force pas l action. Il bloque ou suspend. Pour un public technique, il faut rattacher cette regle aux invariants, aux votes et aux preuves Lean.

Message principal a faire passer

Obsidia doit etre raconte comme une infrastructure de gouvernance du passage a l action.

Version courte utilisable en rendez-vous

Phrase courte : Obsidia doit etre raconte comme une infrastructure de gouvernance du passage a l action.

Version orale : Obsidia met une couche d autorite entre ce qu une IA propose et ce qu un systeme a le droit de faire. Le projet ne cherche pas a parler plus fort qu un modele, il cherche a rendre l action controlable, tracable et verifiable.

Ce qu il faut montrer sans noyer dans la technique

- le role de la couche dans l architecture globale
- la valeur concrete pour l interlocuteur
- le statut reel : runtime, test, freeze, code, fichier, prior audit ou documented-only
- la limite honnete a ne pas masquer
- la question que cette partie doit susciter chez l interlocuteur

Preuve ou element interne mobilisable

Element	Statut	Utilisation dans le discours
server.kernel.sealed.cjs	CONFIRMED_BY_FILE / CODE selon source	montrer qu il existe un coeur separe
POST /kernel/ragnarok	CONFIRMED_BY_FREEZE	montrer l endpoint decisionnel
15 decisions freeze	CONFIRMED_BY_FREEZE	montrer que des decisions ont ete capturees

Limite a dire honnetement

Ne pas transformer un statut partiel en preuve totale. Ne pas dire production-ready. Ne pas dire certifie. Ne pas dire que les couches peripheriques decident. Ne pas dire que la presence d un fichier equivaut a un runtime actif. La credibilite d Obsidia vient aussi de cette discipline de statut.

Question a susciter

Question cible : si vos agents ou vos workflows pouvaient agir dans le reel, quelle couche separe aujourd hui la recommandation, l autorisation, la preuve et le replay ?

Notes de presentation

- garder les details code pour les questions techniques
- montrer le schema d architecture avant les fichiers
- faire ressortir le Kernel et la securite dans chaque explication
- lier chaque prouesse a une limite honnete
- conclure chaque partie par la valeur metier

Separation d autorite

Objectif de cette partie : Rendre visible que chaque couche a un role et une limite.

La securite d Obsidia ne vient pas seulement d une regle. Elle vient d une distribution stricte des roles. Chaque couche peut etre puissante dans son domaine, mais limitee dans son autorite.

Brody explique. Graphiti memorise. Sigma agrege. L API bridge. Les domaines signalent. Le WorkCenter affiche et prepare. Le Kernel gouverne. Les preuves verifient. Cette separation est la colonne vertebrale de la credibilite.

Message principal a faire passer

La securite vient de la separation des roles, des boundaries explicites et de la non-substitution du Kernel par les couches peripheriques.

Version courte utilisable en rendez-vous

Phrase courte : La securite vient de la separation des roles, des boundaries explicites et de la non-substitution du Kernel par les couches peripheriques.

Version orale : Obsidia met une couche d autorite entre ce qu une IA propose et ce qu un systeme a le droit de faire. Le projet ne cherche pas a parler plus fort qu un modele, il cherche a rendre l action controlable, tracable et verifiable.

Ce qu il faut montrer sans noyer dans la technique

- le role de la couche dans l architecture globale
- la valeur concrete pour l interlocuteur
- le statut reel : runtime, test, freeze, code, fichier, prior audit ou documented-only
- la limite honnete a ne pas masquer
- la question que cette partie doit susciter chez l interlocuteur

Preuve ou element interne mobilisable

Element	Statut	Utilisation
API 8000	CONFIRMED_BY_RUNTIME	surface bridge actuelle
Kernel 3001	CONFIRMED_BY_FREEZE	coeur de decision
WorkCenter docs	pre-build	base de construction industrielle

Limite a dire honnetement

Ne pas transformer un statut partiel en preuve totale. Ne pas dire production-ready. Ne pas dire certifie. Ne pas dire que les couches peripheriques decident. Ne pas dire que la presence d un fichier equivaut a un runtime actif. La credibilite d Obsidia vient aussi de cette discipline de statut.

Question a susciter

Question cible : si vos agents ou vos workflows pouvaient agir dans le reel, quelle couche separe aujourd hui la recommandation, l autorisation, la preuve et le replay ?

Notes de presentation

- garder les details code pour les questions techniques
- montrer le schema d architecture avant les fichiers
- faire ressortir le Kernel et la securite dans chaque explication
- lier chaque prouesse a une limite honnete
- conclure chaque partie par la valeur metier

Security by design

Objectif de cette partie : Montrer les boundaries, les interdits et les garanties de non-contournement.

Le support doit montrer les flags de boundary : allowed_to_decide=false, emits_act=false, emits_verdict=false, memory_write=false, graphiti_write=false, kernel_mutation=false, real_action=false.

Ces flags ne sont pas seulement des details de code. Ils traduisent une doctrine : aucun composant peripherique ne doit s octroyer une autorite qui appartient au Kernel ou a une validation humaine.

Message principal a faire passer

La securite vient de la separation des roles, des boundaries explicites et de la non-substitution du Kernel par les couches peripheriques.

Version courte utilisable en rendez-vous

Phrase courte : La securite vient de la separation des roles, des boundaries explicites et de la non-substitution du Kernel par les couches peripheriques.

Version orale : Obsidia met une couche d autorite entre ce qu une IA propose et ce qu un systeme a le droit de faire. Le projet ne cherche pas a parler plus fort qu un modele, il cherche a rendre l action controlable, tracable et verifiable.

Ce qu il faut montrer sans noyer dans la technique

- le role de la couche dans l architecture globale
- la valeur concrete pour l interlocuteur
- le statut reel : runtime, test, freeze, code, fichier, prior audit ou documented-only
- la limite honnete a ne pas masquer
- la question que cette partie doit susciter chez l interlocuteur

Preuve ou element interne mobilisable

Element	Statut	Utilisation
API 8000	CONFIRMED_BY_RUNTIME	surface bridge actuelle
Kernel 3001	CONFIRMED_BY_FREEZE	coeur de decision
WorkCenter docs	pre-build	base de construction industrielle

Limite a dire honnetement

Ne pas transformer un statut partiel en preuve totale. Ne pas dire production-ready. Ne pas dire certifie. Ne pas dire que les couches peripheriques decident. Ne pas dire que la presence d un fichier equivaut a un runtime actif. La credibilite d Obsidia vient aussi de cette discipline de statut.

Question a susciter

Question cible : si vos agents ou vos workflows pouvaient agir dans le reel, quelle couche separe aujourd hui la recommandation, l autorisation, la preuve et le replay ?

Notes de presentation

- garder les details code pour les questions techniques
- montrer le schema d architecture avant les fichiers
- faire ressortir le Kernel et la securite dans chaque explication
- lier chaque prouesse a une limite honnete
- conclure chaque partie par la valeur metier

Les terrains stables : Bank, Trading, GPS/Aviation

Objectif de cette partie : Montrer les trois branchements terrain deja validates par freeze et runtime API.

Les trois terrains deja branches sont essentiels pour susciter l interet. Bank, Trading et GPS/Aviation montrent que l architecture n est pas seulement conceptuelle.

Bank a capture 5 decisions ALLOW S1 en freeze. Trading a capture 5 decisions BLOCK S4 avec contradiction threshold. GPS/Aviation a capture 5 decisions ALLOW S0. Le point commercial n est pas le resultat exact, mais la preuve que la meme architecture de gouvernance peut s appliquer a plusieurs mondes.

La lecture business est simple : Obsidia peut se brancher sur des domaines differents sans changer la doctrine d autorite.

Message principal a faire passer

Les terrains Bank, Trading et GPS/Aviation montrent que la doctrine peut s appliquer a plusieurs mondes concrets.

Version courte utilisable en rendez-vous

Phrase courte : Les terrains Bank, Trading et GPS/Aviation montrent que la doctrine peut s appliquer a plusieurs mondes concrets.

Version orale : Obsidia met une couche d autorite entre ce qu une IA propose et ce qu un systeme a le droit de faire. Le projet ne cherche pas a parler plus fort qu un modele, il cherche a rendre l action controlable, tracable et verifiable.

Ce qu il faut montrer sans noyer dans la technique

- le role de la couche dans l architecture globale
- la valeur concrete pour l interlocuteur
- le statut reel : runtime, test, freeze, code, fichier, prior audit ou documented-only
- la limite honnete a ne pas masquer
- la question que cette partie doit susciter chez l interlocuteur

Preuve ou element interne mobilisable

Domaine	Decision freeze	Angle de vente
Bank	5 ALLOW S1	controle pre-action sur transaction ou flux bancaire
Trading	5 BLOCK S4	blocage face aux contradictions et au risque
GPS / Aviation	5 ALLOW S0	gouvernance sur signal critique terrain

Limite a dire honnetement

Ne pas transformer un statut partiel en preuve totale. Ne pas dire production-ready. Ne pas dire certifie. Ne pas dire que les couches peripheriques decident. Ne pas dire que la presence d un fichier equivaut a un runtime actif. La credibilite d Obsidia vient aussi de cette discipline de statut.

Question a susciter

Question cible : si vos agents ou vos workflows pouvaient agir dans le reel, quelle couche separe aujourd hui la recommandation, l autorisation, la preuve et le replay ?

Notes de presentation

- garder les details code pour les questions techniques
- montrer le schema d architecture avant les fichiers
- faire ressortir le Kernel et la securite dans chaque explication
- lier chaque prouesse a une limite honnete
- conclure chaque partie par la valeur metier

Brody : pseudo-LLM educatif et assistant non souverain

Objectif de cette partie : Expliquer la valeur de Brody sans lui donner d autorite.

Brody doit etre presente comme une couche d intelligence assistante encadree. Il rend Obsidia comprehensible. Il peut lire des audits, expliquer des preuves, former un utilisateur, proposer une memoire candidate ou aider a preparer un patch.

Mais Brody ne decide pas. Il ne valide pas le canon seul. Il n ecrit pas Graphiti directement. Il ne mute pas le Kernel. Il ne produit pas d action reelle.

Le terme pseudo-LLM peut etre compris comme une interface cognitive specialisee, educative et gouvernee autour du projet. Il ne faut pas le vendre comme un nouveau modele de fondation.

Message principal a faire passer

Brody augmente la comprehension et l education, mais reste non souverain.

Version courte utilisable en rendez-vous

Phrase courte : Brody augmente la comprehension et l education, mais reste non souverain.

Version orale : Obsidia met une couche d autorite entre ce qu une IA propose et ce qu un systeme a le droit de faire. Le projet ne cherche pas a parler plus fort qu un modele, il cherche a rendre l action controlable, tracable et verifiable.

Ce qu il faut montrer sans noyer dans la technique

- le role de la couche dans l architecture globale
- la valeur concrete pour l interlocuteur
- le statut reel : runtime, test, freeze, code, fichier, prior audit ou documented-only
- la limite honnete a ne pas masquer
- la question que cette partie doit susciter chez l interlocuteur

Preuve ou element interne mobilisable

Bloc	Statut	Angle
Education Pack V1	124/124 tests PASS	formation et comprehension
Two-Level Guard	CONFIRMED_BY_TEST / FREEZE	protection contre injection d action
memory_write=false	CONFIRMED_BY_TEST	pas de promotion sauvage

Limite a dire honnetement

Ne pas transformer un statut partiel en preuve totale. Ne pas dire production-ready. Ne pas dire certifie. Ne pas dire que les couches peripheriques decident. Ne pas dire que la presence d un fichier equivaut a un runtime actif. La credibilite d Obsidia vient aussi de cette discipline de statut.

Question a susciter

Question cible : si vos agents ou vos workflows pouvaient agir dans le reel, quelle couche separe aujourd hui la recommandation, l autorisation, la preuve et le replay ?

Notes de presentation

- garder les details code pour les questions techniques

- montrer le schema d architecture avant les fichiers
- faire ressortir le Kernel et la securite dans chaque explication
- lier chaque prouesse a une limite honnete
- conclure chaque partie par la valeur metier

Graphiti : memoire readonly non souveraine

Objectif de cette partie : Expliquer la memoire comme sidecar de contexte, pas comme decideur.

Graphiti apporte la memoire, les relations et le contexte. Mais la memoire ne doit jamais devenir l autorite. Une memoire peut aider a comprendre une situation, pas autoriser une action critique.

Le snapshot frozen V20 montre 167 nodes, 477 relations et 20 episodes. Le point important est le mode FROZEN_READONLY : la memoire est exploitable sans devenir decisionnelle.

Message principal a faire passer

La memoire augmente le contexte, mais elle ne doit jamais devenir une autorite d action.

Version courte utilisable en rendez-vous

Phrase courte : La memoire augmente le contexte, mais elle ne doit jamais devenir une autorite d action.

Version orale : Obsidia met une couche d autorite entre ce qu une IA propose et ce qu un systeme a le droit de faire. Le projet ne cherche pas a parler plus fort qu un modele, il cherche a rendre l action controlable, tracable et verifiable.

Ce qu il faut montrer sans noyer dans la technique

- le role de la couche dans l architecture globale
- la valeur concrete pour l interlocuteur
- le statut reel : runtime, test, freeze, code, fichier, prior audit ou documented-only
- la limite honnete a ne pas masquer
- la question que cette partie doit susciter chez l interlocuteur

Preuve ou element interne mobilisable

Element	Statut	Angle
Snapshot V20	167 nodes / 477 relations / 20 episodes	memoire exploitable
FROZEN_READONLY	CONFIRMED_BY_FREEZE	memoire non souveraine
live Neo4j dependency false	CONFIRMED_BY_FREEZE	lecture frozen sans dependance live

Limite a dire honnetement

Ne pas transformer un statut partiel en preuve totale. Ne pas dire production-ready. Ne pas dire certifie. Ne pas dire que les couches peripheriques decident. Ne pas dire que la presence d un fichier equivaut a un runtime actif. La credibilite d Obsidia vient aussi de cette discipline de statut.

Question a susciter

Question cible : si vos agents ou vos workflows pouvaient agir dans le reel, quelle couche separe aujourd hui la recommandation, l autorisation, la preuve et le replay ?

Notes de presentation

- garder les details code pour les questions techniques
- montrer le schema d architecture avant les fichiers
- faire ressortir le Kernel et la securite dans chaque explication
- lier chaque prouesse a une limite honnete
- conclure chaque partie par la valeur metier

Sigma : aggregation et signaux

Objectif de cette partie : Expliquer la couche qui structure les signaux et votes avant le Kernel.

Sigma structure les signaux. Les domaines produisent des informations. Les agents emettent des votes ou claims. Sigma agrege, detecte les contradictions et prepare une entree lisible pour le Kernel.

Sigma ne doit pas etre vendu comme le decideur final. Sa valeur est de rendre les signaux exploitables et comparables avant la decision souveraine.

Message principal a faire passer

Obsidia doit etre raconte comme une infrastructure de gouvernance du passage a l action.

Version courte utilisable en rendez-vous

Phrase courte : Obsidia doit etre raconte comme une infrastructure de gouvernance du passage a l action.

Version orale : Obsidia met une couche d autorite entre ce qu une IA propose et ce qu un systeme a le droit de faire. Le projet ne cherche pas a parler plus fort qu un modele, il cherche a rendre l action controlable, tracable et verifiable.

Ce qu il faut montrer sans noyer dans la technique

- le role de la couche dans l architecture globale
- la valeur concrete pour l interlocuteur
- le statut reel : runtime, test, freeze, code, fichier, prior audit ou documented-only
- la limite honnete a ne pas masquer
- la question que cette partie doit susciter chez l interlocuteur

Preuve ou element interne mobilisable

Element	Statut	Utilisation
API 8000	CONFIRMED_BY_RUNTIME	surface bridge actuelle
Kernel 3001	CONFIRMED_BY_FREEZE	coeur de decision
WorkCenter docs	pre-build	base de construction industrielle

Limite a dire honnetement

Ne pas transformer un statut partiel en preuve totale. Ne pas dire production-ready. Ne pas dire certifie. Ne pas dire que les couches peripheriques decident. Ne pas dire que la presence d un fichier equivaut a un runtime actif. La credibilite d Obsidia vient aussi de cette discipline de statut.

Question a susciter

Question cible : si vos agents ou vos workflows pouvaient agir dans le reel, quelle couche separe aujourd hui la recommandation, l autorisation, la preuve et le replay ?

Notes de presentation

- garder les details code pour les questions techniques
- montrer le schema d architecture avant les fichiers
- faire ressortir le Kernel et la securite dans chaque explication
- lier chaque prouesse a une limite honnete
- conclure chaque partie par la valeur metier

Proof stack Lean / TLA / Merkle / RFC3161

Objectif de cette partie : Presenter les preuves stabilisees et les limites honnetes.

La proof stack est un actif de credibilite. Lean confirme 57 theorems / lemmas avec lake build PASS. Les themes couvrent gouvernance decisionnelle, consensus, hypotheses crypto, Merkle, modele systeme, temporalite, raffinement et sensibilite.

TLA+ existe mais TLC n a pas ete relance dans le recheck. RFC3161 anchor existe mais openssl verify n a pas ete execute dans le recheck. Les tests adversariaux sont documentes comme prior audit. Ces limites doivent etre dites sans faiblesse : c est ce qui rend le projet credible.

Message principal a faire passer

La preuve apporte de la credibilite a condition de respecter les statuts reels.

Version courte utilisable en rendez-vous

Phrase courte : La preuve apporte de la credibilite a condition de respecter les statuts reels.

Version orale : Obsidia met une couche d autorite entre ce qu une IA propose et ce qu un systeme a le droit de faire. Le projet ne cherche pas a parler plus fort qu un modele, il cherche a rendre l action controlable, tracable et verifiable.

Ce qu il faut montrer sans noyer dans la technique

- le role de la couche dans l architecture globale
- la valeur concrete pour l interlocuteur
- le statut reel : runtime, test, freeze, code, fichier, prior audit ou documented-only
- la limite honnete a ne pas masquer
- la question que cette partie doit susciter chez l interlocuteur

Preuve ou element interne mobilisable

Preuve	Statut	Limite
Lean 57	CONFIRMED_BY_TEST	ne prouve pas tout le systeme production
TLA+	PRESENT_BUT_NOT_EXECUTED	TLC a relancer
RFC3161	CONFIRMED_BY_FILE	openssl verify a relancer
Adversarial	CONFIRMED_BY_PRIOR_AUDIT	a relancer dans le repo exact

Limite a dire honnetement

Ne pas transformer un statut partiel en preuve totale. Ne pas dire production-ready. Ne pas dire certifie. Ne pas dire que les couches peripheriques decident. Ne pas dire que la presence d un fichier equivaut a un runtime actif. La credibilite d Obsidia vient aussi de cette discipline de statut.

Question a susciter

Question cible : si vos agents ou vos workflows pouvaient agir dans le reel, quelle couche separe aujourd hui la recommandation, l autorisation, la preuve et le replay ?

Notes de presentation

- garder les details code pour les questions techniques
- montrer le schema d architecture avant les fichiers
- faire ressortir le Kernel et la securite dans chaque explication
- lier chaque prouesse a une limite honnete
- conclure chaque partie par la valeur metier

Pourquoi deterministe

Objectif de cette partie : Defendre le choix structurel : invariants, gates, replay, statuts.

Le choix deterministe repond a un besoin de gouvernance. Dans un systeme critique, il faut pouvoir rejouer, auditer et expliquer pourquoi une decision a ete prise.

Un LLM peut aider a comprendre et explorer. Mais le passage a l action exige des statuts, des priorites, des invariants et des preuves. C est la difference entre intelligence conversationnelle et autorite decisionnelle.

Message principal a faire passer

Obsidia doit etre raconte comme une infrastructure de gouvernance du passage a l action.

Version courte utilisable en rendez-vous

Phrase courte : Obsidia doit etre raconte comme une infrastructure de gouvernance du passage a l action.

Version orale : Obsidia met une couche d autorite entre ce qu une IA propose et ce qu un systeme a le droit de faire. Le projet ne cherche pas a parler plus fort qu un modele, il cherche a rendre l action controlable, tracable et verifiable.

Ce qu il faut montrer sans noyer dans la technique

- le role de la couche dans l architecture globale
- la valeur concrete pour l interlocuteur
- le statut reel : runtime, test, freeze, code, fichier, prior audit ou documented-only
- la limite honnete a ne pas masquer
- la question que cette partie doit susciter chez l interlocuteur

Preuve ou element interne mobilisable

Element	Statut	Utilisation
API 8000	CONFIRMED_BY_RUNTIME	surface bridge actuelle
Kernel 3001	CONFIRMED_BY_FREEZE	coeur de decision
WorkCenter docs	pre-build	base de construction industrielle

Limite a dire honnetement

Ne pas transformer un statut partiel en preuve totale. Ne pas dire production-ready. Ne pas dire certifie. Ne pas dire que les couches peripheriques decident. Ne pas dire que la presence d un fichier equivaut a un runtime actif. La credibilite d Obsidia vient aussi de cette discipline de statut.

Question a susciter

Question cible : si vos agents ou vos workflows pouvaient agir dans le reel, quelle couche separe aujourd hui la recommandation, l autorisation, la preuve et le replay ?

Notes de presentation

- garder les details code pour les questions techniques
- montrer le schema d architecture avant les fichiers
- faire ressortir le Kernel et la securite dans chaque explication
- lier chaque prouesse a une limite honnete
- conclure chaque partie par la valeur metier

Pourquoi pas dependant du big training

Objectif de cette partie : Expliquer l avantage structurel sans pretendre remplacer les LLM.

Le coeur de valeur d Obsidia ne depend pas d un entrainement massif. Il depend d une architecture : Kernel, gates, statuts, boundaries, preuve, replay, memoire gouvernee.

Cela ne signifie pas que les donnees sont inutiles. Cela signifie que la decision critique n est pas abandonnee au volume de donnees ou a la taille d un modele.

Message principal a faire passer

L avantage vient de la structure, de la gouvernance et de la verification, pas de la masse brute seule.

Version courte utilisable en rendez-vous

Phrase courte : L avantage vient de la structure, de la gouvernance et de la verification, pas de la masse brute seule.

Version orale : Obsidia met une couche d autorite entre ce qu une IA propose et ce qu un systeme a le droit de faire. Le projet ne cherche pas a parler plus fort qu un modele, il cherche a rendre l action controlable, tracable et verifiable.

Ce qu il faut montrer sans noyer dans la technique

- le role de la couche dans l architecture globale
- la valeur concrete pour l interlocuteur
- le statut reel : runtime, test, freeze, code, fichier, prior audit ou documented-only
- la limite honnete a ne pas masquer
- la question que cette partie doit susciter chez l interlocuteur

Preuve ou element interne mobilisable

Element	Statut	Utilisation
API 8000	CONFIRMED_BY_RUNTIME	surface bridge actuelle
Kernel 3001	CONFIRMED_BY_FREEZE	coeur de decision
WorkCenter docs	pre-build	base de construction industrielle

Limite a dire honnetement

Ne pas transformer un statut partiel en preuve totale. Ne pas dire production-ready. Ne pas dire certifie. Ne pas dire que les couches peripheriques decident. Ne pas dire que la presence d un fichier equivaut a un runtime actif. La credibilite d Obsidia vient aussi de cette discipline de statut.

Question a susciter

Question cible : si vos agents ou vos workflows pouvaient agir dans le reel, quelle couche separe aujourd hui la recommandation, l autorisation, la preuve et le replay ?

Notes de presentation

- garder les details code pour les questions techniques
- montrer le schema d architecture avant les fichiers
- faire ressortir le Kernel et la securite dans chaque explication
- lier chaque prouesse a une limite honnete
- conclure chaque partie par la valeur metier

Gain calcul / cout : structure > masse

Objectif de cette partie : Montrer pourquoi une approche structurelle reduit la pression GPU et data sur le coeur decisionnel.

L argument cout doit etre presente proprement. Obsidia ne promet pas de remplacer toute infrastructure GPU. Il montre qu une part importante de la valeur peut venir de la structure, pas de la masse brute.

Sur une petite machine, le projet peut deja organiser un Kernel, une API, des domaines, des tests, des preuves, une memoire readonly, un cockpit et des freezes. Cela montre qu une architecture lourde peut naitre sans data center.

Message principal a faire passer

L avantage vient de la structure, de la gouvernance et de la verification, pas de la masse brute seule.

Version courte utilisable en rendez-vous

Phrase courte : L avantage vient de la structure, de la gouvernance et de la verification, pas de la masse brute seule.

Version orale : Obsidia met une couche d autorite entre ce qu une IA propose et ce qu un systeme a le droit de faire. Le projet ne cherche pas a parler plus fort qu un modele, il cherche a rendre l action controlable, tracable et verifiable.

Ce qu il faut montrer sans noyer dans la technique

- le role de la couche dans l architecture globale

- la valeur concrete pour l interlocuteur
- le statut reel : runtime, test, freeze, code, fichier, prior audit ou documented-only
- la limite honnete a ne pas masquer
- la question que cette partie doit susciter chez l interlocuteur

Preuve ou element interne mobilisable

Element	Statut	Utilisation
API 8000	CONFIRMED_BY_RUNTIME	surface bridge actuelle
Kernel 3001	CONFIRMED_BY_FREEZE	coeur de decision
WorkCenter docs	pre-build	base de construction industrielle

Limite a dire honnetement

Ne pas transformer un statut partiel en preuve totale. Ne pas dire production-ready. Ne pas dire certifie. Ne pas dire que les couches peripheriques decident. Ne pas dire que la presence d un fichier equivaut a un runtime actif. La credibilite d Obsidia vient aussi de cette discipline de statut.

Question a susciter

Question cible : si vos agents ou vos workflows pouvaient agir dans le reel, quelle couche separe aujourd hui la recommandation, l autorisation, la preuve et le replay ?

Notes de presentation

- garder les details code pour les questions techniques
- montrer le schema d architecture avant les fichiers
- faire ressortir le Kernel et la securite dans chaque explication
- lier chaque prouesse a une limite honnete
- conclure chaque partie par la valeur metier

Petite machine, architecture lourde

Objectif de cette partie : Montrer ce qui a deja ete construit localement et ce que cela prouve.

Le message est fort : avec une machine locale, il est possible de construire un systeme avance de gouvernance, de preuve et de pilotage. Ce n est pas encore une industrialisation. Mais c est deja une preuve d architecture.

La valeur a mettre en avant est la densite structurelle : beaucoup de couches coherentisees, pas seulement beaucoup de calcul.

Message principal a faire passer

L avantage vient de la structure, de la gouvernance et de la verification, pas de la masse brute seule.

Version courte utilisable en rendez-vous

Phrase courte : L avantage vient de la structure, de la gouvernance et de la verification, pas de la masse brute seule.

Version orale : Obsidia met une couche d autorite entre ce qu une IA propose et ce qu un systeme a le droit de faire. Le projet ne cherche pas a parler plus fort qu un modele, il cherche a rendre l action controlable, tracable et verifiable.

Ce qu il faut montrer sans noyer dans la technique

- le role de la couche dans l architecture globale
- la valeur concrete pour l interlocuteur
- le statut reel : runtime, test, freeze, code, fichier, prior audit ou documented-only
- la limite honnete a ne pas masquer
- la question que cette partie doit susciter chez l interlocuteur

Preuve ou element interne mobilisable

Element	Statut	Utilisation
API 8000	CONFIRMED_BY_RUNTIME	surface bridge actuelle
Kernel 3001	CONFIRMED_BY_FREEZE	coeur de decision
WorkCenter docs	pre-build	base de construction industrielle

Limite a dire honnetement

Ne pas transformer un statut partiel en preuve totale. Ne pas dire production-ready. Ne pas dire certifie. Ne pas dire que les couches peripheriques decident. Ne pas dire que la presence d un fichier equivaut a un runtime actif. La credibilite d Obsidia vient aussi de cette discipline de statut.

Question a susciter

Question cible : si vos agents ou vos workflows pouvaient agir dans le reel, quelle couche separe aujourd hui la recommandation, l autorisation, la preuve et le replay ?

Notes de presentation

- garder les details code pour les questions techniques
- montrer le schema d architecture avant les fichiers
- faire ressortir le Kernel et la securite dans chaque explication
- lier chaque prouesse a une limite honnete
- conclure chaque partie par la valeur metier

Ce qui est deja stabilise

Objectif de cette partie : Synthese des actifs confirmed runtime, test, freeze, code.

Cette section doit etre factuelle. Elle doit lister uniquement ce qui est confirme par runtime, test, freeze, code ou fichier.

Les blocs principaux deja stabilises sont : API FastAPI 8000, Kernel freeze, bridges terrain, Brody Education, Graphiti frozen readonly, Lean 57, proofkit prior audit, contracts Sigma, security boundaries.

Message principal a faire passer

Obsidia doit etre raconte comme une infrastructure de gouvernance du passage a l action.

Version courte utilisable en rendez-vous

Phrase courte : Obsidia doit etre raconte comme une infrastructure de gouvernance du passage a l action.

Version orale : Obsidia met une couche d autorite entre ce qu une IA propose et ce qu un systeme a le droit de faire. Le projet ne cherche pas a parler plus fort qu un modele, il cherche a rendre l action controlable, tracable et verifiable.

Ce qu il faut montrer sans noyer dans la technique

- le role de la couche dans l architecture globale
- la valeur concrete pour l interlocuteur
- le statut reel : runtime, test, freeze, code, fichier, prior audit ou documented-only
- la limite honnete a ne pas masquer
- la question que cette partie doit susciter chez l interlocuteur

Preuve ou element interne mobilisable

Element	Statut	Utilisation
API 8000	CONFIRMED_BY_RUNTIME	surface bridge actuelle
Kernel 3001	CONFIRMED_BY_FREEZE	coeur de decision
WorkCenter docs	pre-build	base de construction industrielle

Limite a dire honnetement

Ne pas transformer un statut partiel en preuve totale. Ne pas dire production-ready. Ne pas dire certifie. Ne pas dire que les couches peripheriques decident. Ne pas dire que la presence d un fichier equivaut a un runtime actif. La credibilite d Obsidia vient aussi de cette discipline de statut.

Question a susciter

Question cible : si vos agents ou vos workflows pouvaient agir dans le reel, quelle couche separe aujourd hui la recommandation, l autorisation, la preuve et le replay ?

Notes de presentation

- garder les details code pour les questions techniques
- montrer le schema d architecture avant les fichiers
- faire ressortir le Kernel et la securite dans chaque explication
- lier chaque prouesse a une limite honnete
- conclure chaque partie par la valeur metier

Limites honnetes

Objectif de cette partie : Dire clairement ce qui reste a relancer, auditer, durcir et certifier.

Cette section protege le projet. Il faut dire explicitement : pas production-ready, audit externe non fait, certification non faite, Kernel ferme au recheck, Graphiti live degrade ou ferme selon moment, TLA/TLC non relance, RFC3161 openssl verify non execute, Ecom non teste live, adversarial tests non relances dans ce repo exact.

Dire ces limites ne diminue pas la valeur. Cela montre que le projet sait distinguer preuve, freeze, runtime et documentation.

Message principal a faire passer

Obsidia doit etre raconte comme une infrastructure de gouvernance du passage a l action.

Version courte utilisable en rendez-vous

Phrase courte : Obsidia doit etre raconte comme une infrastructure de gouvernance du passage a l action.

Version orale : Obsidia met une couche d autorite entre ce qu une IA propose et ce qu un systeme a le droit de faire. Le projet ne cherche pas a parler plus fort qu un modele, il cherche a rendre l action controlable, tracable et verifiable.

Ce qu il faut montrer sans noyer dans la technique

- le role de la couche dans l architecture globale
- la valeur concrete pour l interlocuteur
- le statut reel : runtime, test, freeze, code, fichier, prior audit ou documented-only
- la limite honnete a ne pas masquer
- la question que cette partie doit susciter chez l interlocuteur

Preuve ou element interne mobilisable

Element	Statut	Utilisation
API 8000	CONFIRMED_BY_RUNTIME	surface bridge actuelle
Kernel 3001	CONFIRMED_BY_FREEZE	coeur de decision
WorkCenter docs	pre-build	base de construction industrielle

Limite a dire honnetement

Ne pas transformer un statut partiel en preuve totale. Ne pas dire production-ready. Ne pas dire certifie. Ne pas dire que les couches peripheriques decident. Ne pas dire que la presence d un fichier equivaut a un runtime actif. La credibilite d Obsidia vient aussi de cette discipline de statut.

Question a susciter

Question cible : si vos agents ou vos workflows pouvaient agir dans le reel, quelle couche separe aujourd hui la recommandation, l autorisation, la preuve et le replay ?

Notes de presentation

- garder les details code pour les questions techniques
- montrer le schema d architecture avant les fichiers
- faire ressortir le Kernel et la securite dans chaque explication
- lier chaque prouesse a une limite honnete
- conclure chaque partie par la valeur metier

Etat actuel du projet

Objectif de cette partie : Positionner le projet en POC local avance / pre-industrialisation.

L etat actuel est : POC local avance, recherche appliquee, pre-industrialisation. Le projet possede deja une colonne vertebrale technique, mais doit encore passer par durcissement, audit externe, packaging, education et WorkCenter industriel.

Message principal a faire passer

Obsidia doit etre raconte comme une infrastructure de gouvernance du passage a l action.

Version courte utilisable en rendez-vous

Phrase courte : Obsidia doit etre raconte comme une infrastructure de gouvernance du passage a l action.

Version orale : Obsidia met une couche d autorite entre ce qu une IA propose et ce qu un systeme a le droit de faire. Le projet ne cherche pas a parler plus fort qu un modele, il cherche a rendre l action controlable, tracable et verifiable.

Ce qu il faut montrer sans noyer dans la technique

- le role de la couche dans l architecture globale
- la valeur concrete pour l interlocuteur
- le statut reel : runtime, test, freeze, code, fichier, prior audit ou documented-only
- la limite honnete a ne pas masquer
- la question que cette partie doit susciter chez l interlocuteur

Preuve ou element interne mobilisable

Element	Statut	Utilisation
API 8000	CONFIRMED_BY_RUNTIME	surface bridge actuelle
Kernel 3001	CONFIRMED_BY_FREEZE	coeur de decision
WorkCenter docs	pre-build	base de construction industrielle

Limite a dire honnetement

Ne pas transformer un statut partiel en preuve totale. Ne pas dire production-ready. Ne pas dire certifie. Ne pas dire que les couches peripheriques decident. Ne pas dire que la presence d un fichier equivaut a un runtime actif. La credibilite d Obsidia vient aussi de cette discipline de statut.

Question a susciter

Question cible : si vos agents ou vos workflows pouvaient agir dans le reel, quelle couche separe aujourd hui la recommandation, l autorisation, la preuve et le replay ?

Notes de presentation

- garder les details code pour les questions techniques
- montrer le schema d architecture avant les fichiers

- faire ressortir le Kernel et la securite dans chaque explication
- lier chaque prouesse a une limite honnete
- conclure chaque partie par la valeur metier

Dernieres etapes avant industrialisation

Objectif de cette partie : Lister les paliers avant audit externe, CI, pilot POC, certification.

Les dernieres etapes doivent etre decoupees en paliers. D abord rendre visible avec WorkCenter. Ensuite education poussee. Ensuite durcissement technique. Ensuite audit externe. Ensuite pilot POC controle. Ensuite industrialisation.

Message principal a faire passer

Obsidia doit etre raconte comme une infrastructure de gouvernance du passage a l action.

Version courte utilisable en rendez-vous

Phrase courte : Obsidia doit etre raconte comme une infrastructure de gouvernance du passage a l action.

Version orale : Obsidia met une couche d autorite entre ce qu une IA propose et ce qu un systeme a le droit de faire. Le projet ne cherche pas a parler plus fort qu un modele, il cherche a rendre l action controlable, tracable et verifiable.

Ce qu il faut montrer sans noyer dans la technique

- le role de la couche dans l architecture globale
- la valeur concrete pour l interlocuteur
- le statut reel : runtime, test, freeze, code, fichier, prior audit ou documented-only
- la limite honnete a ne pas masquer
- la question que cette partie doit susciter chez l interlocuteur

Preuve ou element interne mobilisable

Element	Statut	Utilisation
API 8000	CONFIRMED_BY_RUNTIME	surface bridge actuelle
Kernel 3001	CONFIRMED_BY_FREEZE	coeur de decision
WorkCenter docs	pre-build	base de construction industrielle

Limite a dire honnetement

Ne pas transformer un statut partiel en preuve totale. Ne pas dire production-ready. Ne pas dire certifie. Ne pas dire que les couches peripheriques decident. Ne pas dire que la presence d un fichier equivaut a un runtime actif. La credibilite d Obsidia vient aussi de cette discipline de statut.

Question a susciter

Question cible : si vos agents ou vos workflows pouvaient agir dans le reel, quelle couche separe aujourd hui la recommandation, l autorisation, la preuve et le replay ?

Notes de presentation

- garder les details code pour les questions techniques
- montrer le schema d architecture avant les fichiers
- faire ressortir le Kernel et la securite dans chaque explication
- lier chaque prouesse a une limite honnete
- conclure chaque partie par la valeur metier

Roadmap WorkCenter

Objectif de cette partie : Transformer les supports dev en cockpit concret.

Le WorkCenter est le passage entre laboratoire et systeme exploitable. Il doit montrer le Kernel, la securite, les preuves, les domaines, les routes, les memoires, les validations, les freezes et la roadmap.

Les supports dev deja crees servent de base : cahier des charges, checklist, matrice tickets, contrat API, object model, security boundary matrix.

Message principal a faire passer

Obsidia doit etre raconte comme une infrastructure de gouvernance du passage a l action.

Version courte utilisable en rendez-vous

Phrase courte : Obsidia doit etre raconte comme une infrastructure de gouvernance du passage a l action.

Version orale : Obsidia met une couche d autorite entre ce qu une IA propose et ce qu un systeme a le droit de faire. Le projet ne cherche pas a parler plus fort qu un modele, il cherche a rendre l action controlable, tracable et verifiable.

Ce qu il faut montrer sans noyer dans la technique

- le role de la couche dans l architecture globale
- la valeur concrete pour l interlocuteur
- le statut reel : runtime, test, freeze, code, fichier, prior audit ou documented-only
- la limite honnete a ne pas masquer
- la question que cette partie doit susciter chez l interlocuteur

Preuve ou element interne mobilisable

Element	Statut	Utilisation
API 8000	CONFIRMED_BY_RUNTIME	surface bridge actuelle
Kernel 3001	CONFIRMED_BY_FREEZE	coeur de decision
WorkCenter docs	pre-build	base de construction industrielle

Limite a dire honnetement

Ne pas transformer un statut partiel en preuve totale. Ne pas dire production-ready. Ne pas dire certifie. Ne pas dire que les couches peripheriques decident. Ne pas dire que la presence d un fichier equivaut a un runtime actif. La credibilite d Obsidia vient aussi de cette discipline de statut.

Question a susciter

Question cible : si vos agents ou vos workflows pouvaient agir dans le reel, quelle couche separe aujourd hui la recommandation, l autorisation, la preuve et le replay ?

Notes de presentation

- garder les details code pour les questions techniques
- montrer le schema d architecture avant les fichiers
- faire ressortir le Kernel et la securite dans chaque explication
- lier chaque prouesse a une limite honnete
- conclure chaque partie par la valeur metier

Roadmap education poussee

Objectif de cette partie : Structurer Brody Education, modules, parcours publics.

L education est une couche strategique. Elle transforme la complexite en parcours. Brody Education doit former les non-dev, les devs, les auditeurs, les investisseurs et les partenaires domaine.

Message principal a faire passer

Obsidia doit etre raconte comme une infrastructure de gouvernance du passage a l action.

Version courte utilisable en rendez-vous

Phrase courte : Obsidia doit etre raconte comme une infrastructure de gouvernance du passage a l action.

Version orale : Obsidia met une couche d autorite entre ce qu une IA propose et ce qu un systeme a le droit de faire. Le projet ne cherche pas a parler plus fort qu un modele, il cherche a rendre l action controlable, tracable et verifiable.

Ce qu il faut montrer sans noyer dans la technique

- le role de la couche dans l architecture globale
- la valeur concrete pour l interlocuteur
- le statut reel : runtime, test, freeze, code, fichier, prior audit ou documented-only
- la limite honnete a ne pas masquer
- la question que cette partie doit susciter chez l interlocuteur

Preuve ou element interne mobilisable

Element	Statut	Utilisation
API 8000	CONFIRMED_BY_RUNTIME	surface bridge actuelle
Kernel 3001	CONFIRMED_BY_FREEZE	coeur de decision
WorkCenter docs	pre-build	base de construction industrielle

Limite a dire honnetement

Ne pas transformer un statut partiel en preuve totale. Ne pas dire production-ready. Ne pas dire certifie. Ne pas dire que les couches peripheriques decident. Ne pas dire que la presence d un fichier equivaut a un runtime actif. La credibilite d Obsidia vient aussi de cette discipline de statut.

Question a susciter

Question cible : si vos agents ou vos workflows pouvaient agir dans le reel, quelle couche separe aujourd hui la recommandation, l autorisation, la preuve et le replay ?

Notes de presentation

- garder les details code pour les questions techniques
- montrer le schema d architecture avant les fichiers
- faire ressortir le Kernel et la securite dans chaque explication
- lier chaque prouesse a une limite honnete
- conclure chaque partie par la valeur metier

Roadmap audit externe

Objectif de cette partie : Preparer preuves, limitations, test packs, release process.

Avant usage critique, le projet doit produire un pack audit externe : preuves relancees, limites, manifests, hashes, CI, tests adversariaux, verification RFC3161, re-run TLA/TLC, security review.

Message principal a faire passer

Obsidia doit etre raconte comme une infrastructure de gouvernance du passage a l action.

Version courte utilisable en rendez-vous

Phrase courte : Obsidia doit etre raconte comme une infrastructure de gouvernance du passage a l action.

Version orale : Obsidia met une couche d autorite entre ce qu une IA propose et ce qu un systeme a le droit de faire. Le projet ne cherche pas a parler plus fort qu un modele, il cherche a rendre l action controlable, tracable et verifiable.

Ce qu il faut montrer sans noyer dans la technique

- le role de la couche dans l architecture globale

- la valeur concrete pour l interlocuteur
- le statut reel : runtime, test, freeze, code, fichier, prior audit ou documented-only
- la limite honnete a ne pas masquer
- la question que cette partie doit susciter chez l interlocuteur

Preuve ou element interne mobilisable

Element	Statut	Utilisation
API 8000	CONFIRMED_BY_RUNTIME	surface bridge actuelle
Kernel 3001	CONFIRMED_BY_FREEZE	coeur de decision
WorkCenter docs	pre-build	base de construction industrielle

Limite a dire honnetement

Ne pas transformer un statut partiel en preuve totale. Ne pas dire production-ready. Ne pas dire certifie. Ne pas dire que les couches peripheriques decident. Ne pas dire que la presence d un fichier equivaut a un runtime actif. La credibilite d Obsidia vient aussi de cette discipline de statut.

Question a susciter

Question cible : si vos agents ou vos workflows pouvaient agir dans le reel, quelle couche separe aujourd hui la recommandation, l autorisation, la preuve et le replay ?

Notes de presentation

- garder les details code pour les questions techniques
- montrer le schema d architecture avant les fichiers
- faire ressortir le Kernel et la securite dans chaque explication
- lier chaque prouesse a une limite honnete
- conclure chaque partie par la valeur metier

Roadmap Big Tech Workbench

Objectif de cette partie : Decrire la trajectoire vers un environnement de travail gouverne de niveau industriel.

Le Workbench cible n est pas seulement un terminal enrichi. C est un environnement complet ou agents, memoire, preuve, replay, sandbox, education et audit travaillent sous gouvernance.

La difference avec les workbenchs Big Tech doit etre claire : Obsidia ne cherche pas seulement a accelerer l agent, mais a gouverner son droit d agir.

Message principal a faire passer

Obsidia doit etre raconte comme une infrastructure de gouvernance du passage a l action.

Version courte utilisable en rendez-vous

Phrase courte : Obsidia doit etre raconte comme une infrastructure de gouvernance du passage a l action.

Version orale : Obsidia met une couche d autorite entre ce qu une IA propose et ce qu un systeme a le droit de faire. Le projet ne cherche pas a parler plus fort qu un modele, il cherche a rendre l action controlable, tracable et verifiable.

Ce qu il faut montrer sans noyer dans la technique

- le role de la couche dans l architecture globale
- la valeur concrete pour l interlocuteur
- le statut reel : runtime, test, freeze, code, fichier, prior audit ou documented-only
- la limite honnete a ne pas masquer
- la question que cette partie doit susciter chez l interlocuteur

Preuve ou element interne mobilisable

Element	Statut	Utilisation
API 8000	CONFIRMED_BY_RUNTIME	surface bridge actuelle
Kernel 3001	CONFIRMED_BY_FREEZE	coeur de decision
WorkCenter docs	pre-build	base de construction industrielle

Limite a dire honnetement

Ne pas transformer un statut partiel en preuve totale. Ne pas dire production-ready. Ne pas dire certifie. Ne pas dire que les couches peripheriques decident. Ne pas dire que la presence d un fichier equivaut a un runtime actif. La credibilite d Obsidia vient aussi de cette discipline de statut.

Question a susciter

Question cible : si vos agents ou vos workflows pouvaient agir dans le reel, quelle couche separe aujourd hui la recommandation, l autorisation, la preuve et le replay ?

Notes de presentation

- garder les details code pour les questions techniques
- montrer le schema d architecture avant les fichiers
- faire ressortir le Kernel et la securite dans chaque explication
- lier chaque prouesse a une limite honnete
- conclure chaque partie par la valeur metier

Positionnement concurrentiel

Objectif de cette partie : Positionner Obsidia face aux LLM, agent frameworks, MLOps, RAG et monitoring.

Obsidia ne doit pas se positionner frontalement comme un modele plus gros. Il se positionne comme couche de gouvernance autour des systemes qui peuvent agir.

Face aux LLM : complement et controle. Face aux agent frameworks : autorite et preuve. Face au RAG : memoire gouvernee. Face au monitoring : decision pre-action. Face au MLOps : gouvernance du passage a l acte.

Message principal a faire passer

Obsidia doit etre raconte comme une infrastructure de gouvernance du passage a l action.

Version courte utilisable en rendez-vous

Phrase courte : Obsidia doit etre raconte comme une infrastructure de gouvernance du passage a l action.

Version orale : Obsidia met une couche d autorite entre ce qu une IA propose et ce qu un systeme a le droit de faire. Le projet ne cherche pas a parler plus fort qu un modele, il cherche a rendre l action controlable, tracable et verifiable.

Ce qu il faut montrer sans noyer dans la technique

- le role de la couche dans l architecture globale
- la valeur concrete pour l interlocuteur
- le statut reel : runtime, test, freeze, code, fichier, prior audit ou documented-only
- la limite honnete a ne pas masquer
- la question que cette partie doit susciter chez l interlocuteur

Preuve ou element interne mobilisable

Element	Statut	Utilisation
API 8000	CONFIRMED_BY_RUNTIME	surface bridge actuelle
Kernel 3001	CONFIRMED_BY_FREEZE	coeur de decision
WorkCenter docs	pre-build	base de construction industrielle

Limite a dire honnetement

Ne pas transformer un statut partiel en preuve totale. Ne pas dire production-ready. Ne pas dire certifie. Ne pas dire que les couches peripheriques decident. Ne pas dire que la presence d un fichier equivaut a un runtime actif. La credibilite d Obsidia vient aussi de cette discipline de statut.

Question a susciter

Question cible : si vos agents ou vos workflows pouvaient agir dans le reel, quelle couche separe aujourd hui la recommandation, l autorisation, la preuve et le replay ?

Notes de presentation

- garder les details code pour les questions techniques
- montrer le schema d architecture avant les fichiers
- faire ressortir le Kernel et la securite dans chaque explication
- lier chaque prouesse a une limite honnete
- conclure chaque partie par la valeur metier

Pourquoi Obsidia peut rejoindre puis depasser

Objectif de cette partie : Formuler l angle : pas la taille du modele, mais le controle du passage a l action.

Rejoindre signifie construire les outils de travail, de memoire, de preuve et de sandbox au niveau des meilleurs environnements. Depasser signifie apporter une couche que beaucoup de stacks n ont pas encore pleinement : l autorite deterministe du passage a l action.

Message principal a faire passer

Obsidia doit etre raconte comme une infrastructure de gouvernance du passage a l action.

Version courte utilisable en rendez-vous

Phrase courte : Obsidia doit etre raconte comme une infrastructure de gouvernance du passage a l action.

Version orale : Obsidia met une couche d autorite entre ce qu une IA propose et ce qu un systeme a le droit de faire. Le projet ne cherche pas a parler plus fort qu un modele, il cherche a rendre l action controlable, tracable et verifiable.

Ce qu il faut montrer sans noyer dans la technique

- le role de la couche dans l architecture globale
- la valeur concrete pour l interlocuteur
- le statut reel : runtime, test, freeze, code, fichier, prior audit ou documented-only
- la limite honnete a ne pas masquer
- la question que cette partie doit susciter chez l interlocuteur

Preuve ou element interne mobilisable

Element	Statut	Utilisation
API 8000	CONFIRMED_BY_RUNTIME	surface bridge actuelle
Kernel 3001	CONFIRMED_BY_FREEZE	coeur de decision
WorkCenter docs	pre-build	base de construction industrielle

Limite a dire honnetement

Ne pas transformer un statut partiel en preuve totale. Ne pas dire production-ready. Ne pas dire certifie. Ne pas dire que les couches peripheriques decident. Ne pas dire que la presence d un fichier equivaut a un runtime actif. La credibilite d Obsidia vient aussi de cette discipline de statut.

Question a susciter

Question cible : si vos agents ou vos workflows pouvaient agir dans le reel, quelle couche separe aujourd hui la recommandation, l autorisation, la preuve et le replay ?

Notes de presentation

- garder les details code pour les questions techniques
- montrer le schema d architecture avant les fichiers
- faire ressortir le Kernel et la securite dans chaque explication
- lier chaque prouesse a une limite honnete
- conclure chaque partie par la valeur metier

Supports separes par public

Objectif de cette partie : Decliner les messages selon investisseur, banque, CTO, Big Tech, non-tech, education.

Le meme projet doit etre raconte differemment selon l interlocuteur. Un investisseur veut comprendre le marche et la trajectoire. Une banque veut comprendre le risque et l audit. Un CTO veut comprendre l architecture et les limites. Un public Big Tech veut comprendre le changement de paradigme. Un non-tech veut comprendre qui decide.

Message principal a faire passer

Obsidia doit etre raconte comme une infrastructure de gouvernance du passage a l action.

Version courte utilisable en rendez-vous

Phrase courte : Obsidia doit etre raconte comme une infrastructure de gouvernance du passage a l action.

Version orale : Obsidia met une couche d autorite entre ce qu une IA propose et ce qu un systeme a le droit de faire. Le projet ne cherche pas a parler plus fort qu un modele, il cherche a rendre l action controlable, tracable et verifiable.

Ce qu il faut montrer sans noyer dans la technique

- le role de la couche dans l architecture globale
- la valeur concrete pour l interlocuteur
- le statut reel : runtime, test, freeze, code, fichier, prior audit ou documented-only
- la limite honnete a ne pas masquer
- la question que cette partie doit susciter chez l interlocuteur

Preuve ou element interne mobilisable

Element	Statut	Utilisation
API 8000	CONFIRMED_BY_RUNTIME	surface bridge actuelle
Kernel 3001	CONFIRMED_BY_FREEZE	coeur de decision
WorkCenter docs	pre-build	base de construction industrielle

Limite a dire honnetement

Ne pas transformer un statut partiel en preuve totale. Ne pas dire production-ready. Ne pas dire certifie. Ne pas dire que les couches peripheriques decident. Ne pas dire que la presence d un fichier equivaut a un runtime actif. La credibilite d Obsidia vient aussi de cette discipline de statut.

Question a susciter

Question cible : si vos agents ou vos workflows pouvaient agir dans le reel, quelle couche separe aujourd hui la recommandation, l autorisation, la preuve et le replay ?

Notes de presentation

- garder les details code pour les questions techniques
- montrer le schema d architecture avant les fichiers

- faire ressortir le Kernel et la securite dans chaque explication
- lier chaque prouesse a une limite honnete
- conclure chaque partie par la valeur metier

Conclusion

Objectif de cette partie : Gouverner l action, pas seulement generer du texte.

La conclusion doit revenir a la phrase centrale : Obsidia ne cherche pas seulement a generer du texte. Obsidia cherche a gouverner l action.

Dans un monde ou les agents deviennent capables, le point critique n est pas seulement leur intelligence. C est l architecture qui decide quand ils ont le droit d agir.

Message principal a faire passer

Obsidia doit etre raconte comme une infrastructure de gouvernance du passage a l action.

Version courte utilisable en rendez-vous

Phrase courte : Obsidia doit etre raconte comme une infrastructure de gouvernance du passage a l action.

Version orale : Obsidia met une couche d autorite entre ce qu une IA propose et ce qu un systeme a le droit de faire. Le projet ne cherche pas a parler plus fort qu un modele, il cherche a rendre l action controlable, tracable et verifiable.

Ce qu il faut montrer sans noyer dans la technique

- le role de la couche dans l architecture globale
- la valeur concrete pour l interlocuteur
- le statut reel : runtime, test, freeze, code, fichier, prior audit ou documented-only
- la limite honnete a ne pas masquer
- la question que cette partie doit susciter chez l interlocuteur

Preuve ou element interne mobilisable

Element	Statut	Utilisation
API 8000	CONFIRMED_BY_RUNTIME	surface bridge actuelle
Kernel 3001	CONFIRMED_BY_FREEZE	coeur de decision
WorkCenter docs	pre-build	base de construction industrielle

Limite a dire honnetement

Ne pas transformer un statut partiel en preuve totale. Ne pas dire production-ready. Ne pas dire certifie. Ne pas dire que les couches peripheriques decident. Ne pas dire que la presence d un fichier equivaut a un runtime actif. La credibilite d Obsidia vient aussi de cette discipline de statut.

Question a susciter

Question cible : si vos agents ou vos workflows pouvaient agir dans le reel, quelle couche separe aujourd hui la recommandation, l autorisation, la preuve et le replay ?

Notes de presentation

- garder les details code pour les questions techniques
- montrer le schema d architecture avant les fichiers
- faire ressortir le Kernel et la securite dans chaque explication
- lier chaque prouesse a une limite honnete
- conclure chaque partie par la valeur metier

Schema narratif maitre a utiliser dans le support

```
Utilisateur / systeme externe
 -> Signal ou action candidate
   -> Domaine terrain : Bank / Trading / GPS / Aviation / Ecom / CIC / NCP
     -> Agents domaine : observation, risque, contradiction, contexte
       -> Sigma : aggregation, coherence, severity, votes
         -> Kernel X108 : ALLOW / HOLD / BLOCK
           -> Decision envelope : decision_id, trace_id, severity, reason_code
             -> Proof / audit / hash / replay / freeze
               -> WorkCenter : comprehension, validation, education, supervision
```

Ce schema doit apparaitre tot dans le support. Il permet de montrer toutes les couches sans entrer dans le detail des fichiers. Les details viennent en annexe ou en reponse aux questions.

DECLINAISONS PAR PUBLIC

Investisseur / incubateur

Objectif : adapter le discours sans changer la doctrine. Focus principal : marche, timing, actif technique, reduction du risque, trajectoire industrialisation.

Angle d entree

Le marche des agents IA va exiger une couche d autorite. Obsidia se positionne avant l action : il gouverne le passage entre recommandation et transformation reelle. Le projet combine actif technique, preuve, securite, education et WorkCenter industriel.

Pitch 30 secondes

Obsidia / X108 est une architecture de gouvernance pour agents IA. Elle se place entre ce qu une IA propose et ce qu un systeme a le droit de faire. Son Kernel retourne ALLOW, HOLD ou BLOCK, pendant que les autres couches expliquent, memorisent, agregent, auditent ou affichent sans prendre l autorite finale.

Pitch 2 minutes

Aujourd hui, les IA ne font plus seulement du texte. Elles appellent des outils, preparent des actions et s inserent dans des workflows. Obsidia repond au probleme du droit d agir : qui autorise, qui suspend, qui bloque, qui trace, qui prouve et qui rejoue. Le projet possede un Kernel X108, une API bridge, des terrains Bank / Trading / GPS, une couche Brody educative, une memoire Graphiti readonly, une stack de preuves Lean / TLA / Merkle / RFC3161 et une trajectoire WorkCenter pour rendre tout cela exploitable. Le statut reste honnete : POC local avance, pre-industrialisation, pas production-ready sans audit externe.

Ce qu il faut montrer

- le probleme du passage a l action
- le Kernel comme verrou
- la separation d autorite
- les terrains Bank / Trading / GPS
- les preuves et les limites
- la roadmap industrialisation

Ce qu il ne faut pas dire

- production-ready
- certifie
- remplace les LLM
- plus besoin de donnees

- Brody decide
- Graphiti autorise
- l API decide si le Kernel est down

Questions attendues

- Comment le Kernel prend-il une decision ?
- Qu est-ce qui empeche un agent de contourner la gouvernance ?
- Quelles preuves sont formelles et quelles preuves restent a relancer ?
- Comment passer de POC local a industrialisation ?
- Quelle partie est vendable en premier ?

Banque / risque / conformite

Objectif : adapter le discours sans changer la doctrine. Focus principal : autoriser, suspendre, bloquer, tracer, rejouer, auditer avant impact.

Angle d entree

Le point cle est le controle avant impact : autoriser, suspendre, bloquer, tracer et rejouer. Obsidia ne remplace pas la stack bancaire. Il se place comme couche de gouvernance entre signaux, agents, outils et action.

Pitch 30 secondes

Obsidia / X108 est une architecture de gouvernance pour agents IA. Elle se place entre ce qu une IA propose et ce qu un systeme a le droit de faire. Son Kernel retourne ALLOW, HOLD ou BLOCK, pendant que les autres couches expliquent, memorisent, agregent, auditent ou affichent sans prendre l autorite finale.

Pitch 2 minutes

Aujourd hui, les IA ne font plus seulement du texte. Elles appellent des outils, preparent des actions et s inserent dans des workflows. Obsidia repond au probleme du droit d agir : qui autorise, qui suspend, qui bloque, qui trace, qui prouve et qui rejoue. Le projet possede un Kernel X108, une API bridge, des terrains Bank / Trading / GPS, une couche Brody educative, une memoire Graphiti readonly, une stack de preuves Lean / TLA / Merkle / RFC3161 et une trajectoire WorkCenter pour rendre tout cela exploitable. Le statut reste honnete : POC local avance, pre-industrialisation, pas production-ready sans audit externe.

Ce qu il faut montrer

- le probleme du passage a l action
- le Kernel comme verrou
- la separation d autorite
- les terrains Bank / Trading / GPS
- les preuves et les limites
- la roadmap industrialisation

Ce qu il ne faut pas dire

- production-ready
- certifie
- remplace les LLM
- plus besoin de donnees
- Brody decide
- Graphiti autorise
- l API decide si le Kernel est down

Questions attendues

- Comment le Kernel prend-il une decision ?

- Qu est-ce qui empeche un agent de contourner la gouvernance ?
- Quelles preuves sont formelles et quelles preuves restent a relancer ?
- Comment passer de POC local a industrialisation ?
- Quelle partie est vendable en premier ?

CTO / dev senior

Objectif : adapter le discours sans changer la doctrine. Focus principal : architecture, Kernel, API bridge, boundaries, preuves, endpoints, tests.

Angle d entree

Le point cle est la separation d autorite : Kernel souverain, API bridge, domaines, Sigma, Brody non souverain, Graphiti readonly, preuves et freezes. Les statuts doivent etre verifies et non sur-declares.

Pitch 30 secondes

Obsidia / X108 est une architecture de gouvernance pour agents IA. Elle se place entre ce qu une IA propose et ce qu un systeme a le droit de faire. Son Kernel retourne ALLOW, HOLD ou BLOCK, pendant que les autres couches expliquent, memorisent, agregent, auditent ou affichent sans prendre l autorite finale.

Pitch 2 minutes

Aujourd hui, les IA ne font plus seulement du texte. Elles appellent des outils, preparent des actions et s inserent dans des workflows. Obsidia repond au probleme du droit d agir : qui autorise, qui suspend, qui bloque, qui trace, qui prouve et qui rejoue. Le projet possede un Kernel X108, une API bridge, des terrains Bank / Trading / GPS, une couche Brody educative, une memoire Graphiti readonly, une stack de preuves Lean / TLA / Merkle / RFC3161 et une trajectoire WorkCenter pour rendre tout cela exploitable. Le statut reste honnete : POC local avance, pre-industrialisation, pas production-ready sans audit externe.

Ce qu il faut montrer

- le probleme du passage a l action
- le Kernel comme verrou
- la separation d autorite
- les terrains Bank / Trading / GPS
- les preuves et les limites
- la roadmap industrialisation

Ce qu il ne faut pas dire

- production-ready
- certifie
- remplace les LLM
- plus besoin de donnees
- Brody decide
- Graphiti autorise
- l API decide si le Kernel est down

Questions attendues

- Comment le Kernel prend-il une decision ?
- Qu est-ce qui empeche un agent de contourner la gouvernance ?
- Quelles preuves sont formelles et quelles preuves restent a relancer ?
- Comment passer de POC local a industrialisation ?
- Quelle partie est vendable en premier ?

Big Tech / IA infra

Objectif : adapter le discours sans changer la doctrine. Focus principal : workbench gouverne, agents encadres, proof layer, complement aux LLM.

Angle d entree

Le point cle est l infrastructure de gouvernance autour des agents. Obsidia ne rivalise pas par la taille du modele, mais par le controle du passage a l action, le replay et la preuve.

Pitch 30 secondes

Obsidia / X108 est une architecture de gouvernance pour agents IA. Elle se place entre ce qu une IA propose et ce qu un systeme a le droit de faire. Son Kernel retourne ALLOW, HOLD ou BLOCK, pendant que les autres couches expliquent, memorisent, agregent, auditent ou affichent sans prendre l autorite finale.

Pitch 2 minutes

Aujourd hui, les IA ne font plus seulement du texte. Elles appellent des outils, preparent des actions et s inserent dans des workflows. Obsidia repond au probleme du droit d agir : qui autorise, qui suspend, qui bloque, qui trace, qui prouve et qui rejoue. Le projet possede un Kernel X108, une API bridge, des terrains Bank / Trading / GPS, une couche Brody educative, une memoire Graphiti readonly, une stack de preuves Lean / TLA / Merkle / RFC3161 et une trajectoire WorkCenter pour rendre tout cela exploitable. Le statut reste honnete : POC local avance, pre-industrialisation, pas production-ready sans audit externe.

Ce qu il faut montrer

- le probleme du passage a l action
- le Kernel comme verrou
- la separation d autorite
- les terrains Bank / Trading / GPS
- les preuves et les limites
- la roadmap industrialisation

Ce qu il ne faut pas dire

- production-ready
- certifie
- remplace les LLM
- plus besoin de donnees
- Brody decide
- Graphiti autorise
- l API decide si le Kernel est down

Questions attendues

- Comment le Kernel prend-il une decision ?
- Qu est-ce qui empeche un agent de contourner la gouvernance ?
- Quelles preuves sont formelles et quelles preuves restent a relancer ?
- Comment passer de POC local a industrialisation ?
- Quelle partie est vendable en premier ?

Non-tech / partenaire terrain

Objectif : adapter le discours sans changer la doctrine. Focus principal : comprendre qui decide, qui explique, qui memorise, qui prouve.

Angle d entree

Le point cle est de faire comprendre qui a le droit d agir. Obsidia est une tour de controle pour les IA qui peuvent recommander, automatiser ou declencher des processus.

Pitch 30 secondes

Obsidia / X108 est une architecture de gouvernance pour agents IA. Elle se place entre ce qu une IA propose et ce qu un systeme a le droit de faire. Son Kernel retourne ALLOW, HOLD ou BLOCK, pendant que les autres couches expliquent, memorisent, agregent, auditent ou affichent sans prendre l autorite finale.

Pitch 2 minutes

Aujourd hui, les IA ne font plus seulement du texte. Elles appellent des outils, preparent des actions et s inserent dans des workflows. Obsidia repond au probleme du droit d agir : qui autorise, qui suspend, qui bloque, qui trace, qui prouve et qui rejoue. Le projet possede un Kernel X108, une API bridge, des terrains Bank / Trading / GPS, une couche Brody educative, une memoire Graphiti readonly, une stack de preuves Lean / TLA / Merkle / RFC3161 et une trajectoire WorkCenter pour rendre tout cela exploitable. Le statut reste honnete : POC local avance, pre-industrialisation, pas production-ready sans audit externe.

Ce qu il faut montrer

- le probleme du passage a l action
- le Kernel comme verrou
- la separation d autorite
- les terrains Bank / Trading / GPS
- les preuves et les limites
- la roadmap industrialisation

Ce qu il ne faut pas dire

- production-ready
- certifie
- remplace les LLM
- plus besoin de donnees
- Brody decide
- Graphiti autorise
- l API decide si le Kernel est down

Questions attendues

- Comment le Kernel prend-il une decision ?
- Qu est-ce qui empeche un agent de contourner la gouvernance ?
- Quelles preuves sont formelles et quelles preuves restent a relancer ?
- Comment passer de POC local a industrialisation ?
- Quelle partie est vendable en premier ?

Education / formation interne

Objectif : adapter le discours sans changer la doctrine. Focus principal : parcours Brody, modules, exercices, validation, limites.

Angle d entree

Le point cle est la transmission : transformer un systeme complexe en parcours d apprentissage, exercices, validation et comprehension des limites.

Pitch 30 secondes

Obsidia / X108 est une architecture de gouvernance pour agents IA. Elle se place entre ce qu une IA propose et ce qu un systeme a le droit de faire. Son Kernel retourne ALLOW, HOLD ou BLOCK, pendant que les autres couches expliquent, memorisent, agregent, auditent ou affichent sans prendre l autorite finale.

Pitch 2 minutes

Aujourd hui, les IA ne font plus seulement du texte. Elles appellent des outils, preparent des actions et s inserent dans des workflows. Obsidia repond au probleme du droit d agir : qui autorise, qui suspend, qui bloque, qui trace, qui prouve et qui rejoue. Le projet possede un Kernel X108, une API bridge, des terrains Bank / Trading / GPS, une couche Brody educative, une memoire Graphiti readonly, une stack de preuves Lean / TLA / Merkle / RFC3161 et une trajectoire WorkCenter pour rendre tout cela exploitable. Le statut reste honnete : POC local avance, pre-industrialisation, pas production-ready sans audit externe.

Ce qu il faut montrer

- le probleme du passage a l action
- le Kernel comme verrou
- la separation d autorite
- les terrains Bank / Trading / GPS
- les preuves et les limites
- la roadmap industrialisation

Ce qu il ne faut pas dire

- production-ready
- certifie
- remplace les LLM
- plus besoin de donnees
- Brody decide
- Graphiti autorise
- l API decide si le Kernel est down

Questions attendues

- Comment le Kernel prend-il une decision ?
- Qu est-ce qui empeche un agent de contourner la gouvernance ?
- Quelles preuves sont formelles et quelles preuves restent a relancer ?
- Comment passer de POC local a industrialisation ?
- Quelle partie est vendable en premier ?

ANNEXE A - ETAT DE PREUVE ET STATUTS

Cette annexe sert a garder une discipline de preuve. Elle doit etre utilisee quand l interlocuteur demande : qu est-ce qui est vraiment prouve, teste, freeze ou simplement documente ?

Statut	Definition presentation	Usage correct
CONFIRMED_BY_RUNTIME	confirme par une sonde live dans la session auditee	a utiliser pour API 8000 et routes live confirmees
CONFIRMED_BY_TEST	confirme par execution de tests	a utiliser pour Lean lake build et Brody 124/124
CONFIRMED_BY_FREEZE	confirme par un freeze date present	a utiliser pour Kernel, Graphiti, UI, decisions capturees
CONFIRMED_BY_CODE	confirme par lecture code	ne pas presenter comme runtime
CONFIRMED_BY_FILE	confirme par presence d un fichier	ne pas presenter comme verification externe
PRESENT_BUT_NOT_EXECUTED	present mais non relance	a utiliser pour TLA/TLC dans ce recheck
DOCUMENTED_ONLY	mentionne par doc sans preuve	a garder comme hypothese ou roadmap

	runtime/code suffisante	
DO_NOT_CLAIM	absence de preuve ou claim interdit	a exclure des supports vente

ANNEXE B - MATRICE DES COUCHES ET VALEUR

Couche	Role	Valeur	Limite
WorkCenter	comprendre, piloter, valider, auditer	rend le projet exploitable par tiers	ne decide pas
Brody	expliquer, former, contextualiser	education et comprehension	non souverain
Graphiti	memoire readonly, relations, contexte	retrouver et relier la connaissance	ne gouverne pas
Domaines	Bank, Trading, GPS, Ecom, CIC, NCP	brancher la gouvernance au terrain	ne decident pas seuls
Sigma	aggregation, votes, contradictions	structure les signaux	ne remplace pas le Kernel
Kernel X108	decision ALLOW / HOLD / BLOCK	autorite souveraine	doit rester encadre par protocole de changement
Proof stack	Lean, TLA, Merkle, RFC3161	verification, audit, scellage	perimetre de preuve a respecter
Freeze / Audit	capturer et figer des etats	preuve historique et replay	ne pas confondre freeze et runtime actif

ANNEXE C - DETAIL DES PROUESSES TECHNIQUES A VULGARISER

Bloc	Prouesse	Comment le dire sans sur-claim
API 8000	164 routes OpenAPI capturees live	montrer la surface deja structuree
Kernel	15 decisions capturees en freeze	montrer une decision gouvernee existante
Bank	5 ALLOW S1	domaine financier branche
Trading	5 BLOCK S4	contradiction et blocage
GPS / Aviation	5 ALLOW S0	terrain critique branche
Brody	124/124 tests PASS	education et guard readonly
Graphiti	167 nodes / 477 relations	memoire navigable frozen
Lean	57 theorems / lemmas	preuve formelle partielle et verifiee

API 8000

Prouesse : 164 routes OpenAPI capturees live.

Formulation conseillee : montrer la surface deja structuree. Ce point doit etre presente comme un actif stabilise dans son perimetre, pas comme une certification globale.

Questions probables : quel est le perimetre exact ? Quel test l a confirme ? Peut-on le rejouer ? Quelle limite reste ouverte ?

Kernel

Prouesse : 15 decisions capturees en freeze.

Formulation conseillee : montrer une decision gouvernee existante. Ce point doit etre presente comme un actif stabilise dans son perimetre, pas comme une certification globale.

Questions probables : quel est le perimetre exact ? Quel test l a confirme ? Peut-on le rejouer ? Quelle limite reste ouverte ?

Bank

Prouesse : 5 ALLOW S1.

Formulation conseillee : domaine financier branche. Ce point doit etre presente comme un actif stabilise dans son perimetre, pas comme une certification globale.

Questions probables : quel est le perimetre exact ? Quel test l a confirme ? Peut-on le rejouer ? Quelle limite reste ouverte ?

Trading

Prouesse : 5 BLOCK S4.

Formulation conseillee : contradiction et blocage. Ce point doit etre presente comme un actif stabilise dans son perimetre, pas comme une certification globale.

Questions probables : quel est le perimetre exact ? Quel test l a confirme ? Peut-on le rejouer ? Quelle limite reste ouverte ?

GPS / Aviation

Prouesse : 5 ALLOW S0.

Formulation conseillee : terrain critique branche. Ce point doit etre presente comme un actif stabilise dans son perimetre, pas comme une certification globale.

Questions probables : quel est le perimetre exact ? Quel test l a confirme ? Peut-on le rejouer ? Quelle limite reste ouverte ?

Brody

Prouesse : 124/124 tests PASS.

Formulation conseillee : education et guard readonly. Ce point doit etre presente comme un actif stabilise dans son perimetre, pas comme une certification globale.

Questions probables : quel est le perimetre exact ? Quel test l a confirme ? Peut-on le rejouer ? Quelle limite reste ouverte ?

Graphiti

Prouesse : 167 nodes / 477 relations.

Formulation conseillee : memoire navigable frozen. Ce point doit etre presente comme un actif stabilise dans son perimetre, pas comme une certification globale.

Questions probables : quel est le perimetre exact ? Quel test l a confirme ? Peut-on le rejouer ? Quelle limite reste ouverte ?

Lean

Prouesse : 57 theorems / lemmas.

Formulation conseillee : preuve formelle partielle et verifiee. Ce point doit etre presente comme un actif stabilise dans son perimetre, pas comme une certification globale.

Questions probables : quel est le perimetre exact ? Quel test l a confirme ? Peut-on le rejouer ? Quelle limite reste ouverte ?

ANNEXE D - DO NOT CLAIM

Cette annexe est obligatoire pour eviter la survente. Elle doit etre relue avant chaque support public.

- Ne pas dire production-ready.
- Ne pas dire certifie banque, finance, aviation, defense ou critique.
- Ne pas dire securite absolue.
- Ne pas dire que Brody decide.
- Ne pas dire que Graphiti decide.
- Ne pas dire que l API decide quand le Kernel est down.
- Ne pas dire que 57 theoremes prouvent tout le systeme de production.
- Ne pas dire que TLA/TLC a ete relance dans le recheck si ce n est pas le cas.
- Ne pas dire que RFC3161 a ete verifie par openssl dans le recheck si ce n est pas le cas.
- Ne pas dire que Ecom est live si seulement present dans code.
- Ne pas dire que l approche supprime tout besoin de GPU ou de donnees.
- Ne pas dire que le Kernel est intouchable pour toujours : dire evolution via protocole audite.

ANNEXE E - FAQ POUR SUSCITER LES QUESTIONS TECHNIQUES

Question	Reponse courte
Pourquoi ne pas laisser le LLM decider ?	Parce que le LLM peut assister, mais le passage a l action doit etre gouverne par une couche deterministe, tracable et verifiable.
Quelle difference avec un agent framework ?	Un agent framework orchestre des capacites. Obsidia ajoute une autorite de decision et de preuve autour du droit d agir.
Quel est le role du Kernel ?	Le Kernel est le verrou qui retourne ALLOW, HOLD ou BLOCK. Il ne doit pas etre remplace par Brody, Graphiti, l API ou l UI.
Pourquoi Bank / Trading / GPS ?	Parce que ces domaines montrent des types differents de risques : transaction, contradiction marche, signal critique terrain.
Pourquoi parler de petite machine ?	Parce que la valeur du coeur vient de la structure, pas seulement du calcul massif. Cela montre une efficience architecturale.
Que manque-t-il avant industrialisation ?	Audit externe, durcissement, CI complete, verification RFC3161, re-run TLA/TLC, tests adversariaux dans le repo exact, packaging WorkCenter.

ANNEXE F - MINI SCRIPTS DE PRESENTATION

Script 30 secondes

Obsidia / X108 est une architecture de gouvernance pour IA et agents autonomes. Le probleme n est plus seulement de generer une reponse, mais de controler le droit d agir. Le Kernel X108 separe recommandation et action en retournant ALLOW, HOLD ou BLOCK. Autour de lui, Brody explique, Graphiti memorise, Sigma agrege, les domaines signalent et la proof stack permet audit et replay. Le projet est un POC local avance en pre-industrialisation, pas une solution critique certifiee.

Script 2 minutes

Les agents IA passent de la conversation a l action : ils peuvent appeler des outils, declencher des workflows et modifier des etats. Dans ce contexte, la question centrale devient : qui autorise, qui suspend, qui bloque, qui trace et qui prouve ? Obsidia repond par une architecture de gouvernance decisionnelle. Son coeur, le Kernel X108, recoit une action candidate et retourne ALLOW, HOLD ou BLOCK selon une priorite securitaire BLOCK > HOLD > ALLOW. Les autres couches ont des roles stricts : Brody explique, Graphiti memorise en readonly, Sigma agrege, l API bridge sans decider, les domaines Bank / Trading / GPS apportent des terrains, et les preuves Lean / TLA / Merkle / RFC3161 structurent la verification. L avantage n est pas de faire un modele plus gros, mais de construire une couche deterministe de controle du passage a l action. Le projet est deja riche techniquement, mais il reste a industrialiser par WorkCenter, education, audit externe et durcissement.

Script 5 minutes

Commencer par le probleme : les IA deviennent capables d agir. Continuer par le risque : il n existe pas toujours une autorite claire entre recommandation et action. Presenter Obsidia comme la couche de gouvernance du droit d agir. Montrer le Kernel X108 et la regle ALLOW / HOLD / BLOCK. Montrer la separation d autorite : ce qui propose ne decide pas, ce qui memorise ne decide pas, ce qui affiche ne decide pas. Montrer les terrains stabilises : Bank, Trading, GPS/Aviation. Montrer Brody comme education non souveraine et Graphiti comme memoire readonly. Montrer les preuves : API runtime, 164 routes, Brody 124/124, Lean 57, freezes, Graphiti frozen. Dire les limites : pas production-ready, audits a relancer, certification non faite. Conclure : dans un monde d agents capables, Obsidia ne vend pas seulement une IA, il vend le controle du passage a l action.

ANNEXE G - ROADMAP DETAILLEE INDUSTRIALISATION

Palier	Travaux
Palier 1 - WorkCenter de comprehension	build cockpit readonly, afficher Kernel/securite/preuves/domaines/memoire
Palier 2 - Education poussee	Brody Education public, dev, audit, investisseur, domaine
Palier 3 - Durcissement technique	TLA/TLC, RFC3161 openssl, adversarial re-run, Neo4j, Ecom live, CI
Palier 4 - Pre-industrialisation	audit externe, release process, freeze industriel, security review
Palier 5 - Pilot POC controle	cas metier limite, donnees simulees ou anonymisees, criteres d acceptation
Palier 6 - Workbench Big Tech level	agents gouvernes, memory governance, proof viewer, replay lab, sandbox, multimodal, audit center

ANNEXE H - TABLE DE CORRESPONDANCE AVEC LES SUPPORTS DEV

Support	Role	Comment il nourrit le support vente
Cahier des charges DEV	decrit le WorkCenter final	sert a montrer la trajectoire produit
Checklist implementation	ordre V0 a V5	sert a montrer la methode d execution
Matrice tickets	tickets page par page	sert a prouver que le build est decoupe
Contrat API	routes et payloads	sert a rassurer CTO / dev senior
Object Model	donnees et registries	sert a montrer que le systeme est structurable
Security Matrix	autorites et interdits	sert a montrer la securite by design

STORYBOARD DETAILLE DU SUPPORT MAITRE

Cette partie transforme le plan en pages de support. Chaque bloc indique ce que la page doit faire comprendre, ce qu elle doit montrer, ce qu elle doit eviter, et quelle preuve sortir si l interlocuteur demande de la profondeur technique.

Page support 01 - Resume executif

Role de la page : Donner en quelques pages la these du projet, le statut reel, les actifs stabilises et la trajectoire.

Message en haut de page

Cette page doit faire comprendre que obsidia / x108 part d une question simple : quand une ia ou un agent peut proposer une action, qui controle le droit d agir ? la reponse n est pas un modele plus gros, ni un chatbot plus bavard.

Narration conseillee

Commencer par une phrase simple, puis montrer le lien avec le Kernel, la securite ou les preuves. Ne pas entrer dans les fichiers si l interlocuteur ne demande pas. L objectif est de declencher une question de profondeur, pas de faire un cours technique complet.

Elements visuels a prevoir

- schema simple en trois blocs : probleme / solution / preuve
- un badge statut
- une question d ouverture

A dire en rendez-vous

Phrase : Obsidia ne cherche pas a remplacer l intelligence des modeles. Il cherche a encadrer le moment ou une intelligence devient action. Cette page montre une brique de cette chaine.

A ne pas dire

- ne pas annoncer production-ready
- ne pas faire croire qu une couche peripherique decide
- ne pas transformer un freeze en runtime actif permanent
- ne pas masquer les limites auditees

Preuve a sortir seulement si on demande

Preuve : utiliser la matrice de statuts et les supports pre-build pour rattacher chaque claim a runtime, test, freeze, code ou fichier.

Question que la page doit provoquer

Question souhaitee : comment ce bloc se connecte-t-il au reste de l architecture et quel niveau de preuve existe deja ?

Page support 02 - Pourquoi maintenant : les agents IA passent a l action

Role de la page : Faire comprendre que le risque se deplace de la reponse vers l action.

Message en haut de page

Cette page doit faire comprendre que le moment est important parce que les ia ne restent plus seulement dans une zone de texte.

Narration conseillee

Commencer par une phrase simple, puis montrer le lien avec le Kernel, la securite ou les preuves. Ne pas entrer dans les fichiers si l interlocuteur ne demande pas. L objectif est de declencher une question de profondeur, pas de faire un cours technique complet.

Elements visuels a prevoir

- schema simple en trois blocs : probleme / solution / preuve
- un badge statut
- une question d ouverture

A dire en rendez-vous

Phrase : Obsidia ne cherche pas a remplacer l intelligence des modeles. Il cherche a encadrer le moment ou une intelligence devient action. Cette page montre une brique de cette chaine.

A ne pas dire

- ne pas annoncer production-ready
- ne pas faire croire qu une couche peripherique decide
- ne pas transformer un freeze en runtime actif permanent
- ne pas masquer les limites auditees

Preuve a sortir seulement si on demande

Preuve : utiliser la matrice de statuts et les supports pre-build pour rattacher chaque claim a runtime, test, freeze, code ou fichier.

Question que la page doit provoquer

Question souhaitee : comment ce bloc se connecte-t-il au reste de l architecture et quel niveau de preuve existe deja ?

Page support 03 - Le probleme marche

Role de la page : Mettre en mots le manque d autorite claire entre recommandation et action.

Message en haut de page

Cette page doit faire comprendre que les organisations ajoutent des agents, de l automatisation, des dashboards et des assistants, mais la chaine d autorite reste souvent floue.

Narration conseillee

Commencer par une phrase simple, puis montrer le lien avec le Kernel, la securite ou les preuves. Ne pas entrer dans les fichiers si l interlocuteur ne demande pas. L objectif est de declencher une question de profondeur, pas de faire un cours technique complet.

Elements visuels a prevoir

- schema simple en trois blocs : probleme / solution / preuve
- un badge statut
- une question d ouverture

A dire en rendez-vous

Phrase : Obsidia ne cherche pas a remplacer l intelligence des modeles. Il cherche a encadrer le moment ou une intelligence devient action. Cette page montre une brique de cette chaine.

A ne pas dire

- ne pas annoncer production-ready
- ne pas faire croire qu une couche peripherique decide

- ne pas transformer un freeze en runtime actif permanent
- ne pas masquer les limites auditees

Preuve a sortir seulement si on demande

Preuve : utiliser la matrice de statuts et les supports pre-build pour rattacher chaque claim a runtime, test, freeze, code ou fichier.

Question que la page doit provoquer

Question souhaitee : comment ce bloc se connecte-t-il au reste de l architecture et quel niveau de preuve existe deja ?

Page support 04 - Le changement de paradigme IA

Role de la page : Passer du modele generatif a l infrastructure de gouvernance.

Message en haut de page

Cette page doit faire comprendre que le paradigme dominant augmente les capacites : plus de parametres, plus de donnees, plus de contexte, plus d outils.

Narration conseillee

Commencer par une phrase simple, puis montrer le lien avec le Kernel, la securite ou les preuves. Ne pas entrer dans les fichiers si l interlocuteur ne demande pas. L objectif est de declencher une question de profondeur, pas de faire un cours technique complet.

Elements visuels a prevoir

- schema simple en trois blocs : probleme / solution / preuve
- un badge statut
- une question d ouverture

A dire en rendez-vous

Phrase : Obsidia ne cherche pas a remplacer l intelligence des modeles. Il cherche a encadrer le moment ou une intelligence devient action. Cette page montre une brique de cette chaine.

A ne pas dire

- ne pas annoncer production-ready
- ne pas faire croire qu une couche peripherique decide
- ne pas transformer un freeze en runtime actif permanent
- ne pas masquer les limites auditees

Preuve a sortir seulement si on demande

Preuve : utiliser la matrice de statuts et les supports pre-build pour rattacher chaque claim a runtime, test, freeze, code ou fichier.

Question que la page doit provoquer

Question souhaitee : comment ce bloc se connecte-t-il au reste de l architecture et quel niveau de preuve existe deja ?

Page support 05 - Pourquoi les agents ont besoin d une autorite

Role de la page : Montrer que tool-use, memoire, workflows et automatisation exigent une autorite separee.

Message en haut de page

Cette page doit faire comprendre que un agent peut etre tres utile et rester dangereux si son autorite est mal definie.

Narration conseillee

Commencer par une phrase simple, puis montrer le lien avec le Kernel, la securite ou les preuves. Ne pas entrer dans les fichiers si l interlocuteur ne demande pas. L objectif est de declencher une question de profondeur, pas de faire un cours technique complet.

Elements visuels a prevoir

- un bloc Kernel X108 au centre
- des fleches entrantes venant des domaines
- des fleches sortantes vers ALLOW / HOLD / BLOCK
- des cadenas sur API, Brody, Graphiti et WorkCenter

A dire en rendez-vous

Phrase : Obsidia ne cherche pas a remplacer l intelligence des modeles. Il cherche a encadrer le moment ou une intelligence devient action. Cette page montre une brique de cette chaine.

A ne pas dire

- ne pas annoncer production-ready
- ne pas faire croire qu une couche peripherique decide
- ne pas transformer un freeze en runtime actif permanent
- ne pas masquer les limites auditees

Preuve a sortir seulement si on demande

Preuve : decision_authority=KX108_ONLY, endpoint /kernel/ragnarok, boundary flags false, decisions capturees en freeze.

Question que la page doit provoquer

Question souhaitee : comment ce bloc se connecte-t-il au reste de l architecture et quel niveau de preuve existe deja ?

Page support 06 - Ce qu est Obsidia / X108

Role de la page : Definir Obsidia comme architecture de gouvernance decisionnelle.

Message en haut de page

Cette page doit faire comprendre que obsidia / x108 est une architecture de gouvernance decisionnelle pour systemes ia, agents autonomes et workflows critiques.

Narration conseillee

Commencer par une phrase simple, puis montrer le lien avec le Kernel, la securite ou les preuves. Ne pas entrer dans les fichiers si l interlocuteur ne demande pas. L objectif est de declencher une question de profondeur, pas de faire un cours technique complet.

Elements visuels a prevoir

- schema simple en trois blocs : probleme / solution / preuve
- un badge statut
- une question d ouverture

A dire en rendez-vous

Phrase : Obsidia ne cherche pas a remplacer l intelligence des modeles. Il cherche a encadrer le moment ou une intelligence devient action. Cette page montre une brique de cette chaine.

A ne pas dire

- ne pas annoncer production-ready
- ne pas faire croire qu une couche peripherique decide

- ne pas transformer un freeze en runtime actif permanent
- ne pas masquer les limites auditees

Preuve a sortir seulement si on demande

Preuve : utiliser la matrice de statuts et les supports pre-build pour rattacher chaque claim a runtime, test, freeze, code ou fichier.

Question que la page doit provoquer

Question souhaitee : comment ce bloc se connecte-t-il au reste de l architecture et quel niveau de preuve existe deja ?

Page support 07 - Ce qu Obsidia n est pas

Role de la page : Eviter les confusions : chatbot, wrapper, simple UI, modele plus gros, outil de monitoring seul.

Message en haut de page

Cette page doit faire comprendre que obsidia n est pas un chatbot.

Narration conseillee

Commencer par une phrase simple, puis montrer le lien avec le Kernel, la securite ou les preuves. Ne pas entrer dans les fichiers si l interlocuteur ne demande pas. L objectif est de declencher une question de profondeur, pas de faire un cours technique complet.

Elements visuels a prevoir

- schema simple en trois blocs : probleme / solution / preuve
- un badge statut
- une question d ouverture

A dire en rendez-vous

Phrase : Obsidia ne cherche pas a remplacer l intelligence des modeles. Il cherche a encadrer le moment ou une intelligence devient action. Cette page montre une brique de cette chaine.

A ne pas dire

- ne pas annoncer production-ready
- ne pas faire croire qu une couche peripherique decide
- ne pas transformer un freeze en runtime actif permanent
- ne pas masquer les limites auditees

Preuve a sortir seulement si on demande

Preuve : utiliser la matrice de statuts et les supports pre-build pour rattacher chaque claim a runtime, test, freeze, code ou fichier.

Question que la page doit provoquer

Question souhaitee : comment ce bloc se connecte-t-il au reste de l architecture et quel niveau de preuve existe deja ?

Page support 08 - Architecture en une page

Role de la page : Presenter la chaine Signal -> Domaines -> Sigma -> Kernel -> Proof -> Replay -> WorkCenter.

Message en haut de page

Cette page doit faire comprendre que l architecture doit etre lue comme une chaine de gouvernance.

Narration conseillee

Commencer par une phrase simple, puis montrer le lien avec le Kernel, la securite ou les preuves. Ne pas entrer dans les fichiers si l interlocuteur ne demande pas. L objectif est de declencher une question de profondeur, pas de faire un cours technique complet.

Elements visuels a prevoir

- schema simple en trois blocs : probleme / solution / preuve
- un badge statut
- une question d ouverture

A dire en rendez-vous

Phrase : Obsidia ne cherche pas a remplacer l intelligence des modeles. Il cherche a encadrer le moment ou une intelligence devient action. Cette page montre une brique de cette chaine.

A ne pas dire

- ne pas annoncer production-ready
- ne pas faire croire qu une couche peripherique decide
- ne pas transformer un freeze en runtime actif permanent
- ne pas masquer les limites auditees

Preuve a sortir seulement si on demande

Preuve : utiliser la matrice de statuts et les supports pre-build pour rattacher chaque claim a runtime, test, freeze, code ou fichier.

Question que la page doit provoquer

Question souhaitee : comment ce bloc se connecte-t-il au reste de l architecture et quel niveau de preuve existe deja ?

Page support 09 - Le Kernel X108

Role de la page : Montrer le coeur de securite et d autorite.

Message en haut de page

Cette page doit faire comprendre que le kernel est le coeur du support.

Narration conseillee

Commencer par une phrase simple, puis montrer le lien avec le Kernel, la securite ou les preuves. Ne pas entrer dans les fichiers si l interlocuteur ne demande pas. L objectif est de declencher une question de profondeur, pas de faire un cours technique complet.

Elements visuels a prevoir

- un bloc Kernel X108 au centre
- des fleches entrantes venant des domaines
- des fleches sortantes vers ALLOW / HOLD / BLOCK
- des cadenas sur API, Brody, Graphiti et WorkCenter

A dire en rendez-vous

Phrase : Obsidia ne cherche pas a remplacer l intelligence des modeles. Il cherche a encadrer le moment ou une intelligence devient action. Cette page montre une brique de cette chaine.

A ne pas dire

- ne pas annoncer production-ready
- ne pas faire croire qu une couche peripherique decide

- ne pas transformer un freeze en runtime actif permanent
- ne pas masquer les limites auditees

Preuve a sortir seulement si on demande

Preuve : decision_authority=KX108_ONLY, endpoint /kernel/ragnarok, boundary flags false, decisions capturees en freeze.

Question que la page doit provoquer

Question souhaitee : comment ce bloc se connecte-t-il au reste de l architecture et quel niveau de preuve existe deja ?

Page support 10 - La regle BLOCK > HOLD > ALLOW

Role de la page : Expliquer la priorite fail-closed sans surcharge technique.

Message en haut de page

Cette page doit faire comprendre que la regle explique la philosophie de securite.

Narration conseillee

Commencer par une phrase simple, puis montrer le lien avec le Kernel, la securite ou les preuves. Ne pas entrer dans les fichiers si l interlocuteur ne demande pas. L objectif est de declencher une question de profondeur, pas de faire un cours technique complet.

Elements visuels a prevoir

- un bloc Kernel X108 au centre
- des fleches entrantes venant des domaines
- des fleches sortantes vers ALLOW / HOLD / BLOCK
- des cadenas sur API, Brody, Graphiti et WorkCenter

A dire en rendez-vous

Phrase : Obsidia ne cherche pas a remplacer l intelligence des modeles. Il cherche a encadrer le moment ou une intelligence devient action. Cette page montre une brique de cette chaine.

A ne pas dire

- ne pas annoncer production-ready
- ne pas faire croire qu une couche peripherique decide
- ne pas transformer un freeze en runtime actif permanent
- ne pas masquer les limites auditees

Preuve a sortir seulement si on demande

Preuve : decision_authority=KX108_ONLY, endpoint /kernel/ragnarok, boundary flags false, decisions capturees en freeze.

Question que la page doit provoquer

Question souhaitee : comment ce bloc se connecte-t-il au reste de l architecture et quel niveau de preuve existe deja ?

Page support 11 - Separation d autorite

Role de la page : Rendre visible que chaque couche a un role et une limite.

Message en haut de page

Cette page doit faire comprendre que la securite d obsidia ne vient pas seulement d une regle.

Narration conseillee

Commencer par une phrase simple, puis montrer le lien avec le Kernel, la securite ou les preuves. Ne pas entrer dans les fichiers si l interlocuteur ne demande pas. L objectif est de declencher une question de profondeur, pas de faire un cours technique complet.

Elements visuels a prevoir

- un bloc Kernel X108 au centre
- des fleches entrantes venant des domaines
- des fleches sortantes vers ALLOW / HOLD / BLOCK
- des cadenas sur API, Brody, Graphiti et WorkCenter

A dire en rendez-vous

Phrase : Obsidia ne cherche pas a remplacer l intelligence des modeles. Il cherche a encadrer le moment ou une intelligence devient action. Cette page montre une brique de cette chaine.

A ne pas dire

- ne pas annoncer production-ready
- ne pas faire croire qu une couche peripherique decide
- ne pas transformer un freeze en runtime actif permanent
- ne pas masquer les limites auditees

Preuve a sortir seulement si on demande

Preuve : decision_authority=KX108_ONLY, endpoint /kernel/ragnarok, boundary flags false, decisions capturees en freeze.

Question que la page doit provoquer

Question souhaitee : comment ce bloc se connecte-t-il au reste de l architecture et quel niveau de preuve existe deja ?

Page support 12 - Security by design

Role de la page : Montrer les boundaries, les interdits et les garanties de non-contournement.

Message en haut de page

Cette page doit faire comprendre que le support doit montrer les flags de boundary : allowed_to_decide=false, emits_act=false, emits_verdict=false, memory_write=false, graphiti_write=false, kernel_mutation=false, real_action=false.

Narration conseillee

Commencer par une phrase simple, puis montrer le lien avec le Kernel, la securite ou les preuves. Ne pas entrer dans les fichiers si l interlocuteur ne demande pas. L objectif est de declencher une question de profondeur, pas de faire un cours technique complet.

Elements visuels a prevoir

- un bloc Kernel X108 au centre
- des fleches entrantes venant des domaines
- des fleches sortantes vers ALLOW / HOLD / BLOCK
- des cadenas sur API, Brody, Graphiti et WorkCenter

A dire en rendez-vous

Phrase : Obsidia ne cherche pas a remplacer l intelligence des modeles. Il cherche a encadrer le moment ou une intelligence devient action. Cette page montre une brique de cette chaine.

A ne pas dire

- ne pas annoncer production-ready

- ne pas faire croire qu une couche peripherique decide
- ne pas transformer un freeze en runtime actif permanent
- ne pas masquer les limites auditees

Preuve a sortir seulement si on demande

Preuve : decision_authority=KX108_ONLY, endpoint /kernel/ragnarok, boundary flags false, decisions capturees en freeze.

Question que la page doit provoquer

Question souhaitee : comment ce bloc se connecte-t-il au reste de l architecture et quel niveau de preuve existe deja ?

Page support 13 - Les terrains stables : Bank, Trading, GPS/Aviation

Role de la page : Montrer les trois branchements terrain deja validates par freeze et runtime API.

Message en haut de page

Cette page doit faire comprendre que les trois terrains deja branches sont essentiels pour susciter l interet.

Narration conseillee

Commencer par une phrase simple, puis montrer le lien avec le Kernel, la securite ou les preuves. Ne pas entrer dans les fichiers si l interlocuteur ne demande pas. L objectif est de declencher une question de profondeur, pas de faire un cours technique complet.

Elements visuels a prevoir

- trois colonnes Bank / Trading / GPS
- un badge par decision freeze
- une ligne commune vers Kernel X108
- un message : meme doctrine, terrains differents

A dire en rendez-vous

Phrase : Obsidia ne cherche pas a remplacer l intelligence des modeles. Il cherche a encadrer le moment ou une intelligence devient action. Cette page montre une brique de cette chaine.

A ne pas dire

- ne pas annoncer production-ready
- ne pas faire croire qu une couche peripherique decide
- ne pas transformer un freeze en runtime actif permanent
- ne pas masquer les limites auditees

Preuve a sortir seulement si on demande

Preuve : Bank 5 ALLOW S1, Trading 5 BLOCK S4, GPS/Aviation 5 ALLOW S0, bridges API bank/trading/gps confirmes runtime cote API.

Question que la page doit provoquer

Question souhaitee : comment ce bloc se connecte-t-il au reste de l architecture et quel niveau de preuve existe deja ?

Page support 14 - Brody : pseudo-LLM educatif et assistant non souverain

Role de la page : Expliquer la valeur de Brody sans lui donner d autorite.

Message en haut de page

Cette page doit faire comprendre que brody doit etre presente comme une couche d intelligence assistante encadree.

Narration conseillee

Commencer par une phrase simple, puis montrer le lien avec le Kernel, la securite ou les preuves. Ne pas entrer dans les fichiers si l interlocuteur ne demande pas. L objectif est de declencher une question de profondeur, pas de faire un cours technique complet.

Elements visuels a prevoir

- un assistant Brody a gauche
- des bulles expliquer / eduquer / contextualiser
- une barriere rouge devant decide / act / write
- un lien vers Human Validation

A dire en rendez-vous

Phrase : Obsidia ne cherche pas a remplacer l intelligence des modeles. Il cherche a encadrer le moment ou une intelligence devient action. Cette page montre une brique de cette chaine.

A ne pas dire

- ne pas annoncer production-ready
- ne pas faire croire qu une couche peripherique decide
- ne pas transformer un freeze en runtime actif permanent
- ne pas masquer les limites auditees

Preuve a sortir seulement si on demande

Preuve : Brody Education Pack V1 readonly, 124/124 tests PASS, memory_write=false, emits_act=false, pack injection blocked.

Question que la page doit provoquer

Question souhaitee : comment ce bloc se connecte-t-il au reste de l architecture et quel niveau de preuve existe deja ?

Page support 15 - Graphiti : memoire readonly non souveraine

Role de la page : Expliquer la memoire comme sidecar de contexte, pas comme decideur.

Message en haut de page

Cette page doit faire comprendre que graphiti apporte la memoire, les relations et le contexte.

Narration conseillee

Commencer par une phrase simple, puis montrer le lien avec le Kernel, la securite ou les preuves. Ne pas entrer dans les fichiers si l interlocuteur ne demande pas. L objectif est de declencher une question de profondeur, pas de faire un cours technique complet.

Elements visuels a prevoir

- un graphe memoire
- un badge FROZEN_READONLY
- des noeuds relies au contexte
- une barriere vers decision

A dire en rendez-vous

Phrase : Obsidia ne cherche pas a remplacer l intelligence des modeles. Il cherche a encadrer le moment ou une intelligence devient action. Cette page montre une brique de cette chaine.

A ne pas dire

- ne pas annoncer production-ready

- ne pas faire croire qu une couche peripherique decide
- ne pas transformer un freeze en runtime actif permanent
- ne pas masquer les limites auditees

Preuve a sortir seulement si on demande

Preuve : snapshot frozen 167 nodes, 477 relations, 20 episodes, mode FROZEN_READONLY, live Neo4j dependency false dans snapshot.

Question que la page doit provoquer

Question souhaitee : comment ce bloc se connecte-t-il au reste de l architecture et quel niveau de preuve existe deja ?

Page support 16 - Sigma : aggregation et signaux

Role de la page : Expliquer la couche qui structure les signaux et votes avant le Kernel.

Message en haut de page

Cette page doit faire comprendre que sigma structure les signaux.

Narration conseillee

Commencer par une phrase simple, puis montrer le lien avec le Kernel, la securite ou les preuves. Ne pas entrer dans les fichiers si l interlocuteur ne demande pas. L objectif est de declencher une question de profondeur, pas de faire un cours technique complet.

Elements visuels a prevoir

- schema simple en trois blocs : probleme / solution / preuve
- un badge statut
- une question d ouverture

A dire en rendez-vous

Phrase : Obsidia ne cherche pas a remplacer l intelligence des modeles. Il cherche a encadrer le moment ou une intelligence devient action. Cette page montre une brique de cette chaine.

A ne pas dire

- ne pas annoncer production-ready
- ne pas faire croire qu une couche peripherique decide
- ne pas transformer un freeze en runtime actif permanent
- ne pas masquer les limites auditees

Preuve a sortir seulement si on demande

Preuve : utiliser la matrice de statuts et les supports pre-build pour rattacher chaque claim a runtime, test, freeze, code ou fichier.

Question que la page doit provoquer

Question souhaitee : comment ce bloc se connecte-t-il au reste de l architecture et quel niveau de preuve existe deja ?

Page support 17 - Proof stack Lean / TLA / Merkle / RFC3161

Role de la page : Presenter les preuves stabilisees et les limites honnetes.

Message en haut de page

Cette page doit faire comprendre que la proof stack est un actif de credibilite.

Narration conseillee

Commencer par une phrase simple, puis montrer le lien avec le Kernel, la securite ou les preuves. Ne pas entrer dans les fichiers si l interlocuteur ne demande pas. L objectif est de declencher une question de profondeur, pas de faire un cours technique complet.

Elements visuels a prevoir

- une pile Lean / TLA / Merkle / RFC3161
- badges confirmed / present / not run
- une ligne proof -> replay -> audit

A dire en rendez-vous

Phrase : Obsidia ne cherche pas a remplacer l intelligence des modeles. Il cherche a encadrer le moment ou une intelligence devient action. Cette page montre une brique de cette chaine.

A ne pas dire

- ne pas annoncer production-ready
- ne pas faire croire qu une couche peripherique decide
- ne pas transformer un freeze en runtime actif permanent
- ne pas masquer les limites auditees

Preuve a sortir seulement si on demande

Preuve : Lean 57 theorem / lemma, lake build PASS ; TLA present mais TLC a relancer ; RFC3161 present mais openssl verify a relancer.

Question que la page doit provoquer

Question souhaitee : comment ce bloc se connecte-t-il au reste de l architecture et quel niveau de preuve existe deja ?

Page support 18 - Pourquoi deterministe

Role de la page : Defendre le choix structurel : invariants, gates, replay, statuts.

Message en haut de page

Cette page doit faire comprendre que le choix deterministe repond a un besoin de gouvernance.

Narration conseillee

Commencer par une phrase simple, puis montrer le lien avec le Kernel, la securite ou les preuves. Ne pas entrer dans les fichiers si l interlocuteur ne demande pas. L objectif est de declencher une question de profondeur, pas de faire un cours technique complet.

Elements visuels a prevoir

- schema simple en trois blocs : probleme / solution / preuve
- un badge statut
- une question d ouverture

A dire en rendez-vous

Phrase : Obsidia ne cherche pas a remplacer l intelligence des modeles. Il cherche a encadrer le moment ou une intelligence devient action. Cette page montre une brique de cette chaine.

A ne pas dire

- ne pas annoncer production-ready
- ne pas faire croire qu une couche peripherique decide
- ne pas transformer un freeze en runtime actif permanent

- ne pas masquer les limites auditees

Preuve a sortir seulement si on demande

Preuve : utiliser la matrice de statuts et les supports pre-build pour rattacher chaque claim a runtime, test, freeze, code ou fichier.

Question que la page doit provoquer

Question souhaitee : comment ce bloc se connecte-t-il au reste de l architecture et quel niveau de preuve existe deja ?

Page support 19 - Pourquoi pas dependant du big training

Role de la page : Expliquer l avantage structurel sans pretendre remplacer les LLM.

Message en haut de page

Cette page doit faire comprendre que le coeur de valeur d obsidia ne depend pas d un entrainement massif.

Narration conseillee

Commencer par une phrase simple, puis montrer le lien avec le Kernel, la securite ou les preuves. Ne pas entrer dans les fichiers si l interlocuteur ne demande pas. L objectif est de declencher une question de profondeur, pas de faire un cours technique complet.

Elements visuels a prevoir

- schema simple en trois blocs : probleme / solution / preuve
- un badge statut
- une question d ouverture

A dire en rendez-vous

Phrase : Obsidia ne cherche pas a remplacer l intelligence des modeles. Il cherche a encadrer le moment ou une intelligence devient action. Cette page montre une brique de cette chaine.

A ne pas dire

- ne pas annoncer production-ready
- ne pas faire croire qu une couche peripherique decide
- ne pas transformer un freeze en runtime actif permanent
- ne pas masquer les limites auditees

Preuve a sortir seulement si on demande

Preuve : utiliser la matrice de statuts et les supports pre-build pour rattacher chaque claim a runtime, test, freeze, code ou fichier.

Question que la page doit provoquer

Question souhaitee : comment ce bloc se connecte-t-il au reste de l architecture et quel niveau de preuve existe deja ?

Page support 20 - Gain calcul / cout : structure > masse

Role de la page : Montrer pourquoi une approche structurelle reduit la pression GPU et data sur le coeur decisionnel.

Message en haut de page

Cette page doit faire comprendre que l argument cout doit etre presente proprement.

Narration conseillee

Commencer par une phrase simple, puis montrer le lien avec le Kernel, la securite ou les preuves. Ne pas entrer dans les fichiers si l interlocuteur ne demande pas. L objectif est de declencher une question de profondeur, pas de faire un cours technique complet.

Elements visuels a prevoir

- schema simple en trois blocs : probleme / solution / preuve
- un badge statut
- une question d ouverture

A dire en rendez-vous

Phrase : Obsidia ne cherche pas a remplacer l intelligence des modeles. Il cherche a encadrer le moment ou une intelligence devient action. Cette page montre une brique de cette chaine.

A ne pas dire

- ne pas annoncer production-ready
- ne pas faire croire qu une couche peripherique decide
- ne pas transformer un freeze en runtime actif permanent
- ne pas masquer les limites auditees

Preuve a sortir seulement si on demande

Preuve : utiliser la matrice de statuts et les supports pre-build pour rattacher chaque claim a runtime, test, freeze, code ou fichier.

Question que la page doit provoquer

Question souhaitee : comment ce bloc se connecte-t-il au reste de l architecture et quel niveau de preuve existe deja ?

Page support 21 - Petite machine, architecture lourde

Role de la page : Montrer ce qui a deja ete construit localement et ce que cela prouve.

Message en haut de page

Cette page doit faire comprendre que le message est fort : avec une machine locale, il est possible de construire un systeme avance de gouvernance, de preuve et de pilotage.

Narration conseillee

Commencer par une phrase simple, puis montrer le lien avec le Kernel, la securite ou les preuves. Ne pas entrer dans les fichiers si l interlocuteur ne demande pas. L objectif est de declencher une question de profondeur, pas de faire un cours technique complet.

Elements visuels a prevoir

- schema simple en trois blocs : probleme / solution / preuve
- un badge statut
- une question d ouverture

A dire en rendez-vous

Phrase : Obsidia ne cherche pas a remplacer l intelligence des modeles. Il cherche a encadrer le moment ou une intelligence devient action. Cette page montre une brique de cette chaine.

A ne pas dire

- ne pas annoncer production-ready
- ne pas faire croire qu une couche peripherique decide

- ne pas transformer un freeze en runtime actif permanent
- ne pas masquer les limites auditees

Preuve a sortir seulement si on demande

Preuve : utiliser la matrice de statuts et les supports pre-build pour rattacher chaque claim a runtime, test, freeze, code ou fichier.

Question que la page doit provoquer

Question souhaitee : comment ce bloc se connecte-t-il au reste de l architecture et quel niveau de preuve existe deja ?

Page support 22 - Ce qui est deja stabilise

Role de la page : Synthese des actifs confirmed runtime, test, freeze, code.

Message en haut de page

Cette page doit faire comprendre que cette section doit etre factuelle.

Narration conseillee

Commencer par une phrase simple, puis montrer le lien avec le Kernel, la securite ou les preuves. Ne pas entrer dans les fichiers si l interlocuteur ne demande pas. L objectif est de declencher une question de profondeur, pas de faire un cours technique complet.

Elements visuels a prevoir

- schema simple en trois blocs : probleme / solution / preuve
- un badge statut
- une question d ouverture

A dire en rendez-vous

Phrase : Obsidia ne cherche pas a remplacer l intelligence des modeles. Il cherche a encadrer le moment ou une intelligence devient action. Cette page montre une brique de cette chaine.

A ne pas dire

- ne pas annoncer production-ready
- ne pas faire croire qu une couche peripherique decide
- ne pas transformer un freeze en runtime actif permanent
- ne pas masquer les limites auditees

Preuve a sortir seulement si on demande

Preuve : utiliser la matrice de statuts et les supports pre-build pour rattacher chaque claim a runtime, test, freeze, code ou fichier.

Question que la page doit provoquer

Question souhaitee : comment ce bloc se connecte-t-il au reste de l architecture et quel niveau de preuve existe deja ?

Page support 23 - Limites honnetes

Role de la page : Dire clairement ce qui reste a relancer, auditer, durcir et certifier.

Message en haut de page

Cette page doit faire comprendre que cette section protege le projet.

Narration conseillee

Commencer par une phrase simple, puis montrer le lien avec le Kernel, la securite ou les preuves. Ne pas entrer dans les fichiers si l interlocuteur ne demande pas. L objectif est de declencher une question de profondeur, pas de faire un cours technique complet.

Elements visuels a prevoir

- schema simple en trois blocs : probleme / solution / preuve
- un badge statut
- une question d ouverture

A dire en rendez-vous

Phrase : Obsidia ne cherche pas a remplacer l intelligence des modeles. Il cherche a encadrer le moment ou une intelligence devient action. Cette page montre une brique de cette chaine.

A ne pas dire

- ne pas annoncer production-ready
- ne pas faire croire qu une couche peripherique decide
- ne pas transformer un freeze en runtime actif permanent
- ne pas masquer les limites auditees

Preuve a sortir seulement si on demande

Preuve : utiliser la matrice de statuts et les supports pre-build pour rattacher chaque claim a runtime, test, freeze, code ou fichier.

Question que la page doit provoquer

Question souhaitee : comment ce bloc se connecte-t-il au reste de l architecture et quel niveau de preuve existe deja ?

Page support 24 - Etat actuel du projet

Role de la page : Positionner le projet en POC local avance / pre-industrialisation.

Message en haut de page

Cette page doit faire comprendre que l etat actuel est : poc local avance, recherche appliquee, pre-industrialisation.

Narration conseillee

Commencer par une phrase simple, puis montrer le lien avec le Kernel, la securite ou les preuves. Ne pas entrer dans les fichiers si l interlocuteur ne demande pas. L objectif est de declencher une question de profondeur, pas de faire un cours technique complet.

Elements visuels a prevoir

- schema simple en trois blocs : probleme / solution / preuve
- un badge statut
- une question d ouverture

A dire en rendez-vous

Phrase : Obsidia ne cherche pas a remplacer l intelligence des modeles. Il cherche a encadrer le moment ou une intelligence devient action. Cette page montre une brique de cette chaine.

A ne pas dire

- ne pas annoncer production-ready
- ne pas faire croire qu une couche peripherique decide
- ne pas transformer un freeze en runtime actif permanent

- ne pas masquer les limites auditees

Preuve a sortir seulement si on demande

Preuve : utiliser la matrice de statuts et les supports pre-build pour rattacher chaque claim a runtime, test, freeze, code ou fichier.

Question que la page doit provoquer

Question souhaitee : comment ce bloc se connecte-t-il au reste de l architecture et quel niveau de preuve existe deja ?

Page support 25 - Dernieres etapes avant industrialisation

Role de la page : Lister les paliers avant audit externe, CI, pilot POC, certification.

Message en haut de page

Cette page doit faire comprendre que les dernieres etapes doivent etre decoupees en paliers.

Narration conseillee

Commencer par une phrase simple, puis montrer le lien avec le Kernel, la securite ou les preuves. Ne pas entrer dans les fichiers si l interlocuteur ne demande pas. L objectif est de declencher une question de profondeur, pas de faire un cours technique complet.

Elements visuels a prevoir

- schema simple en trois blocs : probleme / solution / preuve
- un badge statut
- une question d ouverture

A dire en rendez-vous

Phrase : Obsidia ne cherche pas a remplacer l intelligence des modeles. Il cherche a encadrer le moment ou une intelligence devient action. Cette page montre une brique de cette chaine.

A ne pas dire

- ne pas annoncer production-ready
- ne pas faire croire qu une couche peripherique decide
- ne pas transformer un freeze en runtime actif permanent
- ne pas masquer les limites auditees

Preuve a sortir seulement si on demande

Preuve : utiliser la matrice de statuts et les supports pre-build pour rattacher chaque claim a runtime, test, freeze, code ou fichier.

Question que la page doit provoquer

Question souhaitee : comment ce bloc se connecte-t-il au reste de l architecture et quel niveau de preuve existe deja ?

Page support 26 - Roadmap WorkCenter

Role de la page : Transformer les supports dev en cockpit concret.

Message en haut de page

Cette page doit faire comprendre que le workcenter est le passage entre laboratoire et systeme exploitable.

Narration conseillee

Commencer par une phrase simple, puis montrer le lien avec le Kernel, la securite ou les preuves. Ne pas entrer dans les fichiers si l interlocuteur ne demande pas. L objectif est de declencher une question de profondeur, pas de faire un cours technique complet.

Elements visuels a prevoir

- sidebar cockpit
- cartes Kernel / Security / Domains / Proof / Memory
- timeline freeze
- panneau validation

A dire en rendez-vous

Phrase : Obsidia ne cherche pas a remplacer l intelligence des modeles. Il cherche a encadrer le moment ou une intelligence devient action. Cette page montre une brique de cette chaine.

A ne pas dire

- ne pas annoncer production-ready
- ne pas faire croire qu une couche peripherique decide
- ne pas transformer un freeze en runtime actif permanent
- ne pas masquer les limites auditees

Preuve a sortir seulement si on demande

Preuve : utiliser la matrice de statuts et les supports pre-build pour rattacher chaque claim a runtime, test, freeze, code ou fichier.

Question que la page doit provoquer

Question souhaitee : comment ce bloc se connecte-t-il au reste de l architecture et quel niveau de preuve existe deja ?

Page support 27 - Roadmap education poussee

Role de la page : Structurer Brody Education, modules, parcours publics.

Message en haut de page

Cette page doit faire comprendre que l education est une couche strategique.

Narration conseillee

Commencer par une phrase simple, puis montrer le lien avec le Kernel, la securite ou les preuves. Ne pas entrer dans les fichiers si l interlocuteur ne demande pas. L objectif est de declencher une question de profondeur, pas de faire un cours technique complet.

Elements visuels a prevoir

- schema simple en trois blocs : probleme / solution / preuve
- un badge statut
- une question d ouverture

A dire en rendez-vous

Phrase : Obsidia ne cherche pas a remplacer l intelligence des modeles. Il cherche a encadrer le moment ou une intelligence devient action. Cette page montre une brique de cette chaine.

A ne pas dire

- ne pas annoncer production-ready
- ne pas faire croire qu une couche peripherique decide

- ne pas transformer un freeze en runtime actif permanent
- ne pas masquer les limites auditees

Preuve a sortir seulement si on demande

Preuve : utiliser la matrice de statuts et les supports pre-build pour rattacher chaque claim a runtime, test, freeze, code ou fichier.

Question que la page doit provoquer

Question souhaitee : comment ce bloc se connecte-t-il au reste de l architecture et quel niveau de preuve existe deja ?

Page support 28 - Roadmap audit externe

Role de la page : Preparer preuves, limitations, test packs, release process.

Message en haut de page

Cette page doit faire comprendre que avant usage critique, le projet doit produire un pack audit externe : preuves relancees, limites, manifests, hashes, ci, tests adversariaux, verification rfc3161, re-run tla/tlc, security review.

Narration conseillee

Commencer par une phrase simple, puis montrer le lien avec le Kernel, la securite ou les preuves. Ne pas entrer dans les fichiers si l interlocuteur ne demande pas. L objectif est de declencher une question de profondeur, pas de faire un cours technique complet.

Elements visuels a prevoir

- schema simple en trois blocs : probleme / solution / preuve
- un badge statut
- une question d ouverture

A dire en rendez-vous

Phrase : Obsidia ne cherche pas a remplacer l intelligence des modeles. Il cherche a encadrer le moment ou une intelligence devient action. Cette page montre une brique de cette chaine.

A ne pas dire

- ne pas annoncer production-ready
- ne pas faire croire qu une couche peripherique decide
- ne pas transformer un freeze en runtime actif permanent
- ne pas masquer les limites auditees

Preuve a sortir seulement si on demande

Preuve : utiliser la matrice de statuts et les supports pre-build pour rattacher chaque claim a runtime, test, freeze, code ou fichier.

Question que la page doit provoquer

Question souhaitee : comment ce bloc se connecte-t-il au reste de l architecture et quel niveau de preuve existe deja ?

Page support 29 - Roadmap Big Tech Workbench

Role de la page : Decrire la trajectoire vers un environnement de travail gouverne de niveau industriel.

Message en haut de page

Cette page doit faire comprendre que le workbench cible n est pas seulement un terminal enrichi.

Narration conseillee

Commencer par une phrase simple, puis montrer le lien avec le Kernel, la securite ou les preuves. Ne pas entrer dans les fichiers si l interlocuteur ne demande pas. L objectif est de declencher une question de profondeur, pas de faire un cours technique complet.

Elements visuels a prevoir

- sidebar cockpit
- cartes Kernel / Security / Domains / Proof / Memory
- timeline freeze
- panneau validation

A dire en rendez-vous

Phrase : Obsidia ne cherche pas a remplacer l intelligence des modeles. Il cherche a encadrer le moment ou une intelligence devient action. Cette page montre une brique de cette chaine.

A ne pas dire

- ne pas annoncer production-ready
- ne pas faire croire qu une couche peripherique decide
- ne pas transformer un freeze en runtime actif permanent
- ne pas masquer les limites auditees

Preuve a sortir seulement si on demande

Preuve : utiliser la matrice de statuts et les supports pre-build pour rattacher chaque claim a runtime, test, freeze, code ou fichier.

Question que la page doit provoquer

Question souhaitee : comment ce bloc se connecte-t-il au reste de l architecture et quel niveau de preuve existe deja ?

Page support 30 - Positionnement concurrentiel

Role de la page : Positionner Obsidia face aux LLM, agent frameworks, MLOps, RAG et monitoring.

Message en haut de page

Cette page doit faire comprendre que obsidia ne doit pas se positionner frontalement comme un modele plus gros.

Narration conseillee

Commencer par une phrase simple, puis montrer le lien avec le Kernel, la securite ou les preuves. Ne pas entrer dans les fichiers si l interlocuteur ne demande pas. L objectif est de declencher une question de profondeur, pas de faire un cours technique complet.

Elements visuels a prevoir

- schema simple en trois blocs : probleme / solution / preuve
- un badge statut
- une question d ouverture

A dire en rendez-vous

Phrase : Obsidia ne cherche pas a remplacer l intelligence des modeles. Il cherche a encadrer le moment ou une intelligence devient action. Cette page montre une brique de cette chaine.

A ne pas dire

- ne pas annoncer production-ready
- ne pas faire croire qu une couche peripherique decide

- ne pas transformer un freeze en runtime actif permanent
- ne pas masquer les limites auditees

Preuve a sortir seulement si on demande

Preuve : utiliser la matrice de statuts et les supports pre-build pour rattacher chaque claim a runtime, test, freeze, code ou fichier.

Question que la page doit provoquer

Question souhaitee : comment ce bloc se connecte-t-il au reste de l architecture et quel niveau de preuve existe deja ?

Page support 31 - Pourquoi Obsidia peut rejoindre puis depasser

Role de la page : Formuler l angle : pas la taille du modele, mais le controle du passage a l action.

Message en haut de page

Cette page doit faire comprendre que rejoindre signifie construire les outils de travail, de memoire, de preuve et de sandbox au niveau des meilleurs environnements.

Narration conseillee

Commencer par une phrase simple, puis montrer le lien avec le Kernel, la securite ou les preuves. Ne pas entrer dans les fichiers si l interlocuteur ne demande pas. L objectif est de declencher une question de profondeur, pas de faire un cours technique complet.

Elements visuels a prevoir

- schema simple en trois blocs : probleme / solution / preuve
- un badge statut
- une question d ouverture

A dire en rendez-vous

Phrase : Obsidia ne cherche pas a remplacer l intelligence des modeles. Il cherche a encadrer le moment ou une intelligence devient action. Cette page montre une brique de cette chaine.

A ne pas dire

- ne pas annoncer production-ready
- ne pas faire croire qu une couche peripherique decide
- ne pas transformer un freeze en runtime actif permanent
- ne pas masquer les limites auditees

Preuve a sortir seulement si on demande

Preuve : utiliser la matrice de statuts et les supports pre-build pour rattacher chaque claim a runtime, test, freeze, code ou fichier.

Question que la page doit provoquer

Question souhaitee : comment ce bloc se connecte-t-il au reste de l architecture et quel niveau de preuve existe deja ?

Page support 32 - Supports separes par public

Role de la page : Decliner les messages selon investisseur, banque, CTO, Big Tech, non-tech, education.

Message en haut de page

Cette page doit faire comprendre que le meme projet doit etre raconte differemment selon l interlocuteur.

Narration conseillee

Commencer par une phrase simple, puis montrer le lien avec le Kernel, la securite ou les preuves. Ne pas entrer dans les fichiers si l interlocuteur ne demande pas. L objectif est de declencher une question de profondeur, pas de faire un cours technique complet.

Elements visuels a prevoir

- schema simple en trois blocs : probleme / solution / preuve
- un badge statut
- une question d ouverture

A dire en rendez-vous

Phrase : Obsidia ne cherche pas a remplacer l intelligence des modeles. Il cherche a encadrer le moment ou une intelligence devient action. Cette page montre une brique de cette chaine.

A ne pas dire

- ne pas annoncer production-ready
- ne pas faire croire qu une couche peripherique decide
- ne pas transformer un freeze en runtime actif permanent
- ne pas masquer les limites auditees

Preuve a sortir seulement si on demande

Preuve : utiliser la matrice de statuts et les supports pre-build pour rattacher chaque claim a runtime, test, freeze, code ou fichier.

Question que la page doit provoquer

Question souhaitee : comment ce bloc se connecte-t-il au reste de l architecture et quel niveau de preuve existe deja ?

Page support 33 - Conclusion

Role de la page : Gouverner l action, pas seulement generer du texte.

Message en haut de page

Cette page doit faire comprendre que la conclusion doit revenir a la phrase centrale : obsidia ne cherche pas seulement a generer du texte.

Narration conseillee

Commencer par une phrase simple, puis montrer le lien avec le Kernel, la securite ou les preuves. Ne pas entrer dans les fichiers si l interlocuteur ne demande pas. L objectif est de declencher une question de profondeur, pas de faire un cours technique complet.

Elements visuels a prevoir

- schema simple en trois blocs : probleme / solution / preuve
- un badge statut
- une question d ouverture

A dire en rendez-vous

Phrase : Obsidia ne cherche pas a remplacer l intelligence des modeles. Il cherche a encadrer le moment ou une intelligence devient action. Cette page montre une brique de cette chaine.

A ne pas dire

- ne pas annoncer production-ready
- ne pas faire croire qu une couche peripherique decide

- ne pas transformer un freeze en runtime actif permanent
- ne pas masquer les limites auditees

Preuve a sortir seulement si on demande

Preuve : utiliser la matrice de statuts et les supports pre-build pour rattacher chaque claim a runtime, test, freeze, code ou fichier.

Question que la page doit provoquer

Question souhaitee : comment ce bloc se connecte-t-il au reste de l architecture et quel niveau de preuve existe deja ?

ARGUMENTAIRE DETAILLE PAR INTERLOCUTEUR

Investisseur / incubateur - argumentaire long

Focus : marche, timing, actif technique, reduction du risque, trajectoire industrialisation.

Objectif psychologique

L objectif est de faire sentir que le marche de la gouvernance des agents arrive, que le projet n est pas une simple idee, et que le chemin vers industrialisation est deja structure par des supports, preuves et paliers.

Angle de premiere slide

Ne pas commencer par le code. Commencer par la question : quand une IA propose une action, qui a le droit de dire oui, non ou attends ?

Trois preuves a montrer

- API 8000 runtime avec 164 routes
- Bank / Trading / GPS deja branches
- Roadmap WorkCenter + education + audit externe

Objection probable

Objection : est-ce juste une surcouche de plus ? Reponse : non, la valeur n est pas l affichage. La valeur est la separation d autorite, la preuve, le replay et la non-substitution du Kernel par les composants peripheriques.

Reponse sans sur-claim

Le projet est un POC local avance. Il possede deja des briques stabilisees, mais doit encore passer par audit externe, durcissement, CI complete, verification des preuves restantes et industrialisation.

Banque / risque / conformite - argumentaire long

Focus : autoriser, suspendre, bloquer, tracer, rejouer, auditer avant impact.

Objectif psychologique

L objectif est de rassurer sur la separation entre recommandation, autorisation, suspension, blocage, trace et replay. La banque ne doit pas entendre : remplacez votre stack. Elle doit entendre : ajoutez une couche de controle avant action.

Angle de premiere slide

Ne pas commencer par le code. Commencer par la question : quand une IA propose une action, qui a le droit de dire oui, non ou attends ?

Trois preuves a montrer

- Bank freeze 5 ALLOW S1

- decision_authority=KX108_ONLY et API BRIDGE_ONLY
- replay / audit / proof stack

Objection probable

Objection : est-ce juste une surcouche de plus ? Reponse : non, la valeur n est pas l affichage. La valeur est la separation d autorite, la preuve, le replay et la non-substitution du Kernel par les composants peripheriques.

Reponse sans sur-claim

Le projet est un POC local avance. Il possede deja des briques stabilisees, mais doit encore passer par audit externe, durcissement, CI complete, verification des preuves restantes et industrialisation.

CTO / dev senior - argumentaire long

Focus : architecture, Kernel, API bridge, boundaries, preuves, endpoints, tests.

Objectif psychologique

L objectif est de declencher une discussion technique controlee : routes, API, Kernel, boundaries, schemas, tests, proofs. Le support ouvre la porte, les annexes repondent.

Angle de premiere slide

Ne pas commencer par le code. Commencer par la question : quand une IA propose une action, qui a le droit de dire oui, non ou attends ?

Trois preuves a montrer

- /api/status boundary flags false
- Kernel /kernel/ragnarok et bridges terrain
- Lean 57 + tests Brody 124/124

Objection probable

Objection : est-ce juste une surcouche de plus ? Reponse : non, la valeur n est pas l affichage. La valeur est la separation d autorite, la preuve, le replay et la non-substitution du Kernel par les composants peripheriques.

Reponse sans sur-claim

Le projet est un POC local avance. Il possede deja des briques stabilisees, mais doit encore passer par audit externe, durcissement, CI complete, verification des preuves restantes et industrialisation.

Big Tech / IA infra - argumentaire long

Focus : workbench gouverne, agents encadres, proof layer, complement aux LLM.

Objectif psychologique

L objectif est de montrer une complementarite disruptive : la Big Tech construit des modeles et workbenchs puissants, Obsidia apporte une doctrine d autorite, preuve, replay et gouvernance du passage a l action.

Angle de premiere slide

Ne pas commencer par le code. Commencer par la question : quand une IA propose une action, qui a le droit de dire oui, non ou attends ?

Trois preuves a montrer

- architecture multi-couche
- separation LLM / memory / Kernel / proof
- WorkCenter cible comme environnement agentique gouverne

Objection probable

Objection : est-ce juste une surcouche de plus ? Reponse : non, la valeur n est pas l affichage. La valeur est la separation d autorite, la preuve, le replay et la non-substitution du Kernel par les composants peripheriques.

Reponse sans sur-claim

Le projet est un POC local avance. Il possede deja des briques stabilisees, mais doit encore passer par audit externe, durcissement, CI complete, verification des preuves restantes et industrialisation.

Non-tech / partenaire terrain - argumentaire long

Focus : comprendre qui decide, qui explique, qui memorise, qui prouve.

Objectif psychologique

L objectif est de rendre le systeme comprehensible : qui peut parler, qui peut memoriser, qui peut afficher, qui peut autoriser, qui peut bloquer.

Angle de premiere slide

Ne pas commencer par le code. Commencer par la question : quand une IA propose une action, qui a le droit de dire oui, non ou attends ?

Trois preuves a montrer

- schema ALLOW / HOLD / BLOCK
- roles separes Brody / Graphiti / Kernel
- exemples Bank / Trading / GPS

Objection probable

Objection : est-ce juste une surcouche de plus ? Reponse : non, la valeur n est pas l affichage. La valeur est la separation d autorite, la preuve, le replay et la non-substitution du Kernel par les composants peripheriques.

Reponse sans sur-claim

Le projet est un POC local avance. Il possede deja des briques stabilisees, mais doit encore passer par audit externe, durcissement, CI complete, verification des preuves restantes et industrialisation.

Education / formation interne - argumentaire long

Focus : parcours Brody, modules, exercices, validation, limites.

Objectif psychologique

L objectif est de transformer Obsidia en systeme transmissible : modules, exercices, validation, parcours et apprentissage progressif.

Angle de premiere slide

Ne pas commencer par le code. Commencer par la question : quand une IA propose une action, qui a le droit de dire oui, non ou attends ?

Trois preuves a montrer

- Brody Education Pack V1
- T1 a T20 modules
- Memory governance et validation humaine

Objection probable

Objection : est-ce juste une surcouche de plus ? Reponse : non, la valeur n est pas l affichage. La valeur est la separation d autorite, la preuve, le replay et la non-substitution du Kernel par les composants peripheriques.

Reponse sans sur-claim

Le projet est un POC local avance. Il possede deja des briques stabilisees, mais doit encore passer par audit externe, durcissement, CI complete, verification des preuves restantes et industrialisation.

CAHIER DE MESSAGES PRETS A EMPLOYER

Theme	Phrase
Message central	Obsidia ne cherche pas a rendre l IA plus bavarde. Il cherche a rendre son passage a l action gouvernable.
Kernel	Le Kernel X108 est le verrou qui decide ALLOW, HOLD ou BLOCK. Les autres couches preparent, expliquent, memorisent ou prouvent.
Securite	La securite vient de la separation d autorite : ce qui aide ne decide pas, ce qui memorise ne decide pas, ce qui affiche ne decide pas.
Terrain	Bank, Trading et GPS/Aviation montrent que la meme doctrine peut se brancher sur plusieurs mondes.
Brody	Brody rend le systeme comprehensible et transmissible, mais ne devient pas le chef.
Graphiti	Graphiti donne de la memoire, pas du pouvoir decisionnel.
Calcul	La puissance ne vient pas seulement du GPU. Elle vient aussi de l organisation des couches et des invariants.
Big Tech	La Big Tech construit les modeles et workbenchs. Obsidia vise la couche qui gouverne leur droit d agir.
Limite	Le projet est avance mais pas production-ready. La difference entre preuve, freeze, runtime et documentation est volontairement explicite.

Message central

Obsidia ne cherche pas a rendre l IA plus bavarde. Il cherche a rendre son passage a l action gouvernable.

Developpement : cette phrase peut etre utilisee comme accroche de slide. Elle doit ensuite etre reliee a une preuve interne et a une limite honnete. Cette discipline evite de vendre du reve et permet de gagner en credibilite face a un interlocuteur exigeant.

Kernel

Le Kernel X108 est le verrou qui decide ALLOW, HOLD ou BLOCK. Les autres couches preparent, expliquent, memorisent ou prouvent.

Developpement : cette phrase peut etre utilisee comme accroche de slide. Elle doit ensuite etre reliee a une preuve interne et a une limite honnete. Cette discipline evite de vendre du reve et permet de gagner en credibilite face a un interlocuteur exigeant.

Securite

La securite vient de la separation d autorite : ce qui aide ne decide pas, ce qui memorise ne decide pas, ce qui affiche ne decide pas.

Developpement : cette phrase peut etre utilisee comme accroche de slide. Elle doit ensuite etre reliee a une preuve interne et a une limite honnete. Cette discipline evite de vendre du reve et permet de gagner en credibilite face a un interlocuteur exigeant.

Terrain

Bank, Trading et GPS/Aviation montrent que la meme doctrine peut se brancher sur plusieurs mondes.

Developpement : cette phrase peut etre utilisee comme accroche de slide. Elle doit ensuite etre reliee a une preuve interne et a une limite honnete. Cette discipline evite de vendre du reve et permet de gagner en credibilite face a un interlocuteur exigeant.

Brody

Brody rend le systeme comprehensible et transmissible, mais ne devient pas le chef.

Developpement : cette phrase peut etre utilisee comme accroche de slide. Elle doit ensuite etre reliee a une preuve interne et a une limite honnete. Cette discipline evite de vendre du reve et permet de gagner en credibilite face a un interlocuteur exigeant.

Graphiti

Graphiti donne de la memoire, pas du pouvoir decisionnel.

Developpement : cette phrase peut etre utilisee comme accroche de slide. Elle doit ensuite etre reliee a une preuve interne et a une limite honnete. Cette discipline evite de vendre du reve et permet de gagner en credibilite face a un interlocuteur exigeant.

Calcul

La puissance ne vient pas seulement du GPU. Elle vient aussi de l organisation des couches et des invariants.

Developpement : cette phrase peut etre utilisee comme accroche de slide. Elle doit ensuite etre reliee a une preuve interne et a une limite honnete. Cette discipline evite de vendre du reve et permet de gagner en credibilite face a un interlocuteur exigeant.

Big Tech

La Big Tech construit les modeles et workbenchs. Obsidia vise la couche qui gouverne leur droit d agir.

Developpement : cette phrase peut etre utilisee comme accroche de slide. Elle doit ensuite etre reliee a une preuve interne et a une limite honnete. Cette discipline evite de vendre du reve et permet de gagner en credibilite face a un interlocuteur exigeant.

Limite

Le projet est avance mais pas production-ready. La difference entre preuve, freeze, runtime et documentation est volontairement explicite.

Developpement : cette phrase peut etre utilisee comme accroche de slide. Elle doit ensuite etre reliee a une preuve interne et a une limite honnete. Cette discipline evite de vendre du reve et permet de gagner en credibilite face a un interlocuteur exigeant.

PLAN DE SUPPORT FINAL RECOMMANDE EN 90 A 120 PAGES

Page	Titre	Role	Profondeur technique
1	Resume executif	Donner en quelques pages la these du projet, le statut reel, les actifs stabilises et la trajectoire.	niveau 1 dans le corps / niveau 2 en annexe

Page	Titre	Role	Profondeur technique
2	Pourquoi maintenant : les agents IA passent a l action	Faire comprendre que le risque se deplace de la reponse vers l action.	niveau 1 dans le corps / niveau 2 en annexe

Page	Titre	Role	Profondeur technique
3	Le probleme marche	Mettre en mots le manque d autorite claire entre recommandation et action.	niveau 1 dans le corps / niveau 2 en annexe

Page	Titre	Role	Profondeur technique
4	Le changement de paradigme IA	Passer du modele generatif a l infrastructure de gouvernance.	niveau 1 dans le corps / niveau 2 en annexe

Page	Titre	Role	Profondeur technique
5	Pourquoi les agents ont besoin d une autorite	Montrer que tool-use, memoire, workflows et automatisation exigent une autorite separee.	niveau 1 dans le corps / niveau 2 en annexe

Page	Titre	Role	Profondeur technique
6	Ce qu est Obsidia / X108	Definir Obsidia comme architecture de gouvernance decisionnelle.	niveau 1 dans le corps / niveau 2 en annexe

Page	Titre	Role	Profondeur technique
7	Ce qu Obsidia n est pas	Eviter les confusions : chatbot, wrapper, simple UI, modele plus gros, outil de monitoring seul.	niveau 1 dans le corps / niveau 2 en annexe

Page	Titre	Role	Profondeur technique
8	Architecture en une page	Presenter la chaine Signal -> Domaines -> Sigma -> Kernel -> Proof -> Replay -> WorkCenter.	niveau 1 dans le corps / niveau 2 en annexe

Page	Titre	Role	Profondeur technique
9	Le Kernel X108	Montrer le coeur de securite et d autorite.	niveau 1 dans le corps / niveau 2 en annexe

Page	Titre	Role	Profondeur technique
10	La regle BLOCK > HOLD > ALLOW	Expliquer la priorite fail-closed sans surcharge technique.	niveau 1 dans le corps / niveau 2 en annexe

Page	Titre	Role	Profondeur technique
11	Separation d autorite	Rendre visible que chaque couche a un role et une limite.	niveau 1 dans le corps / niveau 2 en annexe

Page	Titre	Role	Profondeur technique
12	Security by design	Montrer les boundaries, les interdits et les garanties de non-contournement.	niveau 1 dans le corps / niveau 2 en annexe

Page	Titre	Role	Profondeur technique
13	Les terrains stables : Bank, Trading, GPS/Aviation	Montrer les trois branchements terrain deja validates par freeze et runtime API.	niveau 1 dans le corps / niveau 2 en annexe

Page	Titre	Role	Profondeur technique
14	Brody : pseudo-LLM educatif et assistant non souverain	Expliquer la valeur de Brody sans lui donner d autorite.	niveau 1 dans le corps / niveau 2 en annexe

Page	Titre	Role	Profondeur technique
15	Graphiti : memoire readonly non souveraine	Expliquer la memoire comme sidecar de contexte, pas comme decideur.	niveau 1 dans le corps / niveau 2 en annexe

Page	Titre	Role	Profondeur technique
16	Sigma : aggregation et signaux	Expliquer la couche qui structure les signaux et votes avant le Kernel.	niveau 1 dans le corps / niveau 2 en annexe

Page	Titre	Role	Profondeur technique
17	Proof stack Lean / TLA / Merkle / RFC3161	Presenter les preuves stabilisees et les limites honnetes.	niveau 1 dans le corps / niveau 2 en annexe

Page	Titre	Role	Profondeur technique
18	Pourquoi deterministe	Defendre le choix structurel : invariants, gates, replay, statuts.	niveau 1 dans le corps / niveau 2 en annexe

Page	Titre	Role	Profondeur technique
19	Pourquoi pas dependant du big training	Expliquer l avantage structurel sans pretendre remplacer les LLM.	niveau 1 dans le corps / niveau 2 en annexe

Page	Titre	Role	Profondeur technique
20	Gain calcul / cout : structure > masse	Montrer pourquoi une approche structurelle reduit la pression GPU et data sur le coeur decisionnel.	niveau 1 dans le corps / niveau 2 en annexe

Page	Titre	Role	Profondeur technique
21	Petite machine, architecture lourde	Montrer ce qui a deja ete construit localement et ce que cela prouve.	niveau 1 dans le corps / niveau 2 en annexe

Page	Titre	Role	Profondeur technique

22	Ce qui est deja stabilise	Synthese des actifs confirmed runtime, test, freeze, code.	niveau 1 dans le corps / niveau 2 en annexe

Page	Titre	Role	Profondeur technique
23	Limites honnetes	Dire clairement ce qui reste a relancer, auditer, durcir et certifier.	niveau 1 dans le corps / niveau 2 en annexe

Page	Titre	Role	Profondeur technique
24	Etat actuel du projet	Positionner le projet en POC local avance / pre-industrialisation.	niveau 1 dans le corps / niveau 2 en annexe

Page	Titre	Role	Profondeur technique
25	Dernieres etapes avant industrialisation	Lister les paliers avant audit externe, CI, pilot POC, certification.	niveau 1 dans le corps / niveau 2 en annexe

Page	Titre	Role	Profondeur technique
26	Roadmap WorkCenter	Transformer les supports dev en cockpit concret.	niveau 1 dans le corps / niveau 2 en annexe

Page	Titre	Role	Profondeur technique
27	Roadmap education poussee	Structurer Brody Education, modules, parcours publics.	niveau 1 dans le corps / niveau 2 en annexe

Page	Titre	Role	Profondeur technique
28	Roadmap audit externe	Preparer preuves, limitations, test packs, release process.	niveau 1 dans le corps / niveau 2 en annexe

Page	Titre	Role	Profondeur technique
29	Roadmap Big Tech Workbench	Decrire la trajectoire vers un environnement de travail gouverne de niveau industriel.	niveau 1 dans le corps / niveau 2 en annexe

Page	Titre	Role	Profondeur technique
30	Positionnement concurrentiel	Positionner Obsidia face aux LLM, agent frameworks, MLOps, RAG et monitoring.	niveau 1 dans le corps / niveau 2 en annexe

Page	Titre	Role	Profondeur technique
31	Pourquoi Obsidia peut rejoindre puis depasser	Formuler l angle : pas la taille du modele, mais le controle du passage a l action.	niveau 1 dans le corps / niveau 2 en annexe

Page	Titre	Role	Profondeur technique
32	Supports separes par public	Decliner les messages selon investisseur, banque, CTO, Big Tech, non-tech, education.	niveau 1 dans le corps / niveau 2 en annexe

Page	Titre	Role	Profondeur technique
33	Conclusion	Gouverner l action, pas seulement generer du texte.	niveau 1 dans le corps / niveau 2 en annexe

Les pages principales doivent rester lisibles. Les preuves, statuts, payloads et details techniques doivent etre disponibles en annexe ou dans les supports dev deja produits.

FIN DU SUPPORT

Phrase finale a conserver : Obsidia / X108 ne cherche pas seulement a generer mieux. Il cherche a gouverner le droit d agir.

Support personnel & stratégique - Obsidia / X108

Chemin de vie, vision, architecture, Brody et récit public - Version V3

Auteur : Étienne Aubin
Version : V3 - Chemin de vie, Vision & Brody
Date : Juin 2026

Statut de cette version V3

Ce support remplace la version précédente. Il garde la force du parcours, mais il déplace l'angle : moins d'exposition intime, moins de récit de souffrance, plus de chemin de vie, de vision, de discipline, d'architecture et de responsabilité. Cette V3 ajoute une partie centrale sur Brody : pseudo-LLM éducatif, machine de compréhension, couche de médiation cognitive et interface non souveraine du système.

Il sert à expliquer Étienne Aubin et Obsidia / X108 sans réduire le projet à une histoire personnelle. Le parcours donne le sens. Le Kernel donne la structure. Les preuves donnent la crédibilité. La vision donne la trajectoire.

Principe éditorial : on ne vend pas une fragilité. On montre une transformation : terrain -> observation -> structure -> sécurité -> gouvernance de l'action IA.

1. Objectif du support

Ce document sert à présenter le parcours d'Étienne Aubin et la naissance d'Obsidia / X108 sous un angle public, stratégique et maîtrisé.

Il répond à cinq besoins.

Premier besoin : expliquer d'où vient la vision sans entrer dans des détails trop privés.

Deuxième besoin : montrer qu'Obsidia n'est pas un projet abstrait sorti d'un effet de mode, mais la continuité d'un chemin de vie orienté vers la responsabilité, l'irréversibilité et la sécurité.

Troisième besoin : donner un récit compréhensible par un investisseur, un CTO, un incubateur, une banque ou un partenaire non technique.

Quatrième besoin : relier le parcours personnel au positionnement technique : Kernel X108, séparation d'autorité, Bank / Trading / GPS, Brody, Graphiti, Sigma, Proof stack, WorkCenter.

Cinquième besoin : protéger l'image publique. Le document distingue ce qui peut être dit publiquement, ce qui peut rester semi-public, et ce qui doit rester privé.

Ce support n'est pas un CV classique. Ce n'est pas une confession. Ce n'est pas un pitch technique. C'est un document de trajectoire : il explique pourquoi ce projet existe, pourquoi il a cette forme, et pourquoi il est cohérent avec le chemin d'Étienne.

2. Version courte - Qui est Étienne

Étienne Aubin est un entrepreneur autodidacte et architecte IA indépendant. Il ne vient pas d'un parcours académique classique, ni d'un grand laboratoire, ni de Big Tech. Son chemin s'est construit par le terrain, l'observation, l'entrepreneuriat, l'e-commerce, l'automatisation et une confrontation intense avec les systèmes IA depuis fin 2024.

Son point fort n'est pas d'être un développeur classique. Son point fort est de relier les couches : humain, métier, décision, sécurité, mémoire, preuve, action réelle.

Depuis février 2025, il développe Obsidia / X108 : une architecture de gouvernance décisionnelle pour agents IA et workflows critiques. Son idée centrale est simple à comprendre : quand une IA ou un agent veut agir, il faut une autorité claire pour dire ALLOW, HOLD ou BLOCK.

Sa vision tient en une phrase : les IA ne doivent pas seulement être plus capables ; elles doivent être gouvernées avant d'agir.

3. Version très simple - Explication enfant

Imagine une voiture très intelligente.

Elle sait lire la route. Elle sait accélérer. Elle sait tourner. Elle peut aller très vite. Mais si personne ne vérifie quand elle a le droit d'avancer, elle peut devenir dangereuse.

Obsidia, c'est le système qui vérifie avant que la voiture parte.

Il demande : est-ce autorisé ? Est-ce trop tôt ? Est-ce dangereux ? Est-ce qu'on peut expliquer la décision ? Est-ce qu'on pourra rejouer ce qui s'est passé ?

Le Kernel X108, c'est le feu de sécurité au centre.

Il peut dire :

ALLOW - tu peux passer.

HOLD - attends, il manque quelque chose.

BLOCK - stop, c'est trop risqué.

Ce que construit Étienne, ce n'est donc pas une voiture de plus. C'est le système qui empêche les voitures intelligentes de partir n'importe comment.

4. Version publique - Parcours sans surexposition

Le parcours d'Étienne est non linéaire.

Il n'a pas suivi la voie classique des grandes écoles ou des laboratoires. Il a construit sa manière de penser par le terrain, par l'expérience, par l'observation directe des conséquences, puis par une période d'apprentissage intensif avec l'IA.

Le passage par l'ambulance a laissé une trace importante : dans certains contextes, une décision n'est pas théorique. Elle produit une conséquence réelle. Elle peut être irréversible. On ne peut pas faire comme si tout était corrigeable après coup.

Cette logique est devenue centrale dans Obsidia / X108.

Avant qu'un système IA agisse, il doit être possible de suspendre, de bloquer, de tracer, de prouver, puis de rejouer la décision.

Le parcours personnel n'est donc pas mis en avant pour créer de l'émotion. Il sert à expliquer la racine du projet : la sécurité n'est pas une option esthétique. C'est une architecture.

5. Ce qu'il ne faut plus mettre au centre

La version publique du récit ne doit pas focaliser sur les éléments les plus intimes ou médicaux du parcours.

Il faut éviter trois pièges.

Premier piège : réduire Étienne à ses difficultés. Le vrai sujet n'est pas la difficulté. Le vrai sujet est la transformation d'un chemin non linéaire en architecture cohérente.

Deuxième piège : utiliser des éléments trop privés qui peuvent détourner l'attention du projet. Un investisseur, un CTO ou un partenaire doit retenir la vision, la capacité d'exécution, la discipline de sécurité et le potentiel industriel.

Troisième piège : raconter Obsidia comme une revanche personnelle seulement. L'énergie de revanche existe dans le chemin, mais le projet doit être présenté comme une infrastructure utile, responsable et transférable.

Formulation à privilégier :

Je viens d'un parcours atypique, non académique, construit par le terrain et l'apprentissage intensif. Ce parcours m'a donné une obsession : comprendre comment une décision devient une conséquence réelle. Obsidia est la traduction structurée de cette obsession dans le monde de l'IA agentique.

6. Chronologie structurante

Cette chronologie doit rester simple et lisible.

Octobre 2024 - Découverte opérationnelle de l'IA dans un contexte e-commerce, automatisation, contenu, structuration de projets.

Février 2025 - Déclencheur Obsidia. La question centrale apparaît : si l'IA ressemble de plus en plus à un cerveau fonctionnel, qu'est-ce qui l'empêche de devenir une intelligence générale incontrôlée ? Et surtout : qui contrôle son passage à l'action ?

Février à juin 2025 - Phase de conceptualisation. L'intuition devient architecture. Les premières couches apparaissent : mémoire, décision, invariants, sécurité, action, preuve.

Deuxième semestre 2025 - Rétrécissement progressif. Le projet quitte la simple vision large pour entrer dans des structures plus précises : Kernel, OS, règles, statuts, audit, séparation d'autorité.

2026 - Phase de consolidation technique. Obsidia / X108 passe d'un concept à un système audité localement : API, Kernel, domaines Bank / Trading / GPS, Sigma, Brody, Graphiti, Lean, freezes, WorkCenter.

Juin 2026 - Phase pré-industrialisation. Le Kernel, les preuves, les supports DEV, les matrices d'autorité et les documents de positionnement sont structurés. Le WorkCenter devient la dernière grande surface à assembler avant éducation poussée, packaging et démonstration externe.

7. Le fil rouge : de l'irréversibilité terrain au Kernel X108

Le fil rouge du parcours est l'irréversibilité.

Dans les métiers de terrain, certaines décisions ne sont pas des abstractions. Une action peut changer une situation. Une erreur peut produire une conséquence. Une validation trop rapide peut coûter cher.

Cette intuition est devenue une règle architecturale : une action irréversible ne doit jamais dépendre uniquement d'une sortie probabiliste.

Obsidia / X108 formalise cette idée.

Le modèle peut proposer.

Le domaine peut signaler.

Brody peut expliquer.

Graphiti peut mémoriser.

Sigma peut agréger.

Mais le droit d'agir passe par le Kernel X108.

Le vécu de terrain devient donc un principe de système : entre cognition et action, il faut un verrou.

8. La vision centrale

La vision d'Étienne peut être formulée simplement.

Nous entrons dans une période où les IA ne seront plus seulement des outils de réponse. Elles vont devenir des systèmes capables d'action : appeler des API, modifier des workflows, déclencher des transactions, assister des décisions critiques, piloter des chaînes opérationnelles.

Le problème central ne sera donc plus seulement : est-ce que l'IA répond bien ?

Le problème central deviendra : est-ce qu'elle a le droit d'agir ?

Obsidia répond à ce changement de monde.

Il ne cherche pas à faire un modèle plus gros. Il cherche à construire l'architecture qui gouverne le droit d'agir des modèles, des agents et des workflows.

Formule courte :

La Big Tech construit des capacités. Obsidia construit l'autorité qui contrôle le passage à l'action.

9. Ce qu'Obsidia représente dans le parcours

Obsidia n'est pas seulement un projet technique.

C'est une manière de transformer une pensée non linéaire en système structuré.

C'est une manière de transformer l'observation du chaos en règles vérifiables.

C'est une manière de faire passer l'intuition vers la preuve.

C'est une manière de dire : si l'IA devient plus puissante, il faut la gouverner par architecture, pas seulement par intention.

Mais la présentation publique doit rester sobre. Il ne faut pas enfermer Obsidia dans une histoire personnelle. Le projet doit pouvoir être compris sans connaître toute la vie d'Étienne.

Le bon équilibre est le suivant :

Le parcours explique pourquoi cette architecture existe.

Les preuves expliquent pourquoi elle est crédible.

La roadmap explique pourquoi elle peut devenir industrielle.

10. Positionnement personnel : architecte plutôt que développeur classique

Étienne doit être présenté comme un architecte de systèmes IA, pas comme un développeur logiciel classique.

Son rôle n'est pas de se vendre comme celui qui code tout à la main ligne par ligne. Son rôle est de concevoir une architecture, de coordonner des couches, de contrôler les invariants, de guider les IA copilotes, de tester, d'auditer, de figer et d'orienter le système.

C'est une position moderne : l'architecte IA ne remplace pas tous les spécialistes, mais il donne la forme globale, la doctrine, les limites, les tests, les preuves et l'ordre d'intégration.

Formulation publique :

Je ne viens pas d'un parcours d'ingénieur classique. Mon rôle est celui d'un architecte IA indépendant : je conçois la structure, les règles d'autorité, les flux de décision, les preuves, et j'utilise les IA comme copilotes d'implémentation pour transformer cette vision en système vérifiable.

11. Le passage e-commerce -> IA -> architecture

Le socle initial d'Étienne vient de l'entrepreneuriat, du commerce en ligne, du contenu, de l'automatisation et du test terrain.

Cette base est importante : elle explique pourquoi Obsidia n'est pas seulement théorique. Le projet garde une logique d'usage, de démonstration, de packaging, de mise en marché et de compréhension utilisateur.

L'IA a d'abord été un outil de productivité et de structuration. Puis elle est devenue un miroir de questions plus profondes : mémoire, agents, décision, action, sécurité, preuve, responsabilité.

Le basculement vers Obsidia ne vient pas d'un effet de mode. Il vient d'un constat progressif : plus les IA deviennent capables, plus il faut organiser leur droit d'agir.

Ce chemin donne à Obsidia une double nature :

Vision systémique.

Exécution entrepreneuriale.

12. L'approche autodidacte comme avantage

L'absence de parcours académique classique ne doit pas être présentée comme une faiblesse centrale. Elle peut devenir un avantage si elle est formulée correctement.

Ce qu'elle permet :

- regarder le problème sans être enfermé dans une seule discipline ;
- relier business, terrain, IA, sécurité, preuve, interface et gouvernance ;
- poser des questions simples que les systèmes complexes oublient parfois ;
- construire vite en utilisant l'IA comme accélérateur ;
- rester obsédé par le résultat réel plutôt que par la conformité académique de façade.

Formulation à utiliser :

Je n'ai pas le parcours typique de ce type de projet. C'est justement ce qui m'a poussé à construire autrement : par structure, par preuve, par audit, par tests et par confrontation directe avec les limites du réel.

13. L'ambition sans posture excessive

L'ambition doit être assumée, mais cadrée.

Il faut éviter le ton : je vais battre tout le monde.

Il faut préférer : je construis une trajectoire différente, complémentaire, avec une profondeur spécifique sur la gouvernance de l'action.

Formulation :

Je ne cherche pas à construire un modèle plus gros que la Big Tech. Je construis une couche que ces modèles pourraient avoir besoin d'utiliser : une autorité déterministe, traçable et vérifiable entre recommandation et action.

Autre formulation :

Big Tech a la puissance, les modèles, l'infrastructure et la distribution. Obsidia apporte un angle : la gouvernance déterministe du droit d'agir.

Ce positionnement est plus crédible que l'opposition frontale.

14. Obsidia en une page

Obsidia / X108 est une architecture de gouvernance décisionnelle pour agents IA et workflows critiques.

Le problème traité est le passage entre une recommandation probabiliste et une action réelle.

Le système sépare les rôles :

- les modèles proposent ;
- les domaines signalent ;
- Sigma agrège ;
- Brody explique ;
- Graphiti mémorise ;
- l'API transmet ;
- le WorkCenter affiche et prépare ;
- le Kernel X108 décide ;
- la Proof stack vérifie ;
- les freezes capturent.

Le Kernel X108 retourne trois types de décision :

ALLOW - autorisé.

HOLD - suspendu.

BLOCK - bloqué.

La règle centrale est : BLOCK > HOLD > ALLOW.

L'objectif n'est pas de rendre les agents plus bavards ou plus autonomes. L'objectif est de rendre leur passage à l'action gouverné, auditable, rejouable et contrôlé.

15. Kernel X108 - Le symbole de la vision

Le Kernel X108 doit être présenté comme le cœur de la vision.

Il représente la différence entre une IA qui parle et une IA qui agit.

Il incarne l'idée suivante : le droit d'agir ne doit pas appartenir au composant qui génère la proposition.

Dans le récit personnel, le Kernel est aussi une manière de rendre la discipline plus forte que l'impulsion. Même le concepteur ne doit pas pouvoir contourner facilement les règles. Le projet n'est pas construit sur la confiance dans une personne. Il est construit sur des structures qui limitent les abus, les raccourcis et les passages non contrôlés.

Formulation publique :

Le Kernel X108 est le composant qui empêche le système de confondre capacité et autorisation. Ce n'est pas parce qu'un agent peut agir qu'il a le droit d'agir.

16. Sécurité par séparation d'autorité

La sécurité d'Obsidia ne repose pas sur une simple couche ajoutée après coup.

Elle repose sur une séparation d'autorité dès la conception.

Brody peut expliquer mais ne décide pas.

Graphiti peut mémoriser mais ne décide pas.

Sigma peut agréger mais ne décide pas seul.

L'API peut transmettre mais ne décide pas.

Le WorkCenter peut afficher, préparer et valider selon protocole, mais ne remplace pas le Kernel.

Les domaines Bank, Trading ou GPS peuvent produire des signaux, mais ils ne deviennent pas souverains.

Cette doctrine reflète le chemin d'Étienne : ne pas confondre intensité et décision, vitesse et légitimité, capacité et autorisation.

La sécurité devient une topologie : chacun à sa place, aucun composant périphérique ne capture le pouvoir central.

17. Policy Overlay vs Kernel By Design

La plupart des approches du marché ajoutent des règles, filtres, guardrails, policies ou systèmes de monitoring autour d'un modèle ou d'un framework agent.

Cette logique peut être utile, mais elle reste souvent un overlay : une couche de contrôle ajoutée autour d'un système qui reste centré sur le modèle ou sur l'agent.

Obsidia part d'un autre principe.

L'autorité ne doit pas être ajoutée après coup. Elle doit être séparée par design.

Policy Overlay = filtre ajouté autour d'un système probabiliste.

Kernel By Design = autorité déterministe séparée dès l'architecture.

Phrase à retenir :

Les autres sécurisent souvent autour de l'agent. Obsidia retire l'autorité critique à l'agent et la place dans un Kernel.

18. Bank / Trading / GPS - Le terrain, pas seulement la théorie

Le récit personnel ne doit pas rester abstrait. Il doit être relié aux preuves terrain.

Obsidia a déjà été branché sur plusieurs domaines.

Bank : logique de transaction et de gouvernance financière. Les freezes disponibles montrent des décisions ALLOW S1.

Trading : logique de risque, contradiction et blocage. Les freezes montrent des décisions BLOCK S4 avec reason code CONTRADICTION_THRESHOLD_REACHED.

GPS / Aviation : logique de contexte terrain critique. Les freezes montrent des décisions ALLOW S0.

Ces trois domaines montrent que le Kernel n'est pas seulement une idée. Il peut recevoir des contextes différents, produire une décision gouvernée, et générer une trace exploitable.

Message public :

Obsidia n'est pas encore un produit industriel, mais ce n'est plus seulement une intuition. Des domaines différents sont déjà branchés et documentés dans des preuves locales.

19. Brody - Pseudo-LLM, machine éducative et couche de médiation non souveraine

Brody doit prendre plus de place dans ce support, parce qu'il représente une partie essentielle de la vision d'Étienne : rendre une architecture complexe compréhensible, transmissible et utilisable sans donner le pouvoir à l'assistant.

Brody n'est pas le Kernel.

Brody n'est pas une autorité.

Brody n'est pas une mémoire souveraine.

Brody n'est pas un agent autonome qui agit à la place du système.

Brody est une couche d'explication, d'éducation, de traduction, de navigation et de préparation.

La bonne formule est :

Brody aide à comprendre le système. X108 garde le droit d'agir.

19.1 Pourquoi Brody existe

Obsidia est puissant, mais dense.

Il contient un Kernel, des domaines, Sigma, Graphiti, des proofs, des freezes, des statuts, des documents d'architecture, des routes API, des matrices de sécurité, des décisions rejouables et une roadmap d'industrialisation.

Sans une couche pédagogique, le risque est simple : le système devient trop riche pour être transmis.

Brody existe pour éviter cela.

Il transforme la complexité en parcours lisible.

Il permet à un utilisateur, un investisseur, un développeur, un auditeur ou un partenaire métier de comprendre où il se trouve dans le système.

Il peut expliquer :

- ce qu'est Obsidia ;
- ce qu'est X108 ;
- pourquoi le Kernel est souverain ;
- pourquoi Brody ne décide pas ;
- pourquoi Graphiti est readonly ;
- ce que veut dire CONFIRMED_BY_RUNTIME ;
- ce que veut dire CONFIRMED_BY_FREEZE ;
- ce qui est prouvé ;
- ce qui est seulement documenté ;
- ce qui reste à vérifier ;
- quelle étape vient ensuite.

Brody n'est donc pas un gadget conversationnel.

Brody est l'interface cognitive du projet.

Il sert à rendre le système transmissible.

19.2 Brody comme pseudo-LLM

Le mot pseudo-LLM doit être expliqué simplement.

Un LLM classique génère des réponses à partir d'un modèle probabiliste entraîné à grande échelle.

Brody, dans la vision Obsidia, n'est pas présenté comme un grand modèle propriétaire entraîné de zéro.

Brody est une couche hybride :

- une interface conversationnelle ;

- une mémoire de travail contrôlée ;
- une couche éducative ;
- un lecteur d'audit ;
- un traducteur de complexité ;
- un assistant de parcours ;
- un préparateur de validation ;
- un générateur de candidats, jamais de décisions souveraines.

Il utilise l'intelligence conversationnelle comme surface, mais il est encadré par des règles, des statuts, des boundaries et des invariants.

Formule publique :

Brody ressemble à un assistant intelligent, mais il est construit pour rester non souverain.

Formule technique :

Brody est une interface cognitive readonly / advisory autour du système Obsidia, capable d'expliquer, classifier, préparer, proposer et guider, mais incapable de gouverner l'action.

Formule enfant :

Brody est le professeur qui explique le système. Le Kernel X108 est le feu rouge / orange / vert qui décide si une action peut passer.

19.3 La machination de Brody

Le mot "machination" peut être repris dans le bon sens : non pas manipulation, mais mécanique interne.

La machination de Brody, c'est sa manière de transformer du chaos informationnel en compréhension exploitable.

Entrées possibles :

- conversation ;
- audit ;
- freeze ;
- log terminal ;
- README ;
- glossaire ;
- proof report ;
- route API ;
- capture écran ;
- décision Kernel ;
- contexte Graphiti ;
- retour utilisateur ;
- objectif de session.

Traitement :

- lire ;
- classer ;
- relier ;
- simplifier sans trahir ;
- identifier le statut réel ;
- distinguer preuve, hypothèse, souvenir, intention, blocage ;
- proposer une action suivante ;
- transformer l'information utile en candidat mémoire ;
- demander validation si nécessaire.

Sorties autorisées :

- explication ;
- résumé opérationnel ;
- guide pas à pas ;
- question de clarification ;
- memory candidate ;
- validation request ;
- patch proposal ;
- test proposal ;
- learning module ;
- briefing non-tech ;
- briefing CTO ;
- briefing investisseur ;
- briefing audit.

Sorties interdites :

- décision souveraine ;
- ACT ;
- autorisation d'action réelle ;
- mutation Kernel ;
- écriture Graphiti directe ;
- promotion canonique automatique ;
- contournement de X108 ;
- modification de preuve sans protocole ;
- freeze automatique sans validation.

Phrase à retenir :

Brody transforme la complexité en compréhension, mais ne transforme jamais une intention en autorisation.

19.4 Le cycle Brody

Le cycle Brody peut être présenté en sept étapes.

1. Capture

Brody reçoit une conversation, un fichier, un audit, une demande ou une sortie système.

2. Clarification

Il identifie ce que l'utilisateur cherche vraiment : comprendre, décider d'une prochaine étape, préparer un patch, relire une preuve, former un tiers, ou extraire une mémoire candidate.

3. Classification

Il classe l'information : projet, domaine, preuve, mémoire, bug, route API, sécurité, pitch, roadmap, support, décision, limite, risque.

4. Statut

Il attribue ou rappelle le statut réel : CONFIRMED_BY_RUNTIME, CONFIRMED_BY_TEST, CONFIRMED_BY_FREEZE, CONFIRMED_BY_CODE, DOCUMENTED_ONLY, PRESENT_BUT_NOT_EXECUTED, DO_NOT_CLAIM.

5. Transformation pédagogique

Il rend la matière lisible : explication simple, version enfant, version investisseur, version CTO, version dev, version audit.

6. Proposition non souveraine

Il peut proposer : prochaine commande, prochain ticket, prochain test, prochain support, prochain apprentissage, mais il ne décide pas seul.

7. Validation / mémoire

Si quelque chose doit durer, Brody le transforme en candidat mémoire ou en demande de validation. La promotion finale passe par le processus prévu.

Schéma simple :

Entrée brute -> Brody lit -> Brody classe -> Brody explique -> Brody propose -> Humain / process valide -> X108 garde l'autorité d'action.

19.5 Brody dans le chemin de vie d'Étienne

Brody représente une extension importante du chemin d'Étienne.

Étienne pense par blocs, liens, tensions, inversions, priorités, frictions et architecture.

Cette pensée produit beaucoup de matière.

Le risque est de perdre la matière, de mélanger les statuts, de diluer les décisions, ou de devoir tout réexpliquer à chaque nouvelle session.

Brody sert à faire le pont entre cette pensée vivante et une structure transmissible.

Il aide à :

- stabiliser une session ;
- retrouver le fil ;
- transformer une intuition en bloc ;
- transformer un bloc en support ;
- transformer un support en ticket ;
- transformer un ticket en action préparée ;
- transformer une action préparée en validation ;
- transformer une validation en mémoire durable.

Mais Brody ne prend pas le pouvoir sur la pensée.

Il n'est pas là pour lisser ou remplacer Étienne.

Il est là pour rendre sa vision exploitable par d'autres.

Formulation publique :

Brody est né du besoin de rendre transmissible une architecture très dense, sans laisser l'assistant devenir souverain.

19.6 Brody dans l'architecture Obsidia

Dans l'architecture Obsidia, Brody se situe entre l'humain, la documentation, les traces, la mémoire candidate et le WorkCenter.

Il ne se situe pas dans la couche souveraine.

Carte simplifiée :

Utilisateur / Étienne -> Brody -> explication / candidat / validation request / support / ticket -> WorkCenter -> Human Validation / Process Validation -> Memory Governance -> Kernel X108 uniquement si une action doit être gouvernée.

Brody peut lire autour du Kernel.

Brody peut expliquer le Kernel.

Brody peut aider à préparer un changement de Kernel.

Mais Brody ne modifie jamais le Kernel.

Toute évolution Kernel passe par le Kernel Change Protocol.

19.7 Brody et le WorkCenter

Le WorkCenter doit donner à Brody une place claire.

Pas une place de chef.

Une place de guide.

Dans le cockpit, Brody peut avoir plusieurs modes :

- Learning Mode ;
- Audit Reader Mode ;
- Proof Reader Mode ;
- Domain Reader Mode ;
- Memory Candidate Mode ;
- Patch Proposal Mode ;
- Session Recap Mode ;
- Investor Briefing Mode ;
- CTO Briefing Mode ;
- Non-Tech Explanation Mode.

Chaque mode doit afficher ses limites.

Exemple :

Mode Proof Reader : Brody peut expliquer une preuve, mais il ne peut pas déclarer qu'une preuve est confirmée si le test n'a pas été relancé.

Mode Memory Candidate : Brody peut proposer une mémoire, mais il ne peut pas la promouvoir en canon seul.

Mode Patch Proposal : Brody peut proposer une modification, mais il ne peut pas l'appliquer dans une zone protégée sans validation.

Mode Investor Briefing : Brody peut simplifier, mais il ne doit pas sur-vendre.

19.8 Brody Education - Les parcours T1 à T20

Brody doit aussi être présenté comme un système d'éducation progressive.

L'objectif n'est pas seulement de répondre à des questions.

L'objectif est de former quelqu'un à comprendre Obsidia.

Parcours possibles :

T1 - Comprendre Obsidia / X108.

T2 - Autorité et non-souveraineté.

T3 - Kernel X108.

T4 - API FastAPI readonly.

T5 - Bank domain.

T6 - Trading domain.

T7 - GPS / Aviation / Defense domain.

T8 - Brody education layer.

T9 - Graphiti readonly memory.

T10 - CIC / causal context.

T11 - Proof stack Lean / TLA / Python.

T12 - Freezes / manifests / audit reports.

T13 - Server launch and healthchecks.

T14 - Debugging without breaking Kernel.

T15 - Patch proposal workflow.

T16 - Coding sandbox.

T17 - Replay / decision envelope.

T18 - Market / pitch / non-tech explanation.

T19 - Security boundaries.

T20 - Operational deployment readiness.

Ce parcours donne à Brody une valeur stratégique : il devient la couche qui permet d'industrialiser la compréhension.

Sans Brody, Obsidia peut rester un système profond mais difficile à transmettre.

Avec Brody, Obsidia peut devenir un système apprenable.

19.9 Brody et la différence avec un chatbot classique

Un chatbot classique cherche surtout à répondre.

Brody cherche à orienter correctement dans une architecture.

Un chatbot peut improviser.

Brody doit respecter les statuts.

Un chatbot peut mélanger souvenir, opinion et preuve.

Brody doit séparer :

- ce qui est confirmé ;
- ce qui est figé ;
- ce qui est testé ;
- ce qui est documenté ;
- ce qui reste inconnu ;

- ce qui ne doit pas être affirmé.

Un chatbot peut chercher à satisfaire l'utilisateur.

Brody doit parfois ralentir, refuser, demander validation ou rappeler une limite.

Formule :

Brody n'est pas un assistant flatteur. C'est un assistant cadré par la gouvernance Obsidia.

19.10 Brody comme bénéfice stratégique

Brody apporte cinq bénéfices.

Premier bénéfice : compréhension.

Il rend Obsidia lisible pour des publics différents.

Deuxième bénéfice : transmission.

Il permet de former développeurs, investisseurs, auditeurs, partenaires et futurs utilisateurs.

Troisième bénéfice : mémoire contrôlée.

Il peut transformer une session en mémoire candidate, sans promotion automatique.

Quatrième bénéfice : accélération.

Il aide à produire supports, tickets, plans, tests et rapports plus vite.

Cinquième bénéfice : sécurité.

Parce qu'il est explicitement non souverain, il montre que même la couche cognitive du projet est encadrée.

Phrase investisseur :

Brody est le moteur pédagogique qui rend industrialisable une architecture complexe, sans jamais capter l'autorité du Kernel.

19.11 Ce qu'il faut dire sur Brody selon le public

Investisseur :

Brody rend Obsidia compréhensible et transmissible. C'est la couche éducative qui permet de former, expliquer, préparer et documenter sans donner le pouvoir à l'assistant.

CTO :

Brody est une interface cognitive advisory / readonly. Il peut lire les audits, expliquer les preuves, préparer des tickets ou des candidats mémoire, mais il ne peut pas décider, écrire Graphiti, muter le Kernel ou émettre une action.

Banque / conformité :

Brody peut aider les équipes à comprendre pourquoi une action a été suspendue, bloquée ou autorisée. Mais la décision reste traçable côté Kernel.

Non-tech :

Brody est le professeur. X108 est le feu de sécurité.

Big Tech :

Brody montre comment une interface agentique peut être utile sans devenir souveraine.

19.12 Phrases prêtes à utiliser sur Brody

Brody explique le système, mais ne décide pas pour lui.

Brody transforme la complexité en parcours d'apprentissage.

Brody est un pseudo-LLM cadré : utile pour comprendre, interdit de gouverner.

Brody peut proposer une mémoire candidate, mais il ne peut pas l'imposer au canon.

Brody peut lire une preuve, mais il ne peut pas inventer un statut de preuve.

Brody est la couche qui rend Obsidia transmissible.

Brody n'est pas une autorité. C'est une interface éducative sous contraintes.

19.13 Phrases à éviter sur Brody

Ne pas dire :

- Brody décide ;
- Brody est le cerveau souverain ;
- Brody remplace le Kernel ;
- Brody écrit la mémoire canonique ;
- Brody peut modifier Graphiti ;
- Brody peut muter le Kernel ;
- Brody garantit la vérité ;
- Brody est un LLM propriétaire complet entraîné from scratch ;
- Brody fait tout tout seul.

Dire plutôt :

- Brody assiste ;
- Brody explique ;
- Brody prépare ;
- Brody classe ;
- Brody propose ;
- Brody enseigne ;
- Brody transforme en candidat ;

- Brody reste non souverain.

20. Graphiti - Mémoire contrôlée, non souveraine

Graphiti représente la mémoire et les relations de contexte.

Mais la mémoire n'est pas une autorisation.

Cette distinction est centrale.

Un système peut se souvenir d'un contexte, mais ce souvenir ne doit pas suffire à déclencher une action critique.

Dans Obsidia, Graphiti est traité comme mémoire readonly dans le périmètre stabilisé. Il peut aider à comprendre et relier, mais il ne prend pas le pouvoir.

Phrase simple :

Se souvenir n'est pas autoriser.

Cette phrase est facile à comprendre et utile face à un public non technique.

21. Sigma - Transformer le signal en structure

Sigma sert à faire remonter les signaux des domaines.

Il organise les votes, les contradictions, les sévérités, les scores et les agrégations.

Mais Sigma n'est pas le juge final.

La bonne formule :

Sigma structure le signal. X108 gouverne la décision.

Dans le récit d'Étienne, Sigma représente une capacité importante : transformer le bruit du monde en objets lisibles. Cela rejoint son mode de pensée : observer, relier, classer, séparer, puis laisser le Kernel trancher selon une règle claire.

22. Proof stack - Ne pas seulement promettre, prouver

La Proof stack donne la crédibilité du projet.

Lean 4 permet de vérifier formellement certaines propriétés.

TLA+ permet de modéliser des comportements temporels et système.

Merkle permet de garantir l'intégrité de fichiers ou d'états.

RFC3161 permet d'ancrer temporellement une preuve.

Les freezes capturent des états locaux exploitables.

Le replay permet de rejouer et comprendre une décision.

Le message public doit être prudent : ces preuves ne certifient pas tout le système de production. Elles prouvent des propriétés précises et montrent une culture rare : ne pas demander seulement de croire, mais produire des traces vérifiables.

23. Petite machine, architecture lourde

Un des angles forts du parcours est la construction d'un système ambitieux avec des moyens limités.

Le message n'est pas : plus besoin de GPU, plus besoin de données, plus besoin de modèles.

Le message est plus précis : le cœur de valeur d'Obsidia ne dépend pas uniquement du big training. Il dépend de la structure.

Sur une machine limitée, il est possible d'assembler :

- un Kernel décisionnel ;
- une API bridge ;
- des domaines terrain ;
- des tests ;
- des preuves ;
- des freezes ;
- une mémoire readonly ;
- un assistant éducatif ;
- un WorkCenter en préparation.

Phrase :

La puissance ne vient pas seulement du GPU. Elle vient de l'organisation des couches.

24. La trajectoire solo avec IA copilotes

Le projet a été développé en recherche appliquée indépendante, avec l'IA comme copilote intensif.

Cette trajectoire doit être présentée comme une preuve de vitesse, pas comme une posture héroïque.

Ce que cela montre :

- capacité à apprendre vite ;
- capacité à structurer ;
- capacité à diriger des IA plutôt qu'à être dirigé par elles ;
- capacité à documenter ;
- capacité à auditer ;
- capacité à corriger les erreurs ;
- capacité à produire des supports exploitables ;
- capacité à transformer une vision en architecture.

Formulation investisseur :

Le risque d'exécution reste réel, mais la vitesse de consolidation montre une capacité rare : transformer une intuition architecturale en système auditable.

25. Ce qu'il faut dire selon le public

Investisseur :

Je développe une infrastructure de gouvernance décisionnelle pour agents IA. Le marché construit des agents capables d'agir ; Obsidia construit le Kernel qui vérifie s'ils ont le droit d'agir.

CTO :

Obsidia est une architecture OS-like avec séparation stricte entre cognition, mémoire, agrégation, décision, preuve et interface. La décision souveraine reste dans X108.

Banque / conformité :

Obsidia sépare recommandation et autorisation. Avant une action financière, il peut suspendre, bloquer, tracer et rendre la décision rejouable.

Incubateur :

Je viens d'un parcours atypique et j'ai transformé une vision de sécurité en système de gouvernance IA. Le projet est en POC avancé / pré-industrialisation.

Big Tech :

Vous construisez les capacités. Obsidia construit une couche d'autorité vérifiable pour le passage à l'action.

Non-tech :

Quand une IA veut faire quelque chose dans le monde réel, Obsidia vérifie si elle a le droit de le faire.

26. Ce qu'il ne faut pas dire

Ne pas dire :

- je suis production-ready ;
- je remplace OpenAI, Google ou Microsoft ;
- le système est certifié ;
- tout est prouvé de bout en bout ;
- les preuves Lean prouvent tout le produit industriel ;
- Brody décide ;
- Graphiti décide ;
- le WorkCenter remplace le Kernel ;
- le Kernel peut être modifié depuis l'interface ;

- je n’ai plus besoin de données ni de modèles ;
- je suis le premier mondial ;
- mon parcours personnel suffit à prouver la valeur.

Dire plutôt :

- le projet est en recherche appliquée / POC avancé / pré-industrialisation ;
- certaines briques sont testées, figées ou confirmées ;
- les limites sont documentées ;
- la trajectoire est précoce sur un problème qui devient central ;
- Obsidia est complémentaire aux modèles et frameworks agents ;
- la valeur vient de la gouvernance du passage à l’action.

27. Version courte pour rendez-vous investisseur

Je m’appelle Étienne Aubin. Je viens d’un parcours atypique, terrain et entrepreneurial, pas d’un laboratoire ou d’une grande école. Depuis février 2025, je développe Obsidia / X108, une architecture de gouvernance décisionnelle pour agents IA.

Le problème que je traite est simple : les IA ne font plus seulement des réponses. Elles commencent à appeler des outils, manipuler des workflows et déclencher des actions. Le vrai risque devient donc : qui contrôle leur droit d’agir ?

Obsidia place un Kernel X108 entre recommandation et action. Ce Kernel peut autoriser, suspendre ou bloquer : ALLOW, HOLD, BLOCK. Les autres composants peuvent expliquer, mémoriser, agréger ou afficher, mais ils ne décident pas.

Le projet est en POC avancé / pré-industrialisation. Il a déjà des preuves locales : domaines Bank, Trading, GPS ; API bridge ; Brody education ; Graphiti readonly ; 57 théorèmes Lean ; freezes et documents d’industrialisation.

Je ne construis pas un agent de plus. Je construis la couche qui vérifie si les agents ont le droit d’agir.

28. Version courte pour CTO

Obsidia / X108 est une architecture OS-like de gouvernance pour agents IA et workflows critiques.

La séparation d’autorité est stricte : modèle, mémoire, UI, API, domaines et assistant éducatif restent non souverains. Le Kernel X108 est l’autorité finale : ALLOW, HOLD, BLOCK, avec priorité BLOCK > HOLD > ALLOW.

L’intérêt technique est de ne pas traiter la sécurité comme un simple overlay, mais comme une topologie d’autorité. Les couches sont séparées : Sigma pour l’agrégation, Brody pour l’éducation, Graphiti pour la mémoire readonly, FastAPI pour le bridge, Proof stack pour Lean / TLA / Merkle / RFC3161, WorkCenter pour cockpit industriel.

Le système est encore POC avancé, mais plusieurs briques sont déjà confirmées par runtime, test ou freeze. La prochaine étape est le WorkCenter opérationnel, puis audit externe et packaging.

29. Version courte pour banque / conformité

Obsidia / X108 peut être présenté comme une couche de séparation entre recommandation et autorisation.

Dans un contexte bancaire ou réglementé, une IA peut recommander, scorer ou détecter un signal. Mais l'action ne doit pas être automatique. Il faut pouvoir suspendre, bloquer, tracer, expliquer et rejouer.

X108 répond à cette logique avec un Kernel de décision : ALLOW, HOLD, BLOCK.

L'intérêt pour une banque n'est pas de remplacer les outils existants, mais de s'insérer entre les signaux, les agents, les workflows et l'action réelle.

Le système reste à auditer, durcir et valider domaine par domaine, mais il porte déjà une doctrine cohérente : ce qui recommande ne doit pas autoriser seul.

30. Version personnelle filtrée

Mon parcours n'est pas linéaire. Je ne viens pas d'un chemin académique classique. J'ai appris par le terrain, par l'entrepreneuriat, par l'observation et par une confrontation intensive avec l'IA.

Ce chemin m'a donné une obsession : comprendre comment une décision devient une conséquence réelle.

Obsidia est né de là.

Je ne voulais pas construire un chatbot de plus. Je voulais construire une architecture qui empêche un système intelligent de confondre capacité et autorisation.

Le projet est aussi une manière de transformer une pensée très large, parfois chaotique, en structure : Kernel, statuts, preuves, audit, mémoire contrôlée, séparation des pouvoirs.

Ce n'est pas seulement un projet technique. C'est une vision de responsabilité : si l'IA devient capable d'agir, son droit d'agir doit être gouverné.

31. Section "chemin de vie" - Version longue maîtrisée

Le chemin d'Étienne peut être raconté sans entrer dans l'exposition intime.

Il commence par un parcours non académique, un rapport direct au terrain, une construction entrepreneuriale progressive et une capacité à apprendre par immersion.

L'expérience de terrain a appris une chose centrale : une décision n'est jamais neutre quand elle touche le réel. Cette sensibilité à l'irréversibilité a ensuite rencontré l'IA au moment où les agents commençaient à devenir capables d'agir.

C’est là que la question Obsidia est apparue : si une IA peut proposer, appeler un outil, automatiser un workflow ou déclencher une décision, qui contrôle l’autorisation ?

Le projet s’est construit comme une réponse à cette question.

Ce chemin de vie explique la forme du système :

- refus des décisions opaques ;
- besoin de preuves ;
- séparation stricte des rôles ;
- vérification avant action ;
- mémoire non souveraine ;
- traces et replay ;
- Kernel comme verrou central.

Le parcours n’est pas utilisé comme argument émotionnel. Il sert à expliquer pourquoi cette architecture a émergé sous cette forme.

32. Section “vision du monde”

La vision portée par Obsidia est simple : les systèmes puissants doivent être responsables de leurs passages à l’action.

Le monde accélère : automatisation, agents, finance algorithmique, robots, workflows industriels, IA dans les outils métier. Plus les systèmes deviennent capables, plus la question de l’autorité devient centrale.

Obsidia défend une idée : l’alignement ne doit pas être seulement une intention inscrite dans un prompt ou un modèle. Il doit être une structure.

Structure veut dire :

- règles explicites ;
- frontières d’autorité ;
- décision traçable ;
- preuve ;
- replay ;
- mémoire contrôlée ;
- refus possible ;
- temporalité ;
- protocole de changement.

La vision n’est pas anti-IA. Elle est pro-gouvernance.

L’IA peut devenir plus utile si son passage à l’action est mieux contrôlé.

33. Section “revanche” - Version à cadrer

L’idée de revanche peut exister dans le récit interne, mais elle doit être filtrée dans la version publique.

Version à éviter :

Je veux prouver au monde qu’il avait tort.

Version à utiliser :

Mon parcours m’a donné une énergie de construction. Je veux montrer qu’un chemin atypique peut produire une architecture sérieuse, utile et vérifiable.

Version investisseur :

Ce parcours donne au projet une force d’exécution particulière : peu de moyens, beaucoup de discipline, une vitesse d’apprentissage forte, et une capacité à transformer une vision complexe en livrables concrets.

Il faut donc garder l’intensité, mais retirer le côté confrontation personnelle.

34. Section “industrialisation personnelle”

La prochaine étape du projet est le passage de l’architecture prouvée localement à une surface industrialisable.

Les priorités :

1. Finaliser le WorkCenter / Cockpit.
2. Stabiliser la gouvernance mémoire.
3. Rendre les preuves lisibles pour un tiers.
4. Produire une démo Bank / Trading / GPS claire.
5. Resynchroniser une vitrine GitHub propre.
6. Préparer audit externe.
7. Structurer les parcours d’éducation Brody.
8. Préparer les premiers POC contrôlés.

Cette roadmap est aussi une roadmap personnelle : passer du laboratoire intense à une structure transmissible.

Le projet ne doit plus seulement fonctionner pour Étienne. Il doit devenir compréhensible, testable et exploitable par d’autres.

35. Q&A - Questions difficiles

Question : Vous n’avez pas un parcours d’ingénieur classique. Pourquoi vous croire ?

Réponse : Justement parce que le projet ne repose pas sur une affirmation personnelle. Il repose sur des preuves, des tests, des freezes, des documents d’architecture, des limites explicites et

une trajectoire de consolidation. Mon rôle est celui d'un architecte IA indépendant : je structure, j'oriente, je teste, j'audite et je transforme la vision en système vérifiable.

Question : Êtes-vous production-ready ?

Réponse : Non. Le projet est en recherche appliquée / POC avancé / pré-industrialisation. Aucun usage critique ne doit être fait sans audit externe, durcissement, certification et validation domaine.

Question : Pourquoi ne pas simplement utiliser les guardrails existants ?

Réponse : Les guardrails sont utiles, mais ils sont souvent ajoutés autour du modèle. Obsidia sépare l'autorité dès la conception. Le modèle peut proposer, mais le Kernel décide.

Question : Pourquoi Big Tech ne ferait pas la même chose ?

Réponse : Big Tech construit déjà beaucoup de briques : agents, outils, orchestration, monitoring, sécurité. Obsidia est complémentaire : il formule le problème comme un Kernel de gouvernance du droit d'agir. La valeur est dans la cohérence de l'architecture et la séparation d'autorité.

Question : Le projet est-il trop complexe ?

Réponse : Il est complexe parce que le problème l'est. Le WorkCenter sert précisément à rendre cette complexité lisible : statuts, preuves, domaines, décisions, mémoire, validation et roadmap.

36. Phrases publiques prêtes à utiliser

Je ne construis pas un agent de plus. Je construis la couche qui vérifie si un agent a le droit d'agir.

Ce qui propose ne doit pas décider seul.

Une IA capable n'est pas forcément une IA autorisée.

La question n'est plus seulement : que peut faire l'IA ? La question devient : qu'a-t-elle le droit de faire ?

Obsidia transforme l'action candidate en décision gouvernée : ALLOW, HOLD ou BLOCK.

Le Kernel X108 empêche de confondre capacité et autorisation.

La mémoire informe, mais elle n'autorise pas.

L'assistant explique, mais il ne gouverne pas.

La puissance ne vient pas seulement du GPU. Elle vient aussi de l'organisation des couches.

Mon parcours est atypique, mais le projet ne demande pas d'y croire : il produit des preuves, des tests et des limites documentées.

37. Phrases à garder privées ou à éviter

À éviter dans un support public :

- détails trop intimes ;
- détails médicaux ;
- habitudes personnelles qui peuvent détourner l'attention ;
- formulations de revanche trop frontales ;
- accusations globales contre le monde ;
- promesses historiques excessives ;
- affirmation que tout est terminé ;
- comparaison frontale inutile avec Big Tech.

À remplacer par :

- parcours non linéaire ;
- apprentissage par le terrain ;
- construction indépendante ;
- vision de responsabilité ;
- sécurité par architecture ;
- pré-industrialisation ;
- complémentarité avec Big Tech ;
- discipline d'audit.

38. Version "About founder"

Étienne Aubin est un entrepreneur autodidacte et architecte IA indépendant. Après un parcours terrain et entrepreneurial, il développe depuis février 2025 Obsidia / X108, une architecture de gouvernance décisionnelle pour agents IA et workflows critiques.

Son travail se concentre sur une question centrale : quand une IA passe de la recommandation à l'action, qui contrôle son droit d'agir ?

Obsidia répond par une architecture OS-like centrée sur un Kernel X108, capable de produire des décisions ALLOW, HOLD ou BLOCK, avec séparation stricte entre cognition, mémoire, agrégation, interface, preuve et autorité.

Le projet est actuellement en recherche appliquée / POC avancé / pré-industrialisation, avec des briques déjà testées ou figées : domaines Bank / Trading / GPS, API bridge, Brody Education comme pseudo-LLM éducatif non souverain, Graphiti readonly, preuves Lean, freezes et WorkCenter en cadrage final.

39. Version "lettre personnelle publique"

Je viens d'un parcours atypique. Je n'ai pas suivi la voie classique des grandes écoles ou des laboratoires. J'ai appris par le terrain, par l'entrepreneuriat, par l'observation et par une immersion intense dans l'IA.

Obsidia est né d'une question simple : si les IA deviennent capables d'agir, qui contrôle leur droit d'agir ?

Je ne voulais pas construire un chatbot de plus. Je voulais construire une architecture qui sépare ce qui pense, ce qui mémorise, ce qui affiche, ce qui agrège et ce qui décide.

Le cœur du projet est le Kernel X108. Il ne cherche pas à parler. Il gouverne : ALLOW, HOLD ou BLOCK.

Mon objectif est de rendre le passage à l'action des systèmes IA plus sûr, plus traçable, plus vérifiable et plus responsable.

Le projet n'est pas encore un produit industriel. Mais il a déjà dépassé le stade de l'idée. Il existe en preuves, en tests, en freezes, en domaines branchés et en architecture documentée.

Ce que je construis, c'est une réponse à un monde où les IA deviennent capables. Plus elles deviennent puissantes, plus leur droit d'agir doit être gouverné.

40. Conclusion

Cette version V3 recentre le support sur le chemin de vie, la vision et Brody comme couche de médiation éducative non souveraine.

Le parcours reste présent, mais il n'écrase pas le projet.

Les difficultés ne deviennent pas le sujet principal. Elles sont remplacées par une lecture plus forte : chemin non linéaire, terrain, observation, apprentissage intensif, responsabilité, architecture.

La bonne synthèse est la suivante :

Étienne ne vend pas une histoire personnelle. Il porte une architecture née d'un chemin de vie.

Obsidia / X108 ne vend pas un agent de plus. Il propose une gouvernance du droit d'agir.

Le chemin donne le sens. Le Kernel donne la structure. Les preuves donnent la crédibilité. Le WorkCenter donnera la transmissibilité.

SUPPORT PITCH INVESTISSEUR OBSIDIA / X108

Gouverner le droit d agir des IA
Du Kernel securise au WorkCenter industriel

Version V1 complete - comprehension simple, credibilite technique, pre-industrialisation

Perimetre : sans previsionnel financier detaille ni analyse concurrentielle complete - prevus en V2

Statut projet : recherche appliquee / POC local avance / pre-industrialisation

Principe : comprenable par un enfant, credible pour un investisseur, assez solide pour declencher des questions techniques sans noyer l interlocuteur.

Perimetre : ce support ne traite pas encore le previsionnel financier detaille ni l analyse concurrentielle complete. Ces deux blocs sont prevus en V2.

Statut honnête : Recherche appliquee / POC local avance / pre-industrialisation. Non production-ready sans audit independant, durcissement, certification et validation domaine.

Sources internes utilisees

Source	Usage
README_OBSIDIA_X108_GITHUB_V2(2).md	statut, architecture, Kernel, API, domaines, preuves
README_REALITY_RECHECK_REPORT(2).md	recheck 2026-06-17 : API runtime, Lean, Brody, Graphiti, domaines
OBSIDIA_X108_GLOSSAIRE_INTEGRAL_AUDITE(2).md	definitions auditees, autorite, statuts, invariants, limites
SUPPORT_POSITIONNEMENT_VENTE_INDUSTRIALISATION_OBSIDIA_X108_V1_COMPLETE.md	support maitre positionnement / industrialisation
CAHIER_DES_CHARGES_DEV_COCKPIT_OBSIDIA_X108_COMPLET_V1.md	cahier des charges WorkCenter / Cockpit
CHECKLIST_IMPLEMENTATION_COCKPIT_OBSIDIA_X108_V1_CORRIGEE.md	checklist V0 a V5
MATRICE_IMPLEMENTATION_COCKPIT_OBSIDIA_X108_V1_COMPLETE.md	matrice tickets
CONTRAT_API_WORKCENTER_COCKPIT_OBSIDIA_X108_V1_COMPLETE.md	contrat API WorkCenter
OBJECT_MODEL_DATA_REGISTRY_COCKPIT_OBSIDIA_X108_V1_COMPLETE.md	modeles de donnees
SECURITY_AUTHORITY_BOUNDARY_MATRIX_COCKPIT_OBSIDIA_X108_V1_COMPLETE.md	security boundary matrix

Mode d emploi du support

Ce document n est pas un cahier des charges technique. C est un support de comprehension et de vente. Il explique l histoire, le probleme, la solution, les preuves, les limites et la trajectoire. La technique pure vient seulement si l interlocuteur demande.

La regle de presentation est simple : commencer par une image que tout le monde comprend, puis ouvrir les couches du systeme une par une.

> ***Image directrice : une IA peut proposer une action. Obsidia est le feu rouge, l aiguilleur et la boite noire qui disent si cette action peut passer, doit attendre ou doit etre bloquee.***

```
Pitch en une phrase :

Obsidia / X108 est une couche de gouvernance pour agents IA : elle se place entre recommandation et
action pour autoriser, suspendre, bloquer, tracer, prouver et rejouer les decisions.
```

PARTIE A - RESUME EXECUTIF INVESTISSEUR

A1. Le probleme en mots simples

Les IA savent parler. Les agents IA commencent a agir. Le vrai risque n est plus seulement une mauvaise reponse. Le vrai risque devient une mauvaise action.

Exemple enfant : si un enfant demande a un robot de traverser la route, on ne veut pas que le robot traverse juste parce qu il a compris la phrase. On veut un feu rouge, un adulte, une regle, une preuve et une raison.

Dans une entreprise, c est pareil : si une IA peut envoyer de l argent, lancer un ordre, changer un fichier, declencher un workflow ou influencer une decision critique, il faut une autorite claire.

A2. La solution en mots simples

Obsidia ajoute cette autorite. Il ne cherche pas seulement a produire une bonne reponse. Il gouverne le droit d agir.

```
Signal -> Action candidate -> Kernel X108 -> ALLOW / HOLD / BLOCK -> Trace -> Preuve -> Replay
```

Le Kernel X108 est le coeur : il decide si une action candidate est autorisee, suspendue ou bloquee. Tout le reste aide, explique, memorise, structure ou affiche, mais ne prend pas sa place.

A3. La these investisseur

- Les agents IA vont se multiplier dans les entreprises.
- Plus ils deviennent capables, plus le controle du passage a l action devient critique.
- Le marche aura besoin de couches de gouvernance, de preuve, de replay et d audit.
- Obsidia est positionne sur cette couche : avant l action, pas seulement apres l incident.
- Le projet dispose deja d actifs techniques locaux : Kernel, API bridge, domaines, tests, preuves, memoire readonly, Brody Education, WorkCenter en preparation.

A4. Ce qui rend Obsidia different

Approche classique	Approche Obsidia
Rendre le modele plus gros	Rendre le passage a l action plus gouverne
Faire confiance au raisonnement probabiliste	Separer assistance, memoire, simulation, decision et preuve
Ajouter un agent qui fait tout	Distribuer les roles et verrouiller l autorite
Optimiser la reponse	Controler ALLOW / HOLD / BLOCK
Dependance forte au big training	Valeur centrale par structure deterministe, invariants, gates, preuves, replay

A5. Ce qui est deja stabilise

Bloc	Fait presentable	Valeur pitch
Kernel X108	Kernel X108 : composant souverain de decision, endpoint POST /kernel/ragnarok, port 3001 confirme par freeze, bridge API cable.	Le coeur souverain existe et a capture des decisions en freeze.
API FastAPI	API FastAPI 8000 : V5B, CONFIRMED_BY_RUNTIME, HTTP 200, 164 routes OpenAPI, readonly_dryrun, BRIDGE_ONLY, decision_authority=KX108_ONLY.	La surface API est active, bridge only, readonly, avec 164 routes.
Securite	allowed_to_decide=false, emits_act=false, emits_verdict=false, memory_write=false, graphiti_write=false, kernel_mutation=false, real_action=false.	Les flags empechent la substitution d autorite par l API.
Domaines terrain	Domaines branches : Bank, Trading, GPS/Aviation. Decisions freeze : Bank 5 ALLOW S1, Trading 5 BLOCK S4, GPS/Aviation 5 ALLOW S0.	Bank, Trading, GPS/Aviation ne sont pas juste des idees.
Brody	Brody Education Pack V1 readonly, 124/124 tests PASS, non souverain, memory_write=false, emits_act=false.	Couche educative non souveraine testee.
Graphiti	Graphiti V20 FROZEN_READONLY, 167 nodes, 477 relations, 20 episodes, live	Memoire readonly structuree, non souveraine.

	Neo4j dependency=false dans snapshot frozen.	
Lean	Lean : 57 theoremes / lemmes verifies par lake build PASS.	Une partie des invariants est formalisee et verifiee.

A6. Ce qui reste a finaliser avant industrialisation

- Construire le WorkCenter / Cockpit de comprehension, validation, audit et pilotage.
- Relancer TLA/TLC dans le repo exact et ne pas sur-declarer les anciens resultats.
- Verifier RFC3161 via openssl.
- Relancer les batteries adversariales dans le repo exact actuel.
- Durcir Graphiti live / Neo4j auth si passage au live memory.
- Tester ECOM live et nouveaux domaines.
- Mettre une CI complete, un processus release, un audit externe et un pilot POC controle.

A7. Ce qui manque volontairement dans cette V1

Deux blocs seront ajoutes ensuite :

- Previsionnel financier : couts, run rate, besoins, scenarii, jalons de financement.
- Concurrence : cartographie agent frameworks, MLOps, RAG, LLM platforms, policy engines, GRC, observability, audit AI.

PARTIE B - HISTOIRE SIMPLE POUR EXPLIQUER A UN ENFANT

B1. La ville, les voitures et le feu rouge

Imagine une ville. Dans cette ville, il y a beaucoup de voitures. Les voitures vont de plus en plus vite. Elles voient les routes, elles calculent les trajets, elles savent tourner, accelerer et freiner.

Mais si chaque voiture decide toute seule, la ville devient dangereuse. Il faut des feux rouges, des panneaux, des priorites, des policiers, des traces et parfois une boite noire pour comprendre ce qui s est passe.

Les IA agents sont comme ces voitures. Elles savent de plus en plus faire des choses. Obsidia est le systeme qui dit : cette action peut passer, cette action doit attendre, cette action est interdite.

Dans l image	Dans Obsidia
La voiture	Un agent IA ou un workflow
Le conducteur	Le modele / assistant / outil
Le feu rouge	Le Kernel X108
Les panneaux	Les invariants et policies
La boite noire	Audit / hash / replay / freeze
La carte de la ville	Graphiti / memoire readonly
Le copilote	Brody Education
La tour de controle	WorkCenter / Cockpit

B2. La phrase enfant

Obsidia, c est le systeme qui empeche une IA de faire n importe quoi quand elle commence a agir.

B3. La phrase investisseur

Obsidia se positionne comme une infrastructure de gouvernance ex ante pour agents IA : une couche qui controle le passage entre recommandation et action par decision deterministe, trace, preuve et replay.

PARTIE C - SUPPORT PITCH INVESTISSEUR COMPLET

1. Pourquoi maintenant : les agents IA passent a l action

Version enfant

Les IA deviennent comme des robots qui peuvent appuyer sur des boutons. Avant qu elles appuient, il faut un adulte, un feu rouge et une boite noire.

Message investisseur

Les agents IA ne sont plus seulement des assistants textuels. Ils lisent, appellent des outils, declenchent des scripts, preparent des emails, interrogent des bases, modifient des fichiers et peuvent etre branches a des APIs metier.

Quand une IA parle, l erreur reste souvent informationnelle. Quand une IA agit, l erreur devient operationnelle. Elle peut couter de l argent, abimer une relation client, ouvrir un risque legal ou declencher une chaine automatique difficile a annuler.

Le moment est donc celui du passage de la generation a l action. C est exactement la zone ou Obsidia se place : avant l action, au moment ou le systeme doit decider s il autorise, suspend ou bloque.

Pourquoi c est important

- Faire comprendre que le timing marche est naturel : plus les agents deviennent capables, plus la gouvernance devient inevitable.
- Ne pas dire que tous les agents sont dangereux ; dire que les agents utiles exigent une autorite claire.

Questions que cela doit declencher

- Quel composant decide si plusieurs agents proposent des actions contradictoires ?
- Comment prouvez-vous qu une action a ete autorisee ou bloquee ?
- Comment gerez-vous un agent qui parle bien mais ne doit pas agir ?

Phrase de slide

> ***Les agents IA passent du texte a l action : le marche a besoin d une couche d autorite.***

2. Le probleme : aucune autorite claire entre recommandation et action

Version enfant

Le probleme, c est que parfois on sait qui parle, mais pas qui a le droit de faire.

Message investisseur

Dans beaucoup de systemes, une IA recommande, un script execute, une API transmet et un dashboard affiche. Mais la question de l autorite est dispersee. On sait qui produit un signal, mais pas toujours qui a le droit de transformer ce signal en action.

Obsidia part de ce vide. Entre une recommandation et une action, il faut une couche explicite : qui autorise, qui bloque, qui suspend, qui trace, qui prouve et qui rejoue.

C est une problematique transversale : banque, trading, e-commerce, GPS, aviation, defense, gestion interne, workflows critiques.

Pourquoi c est important

- Mettre le probleme sur la table sans jargon.
- Faire sentir que ce n est pas un marche de gadget mais d infrastructure.

Questions que cela doit declencher

- Le marche le plus urgent est-il finance, banque, conformité ou agent dev ?
- Le systeme est-il un produit horizontal ou un moteur verticalisable ?

Phrase de slide

> ***Entre recommandation et action, il manque un arbitre verifiable.***

3. La solution : Obsidia gouverne le droit d agir

Version enfant

Obsidia est le systeme qui dit oui, attends ou non avant que l IA agisse.

Message investisseur

Obsidia n essaie pas de remplacer les modeles IA. Il ajoute la couche qui manque autour d eux : une couche qui gouverne l action.

Le systeme recoit des signaux, structure le contexte, agrege des votes, appelle le Kernel, produit une decision, conserve une trace et permet le replay.

La solution est lisible : ALLOW pour autoriser, HOLD pour suspendre, BLOCK pour interdire.

Pourquoi c est important

- Definir Obsidia comme infrastructure de gouvernance decisionnelle.
- Ne pas le presenter comme simple chatbot, dashboard ou wrapper.

Questions que cela doit declencher

- A quel endroit exact s insere Obsidia dans une stack entreprise ?
- Comment se branche-t-il sans remplacer toute la stack existante ?

Phrase de slide

> ***Obsidia gouverne le droit d agir : ALLOW, HOLD, BLOCK.***

4. Le coeur : Kernel X108

Version enfant

Le Kernel, c est l arbitre. Il ne raconte pas une histoire : il tranche.

Message investisseur

Le Kernel X108 est le coeur du systeme. C est lui qui porte l autorite finale ALLOW / HOLD / BLOCK.

Les autres couches peuvent etre utiles, mais elles ne deviennent pas souveraines. L API transmet. Brody explique. Graphiti memorise. Sigma agrege. Les domaines signalent. Le Kernel gouverne.

Le Kernel est confirme par freeze avec endpoint POST /kernel/ragnarok et decisions capturees sur plusieurs domaines.

Pourquoi c est important

- Montrer le Kernel tres tot dans le pitch.
- Expliquer le Kernel comme feu rouge / tour de controle / arbitre final.

Questions que cela doit declencher

- Le Kernel est-il interpretable ?
- Quelles garanties formalisees portent sur sa logique ?
- Quelle est la difference entre Kernel runtime et Kernel freeze ?

Phrase de slide

> ***Le Kernel X108 est le verrou souverain.***

5. La securite : separation d autorite

Version enfant

Chaque personne dans l equipe a son role. Celui qui explique ne peut pas devenir le chef par surprise.

Message investisseur

La securite d Obsidia vient de la separation des pouvoirs. Aucun composant non souverain ne doit pouvoir s inventer une autorite.

L API est BRIDGE_ONLY. Les flags captured dans /api/status indiquent notamment allowed_to_decide=false, emits_act=false, emits_verdict=false, memory_write=false, graphiti_write=false, kernel_mutation=false, real_action=false.

Cette separation rend le systeme plus auditable : on sait qui parle, qui memorise, qui agrege, qui decide et qui prouve.

Pourquoi c est important

- Ne jamais laisser croire que Brody ou le cockpit peuvent decider.
- Faire de la separation d autorite un avantage concurrentiel.

Questions que cela doit declencher

- Quelle couche peut ecrire en memoire durable ?
- Comment empecher un agent de contourner le Kernel ?
- Quel protocole autorise une evolution Kernel ?

Phrase de slide

> ***La securite vient de la separation stricte des pouvoirs.***

6. Les terrains stables : Bank / Trading / GPS

Version enfant

Bank, Trading et GPS sont trois terrains d essai : argent, marche, trajectoire.

Message investisseur

Trois terrains rendent le projet concret : Bank, Trading, GPS/Aviation. Ils montrent que le systeme n est pas seulement theorique.

Bank : 5 decisions ALLOW S1 capturees en freeze. Trading : 5 decisions BLOCK S4 avec contradiction threshold. GPS/Aviation : 5 decisions ALLOW S0 capturees en freeze.

Ces domaines montrent la logique multi-terrain : un meme noyau de gouvernance peut traiter des contextes differents sans devenir un agent general qui improvise.

Pourquoi c est important

- Afficher les terrains comme preuve de generalisation controlee.
- Rester precis : decisions freeze, API bridge runtime, Kernel pas declare live permanent si port ferme au recheck.

Questions que cela doit declencher

- Pourquoi ces trois domaines ?
- Quelle est la prochaine verticalisation prioritaire ?
- Comment passe-t-on d un domaine simule a un POC client ?

Phrase de slide

> ***Bank, Trading et GPS prouvent la logique multi-terrain.***

7. Brody : pseudo-LLM educatif et assistant non souverain

Version enfant

Brody est le professeur. Il explique le systeme, mais il ne donne pas l autorisation finale.

Message investisseur

Brody est la couche qui aide l humain a comprendre le systeme. Il peut expliquer, lire les audits, former un utilisateur, preparer des validations et proposer des candidats memoire.

Brody ne decide pas. Il ne valide pas seul. Il ne transforme pas une memoire en canon sans validation. Il ne remplace pas le Kernel.

La valeur de Brody est d humaniser la complexite : il rend lisibles le Kernel, les preuves, les domaines et le WorkCenter.

Pourquoi c est important

- Presenter Brody comme interface cognitive gouvernee.
- Eviter de le vendre comme LLM autonome magique.

Questions que cela doit declencher

- Quelle difference avec un chatbot interne ?
- Comment Brody est-il empeche d agir ?
- Que signifie pseudo-LLM dans l architecture ?

Phrase de slide

Brody explique et eduque, sans devenir souverain.

8. Graphiti : memoire readonly non souveraine

Version enfant

Graphiti est le carnet de notes. Il se souvient, mais il ne decide pas.

Message investisseur

Graphiti organise la memoire et le contexte. Dans l etat stabilise, il est presente en FROZEN_READONLY : il aide a retrouver, relier et contextualiser sans devenir source d autorite.

La memoire est utile mais dangereuse si elle decide. Obsidia pose donc une regle claire : ce qui memorise ne decide pas.

Le snapshot Graphiti V20 contient 167 nodes, 477 relations et 20 episodes dans le freeze, avec live Neo4j dependency=false pour la surface frozen.

Pourquoi c est important

- Montrer la memoire comme avantage de comprehension, pas comme decideur.
- Ne pas sur-vendre Graphiti live si Neo4j auth reste a durcir.

Questions que cela doit declencher

- Comment evitez-vous la memoire sauvage ?
- Quelle difference entre memoire candidate, memoire validee et canon ?

Phrase de slide

Graphiti memorise, mais ne decide pas.

9. Sigma : aggregation et signaux

Version enfant

Sigma range les signaux avant de les donner a l arbitre.

Message investisseur

Sigma est la couche qui structure les signaux avant le Kernel. Les domaines et agents remontent des votes, des severites, des contradictions et des indices de risque.

Sigma aide a transformer un chaos de signaux en objet decisionnel lisible. Mais Sigma ne remplace pas le Kernel. Il prepare et agrege.

Cette couche est importante pour verticaliser : bank, trading, GPS, e-commerce, CIC, NCP et futurs domaines.

Pourquoi c est important

- Expliquer Sigma simplement : une table de tri et d aggregation avant l arbitre final.
- Eviter de faire de Sigma le souverain.

Questions que cela doit declencher

- Comment les agents votent-ils ?
- Comment gerer les contradictions ?
- Comment prioriser BLOCK/HOLD/ALLOW ?

Phrase de slide

Sigma transforme les signaux en contexte decisionnel.

10. Proof stack : Lean / TLA / Merkle / RFC3161

Version enfant

Les preuves sont la boite noire et les controles qui montrent ce qui est vrai.

Message investisseur

Obsidia ne se limite pas a du code applicatif. Il contient une couche de preuve et d audit.

Lean : 57 theoremes / lemmes verifies par lake build PASS. TLA+ : specifications presentes, TLC a relancer dans le recheck exact. Merkle / RFC3161 : integrite et ancrage, avec verification externe a finaliser.

Pour un investisseur, le point n est pas de detailler chaque preuve. Le point est de montrer que la culture du projet est : ne pas affirmer sans statut, ne pas vendre sans limite, ne pas melanger documente et verifie.

Pourquoi c est important

- Mettre en avant la discipline de preuve sans pretendre certification.
- Toujours distinguer confirmed_by_test, confirmed_by_freeze, present_but_not_executed.

Questions que cela doit declencher

- Que prouvent exactement les 57 theoremes ?
- Qu est-ce qui n est pas encore prouve ?
- Quelle sera la strategie d audit externe ?

Phrase de slide

Les preuves rendent les claims auditables.

11. Approche deterministe : pas dependante du big training

Version enfant

Le coeur suit des regles, pas seulement une intuition.

Message investisseur

Le coeur d Obsidia ne repose pas sur le fait d entrainer un modele plus gros. Il repose sur une architecture de decisions, d invariants, de statuts, de gates et de preuves.

Les LLM restent utiles : ils expliquent, assistent, explorent, preparent. Mais la decision critique ne doit pas etre abandonnee a une sortie probabiliste.

Cela deplace l avantage : de la masse brute vers la structure.

Pourquoi c est important

- Formuler prudemment : pas dependant du big training pour le coeur decisionnel, pas pas besoin de donnees pour tout.
- Eviter anti-LLM. Obsidia est complementaire.

Questions que cela doit declencher

- Ou utilisez-vous les LLM ?
- Quelle part est deterministe ?
- Quel est le cout de scaling du coeur ?

Phrase de slide

Le coeur de valeur est deterministe, pas probabiliste.

12. Gain calcul / cout : structure > masse

Version enfant

On ne gagne pas seulement avec une grosse machine. On gagne aussi avec une bonne organisation.

Message investisseur

L approche Obsidia permet de construire beaucoup de valeur avec une petite machine, parce qu une grande partie du systeme est structurelle : registries, gates, tests, preuves, replay, statuts, policies, boundaries.

La puissance ne vient pas seulement du GPU. Elle vient de l organisation des couches. Un gros modele peut produire plus, mais sans gouvernance il peut produire une action dangereuse.

Obsidia cherche donc a reduire la pression sur le calcul massif pour la couche critique : la gouvernance du droit d agir.

Pourquoi c est important

- Ne pas dire plus besoin de GPU. Dire : moins de dependance au GPU pour le coeur de gouvernance.
- Montrer le leverage : petite machine, architecture lourde, preuves locales.

Questions que cela doit declencher

- Quels couts cloud a l echelle ?
- Quels modules demandent du calcul ?
- Comment differencier cout inference et cout gouvernance ?

Phrase de slide

> ***Structure > masse : moins de dependance au big training pour gouverner.***

13. Petite machine, architecture lourde

Version enfant

Comme construire une ville miniature solide avant de construire la grande ville.

Message investisseur

Le projet montre qu il est possible de construire une architecture ambitieuse localement : API, Kernel, domaines, Brody, Graphiti frozen, proofs, WorkCenter specs, freezes et audits.

Ce point est important pour investisseur : la creation de valeur ne depend pas encore d un budget GPU massif. Elle depend d une architecture claire et d un rythme de stabilisation.

L objectif n est pas de rester local. L objectif est de prouver d abord la structure, puis industrialiser proprement.

Pourquoi c est important

- Presenter la petite machine comme preuve de frugalite et de focus structurel.
- Ne pas en faire une limite : la roadmap inclut industrialisation, CI et audit externe.

Questions que cela doit declencher

- Quelle architecture cloud cible ?
- Quels modules restent locaux ?
- Quels couts pour un pilote ?

Phrase de slide

> ***Petite machine, architecture lourde : preuve de frugalite structurelle.***

14. Ce qui est deja stabilise

Version enfant

Plusieurs briques existent deja et ont ete testees ou figees.

Message investisseur

Les blocs stabilises donnent de la credibilite : API runtime, Kernel freeze, domaines branchés, Brody tests, Graphiti frozen, Lean build, boundaries, docs WorkCenter.

La force du projet est la discipline des statuts : confirmé runtime, confirmé test, confirmé freeze, confirmé code, present non execute, documented only.

Cette discipline est vendable car elle evite le discours creux. Elle montre qu Obsidia sait distinguer ambition, preuve et limite.

Pourquoi c est important

- Faire une slide preuve simple.
- Eviter de surcharger avec lignes de code ; mettre les details en annexe.

Questions que cela doit declencher

- Quels actifs sont brevetables ou defendables ?
- Quelles preuves sont independantes ?
- Quel est le prochain jalon de validation ?

Phrase de slide

> ***Le projet dispose deja d actifs stabilises.***

15. Ce qui reste honnetement a finaliser

Version enfant

Il reste des controles a refaire avant de mettre ca dans le monde reel.

Message investisseur

Le projet doit rester honnete : il n est pas production-ready pour des usages critiques sans audit, durcissement, certification et validation domaine.

Les points a finaliser : TLA/TLC, RFC3161 verify, adversarial re-run, Graphiti live, ECOM live, CI, audit externe, release process, pilot POC controle.

Cette honnetete ne diminue pas le projet. Elle le rend credible.

Pourquoi c est important

- Utiliser les limites comme preuve de maturite.
- Ne jamais cacher les statuts present_but_not_executed.

Questions que cela doit declencher

- Combien de temps pour audit externe ?
- Quelles certifications viser ?
- Quel POC client serait le plus propre ?

Phrase de slide

> ***Les limites sont connues et transformees en roadmap.***

16. Roadmap industrialisation

Version enfant

La route industrialisation transforme le labo en outil utilisable par d autres.

Message investisseur

La roadmap doit passer de POC local avance a pilote controle. D abord WorkCenter readonly, ensuite education, ensuite proof/audit re-run, ensuite durcissement, ensuite POC.

L industrialisation ne consiste pas a coder plus vite. Elle consiste a rendre le systeme lisible, testable, auditable, deployable et maintenable par d autres.

Le WorkCenter est donc une etape cle : il transforme un systeme complexe en surface comprenable par dev, non-dev, auditeur et investisseur.

Pourquoi c est important

- Montrer une route en paliers, pas une promesse floue.
- Rattacher chaque palier a une preuve livrable.

Questions que cela doit declencher

- Quel jalon declenche la levee ?

- Quel jalon declenche un pilote ?
- Quel jalon declenche un audit externe ?

Phrase de slide

L industrialisation passe par WorkCenter, CI, audit externe et pilotes controles.

17. Roadmap education poussee

Version enfant

Il faut apprendre aux gens a lire le systeme, comme apprendre le code de la route.

Message investisseur

L education est strategique parce qu Obsidia est une architecture, pas une simple application. Il faut former l utilisateur a comprendre les statuts, les limites, les decisions et les preuves.

Brody Education sert a ca : parcours non-tech, parcours dev, parcours audit, parcours investisseur, parcours domaine.

Un bon investisseur doit comprendre que l education n est pas du marketing annexe. C est une couche d adoption et de securite.

Pourquoi c est important

- Presenter l education comme reduction de friction commerciale et operationnelle.
- Montrer T1-T20 comme base.

Questions que cela doit declencher

- Comment un client apprend a utiliser le systeme ?
- Peut-on faire une certification utilisateur ?
- Quel est le role de Brody dans le support ?

Phrase de slide

L education est une couche d adoption et de securite.

18. Roadmap WorkCenter / Workbench

Version enfant

Le WorkCenter est la tour de controle du projet.

Message investisseur

Le WorkCenter est la surface qui rassemble les pieces : Kernel, securite, domaines, preuves, memoire, validation, session ledger, freeze factory, sandbox et education.

Il ne sert pas seulement a faire joli. Il sert a rendre l architecture pilotable et auditable.

A terme, il doit devenir un workbench gouverne : un environnement ou les agents peuvent aider a travailler, mais ou les limites d autorite restent visibles et imposees.

Pourquoi c est important

- Mettre le WorkCenter comme levier d industrialisation.
- Ne pas le reduire a une interface web.

Questions que cela doit declencher

- Quelles pages V1 ?
- Quelle difference avec Claude Code, Cursor, LangSmith ou un dashboard MLOps ?
- Comment eviter que le workbench devienne souverain ?

Phrase de slide

Le WorkCenter devient la tour de controle gouvernee.

19. Positionnement Big Tech

Version enfant

La Big Tech construit des moteurs enormes. Obsidia construit le systeme de controle autour de l action.

Message investisseur

La Big Tech construit des modeles toujours plus puissants et des workbenches toujours plus integres. Obsidia ne doit pas attaquer frontalement la taille des modeles.

L angle est different : dans un monde ou les modeles deviennent capables, qui controle ce qu ils ont le droit de faire ?

Obsidia peut rejoindre les workbenches modernes en construisant sa propre surface, puis depasser sur l angle gouvernance, preuves et separation d autorite.

Pourquoi c est important

- Positionner Obsidia en complement et couche critique, pas en modele rival au debut.
- Formuler : pas plus gros, mieux gouverne.

Questions que cela doit declencher

- Quelles integrations possibles avec modeles existants ?
- Quelle defensabilite contre les plateformes ?
- Quel module devient le wedge commercial ?

Phrase de slide

> ***Ne pas concurrencer la taille des modeles : controler leur droit d agir.***

20. Supports separes par public

Version enfant

On ne parle pas pareil a un enfant, un banquier, un dev ou un investisseur.

Message investisseur

Le meme projet doit etre explique differemment selon l interlocuteur. Un investisseur veut marche, moat, preuve, roadmap. Une banque veut risque, audit, autorisation. Un CTO veut architecture, interfaces et tests. Un non-tech veut images simples.

Le support maitre doit donc servir de base a plusieurs versions courtes.

La regle : meme verite, langage adapte.

Pourquoi c est important

- Preparer des versions par public apres ce support maitre.
- Ne pas changer les statuts entre supports.

Questions que cela doit declencher

- Quel support envoyer avant rendez-vous ?
- Quel support montrer en live ?
- Quel support laisser apres rendez-vous ?

Phrase de slide

> ***Un seul projet, plusieurs langages selon l interlocuteur.***

21. Conclusion : gouverner l action, pas seulement generer du texte

Version enfant

Le but final : controler l action, pas seulement produire du texte.

Message investisseur

Le coeur du message est simple : l IA entre dans l age de l action. Les entreprises auront besoin d une couche qui controle ce passage.

Obsidia propose cette couche : Kernel, separation d autorite, domaines, memoire readonly, assistant educatif, preuves, replay, WorkCenter.

La promesse n est pas de remplacer tous les modeles. La promesse est de rendre leur action gouvernable.

Pourquoi c est important

- Terminer sur une phrase forte et simple.
- Garder le detail technique pour les questions.

Questions que cela doit declencher

- Quel est le premier pilote ?
- Quel financement pour atteindre l audit externe ?
- Quelle equipe minimale pour industrialiser ?

Phrase de slide

> ***Gouverner l action, pas seulement generer du texte.***

PARTIE D - ARCHITECTURE SIMPLE A MONTRER EN RENDEZ-VOUS

D1. Schema verbal 30 secondes

```
Utilisateur / systeme

  -> demande ou signal

  -> domaine terrain : Bank / Trading / GPS / autres

  -> agents et signaux

  -> Sigma : aggregation

  -> Kernel X108 : ALLOW / HOLD / BLOCK

  -> API bridge : expose sans decider

  -> Proof / audit / hash / replay

  -> WorkCenter : comprehension, validation, pilotage
```

D2. Les 8 couches expliquees simplement

Couche	Explication enfant	Valeur investisseur	Autorite
Interface / WorkCenter	La tour de controle	Rend le systeme lisible et pilotable	Non souverain
Brody	Le professeur	Education, comprehension, support, adoption	Non souverain
Graphiti	Le carnet de memoire	Contexte, liens, navigation dans le savoir	Readonly, non souverain
Domaines terrain	Les mondes reels	Bank, Trading, GPS, Ecom, autres verticales	Signalent
Sigma	La table de tri	Agrege, compare, structure	Non souverain
Kernel X108	L arbitre / feu rouge	Decision ALLOW / HOLD / BLOCK	Souverain
Proof stack	La boite noire verifiee	Lean, TLA, Merkle, RFC3161	Verifie
Freeze / Audit	Le dossier de preuve	Capture l etat et permet replay	Scelle / prouve

D3. Le Kernel doit etre visible

Dans la presentation investisseur, le Kernel ne doit jamais etre perdu au milieu du WorkCenter. Le Kernel est le coeur du projet. Le WorkCenter est la surface qui rend ce coeur comprehensible et operable.

```
REGLE DE SLIDE :

Si une slide parle de securite, elle doit montrer le Kernel.

Si une slide parle de domaines terrain, elle doit montrer que les domaines ne decident pas seuls.

Si une slide parle de Brody ou Graphiti, elle doit montrer qu ils sont non souverains.
```

PARTIE E - PREUVES ET CREDIBILITE SANS NOYER

E1. Table proof-friendly

Element	Statut sobre	A dire	A ne pas dire
API 8000	CONFIRMED_BY_RUNTIME	API active, 164 routes OpenAPI, bridge readonly	API decide
Kernel 3001	CONFIRMED_BY_FREEZE	Kernel confirme par freeze, bridge cable	Kernel live permanent si ferme au recheck
Bank	FREEZE + RUNTIME bridge	5 ALLOW S1 en freeze	Deploiement bancaire reel
Trading	FREEZE + RUNTIME bridge	5 BLOCK S4 en freeze	Systeme trading production
GPS/Aviation	FREEZE + RUNTIME bridge	5 ALLOW S0 en freeze	Certification aviation
Brody	124/124 PASS	Education readonly non souveraine	Agent autonome souverain
Graphiti	FROZEN_READONLY	167 nodes, 477 relations, 20 episodes	Memoire live souveraine
Lean	lake build PASS	57 theoremes / lemmes verifies	Preuve production end-to-end totale
TLA+	Present non relance	Specs presentes, TLC a relancer	TLC revalide dans ce recheck
RFC3161	Anchor present	Ancrage present	openssl verify deja execute dans ce recheck

E2. Formule honnete

Ce n est pas encore un produit certifie. C est un noyau de gouvernance avance, deja structure, teste localement, auditable par statuts, avec une trajectoire claire vers industrialisation.

E3. Le style de preuve a vendre

Le point fort n est pas seulement le nombre de tests. Le point fort est la culture des statuts. Obsidia apprend a dire : ceci est prouve, ceci est confirme par freeze, ceci tourne maintenant, ceci est documente seulement, ceci ne doit pas etre claim.

PARTIE F - ROADMAP INVESTISSEUR SANS PREVISIONNEL CHIFFRE

F1. Phase 1 - WorkCenter de comprehension

- Build Cockpit readonly.
- Afficher Kernel, securite, domaines, preuves, memoire et limites.
- Rendre le projet comprehensible pour un tiers.
- Sortie attendue : WorkCenter V1 demo + freeze + guide utilisateur.

F2. Phase 2 - Education poussee

- Brody Education T1-T20.
- Parcours investisseur, dev, non-tech, auditeur, domaine.
- Exercices, quiz, lectures de preuves, patch proposals sans autorite.
- Sortie attendue : systeme apprenable par un utilisateur externe.

F3. Phase 3 - Durcissement et rechecks

- Relancer TLA/TLC.
- Verifier RFC3161 via openssl.
- Relancer adversarial tests dans le repo exact.
- Durcir Graphiti live / Neo4j auth.
- Tester ECOM live.
- Mettre CI complete et release process.

F4. Phase 4 - Audit externe et pilote controle

- Audit externe independant.
- Pilot POC avec donnees simulees ou anonymisees.
- Criteres metier clairs.
- Security review.
- Plan de certification domaine si secteur critique.

F5. Phase 5 - Workbench niveau Big Tech

- Agents encadres.
- Memory governance.
- Proof viewer.
- Replay lab.
- Sandbox.
- Multimodal.
- Audit center.
- Freeze factory.
- Dashboard autorite et securite.

F6. Bloc a faire plus tard : previsionnel

Cette V1 ne chiffre pas encore le besoin financier. La V2 devra ajouter : budget 3, 6, 12, 18 mois ; equipe minimale ; cout cloud ; cout audit ; cout juridique ; cout certification ; scenarios pilote ; jalons de financement ; revenus possibles ; pricing hypothesis.

F7. Bloc a faire plus tard : concurrence

Cette V1 ne cartographie pas encore la concurrence. La V2 devra comparer Obsidia a : LLM platforms, agent frameworks, copilots dev, RAG/memory tools, MLOps/observability, policy engines, GRC, audit AI, sécurité agentique, workbenches Big Tech.

PARTIE G - VERSIONS PAR PUBLIC

Public	Ce qu il veut comprendre	Message adapte
Investisseur generaliste	Comprendre le timing, le marche, l actif technique, la roadmap et le potentiel de defensabilite.	Obsidia controle le droit d agir des IA. Le projet est encore pre-industriel, mais les briques critiques sont deja structurees.
Investisseur deeptech	Voir le Kernel, les preuves, les statuts, les limites et l industrialisation.	Le projet combine gouvernance deterministe, preuves formelles, boundaries et workbench operable.
Banque / risque / conformite	Voir autorisation, suspension, blocage, trace, replay, audit.	Obsidia se place avant l action pour eviter qu une recommandation devienne execution non gouvernee.
CTO / dev senior	Comprendre architecture, API, Kernel, WorkCenter, tests, JSON, routes.	Tout est separe : API bridge, Kernel souverain, Brody non souverain, Graphiti readonly, proofs explicites.
Big Tech / IA infra	Comprendre l angle complementaire aux LLM et workbenches.	Les grands modeles deviennent capables ; Obsidia controle leur droit d agir.
Non-tech / incubateur	Comprendre sans code.	Obsidia est le feu rouge, la tour de controle et la boite noire des IA qui agissent.

PARTIE H - SCRIPTS DE PITCH

H1. Pitch 20 secondes

Obsidia / X108 est une couche de gouvernance pour agents IA. Quand un systeme IA propose une action, Obsidia decide si l action peut passer, doit attendre ou doit etre bloquee, avec trace, preuve et replay. Le but n est pas de faire un modele plus gros, mais de controler le droit d agir.

H2. Pitch 60 secondes

Les IA passent du texte a l action. Elles peuvent appeler des outils, lancer des workflows et influencer des decisions reelles. Le probleme, c est qu entre une recommandation et une action, l autorite est souvent floue. Obsidia / X108 construit cette couche d autorite. Le Kernel X108 retourne ALLOW, HOLD ou BLOCK. L API expose sans decider. Brody explique sans agir. Graphiti memorise en readonly. Sigma agrege les signaux. Les preuves, les freezes et le replay rendent la chaine auditable. Le projet est encore en pre-industrialisation, mais plusieurs briques sont deja stabilisees : API 8000 runtime, domaines Bank/Trading/GPS branches, Brody 124/124 tests, Graphiti frozen readonly, 57 theoremes Lean verifies.

H3. Pitch 2 minutes

Le marche IA est en train de changer. Les modeles ne se contentent plus de generer du texte ; ils deviennent le centre de workflows, d agents et d outils capables d agir. Dans ce contexte, la question strategique n est plus seulement 'quel modele est le plus intelligent ?' mais 'quelle architecture controle le passage a l action ?'. Obsidia / X108 repond a cette question. C est une architecture de gouvernance decisionnelle. Son coeur, le Kernel X108, tranche une action candidate en ALLOW, HOLD ou BLOCK. Autour de lui, les couches sont separees : l API est bridge only, Brody explique et eduque, Graphiti memorise en readonly, Sigma agrege les signaux, les domaines Bank/Trading/GPS apportent des terrains concrets, et la proof stack rend les claims auditables. L avantage n est pas de rivaliser frontalement avec les grands modeles sur la taille. L avantage est de construire la couche de controle qui deviendra necessaire autour des agents capables. La prochaine etape est de transformer ce noyau en WorkCenter industriel : une tour de controle qui montre l autorite, les preuves, les domaines, la memoire, les validations et les freezes.

H4. Pitch 5 minutes - structure

1. Pourquoi maintenant : les agents passent a l action.
2. Probleme : pas d autorite claire entre recommandation et action.
3. Solution : Obsidia gouverne le droit d agir.
4. Kernel X108 : ALLOW / HOLD / BLOCK.
5. Securite : separation des roles.
6. Terrains : Bank, Trading, GPS.
7. Brody, Graphiti, Sigma : utiles mais non souverains.
8. Proof stack : Lean, TLA, Merkle, RFC3161 avec statuts honnetes.
9. Approche deterministe : structure > masse, pas big training du coeur.
10. Roadmap : WorkCenter, education, durcissement, audit externe, pilote controle.

PARTIE I - QUESTIONS INVESTISSEUR A ANTICIPER

Question	Reponse courte
Est-ce un chatbot ?	Non. Un chatbot parle. Obsidia gouverne le droit d agir. Brody peut expliquer, mais il ne decide pas.
Est-ce un modele IA ?	Non. Obsidia est une architecture de gouvernance autour des agents et workflows. Les LLM peuvent etre utilises comme assistants, pas comme autorite souveraine.
Pourquoi pas juste utiliser un gros modele ?	Parce qu un gros modele peut proposer une action, mais il ne fournit pas automatiquement une autorite deterministe, un replay, un freeze, une preuve et une separation des roles.
Le projet est-il production-ready ?	Non. Il est en recherche appliquee / POC local avance / pre-industrialisation. Il faut audit externe, durcissement, certification et validation domaine avant usage critique.
Qu est-ce qui existe deja ?	Kernel freeze, API runtime 8000 avec 164 routes, domaines Bank/Trading/GPS branches, Brody 124/124 tests PASS, Graphiti frozen readonly, 57 theoremes Lean verifies.
Quel est le moat ?	La combinaison : Kernel de gouvernance, separation d autorite, proof culture, WorkCenter, memory governance, domaines terrain, education et audit/replay.
Pourquoi petite machine ?	Parce que le coeur de gouvernance est structurel : statuts, gates, proofs, replay, registries. Le projet ne depend pas du big training pour produire sa valeur centrale.
Quel premier marche ?	A clarifier selon strategie. Les terrains les plus naturels sont banque/risque/conformite, trading controle, e-commerce gouverne, agents dev/workbench gouvernes.
Que manque-t-il au support ?	Le previsionnel chiffre et la cartographie concurrentielle detaillee seront les blocs V2.

PARTIE J - STRUCTURE DE DECK INVESTISSEUR 20 SLIDES

Slide	Titre	Message
1	Titre	Obsidia / X108 - Gouverner le droit d agir des IA
2	Pourquoi maintenant	Les agents IA passent du texte a l action
3	Probleme	Aucune autorite claire entre recommandation et action
4	Image simple	Feu rouge, tour de controle, boite noire
5	Solution	ALLOW / HOLD / BLOCK avant l action
6	Kernel	Kernel X108, verrou souverain
7	Securite	Separation stricte des roles
8	Architecture	Signal -> domaines -> Sigma -> Kernel -> proof -> WorkCenter
9	Terrains	Bank / Trading / GPS deja branches
10	Brody	Pseudo-LLM educatif non souverain
11	Graphiti	Memoire readonly non souveraine
12	Sigma	Aggregation des signaux
13	Proof stack	Lean / TLA / Merkle / RFC3161
14	Pas big training	Structure deterministe > masse brute pour le coeur
15	Petite machine	Architecture lourde construite frugalement
16	Etat stabilise	API, Kernel, domaines, Brody, Graphiti, Lean
17	Limites	Non production-ready, audits et rechecks restants
18	Roadmap	WorkCenter, education, audit, pilote
19	Vision Big Tech	Workbench gouverne pour agents IA
20	Demande	Ressources, partenaires, audit, pilote, financement - a chiffrer en V2

PARTIE K - DO NOT CLAIM INVESTISSEUR

- Ne pas dire production-ready.
- Ne pas dire certifie banque, finance, aviation, defense ou critique.
- Ne pas dire que Brody decide.
- Ne pas dire que Graphiti decide.
- Ne pas dire que l API decide si le Kernel est down.
- Ne pas dire que Lean prouve tout le systeme de production de bout en bout.
- Ne pas dire que TLA/TLC a ete relance dans le recheck si ce n est pas fait.
- Ne pas dire que RFC3161 a ete verifie par openssl dans le recheck si ce n est pas fait.
- Ne pas dire plus besoin de donnees ou plus besoin de GPU pour tout.
- Dire plutot : le coeur de gouvernance ne depend pas du big training.
- Ne pas dire concurrentiel detaille ni previsionnel final dans cette V1.

PARTIE L - CONCLUSION

Obsidia / X108 doit etre presente comme une reponse a une question qui devient centrale : quand les IA deviennent capables d agir, qui controle leur droit d action ?

Le projet n est pas seulement un ensemble de scripts. C est une architecture : Kernel, securite, domaines, Brody, Graphiti, Sigma, proofs, freezes et WorkCenter. La valeur vient de l organisation des couches.

La formule finale est simple :

> ***Obsidia ne cherche pas seulement a rendre l IA plus intelligente. Obsidia cherche a rendre l action des IA gouvernable.***

FRIA PILOTE
Obsidia X108 - Scoring crédit

Évaluation d’impact sur les droits fondamentaux

Scénario hypothétique - aucun déploiement bancaire réel

Document de travail - 3 août 2026

Cette FRIA est une évaluation pilote fondée sur un scénario hypothétique. Obsidia n’est pas déployé dans une banque et aucune donnée client réelle n’est utilisée.

Voici la version remplie en prenant le cas A — scoring crédit. Important : il s'agit d'une FRIA pilote fondée sur un scénario hypothétique, car Obsidia n'est pas encore déployé dans une banque et aucune donnée client réelle n'est utilisée.

1. Contexte du projet

1.1 Nom du projet et fournisseur

- Nom : Obsidia X108 Proofs
- Fournisseur envisagé : Obsidia Lab
- Porteur du projet : Étienne Aubin, France
- Stade : recherche appliquée, POC local avancé et pré-industrialisation

1.2 Nature du système

Obsidia X108 est une architecture de gouvernance décisionnelle destinée à contrôler le passage entre cognition, intention, décision, action et preuve.

Son principe central est la séparation entre :

- les composants qui analysent, proposent ou prédisent ;
- les composants qui traduisent le contexte métier ;
- le kernel X-108, seul autorisé à produire une décision canonique ;
- les mécanismes d'exécution ;
- les preuves et traces permettant l'audit et le replay.

Un modèle probabiliste, Brody, la mémoire, Sigma, Obsidure ou un domaine métier ne peut pas devenir souverain. Cette règle est exprimée par `KX108_ONLY`.

GuardX108 examine notamment les contradictions, les inconnues, la confiance disponible et les indicateurs de risque. Le système peut produire :

- `HOLD` lorsque les informations ou validations sont insuffisantes ;
- `BLOCK` lorsqu'une structure, une action ou un niveau de risque est interdit ;
- `ACT` seulement lorsque les conditions prévues sont satisfaites.

Dans l'état actuel, les actions réelles restent désactivées, limitées ou soumises à validation humaine.

1.3 Statut actuel

Obsidia n'est pas encore déployé en production chez un établissement bancaire ou un client final.

Les éléments actuellement disponibles relèvent principalement :

- du développement local ;
- des tests unitaires et d'intégration ;
- des campagnes de vérification Lean ;
- de spécifications TLA+/TLC selon les composants ;
- de tests d'invariants, de refus et de replay ;
- de composants en lecture seule ou en mode `BRIDGE_ONLY` ;
- de démonstrateurs techniques bornés.

Cette FRIA ne constitue donc ni une évaluation d'un déploiement réel, ni une déclaration de conformité.

2. Cas d’usage cible

2.1 Cas retenu

Le cas retenu est :

> Obsidia comme couche de gouvernance d’un système bancaire évaluant la solvabilité d’une personne physique ou contribuant à une décision d’octroi de crédit.

Dans ce scénario hypothétique, un établissement bancaire utilise un système de scoring pour analyser une demande de crédit. Le moteur de scoring produit une estimation ou une recommandation. Obsidia reçoit ensuite un état métier structuré contenant notamment :

- la finalité de la demande ;
- les données et signaux autorisés ;
- le résultat du moteur de scoring ;
- les niveaux de confiance ;
- les inconnues ;
- les contradictions éventuelles ;
- les règles métier et réglementaires ;
- les validations nécessaires ;
- les indicateurs de risque.

Les décisions concernées pourraient être :

- demander des informations complémentaires ;
- transmettre le dossier à un analyste ;
- empêcher une décision automatique ;
- bloquer un chemin décisionnel interdit ;
- autoriser la poursuite vers une validation humaine.

Le demandeur de crédit est la principale personne concernée. Les employés de la banque chargés de la validation peuvent également être affectés par l’interface, les alertes et la répartition des responsabilités.

2.2 Rôle d’Obsidia

Dans le scénario pilote, Obsidia intervient :

1. après la collecte et la préparation des données ;
2. après ou autour du moteur de scoring ;
3. avant toute décision bancaire produisant un effet sur la personne ;
4. dans la boucle de contrôle et de validation ;
5. dans la génération des traces et preuves d’exécution.

Obsidia ne calcule pas nécessairement lui-même la solvabilité et ne remplace pas automatiquement le décideur bancaire.

Pour le pilote, son rôle doit rester :

- pré-exécution ;
- gouvernance et contrôle ;

- lecture seule ou recommandation encadrée ;
- sans rejet autonome définitif ;
- avec validation humaine obligatoire.

3. Personnes concernées

3.1 Catégories de personnes

Les catégories potentiellement concernées sont :

- demandeurs de crédit ;
- clients existants sollicitant une modification de crédit ;
- cautions ou co-emprunteurs ;
- employés bancaires analysant les dossiers ;
- équipes conformité, risques, audit et réclamations.

3.2 Groupes particulièrement vulnérables

Les groupes susceptibles d’être davantage exposés comprennent notamment :

- personnes en situation de précarité financière ;
- personnes surendettées ;
- personnes en situation de handicap ;
- personnes âgées ;
- personnes sous tutelle ou curatelle ;
- personnes ayant un historique financier limité ;
- nouveaux arrivants ou personnes dont les données sont incomplètes ;
- populations susceptibles de subir des discriminations directes ou indirectes.

La liste définitive dépendra du produit bancaire, du territoire et des données réellement utilisées.

3.3 Volume estimé

Le volume n’est pas déterminable à ce stade.

Obsidia ne dispose actuellement d’aucun déploiement bancaire réel permettant d’indiquer un nombre de personnes concernées. Le futur déployeur devra documenter :

- le volume annuel de dossiers ;
- le nombre de décisions assistées ;
- le nombre de personnes physiques concernées ;
- le nombre de décisions produisant un effet défavorable ;
- les catégories démographiques ou socio-économiques pertinentes.

4. Droits fondamentaux en jeu

Droit	Pertinence	Risque principal	Probabilité initiale	Gravité
Vie privée et données personnelles	Élevée	Collecte excessive, réutilisation hors finalité, données erronées ou obsolètes	Moyenne	Élevée
Non-discrimination	Élevée	Biais directs, variables proxy, traitement défavorable de certains groupes	Moyenne	Élevée
Égalité de traitement	Élevée	Dossiers similaires traités différemment sans justification	Moyenne	Élevée
Dignité et autonomie	Élevée	Réduction d'une personne à un score ou à un profil probabiliste	Moyenne	Élevée
Droit à une bonne administration et à un traitement équitable	Élevée	Décision opaque, incohérente ou impossible à contester	Moyenne	Élevée
Droit à un recours effectif	Élevée	Absence de canal humain, de preuve ou d'explication exploitable	Moyenne	Élevée
Liberté d'expression et d'information	Partielle	Information insuffisante sur l'usage de l'IA ou les facteurs de décision	Faible à moyenne	Moyenne
Droits des travailleurs	Partielle	Pression à suivre automatiquement la recommandation du système	Moyenne	Moyenne
Droits de l'enfant	À déterminer	Possible exposition indirecte dans certains produits ou dossiers familiaux	Indéterminée	Potentiellement élevée

Principaux scénarios de risque

RFD-01 — Discrimination indirecte

Une variable apparemment neutre peut agir comme proxy de l'origine, du sexe, de la situation sociale, du handicap ou du lieu de résidence.

RFD-02 — Données inexactes ou périmées

Une information erronée ou devenue obsolète peut conduire à une évaluation injustifiée.

RFD-03 — Décision automatisée opaque

La personne peut recevoir un refus sans comprendre les facteurs déterminants ni pouvoir corriger une erreur.

RFD-04 — Automation bias

L'analyste humain peut suivre la recommandation du système sans exercer de contrôle réel.

RFD-05 — Utilisation hors finalité

Des données collectées pour une fonction peuvent être réutilisées pour le scoring sans base ou justification appropriée.

RFD-06 — Absence de recours effectif

Les traces peuvent être insuffisantes, trop techniques ou inaccessibles à la personne concernée.

RFD-07 — Défaillance systémique

Une erreur de règle, de traduction métier ou de version peut affecter un grand nombre de dossiers.

5. Mesures intégrées ou prévues dans Obsidia

Mesure	Risque atténué	Statut
Séparation intelligence/autorité	Décision souveraine produite par un modèle probabiliste	Acquis techniquement
KX108_ONLY	Contournement de l'autorité canonique	Acquis techniquement
HOLD	Décision malgré des informations insuffisantes	Acquis techniquement
BLOCK	Exécution d'un chemin interdit ou contradictoire	Acquis techniquement
GuardX108	Contradictions, inconnues, confiance insuffisante, risques structurels	Acquis techniquement
Absence d'ACT implicite	Passage silencieux de recommandation à exécution	Acquis techniquement
Validation humaine	Décision défavorable entièrement automatique	Partiel
Receipts, événements et replay	Impossibilité de reconstruire une décision	Acquis techniquement, couverture à auditer
Provenance et fraîcheur de la mémoire	Utilisation de données sans origine ou obsolètes	Partiel
Sandbox et dry-run	Mutation ou activation non contrôlée	Acquis techniquement
Lean et TLA+/TLC	Violation de certaines propriétés formalisées	Partiel selon la couverture
Sigma	Détection d'instabilité ou d'incohérence après Guard	Partiel
Mécanismes de recours	Impossibilité de contester ou corriger	Manquant organisationnellement
Politique de conservation	Conservation excessive ou insuffisante	Manquant organisationnellement
Tests de discrimination	Biais directs ou indirects	À construire pour le domaine crédit

5.1 Séparation intelligence/autorité

Le moteur de scoring ou le LLM peut produire une analyse, mais il ne devient pas l'autorité finale. Cette séparation réduit le risque qu'une sortie probabiliste soit automatiquement transformée en décision bancaire.

Limite : il faudra démontrer que cette séparation est respectée dans toutes les intégrations, interfaces et configurations de production.

5.2 Mécanismes de refus

GuardX108 peut produire HOLD lorsque :

- les données sont incomplètes ;
- leur fraîcheur est insuffisante ;
- des contradictions existent ;
- le niveau de confiance est trop faible ;
- une validation humaine manque.

Il peut produire BLOCK lorsqu'un invariant ou une règle interdite est violé.

Pour le crédit, il faudra ajouter des invariants métier et droits fondamentaux, notamment :

- aucune décision défavorable sur une donnée non vérifiée ;

- aucune variable interdite ou proxy non contrôlé ;
- aucune absence d’explication minimale ;
- aucun rejet autonome lorsque la supervision humaine est obligatoire.

5.3 Supervision humaine

Le pilote doit imposer une validation humaine avant toute décision défavorable.

Le superviseur doit pouvoir :

- interrompre le traitement ;
- demander des informations complémentaires ;
- corriger une donnée ;
- contredire la recommandation ;
- transmettre le dossier à un niveau supérieur ;
- documenter la justification de son intervention.

La présence d’un humain ne suffit pas. Son autorité, sa compétence, son temps disponible et son indépendance doivent être démontrés.

5.4 Traçabilité et auditabilité

Obsidia prévoit ou possède des mécanismes de :

- journalisation d’événements ;
- enveloppes décisionnelles ;
- receipts ;
- identifiants de session ;
- hashes et manifests ;
- replay ;
- traçabilité des validations humaines.

Les fonctions précises de signature, de Merkle, d’ancrage temporel et leur couverture doivent être confirmées directement dans le dépôt avant d’être déclarées acquises.

Les traces devront permettre de savoir :

- quelle version du système a traité le dossier ;
- quelles données et règles ont été utilisées ;
- quels composants ont proposé ou contrôlé la décision ;
- pourquoi un `HOLD`, `BLOCK` ou passage en validation a été produit ;
- quel humain est intervenu ;
- quelles corrections ou contestations ont suivi.

5.5 Privacy-by-design et gouvernance des données

La mémoire est conçue comme non souveraine. Une information ne devient pas automatiquement vraie parce qu’elle est enregistrée.

Le pipeline envisagé est :

```
BRUT → candidat → validation/provenance → canonique → projection
```

Les propriétés telles que `RightsEnvelope`, la couche de preuve physique respectueuse de la vie privée, la suppression obligatoire ou la neutralisation du poison doivent être auditées et reliées à des implémentations précises avant d'être qualifiées d'acquises.

Pour le pilote crédit, il faudra formaliser :

- minimisation ;
- base et finalité du traitement ;
- provenance ;
- qualité ;
- fraîcheur ;
- correction ;
- accès ;
- durée de conservation ;
- suppression ;
- gestion des variables sensibles et proxies.

5.6 Transparence et explicabilité

Les refus structurés, les diagnostics de frontière, les limites documentées et le replay peuvent contribuer à l'explication.

Cependant, une preuve technique complète n'est pas automatiquement une explication intelligible pour une personne concernée.

Il faudra produire deux niveaux distincts :

- une explication technique pour l'audit ;
- une explication claire pour le client, exposant les facteurs déterminants, les limites et les moyens de recours.

5.7 Autres mesures

Les mesures supplémentaires pertinentes sont :

- sandbox avant activation ;
- tests adversariaux ;
- tests de non-contournement ;
- contrôle des mutations par Obsidure ;
- vérification Lean ;
- analyse TLA+/TLC ;
- gel de version ;
- séparation développement, validation et production ;
- red team spécialisée dans les biais de crédit.

Le principe « win-win-win » ou le double filtre éthique peut servir de doctrine interne, mais il ne remplace pas une analyse juridique des droits, des risques et des mesures.

6. Procédures organisationnelles

6.1 Gestion des incidents

Sigma, GuardX108, les alertes, les événements, les receipts et le replay peuvent détecter ou documenter certaines anomalies.

Une procédure complète doit encore définir :

- les critères d’incident ;
- les niveaux de gravité ;
- le responsable du triage ;
- les délais d’escalade ;
- les conditions de gel ;
- les autorités à notifier ;
- les mesures correctives ;
- l’information des personnes ;
- la conservation des preuves ;
- la réouverture du système.

Cette procédure n’est pas encore formalisée au niveau réglementaire.

6.2 Droits des personnes

Le futur déployeur bancaire devra mettre en place un canal permettant à une personne de :

- savoir qu’un système d’IA participe à l’évaluation ;
- demander une explication compréhensible ;
- contester la décision ;
- demander une révision humaine ;
- corriger une donnée inexacte ;
- exercer ses droits relatifs aux données ;
- déposer une réclamation ;
- recevoir une réponse dans un délai défini.

Obsidia pourra fournir les traces nécessaires, mais ne doit pas être présenté comme exerçant lui-même les responsabilités légales de la banque.

6.3 Rôles et responsabilités

Obsidia Lab

Responsable notamment de :

- la documentation du produit ;
- ses limites et instructions d’utilisation ;
- la maintenance ;
- les preuves techniques ;
- la sécurité de ses composants ;
- la gestion de ses versions ;
- la coopération avec le déployeur.

Déployeur bancaire

Responsable notamment de :

- la finalité d’utilisation ;

- la qualité des données d’entrée ;
- la supervision humaine ;
- la relation avec les personnes concernées ;
- la FRIA réelle ;
- l’AIPD lorsque nécessaire ;
- les recours ;
- la surveillance du déploiement ;
- les notifications relevant de sa responsabilité.

Rôles encore à nommer

- responsable conformité IA ;
- responsable de la FRIA ;
- DPO ;
- responsable des risques ;
- responsable sécurité ;
- responsable des incidents ;
- autorité humaine de validation.

7. Consultation et gouvernance

7.1 Parties consultées

À ce stade, aucune consultation formelle et documentée n’est établie avec :

- une banque déployeuse ;
- un DPO bancaire ;
- des représentants de clients ;
- des associations de consommateurs ;
- une autorité de surveillance ;
- un organisme d’évaluation ;
- un spécialiste de la non-discrimination financière.

Ces consultations restent à organiser pour transformer le scénario en FRIA réelle.

7.2 Gouvernance interne

Obsidia possède déjà des éléments de gouvernance technique :

- checklists ;
- tests ;
- manifests ;
- audits ;
- jalons de développement ;
- critères d’acceptation ;

- séparation des rôles entre proposition, preuve et décision ;
- contrôles avant intégration.

Ces éléments doivent être consolidés dans un système de gestion de la qualité couvrant :

- responsabilités ;
- gestion des changements ;
- incidents ;
- fournisseurs ;
- documentation ;
- risques ;
- actions correctives ;
- surveillance post-déploiement.

8. Suivi dans le temps

8.1 Mise à jour de la FRIA

La FRIA devra être réévaluée lorsque :

- la finalité du système change ;
- un nouveau produit de crédit est ajouté ;
- une nouvelle source de données est intégrée ;
- le modèle de scoring est remplacé ;
- une règle ou un seuil change ;
- l'autonomie du système augmente ;
- une nouvelle population est concernée ;
- un incident survient ;
- une discrimination ou dérive est détectée ;
- les mesures de supervision humaine évoluent.

Chaque mise à jour devra être reliée à :

- une version produit ;
- un commit ou artefact identifiable ;
- une campagne de tests ;
- une décision d'approbation ;
- une date d'entrée en application.

8.2 Indicateurs de suivi

Les indicateurs proposés sont :

1. taux de HOLD et motifs associés ;
2. taux de BLOCK par invariant ou catégorie de risque ;
3. taux de décisions humaines contraires à la recommandation initiale ;

4. nombre de contestations, corrections et révisions favorables ;
5. écarts de décisions et de taux d’erreur entre groupes pertinents ;
6. nombre d’incidents et délai moyen de traitement ;
7. nombre de décisions impossibles à rejouer ou expliquer ;
8. fréquence des données périmées, incomplètes ou contradictoires ;
9. taux de faux positifs et faux négatifs du contrôle ;
10. volume de décisions défavorables prises sans seconde validation.

Les indicateurs de groupes devront être définis avec une base juridique et des protections adaptées.

9. Conclusion et recommandations

9.1 Risques majeurs

Les principaux risques sont :

1. discrimination directe ou indirecte ;
2. décision fondée sur des données erronées, incomplètes ou périmées ;
3. absence d’explication et de recours effectif ;
4. dépendance excessive de l’opérateur humain à la recommandation ;
5. défaillance systémique affectant de nombreux dossiers ;
6. collecte ou réutilisation excessive de données.

9.2 Mesures prioritaires

Les priorités sont :

1. geler précisément le scénario de scoring et le rôle d’Obsidia ;
2. produire un registre des données, variables et proxies ;
3. formaliser les tests de discrimination et les seuils d’acceptation ;
4. définir la supervision humaine et la procédure de recours ;
5. produire la procédure d’incident et la politique de conservation ;
6. relier chaque mécanisme annoncé à une preuve, un test et une version ;
7. construire le dossier technique et le kit FRIA destiné au déployeur.

9.3 Appréciation globale

L’architecture d’Obsidia est structurellement compatible avec plusieurs objectifs de protection des droits fondamentaux :

- séparation entre intelligence et autorité ;
- refus explicite ;
- absence d’action implicite ;
- supervision humaine ;
- traçabilité ;
- replay ;
- contrôle des contradictions et inconnues.

Cependant, cette compatibilité architecturale ne démontre pas encore la conformité d’un système bancaire réel.

Les conditions principales sont :

- qualification réglementaire du cas d’usage ;
- données et modèles réels documentés ;
- tests de biais et de robustesse ;
- supervision humaine effective ;
- recours accessible ;
- responsabilités contractuelles claires ;
- procédures d’incident et de conservation ;
- dossier de preuves relié au runtime ;
- FRIA finalisée et validée par le déployeur.

Conclusion finale

Cette FRIA pilote montre qu’Obsidia peut devenir une infrastructure facilitant fortement l’évaluation des droits fondamentaux. Sa valeur ne réside pas seulement dans la production d’un document : elle réside dans la possibilité de relier chaque risque à un garde-fou, chaque garde-fou à une règle d’exécution, et chaque décision à une preuve auditable.

La FRIA définitive ne pourra néanmoins être produite qu’avec un établissement bancaire, un scénario gelé, des données identifiées et des responsabilités formellement attribuées.

Annexe A. Tableau de mapping FRIA - Cas scoring crédit (solvabilité)

Légende

Symbole	Signification
[ACQUIS]	Mécanisme acquis techniquement, déjà implémenté et testé dans Obsidia
[PARTIEL]	Mécanisme partiellement implémenté ou nécessitant consolidation
[À CONSTRUIRE]	Mécanisme à construire ou à formaliser pour le domaine crédit
[MANQUANT]	Mécanisme manquant organisationnellement : procédure, politique, etc.

Mapping risques ↔ garde-fous Obsidia ↔ obligations déployeur

Risque FRIA (RFD)	Description du risque	Garde-fous Obsidia	Statut	Obligations déployeur (banque)
RFD-01 - Discrimination indirecte	Variable apparemment neutre agissant comme proxy : origine, sexe, situation sociale, handicap ou lieu de résidence.	Séparation intelligence/autorité : le modèle de scoring ne devient pas souverain. KX108_ONLY : aucun contournement de l'autorité canonique. GuardX108 : détection de contradictions, inconnues et risques structurels. HOLD : suspension si confiance insuffisante ou variables suspectes. BLOCK : interdiction si invariant anti-discrimination violé.	[ACQUIS] Architecture. [À CONSTRUIRE] Invariants métier à construire.	Réaliser les tests de discrimination directs et indirects. Définir les seuils d'acceptation par groupe. Documenter les variables interdites et proxies. Renforcer la supervision humaine des décisions sensibles. Réaliser une AIPD si données sensibles.
RFD-02 - Données inexactes ou périmées	Une information erronée ou obsolète conduit à une évaluation injustifiée.	Provenance et fraîcheur de la mémoire. Triage BRUT → candidat → canonique. GuardX108 : détection des données incomplètes ou obsolètes. HOLD si fraîcheur insuffisante. Receipts, logs et replay pour tracer l'origine.	[PARTIEL] Concepts présents, à relier aux implémentations précises.	Définir une politique de qualité des données : minimisation, exactitude, mise à jour. Créer la procédure de correction des données erronées. Conservation et suppression selon le RGPD. Informer les personnes sur les données utilisées.
RFD-03 - Décision automatisée opaque	Refus sans compréhension des facteurs déterminants ni possibilité de correction.	Receipts, événements et replay. Session Ledger avec trace_id et span_id. Refus structuré : HOLD/BLOCK avec diagnostic. Reverse OS pour l'explicabilité technique.	[ACQUIS] Traçabilité technique. [PARTIEL] Explication intelligible à construire.	Produire une explication claire pour le client : facteurs déterminants et limites. Créer un canal de contestation et de révision humaine. Respecter le droit à l'explication : IA Act et RGPD. Mettre en place une procédure de recours accessible.
RFD-04 - Automation bias	L'analyste humain suit automatiquement la recommandation sans contrôle réel.	Séparation intelligence/autorité. Validation humaine obligatoire pour le pilote. Journal des décisions humaines. Possibilité de pause, annulation et correction.	[ACQUIS] Mécanismes présents. [PARTIEL] Effectivité à démontrer en production.	Former les analystes au biais d'automatisation. Définir la procédure de validation humaine : compétence, indépendance et temps. Mesurer le taux de validations contraires aux recommandations. Auditer régulièrement les pratiques.

Risque FRIA (RFD)	Description du risque	Garde-fous Obsidia	Statut	Obligations déployeur (banque)
RFD-05 - Utilisation hors finalité	Données collectées pour une fonction puis réutilisées pour le scoring sans base.	RightsEnvelope : conservation, suppression et interdictions. Mémoire non souveraine : donnée ≠ vérité. Provenance et contractualisation des usages.	[PARTIEL] Concepts présents, à formaliser juridiquement.	Documenter la base légale et la finalité de chaque traitement. Réaliser AIPD/FRIA pour les réutilisations. Tenir le registre des traitements RGPD. Contrôler les accès et auditer les usages.
RFD-06 - Absence de recours effectif	Traces insuffisantes, trop techniques ou inaccessibles.	Receipts signés, logs et replay. Session Ledger exhaustif. Cognitive Debugger pour inspection.	[ACQUIS] Traçabilité. [MANQUANT] Canal de recours à construire.	Mettre en place un canal de contestation : délai et responsable. Produire une explication intelligible et non technique. Définir la procédure de révision humaine. Informer sur les droits RGPD et IA Act.
RFD-07 - Défaillance systémique	Erreur de règle, traduction métier ou version affectant de nombreux dossiers.	Sandbox et dry-run avant activation. Tests unitaires, intégration et E2E. Lean et TLA+/TLC pour les propriétés formalisées. Sigma : détection d'instabilité et d'incohérence. Gel de version : freezes.	[ACQUIS] Tests et preuves. [PARTIEL] Couverture à auditer pour le domaine crédit.	Formaliser la gestion des incidents : critères, gravité et escalade. Créer le plan de rollback et de continuité d'activité. Informer les personnes affectées. Notifier les autorités en cas d'incident grave.

Synthèse par catégorie

Catégorie	Couverture Obsidia	Travaux restants
Séparation intelligence/autorité	[ACQUIS] Acquis	Démontrer le respect dans toutes les intégrations de production.
Mécanismes de refus HOLD/BLOCK	[ACQUIS] Acquis	Ajouter les invariants métier et droits fondamentaux pour le crédit.
Supervision humaine	[ACQUIS] Mécanismes acquis. [PARTIEL] Effectivité à démontrer.	Définir compétence, indépendance et temps disponible des validateurs.
Traçabilité et replay	[ACQUIS] Acquis	Relier chaque fonction à une implémentation précise : signature, Merkle, RFC3161.
Privacy-by-design	[PARTIEL] Partiel	Auditer et relier RightsEnvelope, couche privacy et conservation à des implémentations.
Explicabilité	[ACQUIS] Technique. [PARTIEL] Intelligible.	Produire deux niveaux : explication technique pour l'audit et explication client.
Tests de discrimination	[À CONSTRUIRE] À construire	Développer tests spécifiques aux biais crédit, seuils et métriques.
Procédure d'incident	[MANQUANT] Manquant	Formaliser critères, délais, autorités et information des personnes.
Politique de conservation	[MANQUANT] Manquant	Définir durées applicables, suppression et archivage.

Catégorie	Couverture Obsidia	Travaux restants
Canal de recours	[MANQUANT] Manquant	Mettre en place contestation, révision et droits RGPD.

Recommandations prioritaires

1. Geler le scénario de scoring : finalité exacte, données, variables et règles métier.
2. Construire les invariants droits fondamentaux : interdiction des variables sensibles, proxies non contrôlés et absence d’explication.
3. Formaliser les tests de discrimination : métriques, seuils, campagnes de tests et red team.
4. Définir la supervision humaine : qui valide, comment, avec quel temps, quelle indépendance et quel recours.
5. Produire la procédure d’incident et de conservation : critères, délais, autorités et durées.
6. Relier chaque mécanisme à une preuve : test, commit, version et propriété formalisée.
7. Construire le kit FRIA déployeur : dossier technique, explications et modèles de procédures.

Limite documentaire

Cette FRIA pilote ne constitue ni une FRIA réglementaire finalisée par un déployeur bancaire, ni une déclaration de conformité. Elle devra être complétée avec un scénario gelé, les données réelles, les responsabilités attribuées et les procédures du déployeur.

OBSIDIA X108
État d’alignement avec l’AI Act

Cartographie technique, écarts réglementaires et plan de fermeture

Version de travail - 3 août 2026

Périmètre : Obsidia X108 Proofs / architecture de gouvernance décisionnelle

Position nette

Obsidia présente déjà des fondations techniques fortes pour la gouvernance d’un système d’IA : autorité séparée de la cognition, refus explicite, traçabilité, replay, contrôle humain et preuves formelles. En revanche, le projet ne peut pas encore être déclaré juridiquement conforme pour une mise sur le marché à haut risque. L’écart principal est documentaire, organisationnel et lié à la qualification de chaque cas d’usage.

Ce document est un état de préparation, pas un avis juridique, une certification, une déclaration UE de conformité ni une qualification définitive au titre de l’article 6.

1. Cadre et méthode

Référentiel principal : Règlement (UE) 2024/1689, complété par les pages officielles de la Commission européenne disponibles au 3 août 2026. L’AI Act s’applique par étapes. Les interdictions et obligations de maîtrise de l’IA s’appliquent depuis le 2 février 2025 ; les obligations GPAI et une partie de la gouvernance depuis le 2 août 2025 ; les obligations de transparence de l’article 50 depuis le 2 août 2026. Après l’AI Omnibus entré en vigueur le 27 juillet 2026, les règles haut risque de l’Annexe III s’appliquent à partir du 2 décembre 2027 et celles relatives aux systèmes intégrés à certains produits réglementés à partir du 2 août 2028.

Statut	Lecture exacte
ACQUIS TECHNIQUE	Capacité présente/testée selon les éléments fournis. Ne vaut pas conformité juridique.
PARTIEL	Base existante mais couverture, preuve, procédure ou généralisation incomplète.
MANQUANT ORG.	Livrable ou procédure réglementaire non matérialisé.
À VÉRIFIER	Impossible à figer sans audit direct du dépôt et du futur produit.

Corrections apportées aux tableaux sources

• Un secteur n’est pas automatiquement ‘haut risque’ : la finalité prévue et la catégorie exacte de l’article 6 / Annexes I ou III doivent être qualifiées scénario par scénario.
• Une sandbox de développement n’est pas un bac à sable réglementaire.
• Un audit externe ou un organisme notifié n’est pas systématiquement requis pour toutes les configurations ; la voie d’évaluation dépend du système et du cadre applicable.
• L’article 17 exige un système de gestion de la qualité, pas obligatoirement une certification ISO 9001.
• Une architecture de blocage ne démontre pas à elle seule le respect de l’article 5.
• Les chiffres de tests, théorèmes et composants doivent rester rattachés à un commit, une campagne et un rapport reproductible.

2. Matrice principale d’alignement

Lecture : le statut porte sur la préparation d’Obsidia, pas sur une conformité déclarée.

Exigence / articles	État Obsidia	Statut	Preuve / réserve
Qualification du système (art. 3, 6 ; ann. I/III)	Obsidia est une couche de gouvernance ; les cibles banque, trading, GPS, aviation et défense couvrent des usages hétérogènes.	PARTIEL	Créer une fiche de finalité prévue par produit et cas d’usage. Aucun classement global n’est défendable aujourd’hui.
Pratiques interdites (art. 5)	KX108_ONLY, no bypass, HOLD/BLOCK et absence d’ACT implicite réduisent certains chemins de dérive.	PARTIEL	Il faut une matrice explicite des pratiques interdites et des tests par usage. L’architecture seule ne suffit pas.
Maîtrise de l’IA (art. 4)	Documentation, terminal et couche éducative existent conceptuellement.	PARTIEL	Programme, rôles, compétences attendues et preuves de formation non formalisés.
Gestion des risques (art. 9)	GuardX108, contradictions, unknowns, confidence, risk flags, HOLD/BLOCK ; Sigma surveille et gèle sans décider.	ACQUIS TECHNIQUE	Très bonne fondation ex ante ; reste à la convertir en processus continu de cycle de vie avec critères d’acceptation.
Données et gouvernance (art. 10)	Mémoire non souveraine, provenance/fraîcheur, BRUT → candidat → canonique, projection readonly.	PARTIEL	Manquent le registre des jeux de données, les critères qualité/représentativité, biais, nettoyage, versionnage et responsabilités.
Documentation technique (art. 11 ; ann. IV)	README, specs, contrats runtime, manifests, preuves, tests, receipts et limites fournissent une matière riche.	PARTIEL	Pas encore de dossier Annexe IV consolidé, versionné et relié à une configuration produit gelée.
Enregistrements (art. 12, 19)	Événements, receipts, enveloppes, hashes, manifests, sessions et replay ; certaines couches append-only.	ACQUIS TECHNIQUE	Auditer couverture, horodatage, intégrité, accès, export, durées de conservation et données personnelles.
Transparence / instructions (art. 13)	Limites, non-souveraineté et rôles sont explicités dans la doctrine.	PARTIEL	Produire des instructions déployeur : finalité, précision, limites, supervision, logs, données d’entrée, maintenance.
Surveillance humaine (art. 14)	Validation humaine avant action réelle, HOLD, rollback, sandbox ; runtime ACT désactivé ou borné.	ACQUIS TECHNIQUE	Définir compétence, autorité, charge cognitive, anti-automation bias, escalade et preuve d’intervention.
Exactitude, robustesse, cybersécurité (art. 15)	Tests Python/intégration/replay, Lean, TLA+/TLC, adversarial tests et fail-closed.	PARTIEL	Geler métriques par contexte, seuils, threat model, sécurité supply-chain, pentest et gestion vulnérabilités.
Système qualité (art. 17)	Checklists, audits, Definition of Done, manifests, workflow contrôlé de mutation.	PARTIEL	Créer un QMS produit couvrant responsabilités, changements, fournisseurs, non-conformités, CAPA et surveillance.
Conservation (art. 18)	Git, journaux et manifests donnent la capacité technique d’archiver.	MANQUANT ORG.	Politique de conservation, durée applicable, gel de versions, droits d’accès et destruction à formaliser.
Actions correctives / coopération (art. 20-21)	Gel, rollback, correction contrôlée et diagnostic existent techniquement.	PARTIEL	Procédure de rappel/retrait/suspension et canal autorités non formalisés.
Obligations déployeur (art. 26)	Human Validation Queue et contrats readonly peuvent supporter le contrôle déployeur.	PARTIEL	À transformer en pack déployeur : responsabilités, monitoring, inputs, logs, incidents, information des personnes.

Exigence / articles	État Obsidia	Statut	Preuve / réserve
FRIA (art. 27)	Fondations de privacy et gouvernance présentes.	MANQUANT ORG.	La FRIA dépend du déployeur et d’un cas concret ; produire un modèle puis une FRIA par déploiement concerné.
Évaluation de conformité (art. 43)	Audits et preuves internes présents.	MANQUANT ORG.	Choisir la voie applicable après qualification. Un organisme notifié n’est pas automatiquement requis.
Déclaration UE / CE / enregistrement (art. 47-49, 71)	Non réalisés au stade R&D;/pré-industrialisation.	MANQUANT ORG.	À produire seulement pour la configuration mise sur le marché et lorsque les obligations applicables sont déterminées.
Transparence interaction/contenus (art. 50)	Brody et les futures fonctions de génération peuvent entrer dans le champ selon l’interface et la sortie.	PARTIEL	Depuis le 2 août 2026 : mécanismes d’information/étiquetage à implémenter selon le cas.
Surveillance post-marché (art. 72)	Sigma, alertes, audit et replay forment un socle de détection.	PARTIEL	Plan post-market, collecte terrain, tendances, indicateurs, périodicité et gouvernance manquants.
Incidents graves (art. 73)	Diagnostic, gel et preuves techniques disponibles.	MANQUANT ORG.	Définir qualification, délais, destinataires, dossier, responsabilités, communication et suivi.
Réclamations / explication / lanceurs (art. 85-87)	Aucun dispositif organisationnel complet identifié.	MANQUANT ORG.	Créer canaux, délais, traçabilité, protection, réponse et articulation avec l’explication des décisions.
GPAI (art. 51-56)	Le kernel Obsidia n’est pas présenté comme un modèle GPAI ; Brody est décrit comme LLM propriétaire.	À VÉRIFIER	Qualifier Brody séparément : architecture, modalités de distribution, capacités, entraînement et rôle dans la chaîne de valeur.

3. État des briques Obsidia

Brique	État constaté	Portée réglementaire	Limite actuelle
Kernel X-108 / GuardX108	Exécutable et testé ; HOLD/BLOCK/ACT ; KX108_ONLY.	Risque, supervision, robustesse, actions correctives.	Une propriété architecturale n'est pas une preuve de conformité d'un usage.
Sigma	Surveillance post-Guard : entropie, dissipation, instabilité, cohérence, anti-mismatch.	Monitoring, post-market, alertes.	Ne décide pas ; procédures et seuils réglementaires à formaliser.
Mémoire	Structurée, non souveraine, projection readonly ; provenance et fraîcheur prévues.	Gouvernance des données, traçabilité.	Audit complet sessions/canonisation/automatisation encore ouvert.
Brody	LLM propriétaire reliant la mémoire ; non souverain.	Transparence, GPAI éventuel, données, cybersécurité.	Qualification juridique séparée impossible sans dossier modèle complet.
Obsidure + Lean	Obsidure propose ; Lean vérifie ; aucun ne décide. Workflow sandbox → check → intégration.	Robustesse, change control, preuves.	Preuve sur modèle/artefact à relier explicitement au runtime et au commit.
TLA+/TLC	Spécifications et exploration d'états selon les composants.	Transitions, invariants, sûreté.	Couverture et correspondance runtime à auditer fichier par fichier.
API / Cockpit	API BRIDGE_ONLY ; emits_act=false ; cockpit V0 readonly.	Interfaces, contrôle d'accès, information déployeur.	Brody/Reverse/Obsidure pas entièrement branchés.
Receipts / replay	Enveloppes, événements, hashes, manifests, replay.	Logs, auditabilité, preuve d'action.	Rétention, confidentialité, export et couverture complète à démontrer.
Path Compute / Gencoin	Path Compute V0 désactivé ; Gencoin shadow readonly.	Hors périmètre produit actif à ce stade.	Ne pas les inclure dans une déclaration produit avant activation et qualification.

Preuves quantitatives à employer avec discipline

Le benchmark public AMD mentionne 369 tests PASS, 180/180 invariants, 15 appels distants évités sur 18, environ 75 % de tokens évités et 0/8 violation dans la campagne concernée. Ces résultats sont pertinents comme preuve de démonstrateur borné. Ils ne doivent pas être généralisés à Obsidia entier ni être présentés comme validation réglementaire.

4. Tableau créé : reste à faire

Ce registre regroupe les fermetures nécessaires. P0 = bloque la qualification ou une mise sur le marché ; P1 = nécessaire avant pilote encadré ; P2 = industrialisation continue.

ID	Priorité	Livrable à produire	Critère de fermeture	Dépendance / propriétaire
R-01	P0	Registre des produits et finalités prévues	Une fiche gelée par configuration : utilisateur, décision, données, action, territoire, version.	Architecture + juridique
R-02	P0	Qualification article 6 / Annexes	Note argumentée par cas : banque, trading, GPS, aviation, défense ; exclusions et hypothèses tracées.	R-01 + conseil juridique
R-03	P0	Matrice article 5	Chaque pratique interdite : applicable/non applicable, contrôle préventif, test, preuve, responsable.	R-01
R-04	P0	Dossier technique Annexe IV	Dossier complet, versionné, indexé, relié à un commit et à une configuration runtime reproductible.	R-01/R-02 + tech
R-05	P0	Evidence registry	Chaque revendication pointe vers test, rapport, hash, commit, périmètre et date ; aucune métrique orpheline.	Tech / audit
R-06	P0	QMS AI Act	Procédures approuvées : rôles, conception, tests, fournisseurs, changements, non-conformités, CAPA, post-market.	Direction + qualité
R-07	P0	Plan de gestion des risques	Hazard log vivant, mesures, risques résiduels, seuils d’acceptation, validation et revue périodique.	R-02 + domaines
R-08	P0	Voie d’évaluation de conformité	Décision documentée : contrôle interne ou tiers selon la configuration ; organisme identifié si requis.	R-02/R-04/R-06
R-09	P1	Registre données et mémoire	Datasheets, provenance, licences, qualité, biais, fraîcheur, versionnage, accès, suppression et tests.	Data / mémoire
R-10	P1	Threat model + cybersécurité	Modèle de menace, SBOM, secrets, dépendances, vulnérabilités, pentest, patching, incident security.	Sécurité
R-11	P1	Pack supervision humaine	Rôles, compétences, autorité d’arrêt, interfaces, charge, anti-biais d’automatisation, journal de validation.	UX + métier
R-12	P1	Instructions déployeur	Finalité, inputs, métriques, limites, intégrations, logs, supervision, maintenance, incidents et recours.	R-04 + produit
R-13	P1	Politique de conservation	Durées applicables, legal hold, intégrité, chiffrement, accès, export, destruction, preuve de purge.	Juridique + sécurité
R-14	P1	Procédure incidents graves	Critères, horloge, triage, autorités, dossier, corrections, communication, exercices de simulation.	Qualité + juridique
R-15	P1	Plan post-market	Indicateurs terrain, collecte, seuils, tendances, revue, actions correctives, gouvernance et rapports.	R-06/R-14
R-16	P1	Modèle FRIA + DPIA	Templates validés et exercice complet sur un pilote concret ; articulation RGPD documentée.	Déployeur + DPO/juridique
R-17	P1	Transparence article 50	Information machine, marquage/étiquetage des sorties applicables, tests UX et preuves de conformité.	Brody + interfaces

ID	Priorité	Livrable à produire	Critère de fermeture	Dépendance / propriétaire
R-18	P1	Qualification autonome de Brody	Décision documentée sur système/modèle/GPAI, chaîne de valeur, distribution, données, obligations propres.	Équipe modèle + juridique
R-19	P2	Canaux autorité, plainte et explication	SLA, identité responsable, traçabilité, accès aux preuves, réponse intelligible, protection lanceurs.	Support + juridique
R-20	P2	Pilote réglementaire borné	Un seul usage, données maîtrisées, ACT désactivé ou contrôlé, critères entrée/sortie, rapport final.	Tous P0 + principaux P1
R-21	P2	Déclaration UE / CE / base UE	Produits uniquement quand applicables, après évaluation et avant mise sur le marché.	R-08 + conformité

5. Ordre d’exécution recommandé

L’ordre vise à éviter de documenter un produit qui change encore de périmètre.

Lot	Objectif	Sortie gelée	Go / No-Go
A - Périmètre	Séparer les produits et scénarios ; choisir un pilote borné.	R-01, R-02, R-03	Aucune conformité sans finalité prévue stable.
B - Preuves	Relier architecture, runtime, tests et claims.	R-04, R-05, R-07	Tous les claims critiques ont une preuve reproductible.
C - Organisation	Créer la colonne vertébrale qualité et responsabilité.	R-06, R-08, R-13, R-14	Rôles, procédures et voie de conformité approuvés.
D - Produit	Fermer données, sécurité, humain, déployeur et transparence.	R-09 à R-12, R-17, R-18	Pilote techniquement et juridiquement borné.
E - Terrain	Exécuter un pilote contrôlé et mesurer les écarts.	R-15, R-16, R-19, R-20	Rapport pilote + risques résiduels acceptés.
F - Marché	Finaliser les formalités applicables.	R-21	Mise sur le marché seulement après fermeture documentée.

Recommandation structurante

Ne pas tenter une ‘conformité Obsidia globale’. Geler d’abord une configuration minimale : un domaine, une finalité, un type d’utilisateur, une chaîne de données, un mode d’action et un niveau de supervision. La machinerie générale restera le socle ; le dossier réglementaire portera sur le produit effectivement mis sur le marché.

Premier pilote conseillé du point de vue conformité

Un mode d’aide à la décision strictement readonly, sans ACT autonome, avec données non personnelles ou synthétiques, humain responsable, receipts complets et périmètre contractuel fermé. Ce choix ne supprime pas les obligations applicables, mais réduit l’ambiguïté et permet de prouver la chaîne complète avant d’ouvrir les domaines les plus sensibles.

6. Conclusion de positionnement

Obsidia n’est pas ‘conforme IA Act’ aujourd’hui. Il est toutefois mieux préparé que beaucoup de prototypes sur le noyau technique de gouvernance : séparation cognition/autorité, défaut de souveraineté du LLM, refus explicite, preuve avant action, logs, replay, supervision humaine et formalisation.

Le chantier restant n’est pas une couche cosmétique de conformité. Il consiste à convertir cette architecture en système de management, dossier produit, procédures opposables et preuves attachées à une configuration exacte. C’est précisément là que le projet peut transformer sa profondeur technique en avantage industriel auditable.

Sources officielles

[1] Règlement (UE) 2024/1689 - EUR-Lex : texte officiel.
[2] Commission européenne - Cadre réglementaire de l’IA : présentation et calendrier.
[3] Commission européenne - AI Omnibus, entrée en vigueur le 27 juillet 2026 : mise à jour.
[4] Commission européenne - Standardisation de l’AI Act : standards et présomption de conformité.
[5] Commission européenne - Transparence des contenus générés : article 50.

Sources projet

Deux documents fournis dans cette conversation : (a) tableau article par article initial ; (b) comparatif IA Act / Obsidia X108 Proofs. Les affirmations projet ont été recalibrées avec les précisions techniques fournies : état R&D;/pré-industrialisation, runtime readonly/bridge-only, preuves et campagnes bornées.

Limite

Ce document ne remplace ni l’audit du dépôt, ni l’analyse d’un produit gelé, ni un conseil juridique spécialisé. Les statuts doivent être mis à jour à chaque changement de finalité, modèle, domaine, données, interface, autonomie ou chaîne de déploiement.